

Стюарт Рассел, Питер Норвиг

Искусственный интеллект: современный подход, 2-е изд.

Часть II. Решение проблем

Содержание

Часть II. Решение проблем	1
Глава 3. Решение проблем посредством поиска	1
Глава 4. Информированный поиск и исследование пространства состояний	43
Глава 5. Задачи удовлетворения ограничений	100
Глава 6. Поиск в условиях противодействия	132

Часть II. Решение проблем

состояний

Глава 3. Решение проблем посредством поиска

3 Решение проблем посредством ПОИСКА В этой главе показано, каким образом агент может найти последовательность действий, позволяющую достичь его целей в тех условиях, когда единственного действия для этого недостаточно.

Простейшими агентами, которые рассматривались в главе 2, были рефлексные агенты, функционирование которых основано на непосредственном отображении состояний в действия. Подобные агенты не могут успешно действовать в тех вариантах среды, где такие отображения были бы слишком большими, чтобы можно было обеспечить их хранение, а изучение отображений потребовало бы слишком много времени. Агенты на основе цели, с другой стороны, способны достичь успеха, рассматривая будущие действия и оценивая желательность их результатов.

Данная глава посвящена описанию одной разновидности агента на основе цели, называемой $\wedge$ агентом, решающим задачи. Агенты, решающие задачи, определяют, что делать, находя последовательности действий, которые ведут к желаемым состояниям. Начнем изложение этой темы с точного определения элементов, из которых состоит "задача" и ее "решение", и приведем несколько примеров для иллюстрации этих определений. Затем представим несколько алгоритмов поиска общего назначения, которые могут использоваться для решения подобных задач, и проведем сравнение преимуществ и недостатков каждого алгоритма. Эти алгоритмы являются неинформированными в том смысле, что в них не используется какая-либо информация о рассматриваемой задаче, кроме ее определения. В главе 4 речь идет об информированных алгоритмах поиска, в которых используются определенные сведения о том, где следует искать решения.

В данной главе применяются понятия из области анализа алгоритмов. Читатели, незнакомые с понятиями асимптотической сложности (т.е. с системой обозначений $O()$ и NP-полноты, должны обратиться к приложению А.

3.1. Агенты, решающие задачи Предполагается, что интеллектуальные агенты обладают способностью максимизировать свои показатели производительности. Как уже упоминалось в главе 2, реа-

лизация указанного свойства в определенной степени упрощается, если агент способен принять на себя обязанность достичь цели и стремиться к ее удовлетворению.

Вначале рассмотрим, как и почему агент может приобрести такую способность.

Представьте себе, что некоторый агент находится в городе Арад, Румыния, и проводит свой отпуск в качестве туриста. Показатели производительности агента включают много компонентов: он хочет улучшить свой загар, усовершенствовать знание румынского языка, осмотреть достопримечательности, насладиться ночной жизнью (со всеми ее привлекательными сторонами), избежать неприятностей и т.д. Эта задача принятия решения является сложной; для ее выполнения необходимо учитывать множество компромиссов и внимательно читать путеводители. Кроме того, предположим, что у агента имеется не подлежащий возмещению билет для вылета из Бухареста на следующий день. В данном случае для агента имеет смысл стремиться к достижению цели попасть в Бухарест. Способы действий, не позволяющие вовремя попасть в Бухарест, могут быть отвергнуты без дальнейшего рассмотрения и поэтому задача принятия решения агентом значительно упрощается. Цели позволяют организовать поведение, ограничивая выбор промежуточных этапов, которые пытается осуществить агент. Первым шагом в решении задачи является ^ формулировка цели с учетом текущей ситуации и показателей производительности агента.

Мы будем рассматривать цель как множество состояний мира, а именно тех состояний, в которых достигается такая цель. Задача агента состоит в том, чтобы определить, какая последовательность действий приведет его в целевое состояние. Прежде чем это сделать, агент должен определить, какого рода действия и состояния ему необходимо рассмотреть. Но если бы агент пытался рассматривать действия на уровне "перемещения левой ноги вперед на один сантиметр" или "поворота рулевого колеса на один градус влево", то, по-видимому, так и не смог бы выехать с автомобильной стоянки, не говоря уже о своевременном прибытии в Бухарест, поскольку на таком уровне детализации мир обладает слишком большой неопределенностью, а решение должно состоять из слишком многих этапов. ^ Формулировка задачи представляет собой процесс определения того, какие действия и состояния следует рассматривать с учетом некоторой цели. Этот процесс

будет описан более подробно немного позднее. А на данный момент предположим, что агент будет рассматривать действия на уровне автомобильной поездки из одного крупного города в другой. Таким образом, состояния, рассматриваемые агентом, соответствуют его пребыванию в конкретном городе¹.

Итак, наш агент поставил перед собой цель доехать на автомобиле до Бухареста и определяет, куда отправиться для этого из Арада. Из этого города ведут три дороги: в Сибиу, Тимишоару и Зеринд. Прибытие в какой-либо из этих городов не представляет собой достижение намеченной цели, поэтому агент, не очень знакомый с географией Румынии, не будет знать, по какой дороге он должен следовать². Иными словами, агент не знает, какое из его возможных действий является наилучшим, поскольку не обладает достаточными знаниями о состоянии, возникающем в результа¹ Обратите внимание на то, что каждое из этих "состояний" фактически соответствует большому множеству состояний мира, поскольку в состоянии реального мира должен быть определен каждый аспект действительности. Важно всегда учитывать различие между состояниями, которые рассматриваются в проблемной области решения задач, и состояниями реального мира.

² Авторы предполагают, что большинство читателей окажутся в таком же положении и вполне могут представить себя такими же беспомощными, как и наш агент. Приносим свои извинения румынским читателям, которые не смогут воспользоваться преимуществами использованного здесь педагогического приема. те выполнения каждого действия. Если агент не получит дополнительных знаний, то окажется в тупике. Самое лучшее, что он может сделать, — это выбрать одно из указанных действий случайным образом.

Но предположим, что у агента есть карта Румынии, либо на бумаге, либо в его памяти. Карта способна обеспечить агента информацией о состояниях, в которых он может оказаться, и о действиях, которые он способен предпринять. Агент имеет возможность воспользоваться этой информацией для определения последовательных этапов гипотетического путешествия через каждый из этих трех городов в попытке найти такой путь, который в конечном итоге приведет его в Бухарест. Найдя на карте путь от Арада до Бухареста, агент может достичь своей цели, осуществляя действия по вождению автомобиля, которые соответствуют этапам этого путешествия. Вообще говоря, $s^{\wedge}$ = агент, имеющий несколько непосредственных вариантов выбора с неизвестной стоимостью, может решить, что делать, исследуя вначале различные возможные последовательности действий, которые ведут к состояниям с известной стоимостью, а затем выбирая из них наилучшую последовательность.

Описанный процесс определения такой последовательности называется $s^{\wedge}$ поиском.

Любой алгоритм поиска принимает в качестве входных данных некоторую задачу и возвращает $\wedge$ решение в форме последовательности действий. После того как решение найдено, могут быть осуществлены действия, рекомендованные этим алгоритмом. Такое осуществление происходит на стадии $\wedge$ выполнения. Итак, для рассматриваемого агента можно применить простой проект, позволяющий применить принцип "сформулировать, найти, выполнить", как показано в листинге 3.1. После формулировки цели и решаемой задачи агент вызывает процедуру поиска для решения этой задачи. Затем он использует полученное решение для руководства своими действиями, выполняя в качестве следующего предпринимаемого мероприятия все, что рекомендовано в решении (как правило, таковым является первое действие последовательности), а затем удаляет этот этап из последовательности. Сразу после выполнения этого решения агент формулирует новую цель.

Листинг 3.1. Простой агент, решающий задачу. Вначале он формулирует цель и задачу, затем ищет последовательность действий, позволяющую решить эту задачу, и, наконец, осуществляет действия одно за другим. Закончив свою работу, агент формулирует другую цель и начинает сначала. Обратите внимание на то, что при выполнении очередной последовательности действий агент игнорирует данные своих актов восприятия, поскольку предполагает, что найденное им решение всегда выполнимо

```
function Simple-Problem-Solving-Agent(percept) returns действие action
inputs: percept, результат восприятия static: seq, последовательность действий,
первоначально пустая state, некоторое описание текущего состояния мира goal,
цель, первоначально неопределенная problem, формулировка задачи state  $\leftarrow$ 
Update-State (state, percept) if последовательность seq пуста then do goal  $\leftarrow$ 
Formulate-Goal(state) problem  $\leftarrow$  Formulate-Problem(state, goal) seq  $\leftarrow$  Search
(problem) action  $\leftarrow$  First(seq) seq  $\leftarrow$  Rest (seq) return action
```

Вначале опишем процесс составления формулировки задачи, а затем посвятим основную часть данной главы рассмотрению различных алгоритмов для функции Search. В этой главе осуществление функций Update-State и Formulate-Goal дополнительно не рассматривается.

Прежде чем перейти к подробному описанию, сделаем краткую паузу, чтобы определить, в чем агенты, решающие задачи, соответствуют описанию агентов и вариантов среды, приведенному в главе 2. В проекте агента, показанном в листинге 3.1, предполагается, что данная среда является статической, поскольку этапы формулировки и решения задачи осуществляются без учета каких-либо изменений, которые могут произойти в данной среде. Кроме того, в этом проекте агента допускается, что начальное состояние известно; получить такие сведения проще всего, если среда является наблюдаемой. К тому же в самой идее перечисления "альтернативных стратегий" заключается предположение, что среда может

рассматриваться как дискретная. Последней и наиболее важной особенностью является следующее: в проекте агента предполагается, что среда является детерминированной. Решения задач представляют собой единственные последовательности действий, поэтому в них не могут учитываться какие-либо неожиданные события; более того, решения выполняются без учета результатов каких-либо актов восприятия! Агент, выполняющий свои планы, образно говоря, с закрытыми глазами, должен быть вполне уверенным в том, что он делает. (В теории управления подобный проект именуется системой $\wedge$ с разомкнутой обратной связью, поскольку игнорирование результатов восприятия приводит к нарушению обратной связи между агентом и средой.) Все эти предположения означают, что мы имеем дело с простейшими разновидностями среды, и в этом состоит одна из причин, по которой настоящая глава помещена в самом начале данной книги. В разделе 3.6 дано краткое описание того, что произойдет, если будут исключены допущения о наблюдаемости и детерминированности, а в главах 12 и 17 приведено гораздо более подробное изложение соответствующей темы.

Хорошо структурированные задачи и решения $\wedge$ Задача может быть формально определена с помощью четырех компонентов, описанных ниже.

- $\wedge$ Начальное состояние, в котором агент приступает к работе. Например, начальное состояние для нашего агента в Румынии может быть описано как пребывание в Араде, $In(Arad)$.
- Описание возможных действий, доступных агенту. В наиболее общей формулировке³ используется $\wedge$ функция определения преемника. Если задано конкретное состояние x , то функция $Successor-Fn(x)$ возвращает множество упорядоченных пар $\langle action, successor \rangle$, где каждое действие $action$ представляет собой одно из допустимых действий в состоянии x , а каждый преемник $successor$ представляет собой состояние, которое может быть достигнуто из x путем применения этого действия. Например, из состояния 3 В альтернативной формулировке используется множество операторов, которые могут быть применены к некоторому состоянию для выработки преемников.

$in(Arad)$ функция определения преемника для данной задачи проезда по Румынии возвратит следующее: $\{ \langle Go(Sibiu) , In(Sibiu) \rangle , \langle Go(Timisoara) , In(Timisoara) \rangle , \langle Go(Zerind), In(Zerind) \rangle \}$ Начальное состояние и функция определения преемника, вместе взятые, неявно задают $\wedge$ пространство состояний данной задачи — множество всех состояний, достижимых из начального состояния. Пространство состояний образует граф, узлами которого являются состояния, а дугами между узлами — действия. (Карта Румынии, показанная на рис. 3.1, может интерпретироваться как граф пространства состояний, если каждая дорога рассматривается как обозначающая два действия проезда на автомобиле, по одному в каждом направлении.) $\wedge$ Путем в пространстве состояний является последовательность состояний, соединенных последовательностью действий.

Oradea I (Орадя) Neamt (Нямц) Iasi (Яссы) Arad (Арад) Vaslui (Васлуй) Hirsova (Хыршова) Drobeta (Дробета) Craiova (Крайова) Giurgiu (Джурджу) Eforie (Эфорие)

Рис. 3.1. Упрощенная дорожная карта части Румынии ^ Проверка цели, позволяющая определить, является ли данное конкретное состояние целевым состоянием. Иногда имеется явно заданное множество возможных целевых состояний, и эта проверка сводится просто к определению того, является ли данное состояние одним из них. Цель рассматриваемого агента в Румынии представляет собой одноэлементное множество $\{In \{Bucharest\}\}$. Иногда цель задается в виде абстрактного свойства, а не явно перечисленного множества состояний. Например, в шахматах цель состоит в достижении состояния, называемого "матом", в котором король противника атакован и не может уклониться от удара.

Функция ^ стоимости пути, которая назначает числовое значение стоимости каждому пути. Агент, решающий задачу, выбирает функцию стоимости, которая соответствует его собственным показателям производительности. Для данного агента, пытающегося попасть в Бухарест, важнее всего время, поэтому стоимость пути может измеряться длиной в километрах. В настоящей главе предполагается, что стоимость пути может быть описана как сумма стоимостей отдельных действий, выполняемых вдоль этого пути. ^ Стоимость этапа, связанного с выполнением действия а для перехода из состояния x в состояние y , обозначается как $c(x, a, y)$. Стоимости этапов для Румынии показаны на рис. 3.1 в виде дорожных расстояний. Предполагается, что стоимости этапов являются неотрицательными⁴.

Описанные выше элементы определяют задачу и могут быть собраны вместе в единую структуру данных, которая передается в качестве входных данных в алгоритм решения задачи. Решением задачи является путь от начального состояния до целевого состояния. Качество решения измеряется с помощью функции стоимости пути, а * & оптимальным решением является такое решение, которое имеет наименьшую стоимость пути среди всех прочих решений.

Формулировка задачи В предыдущем разделе была предложена формулировка задачи переезда в Бухарест в терминах начального состояния, функции определения преемника, проверки цели и стоимости пути. Эта формулировка кажется приемлемой, но в ней все же не учитываются слишком многие аспекты реального мира. Сравним простое выбранное нами описание состояния, $In \{Arad\}$, с действительным путешествием по стране, в котором состояние мира должно учитывать так много факторов: попутчиков, текущие радиопередачи, вид из окна, наличие поблизости представителей сил правопорядка, расстояние до ближайшей остановки на отдых, состояние дороги, погоду и т.д. Все эти соображения исключены из наших описаний состояния, поскольку они не имеют отношения к задаче поиска маршрута поездки в Бухарест. Процесс удаления деталей из представления называется "> абстрагированием.

Кроме абстрагирования описаний состояний, необходимо абстрагировать сами действия. Любое действие по вождению влечет за собой много следствий. Такие действия не только изменяют местонахождение автомобиля и его пассажиров, но и занимают время, приводят к потреблению топлива, влекут за собой загрязнение окружающей среды и влияют на самого агента (как говорится, путешествия расширяют кругозор). В нашей формулировке учитывается только изменение местонахождения.

Кроме того, существует много действий, которые мы вообще не рассматриваем: включение радиоприемника, осмотр окрестностей из окна, замедление движения по требованию дорожной полиции и т.д. К тому же, безусловно, действия здесь не задаются на уровне "поворота рулевого колеса влево на три градуса".

Можно ли достичь большей точности определения приемлемого уровня абстракции? Будем считать, что выбранные нами абстрактные состояния и действия соответствуют большим множествам более подробных описаний состояния мира, а также более детализированным последовательностям действий. Теперь рассмотрим решение абстрактной задачи, например поиска пути от Арада до Бухареста через города Сибиу, Рымнику-Вылча и Питешти. Это абстрактное решение соответствует Последствия, связанные с применением отрицательных стоимостей, рассматриваются в упр. 3.17.

большому количеству более подробных инструкций по преодолению пути.

Например, можно вести автомобиль между городами Сибиу и Рымнику-Вылча с включенным радиоприемником, а затем выключить его на всю оставшуюся часть путешествия. Абстракция является действительной, если мы можем преобразовать любое абстрактное решение в такое решение, которое подходит для более детально описанного мира; достаточным условием является то, что для любого детализированного состояния, которое представляет собой "пребывание в городе Арад", существует детализированный путь в некоторое состояние, соответствующее "пребыванию в городе Сибиу", и т.д. Абстракция является полезной, если осуществление каждого из действий, предусмотренных в решении, становится проще по сравнению с первоначальной задачей; в этом случае действия являются достаточно простыми для того, чтобы они могли быть выполнены без дальнейшего поиска или планирования со стороны обычного агента, занимающегося вождением автомобиля. Поэтому выбор хорошей абстракции сводится к исключению максимально возможного количества подробностей и вместе с тем к сохранению способности оставаться действительной и обеспечению того, чтобы абстрактные действия были легко осуществимыми. Если бы не было возможности создавать полезные абстракции, то интеллектуальные агенты не могли бы успешно действовать в реальном мире.

3.2. Примеры задач Описанный подход к решению задач был применен в очень многих вариантах экспериментальной среды. В этом разделе перечислены некоторые из наиболее известных примеров решения задач, которые подразделяются на два типа — упрощенные и реальные задачи. ^ Упрощенная задача предназначена для иллюстрации или проверки различных методов решения задач. Ей может быть дано краткое, точное описание. Это означает, что такая задача может легко использоваться разными исследователями для сравнения производительности алгоритмов. ^ Реальной задачей называется такая задача, решение которой действительно требуется людям. Как правило, такие задачи не имеют единого приемлемого для всех описания, но мы попытаемся показать, как обычно выглядят их формулировки.

Упрощенные задачи В качестве первого примера рассмотрим мир пылесоса, впервые представленный в главе 2 (см. рис. 2.2). Деятельность в этом мире можно сформулировать в качестве задачи, как описано ниже.

- Состояния. Агент находится в одном из двух местонахождений, в каждом из которых может присутствовать или не присутствовать мусор. Поэтому существует $2 \times 2 = 4$ возможных состояний мира.
- Начальное состояние. В качестве начального состояния может быть назначено любое состояние.
- Функция определения преемника. Эта функция вырабатывает допустимые состояния, которые являются следствием попыток выполнения трех действий (Left, Right и Suck). Полное пространство состояний показано на рис. 3.2.

Рис. 3.2. Пространство состояний для мира пылесоса. Дуги обозначают действия: L=Left, R=Right, S=Suck

- Проверка цели. Эта проверка сводится к определению того, являются ли чистыми все квадраты.
- Стоимость пути. Стоимость каждого этапа равна 1, поэтому стоимость пути представляет собой количество этапов в этом пути.

По сравнению с задачей реального мира эта упрощенная задача характеризуется различными местонахождениями, возможностью определять наличие мусора, надежной очисткой, а также сохранением достигнутого состояния после очистки.

(В разделе 3.6 некоторые из этих допущений будут исключены.) Необходимо учитывать, что состояние определяется и местонахождением агента, и наличием мусора.

В более крупной среде с p местонахождениями имеется p^2 состояний. ^ Задача игры в восемь, экземпляр которой показан на рис. 3.3, состоит из доски 3×3 с восемью пронумерованными фишками и с одним пустым участком. Фишка, смежная с пустым участком, может быть передвинута на этот участок. Требуется достичь указанного целевого состояния, подобного тому, которое показано в правой части рисунка. Стандартная формулировка этой задачи приведена ниже.

- Состояния. Описание состояния определяет местонахождение каждой из этих восьми фишек и пустого участка на одном из девяти квадратов.
- Начальное состояние. В качестве

начального может быть определено любое состояние. Необходимо отметить, что любая заданная цель может быть достигнута точно из половины возможных начальных состояний (см. упр. 3.4). • Функция определения преемника. Эта функция формирует допустимые состояния, которые являются результатом попыток осуществления указанных четырех действий (теоретически возможных ходов Left, Right, Up или Down). • Проверка цели. Она позволяет определить, соответствует ли данное состояние целевой конфигурации, показанной на рис. 3.3. (Возможны также другие целевые конфигурации.)

Часть П. Решение проблем 1 | 1 1 2 Начальное состояние Целевое состояние Рис. 3.3. Типичный экземпляр задачи игры в восемь • Стоимость пути. Каждый этап имеет стоимость 1, поэтому стоимость пути равна количеству этапов в пути.

Какие абстракции здесь предусмотрены? Действия абстрагируются до уровня указания их начального и конечного состояний, при этом игнорируются промежуточные положения в процессе перемещения фишки. В ходе создания абстрактного описания исключены такие действия, как встряхивание доски, позволяющее передвинуть застрявшую фишку, или извлечение фишек с помощью ножа и повторное укладывание их в ящик. Исключено также описание правил игры, что позволяет обойтись без рассмотрения подробных сведений о физических манипуляциях.

Задача игры в восемь относится к семейству $\wedge$ задач со скользящими фишками, которые часто используются в искусственном интеллекте для проверки новых алгоритмов поиска. Известно, что этот общий класс задач является NP-полным, поэтому вряд ли можно надеяться найти методы, которые в наихудшем случае были бы намного лучше по сравнению с алгоритмами поиска, описанными в этой и следующей главах. Задача игры в восемь имеет $9!/2 = 181\,440$ достижимых состояний и легко решается. Задача игры в пятнадцать (на доске 4x4) имеет около 1,3 триллиона состояний, и случайно выбранные ее экземпляры могут быть решены оптимальным образом за несколько миллисекунд с помощью наилучших алгоритмов поиска.

Задача игры в двадцать четыре (на доске 5x5) имеет количество состояний около 1025, и случайно выбранные ее экземпляры все еще весьма нелегко решить оптимальным образом с применением современных компьютеров и алгоритмов.

Цель $\wedge$ задачи с восемью ферзями состоит в размещении восьми ферзей на шахматной доске таким образом, чтобы ни один ферзь не нападал на любого другого.

(Ферзь атакует любую фигуру, находящуюся на одной и той же с ним горизонтали, вертикали или диагонали.) На рис. 3.4 показана неудачная попытка поиска решения:

ферзь, находящийся на самой правой вертикали, атакован ферзем, который находится вверху слева.

Несмотря на то что существуют эффективные специализированные алгоритмы решения этой задачи и всего семейства задач с p ферзями, она по-прежнему остается интересной экспериментальной задачей для алгоритмов поиска. Для нее применяются формулировки двух основных типов. В $\wedge$ инкрементной формулировке предусматривается использование операторов, которые дополняют описание состояния, начиная с пустого состояния; для задачи с восемью ферзями это означает, что каждое действие приводит к добавлению к этому состоянию еще одного ферзя.

Рис. 3.4. Почти готовое решение задачи с восемью ферзями (поиск окончательного решения оставляем читателю в качестве упражнения) $\wedge$ Формулировка полного состояния начинается с установки на доску всех восьми ферзей и предусматривает их дальнейшее перемещение. В том и другом случае стоимость пути не представляет интереса, поскольку важно лишь достигнуть конечного состояния. Первая инкрементная формулировка, которая может применяться при осуществлении попыток решения этой задачи, приведена ниже.

- Состояния. Состоянием является любое расположение ферзей на доске в количестве от 0 до 8.
- Начальное состояние. Отсутствие ферзей на доске.
- Функция определения преемника. Установка ферзя на любой пустой клетке.
- Проверка цели. На доске находится восемь ферзей, и ни один из них не атакован.

В этой формулировке требуется проверить 64-63...-57-3х1014 возможных последовательностей. В лучшей формулировке должно быть запрещено помещать ферзя на любую клетку, которая уже атакована, следующим образом.

- Состояния. Состояниями являются расположения с p ферзями $@<l<8$), по одному ферзю в каждой из находящихся слева p вертикалей, притом что ни один ферзь не нападает на другого.
- Функция определения преемника. Установка ферзя на любой клетке в находящейся слева пустой вертикали таким образом, чтобы он не был атакован каким-либо другим ферзем.

Эта формулировка позволяет сократить пространство состояний задачи с восемью ферзями с 3х1014 до 2057, и поиск решений значительно упрощается. С другой стороны, для 100 ферзей первоначальная формулировка определяет приблизительно 10400 состояний, а улучшенная формулировка — около 1052 состояний (см. упр. 3.5).

Это — колоссальное сокращение, но улучшенное пространство состояний все еще слишком велико для того, чтобы с ним могли справиться алгоритмы, рассматривае-

мые в данной главе. В главе 4 описана формулировка полного состояния, а в главе 5 приведен простой алгоритм, который позволяет легко решить задачу даже с миллионом ферзей.

Реальные задачи Выше уже было показано, как можно определить $\wedge$ задачу поиска маршрута в терминах заданных местонахождений и переходов по связям между ними.

Алгоритмы поиска маршрута используются в самых разных приложениях, таких как системы маршрутизации в компьютерных сетях, системы планирования военных операций и авиапутешествий. Обычно процесс определения таких задач является трудоемким.

Рассмотрим упрощенный пример задачи планирования авиапутешествий, который задан следующим образом.

- Состояния. Каждое состояние представлено местонахождением (например, аэропортом) и текущим временем.
- Начальное состояние. Оно задается в условии задачи.
- Функция определения преемника. Эта функция возвращает состояния, которые следуют из выполнения любого полета, указанного в расписании (возможно, с дополнительным указанием класса и места), отправления позже по сравнению с текущим временем с учетом продолжительности переезда внутри самого аэропорта, а также из одного аэропорта в другой.
- Проверка цели. Находимся ли мы в месте назначения к некоторому заранее заданному времени?
- Стоимость пути. Зависит от стоимости билета, времени ожидания, продолжительности полета, таможенных и иммиграционных процедур, комфортности места, времени суток, типа самолета, скидок для постоянных пассажиров и т.д.

В коммерческих консультационных системах планирования путешествий используется формулировка задачи такого типа со многими дополнительными усложнениями, которые требуются для учета чрезвычайно запутанных структур определения платы за проезд, применяемых в авиакомпаниях. Но любой опытный путешественник знает, что не все авиапутешествия проходят согласно плану. Действительно качественная система должна предусматривать планы действий в непредвиденных ситуациях (такие как страховочное резервирование билетов на альтернативные рейсы) в такой степени, которая соответствует стоимости и вероятности нарушения первоначального плана. $\wedge$ Задачи планирования обхода тесно связаны с задачами поиска маршрута, но с одним важным исключением. Рассмотрим, например, задачу: "Посетить каждый город, показанный на рис. 3.1, по меньшей мере один раз, начав и окончив путешествие в Бухаресте". Как и при поиске маршрута, действия соответствуют поездкам из одного смежного города в другой. Но пространство состояний является совершенно другим. Каждое состояние должно включать не только текущее местонахождение, но также и множество городов, которые посетил агент. Поэтому первоначальным состоянием должно быть

"В Бухаресте; посещен {Бухарест}", а типичным промежуточным состоянием — "В Васлуй; посещены {Бухарест, Урзичени, Васлуй}", тогда как проверка цели должна предусматривать определение того, находится ли агент в Бухаресте и посетил ли он все 20 городов.

Одной из задач планирования обхода является [^] задача коммивояжера (Traveling Salesperson Problem — TSP), по условию которой каждый город должен быть посещен только один раз. Назначение ее состоит в том, чтобы найти самый короткий путь обхода. Как известно, эта задача является NP-трудной, но на улучшение возможностей алгоритмов TSP были затрачены колоссальные усилия. Кроме планирования поездок коммивояжеров, эти алгоритмы использовались для решения таких задач, как планирование перемещений автоматических сверл при отработке печатных плат и организация работы средств снабжения в производственных цехах.

Задача [^] компоновки СБИС требует позиционирования миллионов компонентов и соединений на микросхеме для минимизации площади, схемных задержек, паразитных емкостей и максимизации выхода готовой продукции. Задача компоновки следует за этапом логического проектирования и обычно подразделяется на две части: компоновка ячеек и маршрутизация каналов. При компоновке ячеек простейшие компоненты схемы группируются по ячейкам, каждая из которых выполняет некоторую известную функцию. Каждая ячейка имеет постоянную форму (размеры и площадь) и требует создания определенного количества соединений с каждой из остальных ячеек. Требуется разместить ячейки на микросхеме таким образом, чтобы они не перекрывались и оставалось место для прокладки соединительных проводов между ячейками. При маршрутизации каналов происходит поиск конкретного маршрута для каждого проводника через зазоры между ячейками. Эти задачи поиска являются чрезвычайно сложными, но затраты на их решение, безусловно, оправдываются. В главе 4 приведены некоторые алгоритмы, позволяющие решать эти задачи. [^] Задача управления навигацией робота представляет собой обобщение описанной выше задачи поиска маршрута. В этой задаче вместо дискретного множества маршрутов рассматривается ситуация, в которой робот может перемещаться в непрерывном пространстве с бесконечным (в принципе) множеством возможных действий и состояний. Если требуется обеспечить циклическое перемещение робота по плоской поверхности, то пространство фактически может рассматриваться как двухмерное, а если робот оборудован верхними и нижними конечностями или колесами, которыми также необходимо управлять, то пространство поиска становится многомерным. Даже для того чтобы сделать это пространство поиска конечным, требуются весьма развитые методы. Некоторые из этих методов рассматриваются в главе 25. Изначальная сложность задачи усугубляется тем, что при управлении реальными роботами

необходимо учитывать ошибки в показаниях датчиков, а также отклонения в работе двигательных средств управления.

Решение задачи $\wedge$ автоматического упорядочения сборки сложных объектов роботом было впервые продемонстрировано на примере робота Freddy [1044]. С тех пор прогресс в этой области происходил медленно, но уверенно, и в настоящее время достигнуто такое положение, что стала экономически выгодной сборка таких неординарных объектов, как электродвигатели. В задачах сборки цель состоит в определении последовательности, в которой должны быть собраны детали некоторого объекта. Если выбрана неправильная последовательность, то в дальнейшем нельзя будет найти способ добавления некоторой детали к этой последовательности без отмены определенной части уже выполненной работы. Проверка возможности выполнения некоторого этапа в последовательности представляет собой сложную геометрическую задачу поиска, тесно связанную с задачей навигации робота. Поэтому одним из дорогостоящих этапов решения задачи упорядочения сборки является

формирование допустимых преемников. Любой практически применимый алгоритм должен предотвращать необходимость поиска во всем пространстве состояний, за исключением крошечной его части. Еще одной важной задачей сборки является $\wedge$ проектирование молекулы белка, цель которой состоит в определении последовательности аминокислот, способных сложиться в трехмерный белок с нужными свойствами, предназначенный для лечения некоторых заболеваний.

В последние годы выросла потребность в создании программных роботов, которые осуществляют $\wedge$ поиск в Internet, находя ответы на вопросы, отыскивая требуемую информацию или совершая торговые сделки. Это — хорошее приложение для методов поиска, поскольку Internet легко представить концептуально в виде графа, состоящего из узлов (страниц), соединенных с помощью ссылок. Полное описание задачи поиска в Internet отложим до главы 10.

3.3. Поиск РЕШЕНИЙ Сформулировав определенные задачи, необходимо найти их решение. Такая цель достигается посредством поиска в пространстве состояний. В настоящей главе рассматриваются методы поиска, в которых используются явно заданное $\wedge$ дерево поиска, создаваемое с помощью начального состояния и функции определения преемника, которые совместно задают пространство состояний. Вообще говоря, вместо дерева поиска может применяться граф поиска, если одно и то же состояние может быть достигнуто с помощью многих путей. Это важное дополнение рассматривается в разделе 3.5.

На рис. 3.5 показаны некоторые расширения дерева поиска, предназначенного для определения маршрута от Арада до Бухареста. Корнем этого дерева поиска является

^поисковый узел, соответствующий начальному состоянию, In(Arad). Первый этап состоит в проверке того, является ли это состояние целевым. Безусловно, что оно не является таковым, но необходимо предусмотреть соответствующую проверку, чтобы можно было решать задачи, содержащие в себе готовое решение, такие как "начав путешествие с города Арад, прибыть в город Арад". А в данном случае текущее состояние не является целевым, поэтому необходимо рассмотреть некоторые другие состояния. Такой этап осуществляется путем ^ разворачивания текущего состояния, т.е. применения функции определения преемника к текущему состоянию для ^ формирования в результате этого нового множества состояний. В данном случае будут получены три новых состояния: In (Sibiu), In (Timisoara) и In (Zerind). Теперь необходимо определить, какой из этих трех вариантов следует рассматривать дальше.

В этом и состоит суть поиска — пока что проверить один вариант и отложить другие в сторону, на тот случай, что первый вариант не приведет к решению. Предположим, что вначале выбран город Сибиу. Проведем проверку для определения того, соответствует ли он целевому состоянию (не соответствует), а затем развернем узел Sibiu для получения состояний In(Arad), In(Fagaras), In(Oradea) и In(RimnicuVilcea). После этого можно выбрать любое из этих четырех состояний либо вернуться и выбрать узел Timisoara или Zerind. Необходимо снова и снова выбирать, проверять и разворачивать узлы до тех пор, пока не будет найдено решение или не останется больше состояний, которые можно было бы развернуть. Порядок, в котором происходит разворачивание состояний, определяется ^ стратегией поиска. Общий алгоритм поиска в дереве неформально представлен d листинге 3.2.

а) Начальное состояние C Sibiu ^ C ^ J'imisoara.^ C Zennd J ;** Arad ~*> l'^Fagaras^ ?* Oradca*} cfltimnicu \Л1а?> C.. Arad } C _Lugoj ^ Arad Oradca б) После разворачивания узла Arad c*^ Aradmj) i'^Fagang^ l'^Oradea^ {JjmnfcuVUoejp C.^Arad^ Cj-UfIOJ^} Arad > C Oradca } в) После разворачивания узла Sibiu Arad > C Oradca } Pwc. J. 5. Частично развернутые деревья поиска, предназначенные для определения маршрута от Арада до Бухареста. Развернутые узлы затенены; узлы, которые были сформированы, но еще не развернуты, выделены полужирным контуром; узлы, которые еще не были сформированы, обозначены тонкими штриховыми линиями

Листинг 3.2. Неформальное описание общего алгоритма поиска в дереве

```
function Tree-Search(problem, strategy) returns решение solution или индикатор неудачи failure
    инициализировать дерево поиска с использованием начального состояния задачи problem
    loop do
        if нет кандидатов на разворачивание then return индикатор неудачи failure
        выбрать листовой узел для разворачивания в соответствии со стратегией strategy
        if этот узел содержит целевое состояние then return соответствующее решение solution
        else развернуть этот узел и добавить полученные узлы к дереву
```


поиска Необходимо учитывать различие между пространством состояний и деревом поиска.

В пространстве состояний для задачи поиска маршрута имеется только 20 состояний, по одному для каждого города. Но количество путей в этом пространстве состояний является бесконечным, поэтому дерево поиска имеет бесконечное количество узлов. Напри-

мер, первыми тремя путями любой бесконечной последовательности путей являются маршруты Арад—Сибиу, Арад—Сибиу—Арад, Арад—Сибиу—Арад—Сибиу. (Безусловно, качественный алгоритм поиска должен исключать возможность формирования таких повторяющихся путей; в разделе 3.5 показано, как этого добиться.) Существует множество способов представления узлов, но здесь предполагается, что узел представляет собой структуру данных с пятью компонентами, которые описаны ниже. • State. Состояние в пространстве состояний, которому соответствует данный узел. • Parent-Node. Узел в дереве поиска, применявшийся для формирования данного узла (родительский узел). • Action. Действие, которое было применено к родительскому узлу для формирования данного узла. • Path-Cost. Стоимость пути (от начального состояния до данного узла), показанного с помощью указателей родительских узлов, которую принято обозначать как $d(n)$. • Depth. Количество этапов пути от начального состояния, называемое также глубиной.

Необходимо учитывать различие между узлами и состояниями. Узел — это учетная структура данных, применяемая для представления дерева поиска, а состояние соответствует конфигурации мира. Поэтому узлы лежат на конкретных путях, которые определены с помощью указателей Parent-Node, а состояния — нет. Кроме того, два разных узла могут включать одно и то же состояние мира, если это состояние формируется с помощью двух различных путей поиска. Структура данных узла показана на рис. 3.6.

Action = right Depth = 6 Path-Cost = 6 Рис. 3.6. Узлы представляют собой структуры данных, с помощью которых формируется дерево поиска. Каждый узел имеет родительский узел, содержит данные о состоянии и имеет различные вспомогательные поля.

Стрелки направлены от дочернего узла к родительскому. Необходимо также представить коллекцию узлов, которые были сформированы, но еще не развернуты; такая ко- ллекция называется >& периферией. Каждый элемент

периферии представляет собой ^ листовой узел, т.е. узел, не имеющий преемников в дереве. На рис. 3.5 периферия каждого дерева состоит из узлов с полужирными контурами. Простейшим представлением периферии может служить множество узлов. Тогда стратегия поиска должна быть выражена в виде функции, которая

выбирает определенным образом из этого множества следующий узел, подлежащий развертыванию. Хотя данный подход концептуально является несложным, он может оказаться дорогостоящим с вычислительной точки зрения, поскольку функцию, предусмотренную в этой стратегии, возможно, придется применить к каждому элементу в указанном множестве для выбора наилучшего из них. Поэтому предполагается, что коллекция узлов реализована в виде $\wedge$ очереди. Операции, применимые к любой очереди, состоят в следующем.

- **Make-Queue (element,...)**. Создает очередь с заданным элементом (элементами).
- **Empty? (queue)**. Возвращает истинное значение, только если в очереди больше нет элементов.
- **First (queue)**. Возвращает первый элемент в очереди.
- **Remove-First (queue)**. Возвращает элемент First (queue) и удаляет его из очереди.
- **Insert (element, queue)**. Вставляет элемент в очередь и возвращает результирующую очередь. (Ниже будет показано, что в очередях различных типов вставка элементов осуществляется в различном порядке.)
- **Insert-All (elements, queue)**. Вставляет множество элементов в очередь и возвращает результирующую очередь.

С помощью этих определений мы можем записать более формальную версию общего алгоритма поиска в дереве, показанную в листинге 3.3.

Листинг 3.3. Общий алгоритм поиска в дереве. Следует учитывать, что фактический параметр fringe (периферия) должен представлять собой пустую очередь, а порядок поиска зависит от типа очереди. Функция Solution возвращает последовательность действий, полученную путем прохождения по указателям на родительские узлы в обратном направлении, к корню

```
function Tree-Search(problem, fringe) returns
  решение solution или индикатор неудачи failure
  fringe  $\leftarrow$  Insert(Make-Node(Initial-State[problem]), fringe)
  loop do
    if Empty?(fringe) then return индикатор неудачи failure
    node  $\leftarrow$  Remove-First(fringe)
    if Goal-Test[problem] применительно к State[node]
      завершается успешно then return Solution(node)
    fringe  $\leftarrow$  Insert-All(Expand(node, problem), fringe)
  function Expand(node, problem) returns множество узлов successors
  successors  $\leftarrow$  пустое множество
  for each <action, result> in Successor-Fn[problem](State[node]) do
    s  $\leftarrow$  новый узел
```

```
State[s]  $\leftarrow$  result
Parent-Node[s]  $\leftarrow$  node
Action[s]  $\leftarrow$  action
Path-Cost[s]  $\leftarrow$  Path-Cost[node] + Step-Cost(node, action, s)
Depth[s]  $\leftarrow$  Depth[node] + 1
добавить узел s к множеству successors
return successors
```

Измерение производительности решения задачи

Результатом применения любого алгоритма решения задачи является либо неудачное завершение, либо получение решения. (Некоторые алгоритмы могут входить в бесконечный цикл и не возвращать никакого результата.) Мы будем оценивать производительность алгоритма с помощью четырех показателей, описанных ниже.

- $\wedge$ **Полнота**. Гарантирует ли алгоритм обнаружение решения, если оно имеется?
- $\wedge$ **Оптимальность**. Обеспечивает ли данная стратегия нахождение оптимального решения, в соответствии с определением, приведенным на с. 115?
- $\wedge$

Временная сложность. За какое время алгоритм находит решение? • $\wedge$
Пространственная сложность. Какой объем памяти необходим для осуществления поиска?

Временная и пространственная сложность всегда анализируются с учетом определенного критерия измерения сложности задачи. В теоретической компьютерной науке типичным критерием является размер графа пространства состояний, поскольку этот граф рассматривается как явно заданная структура данных, которая является входной для программы поиска. (Примером этого может служить карта Румынии.) В искусственном интеллекте, где граф представлен неявно с помощью начального состояния и функции определения преемника и часто является бесконечным, сложность выражается в терминах трех величин: b — B , коэффициент ветвления или максимальное количество преемников любого узла, d — глубина самого поверхностного целевого узла и t — максимальная длина любого пути в пространстве состояний.

Временная сложность часто измеряется в терминах количества узлов, вырабатываемых в процессе поиска, а пространственная сложность — в терминах максимального количества узлов, хранимых в памяти.

Чтобы оценить эффективность любого алгоритма поиска, можно рассматривать только $\wedge$ стоимость поиска, которая обычно зависит от временной сложности, но может также включать выражение для оценки использования памяти, или применять $\wedge$ суммарную стоимость, в которой объединяется стоимость поиска и стоимость пути найденного решения. Для задачи поиска маршрута от Арада до Бухареста стоимость поиска представляет собой количество времени, затраченного на этот поиск, а стоимость решения выражает общую длину пути в километрах. Поэтому для вычисления B в некоторых работах вместо этого для измерения временной сложности применяется количество операций развертывания узлов. Эти два критерия различаются не больше, чем на коэффициент b . С точки зрения авторов, время выполнения операции развертывания узла растет пропорционально количеству узлов, вырабатываемых при этом развертывании.

числения суммарной стоимости нам придется складывать километры и миллисекунды. Между этими двумя единицами измерения не определен "официальный курс обмена", но в данном случае было бы резонно преобразовывать километры в миллисекунды с использованием оценки средней скорости автомобиля (поскольку для данного агента важным является именно время). Это позволяет рассматриваемому агенту найти оптимальную точку компромисса, в которой дальнейшие вычисления для поиска более короткого пути становятся непродуктивными. Описание более общей задачи поиска компромисса между различными ценностями будет продолжено в главе 16.

3.4. Стратегии неинформированного поиска В данном разделе рассматриваются пять стратегий поиска, которые известны под названием ^неинформированного поиска (называемого также слепым поиском).

Этот термин означает, что в данных стратегиях не используется дополнительная информация о состояниях, кроме той, которая представлена в определении задачи.

Все, на что они способны, — вырабатывать преемников и отличать целевое состояние от нецелевого. Стратегии, позволяющие определить, является ли одно нецелевое состояние "более многообещающим" по сравнению с другим, называются стратегиями ^информированного поиска, или ^эвристического поиска; они рассматриваются в главе 4. Все стратегии поиска различаются тем, в каком порядке происходит развертывание узлов.

Поиск в ширину ^ Поиск в ширину — это простая стратегия, в которой вначале развертывается корневой узел, затем — все преемники корневого узла, после этого развертываются преемники этих преемников и т.д. Вообще говоря, при поиске в ширину, прежде чем происходит развертывание каких-либо узлов на следующем уровне, развертываются все узлы на данной конкретной глубине в дереве поиска.

Поиск в ширину может быть реализован путем вызова процедуры Tree-Search с пустой периферией, которая представляет собой последовательную очередь (FirstIn-First-Out — FIFO), гарантирующую, что прежде всего будут развернуты узлы, которые должны посещаться первыми. Иными словами, к организации поиска в глубину приводит вызов процедуры Tree-Search (problem, FIFO-Queue ()). В очереди FIFO предусмотрена вставка всех вновь сформированных преемников в конец очереди, а это означает, что поверхностные узлы развертываются прежде, чем более глубокие. На рис. 3.7 показан ход поиска в простом бинарном дереве.

Ч(СЧ; X^ \> 'ф; 'ф v.F.' V.GJ ф; :E: T; 'G) Рис. 3.7. Поиск в ширину в простом бинарном дереве. На каждом этапе узел, подлежащий развертыванию в следующую очередь, обозначается маркером

Проведем оценку поиска в ширину с использованием четырех критериев, описанных в предыдущем разделе. Вполне очевидно, что этот поиск является полным — если самый поверхностный целевой узел находится на некоторой конечной глубине d , то поиск в ширину в конечном итоге позволяет его обнаружить после развертывания всех более поверхностных узлов (при условии, что коэффициент ветвления b является конечным). Самый поверхностный целевой узел не обязательно является оптимальным; формально поиск в ширину будет оптимальным, если стоимость пути выражается в виде неубывающей функции глубины узла. (Например, такое предположение оправдывается, если все действия имеют одинаковую стоимость.) До сих пор приведенное выше описание поиска в ширину не предвещало никаких неприятностей. Но такая стратегия не всегда

является оптимальной; чтобы понять, с чем это связано, необходимо определить, какое количество времени и какой объем памяти требуются для выполнения поиска. Для этого рассмотрим гипотетическое пространство состояний, в котором каждое состояние имеет b преемников.

Корень этого дерева поиска вырабатывает b узлов на первом уровне, каждый из которых вырабатывает еще b узлов, что соответствует общему количеству узлов на втором уровне, равному b^2 . Каждый из них также вырабатывает b узлов, что приводит к получению b^3 узлов на третьем уровне, и т.д. А теперь предположим, что решение находится на глубине d . В наихудшем случае на уровне d необходимо развернуть все узлы, кроме последнего (поскольку сам целевой узел не разворачивается), что приводит к выработке $bd+1-b$ узлов на уровне $d+1$. Это означает, что общее количество выработанных узлов равно:

$b + b^2 + b^3 + \dots + bd + (bd+1-b) = O(bd+1)$ Каждый выработанный узел должен оставаться в памяти, поскольку он либо относится к периферии, либо является предком периферийного узла. Итак, пространственная сложность становится такой же, как и временная (с учетом добавления одного узла, соответствующего корню).

Поэтому исследователи, выполняющие анализ сложности алгоритма, огорчаются (или восхищаются, если им нравится преодолевать трудности), столкнувшись с экспоненциальными оценками сложности, такими как $O(bd+1)$. В табл. 3.1 показано, с чем это связано. В ней приведены требования ко времени и к объему памяти при поиске в ширину с коэффициентом ветвления $b=10$ для различных значений глубины решения d . При составлении этой таблицы предполагалось, что в секунду может быть сформировано 10 000 узлов, а для каждого узла требуется 1000 байтов памяти.

Этим предположением приблизительно соответствуют многие задачи поиска при их решении на любом современном персональном компьютере (с учетом повышающего или понижающего коэффициента 100).

На основании табл. 3.1 можно сделать два важных вывода. Прежде всего, c^h при поиске в ширину наиболее сложной проблемой по сравнению со значительным временем выполнения является обеспечение потребностей в памяти. Затраты времени, равные 31 часу, не кажутся слишком значительными при ожидании решения важной задачи с глубиной 8, но лишь немногие компьютеры имеют терабайт оперативной памяти, который для этого требуется. К счастью, существуют другие стратегии поиска, которые требуют меньше памяти.

Второй вывод состоит в том, что требования ко времени все еще остаются важным фактором. Если рассматриваемая задача имеет решение на глубине 12, то (с учетом при-

нятых предположений) потребуется 35 лет на поиск в ширину (а в действительности на любой неинформированный поиск), чтобы найти ее решение. Вообще говоря, s^A = задачи поиска с экспоненциальной сложностью невозможно решить с помощью неинформированных методов во всех экземплярах этих задан, кроме самых небольших.

Таблица 3.1. Потребности во времени и объеме памяти для поиска в ширину. Приведенные здесь данные получены при следующих предположениях: коэффициент ветвления — $b=10$; скорость формирования — 10 000 узлов/секунда; объем памяти — 1000 байтов/узел Глубина Количество узлов 111 100 ю7 ю9 ю11 ю13 ю15 Время 0,11 секунды 11 секунд 19 минут 31 час 129 суток 35 лет 3523 года Память 1 мегабайт 106 мегабайтов 10 гигабайтов 1 терабайт 101 терабайт 10 петабайтов 1 эксабайт Поиск по критерию стоимости Поиск в ширину является оптимальным, если стоимости всех этапов равны, поскольку в нем всегда развертывается самый поверхностный неразвернутый узел.

С помощью простого дополнения можно создать алгоритм, который является оптимальным при любой функции определения стоимости этапа. Вместо развертывания самого поверхностного узла A поиск по критерию стоимости обеспечивает развертывание узла p с наименьшей стоимостью пути. Обратите внимание на то, что, если стоимости всех этапов равны, такой поиск идентичен поиску в ширину.

При поиске по критерию стоимости учитывается не количество этапов, имеющих в пути, а только их суммарная стоимость. Поэтому процедура этого поиска может войти в бесконечный цикл, если окажется, что в ней развернут узел, имеющий действие с нулевой стоимостью, которое снова указывает на то же состояние (например, действие NoOp). Можно гарантировать полноту поиска при условии, что стоимость каждого этапа больше или равна некоторой небольшой положительной константе δ . Это условие является также достаточным для обеспечения оптимальности. Оно означает, что стоимость пути всегда возрастает по мере прохождения по этому пути. Из данного свойства легко определить, что такой алгоритм развертывает узлы в порядке возрастания стоимости пути. Поэтому первый целевой узел, выбранный для развертывания, представляет собой оптимальное решение. (Напомним, что в процедуре Tree-Search проверка цели применяется только к тем узлам, которые выбраны для развертывания.) Рекомендуем читателю попытаться воспользоваться этим алгоритмом для поиска кратчайшего пути до Бухареста.

Поиск по критерию стоимости направляется с учетом стоимостей путей, а не значений глубины в дереве поиска, поэтому его сложность не может быть легко охарактеризована в терминах b и d . Вместо этого предположим, что C^* — стоимость оптимального решения, и допустим, что стоимость каждого действия составляет, по

меньшей мере, 8. Это означает, что временная и пространственная сложность этого алгоритма в наихудшем случае составляет $O(b^{*l})$, т.е. может быть намного больше, чем bd . Это связано с тем, что процедуры поиска по критерию стоимости

могут и часто выполняют проверку больших деревьев, состоящих из мелких этапов, прежде чем перейти к исследованию путей, в которые входят крупные, но, возможно, более полезные этапы. Безусловно, если все стоимости этапов равны, то выражение b^{*l} сводится к bd .

Поиск в глубину При $\wedge$ поиске в глубину всегда разворачивается самый глубокий узел в текущей периферии дерева поиска. Ход такого поиска показан на рис. 3.8. Поиск непосредственно переходит на самый глубокий уровень дерева поиска, на котором узлы не имеют преемников. По мере того как эти узлы разворачиваются, они удаляются из периферии, поэтому в дальнейшем поиск "возобновляется" со следующего самого поверхностного узла, который все еще имеет неисследованных преемников.

Со) 'О чу {& и ^ @ %* {л? и? Ц) >© Qp да ч>) ® CD Ф С© СО ® Ф © @ (l) Ф Ф СО ® ® ® ® Ф Ф СО @ ® @ © '6? • А) J\$ 'p) • >© '\$ 'IV >o-o со d) (к) СО @ ® @ •>© Ф © CD @ ® ® •• ® © СО @ # Ф) •••• со ё ё @ CD % ® СО) Рис. J. 8. Поиск в глубину в бинарном дереве. Узлы, которые были развернуты и не имеют потомков в этой периферии, могут быть удалены из памяти; эти узлы обозначены черным цветом. Предполагается, что узлы на глубине 3 не имеют преемников и единственным целевым узлом является м Эта стратегия может быть реализована в процедуре Tree-Search с помощью очереди UFO (Last-In-First-Out), называемой также стеком. В качестве альтернативы способу реализации на основе процедуры Tree-Search поиск в глубину часто реализуют с помощью рекурсивной функции, вызывающей саму себя в каждом из дочерних узлов по очереди. (Рекурсивный алгоритм поиска в глубину, в котором предусмотрен предел глубины, приведен в листинге 3.4.)

Поиск в глубину имеет очень скромные потребности в памяти. Он требует хранения только единственного пути от корня до листового узла, наряду с оставшимися неразвернутыми сестринскими узлами для каждого узла пути. После того как был развернут некоторый узел, он может быть удален из памяти, коль скоро будут полностью исследованы все его потомки (см. рис. 3.8). Для пространства состояний с коэффициентом ветвления b и максимальной глубиной t поиск в глубину требует хранения только $bt+1$ узлов. Используя такие же предположения, как и в табл. 3.1, и допуская, что узлы, находящиеся на той же глубине, что и целевой узел, не имеют преемников, авторы определили, что на глубине $d=12$ для поиска в глубину требуется 118 килобайтов вместо 10 петабайтов, т.е. потребность в пространстве уменьшается примерно в 10 миллиардов раз.

В одном из вариантов поиска в глубину, называемом $\wedge$ поиском с возвратами, используется еще меньше памяти. При поиске с возвратами каждый раз формируется только один преемник, а не все преемники; в каждом частично развернутом узле запоминается информация о том, какой преемник должен быть сформирован следующим. Таким образом, требуется только $O(t)$ памяти, а не $O(bm)$. При поиске с возвратами применяется еще один прием, позволяющий экономить память (и время); идея его состоит в том, чтобы при формировании преемника должно непосредственно модифицироваться описание текущего состояния, а не осуществляться его предварительное копирование. При этом потребность в памяти сокращается до объема, необходимого для хранения только одного описания состояния и $O(l?)$ действий. Но для успешного применения данного приема нужно иметь возможность отменять каждую модификацию при возврате, выполняемом для формирования следующего преемника. При решении задач с объемными описаниями состояния, таких как роботизированная сборка, применение указанных методов модификации состояний становится важнейшим фактором успеха.

Недостатком поиска в глубину является то, что в нем может быть сделан неправильный выбор и переход в тупиковую ситуацию, связанную с прохождением вниз по очень длинному (или даже бесконечному) пути, притом что другой вариант мог бы привести к решению, находящемуся недалеко от корня дерева поиска.

Например, на рис. 3.8 поиск в глубину потребовал бы исследования всего левого поддерева, даже если бы целевым узлом был узел C , находящийся в правом поддереве. А если бы целевым узлом был также узел «J», менее приемлемый по сравнению с узлом C , то поиск в глубину возвратил бы в качестве решения именно его; это означает, что поиск в глубину не является оптимальным. Кроме того, если бы левое поддерево имело неограниченную глубину, но не содержало бы решений, то поиск в глубину так никогда бы и не закончился; это означает, что данный алгоритм — не полный. В наихудшем случае поиск в глубину формирует все $O(lf1)$ узлов в дереве поиска, где t — максимальная глубина любого узла. Следует отметить, что t может оказаться гораздо больше по сравнению с d (глубиной самого поверхностного решения) и является бесконечным, если дерево имеет неограниченную глубину.

Поиск с ограничением глубины Проблему неограниченных деревьев можно решить, предусматривая применение во время поиска в глубину заранее определенного предела глубины L . Это означает, что узлы на глубине ? рассматриваются таким образом, как если бы они не имели

преемников. Такой подход называется $\wedge$ поиском с ограничением глубины.

Применение предела глубины позволяет решить проблему бесконечного пути. К сожалению, в этом подходе также вводится дополнительный источник неполноты,

если будет выбрано значение $? < d$, иными словами, если самая поверхностная цель выходит за пределы глубины. (Такая ситуация вполне вероятна, если значение d неизвестно.) Кроме того, поиск с ограничением глубины будет неоптимальным при выборе значения $? > d$. Его временная сложность равна $O(b^d)$, а пространственная сложность — $O(b)$. Поиск в глубину может рассматриваться как частный случай поиска с ограничением глубины, при котором $? = \infty$.

Иногда выбор пределов глубины может быть основан на лучшем понимании задачи. Например, допустим, что на рассматриваемой карте Румынии имеется 20 городов. Поэтому известно, что если решение существует, то должно иметь длину не больше 19; это означает, что одним из возможных вариантов является $? = 19$. Но в действительности при внимательном изучении этой карты можно обнаружить, что любой город может быть достигнут из любого другого города не больше чем за 9 этапов. Это число, известное как $? \text{ диаметр пространства состояний}$, предоставляет нам лучший предел глубины, который ведет к более эффективному поиску с ограничением глубины. Но в большинстве задач приемлемый предел глубины остается неизвестным до тех пор, пока не будет решена сама задача.

Поиск с ограничением глубины может быть реализован как простая модификация общего алгоритма поиска в дереве или рекурсивного алгоритма поиска в глубину. Псевдокод реализации рекурсивного поиска с ограничением глубины приведен в листинге 3.4. Обратите внимание на то, что поиск с ограничением глубины может приводить к неудачным завершениям двух типов: стандартное значение `failure` указывает на отсутствие решения, а значение `cutoff` свидетельствует о том, что на заданном пределе глубины решения нет.

Листинг 3.4. Рекурсивная реализация поиска с ограничением глубины

```
function Depth-Limited-Search(problem, limit) returns решение result или индикатор неудачи failure/cutoff
return Recursive-DLS(Make-Node(Initial-State[problem]), problem, limit)
function Recursive-DLS(node, problem, limit) returns решение result или индикатор неудачи failure/cutoff
    cutoff_occurred? <— ложное значение
    if Goal-Test[problem](State[node]) then return Solution(node)
    else if Depth[node] = limit then return индикатор неудачи cutoff
    else for each преемник successor in Expand(node, problem) do
        result <— Recursive-DLS(successor, problem, limit)
        if result = cutoff then cutoff_occurred? <— истинное значение
    else if result  $\Phi$  failure then return решение result
    if cutoff_occurred? then return индикатор неудачи cutoff
    else return индикатор неудачи failure
```

Поиск в глубину с итеративным углублением $? \text{ Поиск с итеративным углублением}$ (или, точнее, поиск в глубину с итеративным углублением) представляет собой общую стратегию, часто применяемую в сочетании с поиском в глубину, которая позволяет найти наилучший предел глубины. Это достигается путем постепенного увеличения предела (который вначале становится равным 0, затем 1, затем 2 и т.д.)

до тех пор, пока не будет найдена цель. Такое событие происходит после того, как предел глубины достигает значения d , глубины самого поверхностного целевого узла. Данный алгоритм приведен в листинге 3.5. В поиске с итеративным углублением сочетаются преимущества поиска в глубину и поиска в ширину. Как и поиск в глубину, он характеризуется очень скромными требованиями к памяти, а именно, значением $O(bd)$. Как и поиск в ширину, он является полным, если коэффициент ветвления конечен, и оптимальным, если стоимость пути представляет собой неубывающую функцию глубины узла. На рис. 3.9 показаны четыре итерации применения процедуры Iterative-Deepening-Search к бинарному дереву поиска, где решение найдено в четвертой итерации.

Листинг 3.5. Алгоритм поиска с итеративным углублением, в котором повторно применяется поиск с ограничением глубины при последовательном увеличении пределов. Он завершает свою работу после того, как обнаруживается решение, или процедура поиска с ограничением глубины возвращает значение failure, а это означает, что решение не существует

```
function Iterative-Deepening-Search(problem)
    returns решение result или индикатор неудачи failure
inputs: problem, задача for depth f0 to w
do result ← Depth-Limited-Search(problem, depth)
if result ≠ cutoff then return
    решение result
```

Поиск с итеративным углублением может на первый взгляд показаться расточительным, поскольку одни и те же состояния формируются несколько раз. Но, как оказалось, такие повторные операции не являются слишком дорогостоящими.

Причина этого состоит в том, что в дереве поиска с одним и тем же (или почти одним и тем же) коэффициентом ветвления на каждом уровне большинство узлов находится на нижнем уровне, поэтому не имеет большого значения то, что узлы на верхних уровнях формируются многократно. В поиске с итеративным углублением узлы на нижнем уровне (с глубиной d) формируются один раз, те узлы, которые находятся на уровне, предшествующем нижнему, формируются дважды, и т.д., вплоть до дочерних узлов корневого узла, которые формируются d раз. Поэтому общее количество формируемых узлов выражается следующей формулой:

$W(IDS) = (d)b + (d-1)b^2 + \dots + (1)b^d$ которая соответствует временной сложности порядка $O(bd)$. Это количество можно сравнить с количеством узлов, формируемых при поиске в ширину:

$$N(BFS) = b + b^2 + \dots + b^d + (b^{d+1} - b)$$

Предел = 0 > ® s Предел = 1 н ® ч > < ir чэ ш ^*® вг ^т Предел = 2 > ® ч «от ТЭ Шг иу иу >. (.; им цу Cf; уз; w v v» V/ v/ yv y/ w да w /w% АЛ ЛЛ АА Предел = 3 Н ® v •fir т© tfy йй (f; »p) «KJ •?; Ой Su >E) Ш ® ® У> Уч У/ Уч >»' >-> l-»' Уч X >«г \-> У/ ® sD Cy 0D Ф '*M> w © ® CO CS & (B \$4) CN' ® '51/ CO O; иу (C; <m; йу ш> ^ (н) (D СИ (Ю ® (S3) ® (o, >У Уч. W V»: Уч; Уч У/ УУ Уч Vv Уч. Уч. W >-ч. W)-ч. У». У/ w У/ W У/ W У/ тит У<

Рис. 3.9. Четыре итерации поиска с итеративным углублением в бинарном дереве. Следует отметить, что при поиске в ширину некоторые узлы формируются на глубине $d+1$, а при итеративном углублении этого не происходит. Результатом является то, что поиск с итеративным углублением фактически осуществляется быстрее, чем поиск в ширину, несмотря на повторное формирование состояний. Например, если $b=10$ и $d=5$, то соответствующие оценки количества узлов принимают следующие значения:

$W(IDS) = 50 + 400 + 3000 + 20\,000 + 100\,000 = 123\,450$
 $N\{BFS\} = 10 + 100 + 1000 + 10\,000 + 100\,000 + 999\,990 = 1\,111\,100$
 ~ Вообще говоря, итеративное углубление — это предпочтительный метод неинформированного поиска при тех условиях, когда имеется большое пространство поиска, а глубина решения неизвестна.

Поиск с итеративным углублением аналогичен поиску в ширину в том отношении, что в нем при каждой итерации перед переходом на следующий уровень исследуется полный уровень новых узлов. На первый взгляд может показаться целесообразным разработка итеративного аналога поиска по критерию стоимости, который

унаследовал бы от последнего алгоритма гарантии оптимальности, позволяя вместе с тем исключить его высокие требования к памяти. Идея состоит в том, чтобы вместо увеличивающихся пределов глубины использовались увеличивающиеся пределы стоимости пути. Результирующий алгоритм, получивший название A^* поиска с итеративным удлинением, рассматривается в упр. 3.11. Но, к сожалению, было установлено, что поиск с итеративным удлинением характеризуется более существенными издержками, чем поиск по критерию стоимости.

Двунаправленный поиск В основе двунаправленного поиска лежит такая идея, что можно одновременно проводить два поиска (в прямом направлении, от начального состояния, и в обратном направлении, от цели), останавливаясь после того, как два процесса поиска встретятся на середине (рис. 3.10). Дело в том, что значение $bd/2 + bd/2$ гораздо меньше, чем bd , или, как показано на этом рисунке, площадь двух небольших кругов меньше площади одного большого круга с центром в начале поиска, который охватывает цель поиска.

Рис. 3.10. Схематическое представление двунаправленного поиска в том состоянии, когда он должен вскоре завершиться успешно после того, когда одна из ветвей, исходящих из начального узла, встретится с ветвью из целевого узла. Двунаправленный поиск реализуется с помощью метода, в котором предусматривается проверка в одном или в обоих процессах поиска каждого узла перед его развертыванием для определения того, не находится ли он на периферии другого дерева поиска; в случае положительного результата проверки решение найдено. Например, если задача имеет решение на глубине $d=6$ и в каждом

направлении осуществляется поиск в ширину с последовательным развертыванием по одному узлу, то в самом худшем случае эти два процесса поиска встретятся, если в каждом из них будут развернуты все узлы на глубине 3, кроме одного. Это означает, что при $b=10$ будет сформировано общее количество узлов, равное 22 200, а не 11 111 100, как при стандартном поиске в ширину. Проверка принадлежности узла к другому дереву поиска может быть выполнена за постоянное время с помощью хэш-таблицы, поэтому временная сложность двунаправленного поиска определяется как $O(bd/2)$. В памяти необходимо хранить по крайней мере одно из деревьев поиска, для того, чтобы можно было выполнить проверку принадлежности к другому дереву, поэтому пространственная сложность также определяется как $O(bd/2)$. Такие требования к пространству являются одним из наиболее существенных недостатков двунаправленного поиска.

Этот алгоритм является полным и оптимальным (при единообразных стоимостях эта-

пов), если оба процесса поиска осуществляются в ширину; другие сочетания методов могут характеризоваться отсутствием полноты, оптимальности или того и другого.

Благодаря такому уменьшению временной сложности двунаправленный поиск становится привлекательным, но как организовать поиск в обратном направлении? Это не так легко, как кажется на первый взгляд. Допустим, что $\wedge$ предшественниками узла l , определяемыми с помощью функции $Pred(n)$, являются все те узлы, для которых p служит преемником. Для двунаправленного поиска требуется, чтобы функция определения предшественника $Pred\{n\}$ была эффективно вычислимой. Простейшим является такой случай, когда все действия в пространстве состояний обратимы таким образом, что $Pred(n) = Succ^{-1}(n)$, а другие случаи могут потребовать проявить значительную изобретательность.

Рассмотрим вопрос о том, что подразумевается под понятием "цель" при поиске "в обратном направлении от цели". В задачах игры в восемь и поиска маршрута в Румынии имеется только одно целевое состояние, поэтому обратный поиск весьма напоминает прямой поиск. Если же имеется несколько явно перечисленных целевых состояний (например, два показанных на рис. 3.2 целевых состояния, в которых квадраты пола не содержат мусор), то может быть создано новое фиктивное целевое состояние, непосредственными предшественниками которого являются все фактические целевые состояния. Иным образом, формирования некоторых избыточных узлов можно избежать, рассматривая множество целевых состояний как единственное целевое состояние, каждым из предшественников которого также является множество состояний, а именно, множество состояний, имеющее соответствующего преемника в множестве целевых состояний (см. также раздел 3.6).

Наиболее сложным случаем для двунаправленного поиска является такая задача, в которой для проверки цели дано только неявное описание некоторого (возможно, большого) множества целевых состояний, например всех состояний, соответствующих проверке цели "мат" в шахматах. При обратном поиске потребовалось бы создать компактные описания "всех состояний, которые позволяют поставить мат с помощью хода $л^{\wedge}$ ", и т.д.; и эти описания нужно было бы сверять с состояниями, формируемыми при прямом поиске. Общего способа эффективного решения такой проблемы не существует.

Сравнение стратегий неинформированного поиска В табл. 3.2 приведено сравнение стратегий поиска в терминах четырех критериев оценки, сформулированных в разделе 3.4.

3.5. Предотвращение формирования повторяющихся состояний Вплоть до этого момента мы рассматривали все аспекты поиска, но игнорировали одно из наиболее важных усложнений в процессе поиска — вероятность появления непроизводительных затрат времени при разворачивании состояний, которые уже встречались и были развернуты перед этим. При решении некоторых задач такая ситуация никогда не возникает; в них пространство состояний представляет собой дерево и поэтому имеется только один путь к каждому состоянию. В частности, эффективной является формулировка задачи с восемью ферзями (в которой каждый новый ферзь помещается на самый левый пустой вертикальный ряд), и ее эффективность в значительной степени обусловлена именно этим — каждое состояние может быть достигнуто только по одному пути. А если бы задача с восемью ферзями была сформулирована таким образом, что любого ферзя разрешалось бы ставить на любую вертикаль, то каждого состояния с p ферзями можно было бы достичь с помощью $p!$ различных путей.

Таблица 3.2. Оценка стратегий поиска, где β — коэффициент ветвления; d — глубина самого поверхностного решения; t — максимальная глубина дерева поиска; l — предел глубины. Предостережения, обозначенные строчными буквами, означают следующее: а — полный, если коэффициент ветвления β конечен; б — полный, если стоимость каждого этапа $> e$ при некотором положительном значении e ; в — оптимальный, если стоимости всех этапов являются одинаковыми; г — применимый, если в обоих направлениях осуществляется поиск в ширину Характеристика Полнота Временная сложность Пространственная сложность Оптимальность Поиск в ширину Даа $O(b^{b+1})$ SXb^{b*1} Дав Поиск по критерию стоимости ДаЗ'б $O(bl^4-c*/d)$ $o(bUc-/ei)$ Да Поиск в глубину Нет $0A3^*$ $O(\beta t)$ Нет Поиск с раничением глубины Нет $0(\beta e)$ $O(bl)$ Нет Поиск с итеративным углублением Даа $o(bd)$ $O(bd)$ Дав Двунаправленный поиск (если он применим) Даа'г $o\{bdn\}$ $0(bdn)$ Да""г В некоторых задачах повторяющиеся состояния являются неизбежными. К ним относятся все задачи, в которых действия являются обратимыми, такие как задачи поиска маршрута и игры со скользящими

фишками. Деревья поиска для этих проблем бесконечны, но если мы отсечем некоторые из повторяющихся состояний, то сможем уменьшить дерево поиска до конечного размера, формируя только ту часть дерева, которая охватывает весь граф пространства состояний. Рассматривая только часть дерева поиска вплоть до некоторой постоянной глубины, можно легко обнаружить ситуации, в которых удаление повторяющихся состояний позволяет достичь экспоненциального уменьшения стоимости поиска. В крайнем случае пространство состояний размером $d+1$ (рис. 3.11, а) становится деревом с $2d$ листьями (рис. 3.11, б).

Более реалистичным примером может служить $\wedge$ прямоугольная решетка, как показано на рис. 3.11, в. В решетке каждое состояние имеет четырех приемников, поэтому дерево поиска, включающее повторяющиеся состояния, имеет $4d$ листьев, но существует приблизительно только $2 \cdot d^2$ различных состояний с d этапами достижения любого конкретного состояния. Для $d=20$ это означает, что существует около триллиона узлов, но лишь примерно 800 различных состояний.

Таким образом, неспособность алгоритма обнаруживать повторяющиеся состояния может послужить причиной того, что разрешимая задача станет неразрешимой.

Такое обнаружение обычно сводится к тому, что узел, подлежащий развертыванию, сравнивается с теми узлами, которые уже были развернуты; если обнаружено совпадение, то алгоритм распознал наличие двух путей в одно и то же состояние и может отбросить один из них.

При поиске в глубину в памяти хранятся только те узлы, которые лежат на пути от корня до текущего узла. Сравнение этих узлов с текущим узлом позволяет алгоритму обнаружить закливающиеся пути, которые могут быть немедленно отброшены. Это позволяет обеспечить, чтобы конечные пространства состояний не превращались в бесконечные деревья поиска из-за циклов, но, к сожалению, не дает возможности предотвратить экспоненциальное разрастание нециклических путей в задачах, подобных приведенным на рис. 3.11. Единственный способ предотвращения этого состоит в том, что в памяти нужно хранить больше узлов. В этом заключается фундаментальный компромисс между пространством и временем.

$s^{\wedge}$ Алгоритмы, которые забывают свою историю, обречены на то, чтобы ее повторять. а) б) в) Рис. 3.11. Пространства состояний, которые формируют экспоненциально более крупные деревья поиска: пространство состояний, в котором имеются два возможных действия, ведущих от А к В, два — от В к С и т.д.; это пространство состояний содержит $d + 1$ состояний, где d — максимальная глубина (а); дерево поиска, которое имеет $2d$ ветвей, соответствующих $2d$ путям через это пространство (б); пространство состояний в виде прямоугольной решетки (в); состояния, находящиеся в пределах 2 этапов от начального состояния (А), обозначены серым цветом Если некоторый алгоритм запоминает каждое состояние, которое он посетил, то может рассматриваться как непосредственно исследующий

граф пространства состояний. В частности, можно модифицировать общий алгоритм Tree-Search, чтобы включить в него структуру данных, называемую $\wedge$ закрытым списком, в котором хранится каждый развернутый узел. (Периферию, состоящую из неразвернутых узлов, иногда называют $\wedge$ открытым списком.) Если текущий узел совпадает с любым узлом из закрытого списка, то не разворачивается, а отбрасывается. Этому новому алгоритму присвоено название алгоритма поиска в графе, Graph-Search (листинг 3.6). При решении задач со многими повторяющимися состояниями алгоритм Graph-Search является намного более эффективным по сравнению с Tree-Search. В наихудшем случае предъявляемые им требования к времени и пространству пропорциональны размеру пространства состояний. Эта величина может оказаться намного меньше, чем $O(bd)$.

Вопрос о том, оптимален ли поиск в графе, остается сложным. Выше было указано, что появление повторяющегося состояния соответствует обнаружению алгоритмом двух путей в одно и то же состояние. Алгоритм Graph-Search, приведенный в листинге 3.6, всегда отбрасывает вновь обнаруженный путь и оставляет первоначальный; очевидно, что если этот вновь обнаруженный путь короче, чем первоначальный, то алгоритм Graph-Search может упустить оптимальное решение. К счастью,

можно показать (упр. 3.12), что этого не может случиться, если используется либо поиск по критерию стоимости, либо поиск в ширину с постоянными стоимостями этапов; таким образом, эти две оптимальные стратегии поиска в дереве являются также оптимальными стратегиями поиска в графе. При поиске с итеративным углублением, с другой стороны, используется разворачивание в глубину, поэтому этот алгоритм вполне может проследовать к некоторому узлу по неоптимальному пути, прежде чем найти оптимальный. Это означает, что при поиске в графе с итеративным углублением необходимо проверять, не является ли вновь обнаруженный путь к узлу лучшим, чем первоначальный, и в случае положительного ответа в нем может потребоваться пересматривать значения глубины и стоимости путей для потомков этого узла.

Листинг 3.6. Общий алгоритм поиска в графе. Множество closed может быть реализовано с помощью хэш-таблицы для обеспечения эффективной проверки повторяющихся состояний.

В этом алгоритме предполагается, что первый найденный путь к состоянию s является наименее дорогостоящим (см. текст)
`function Graph-Search(problem, fringe)`
`returns` решение или индикатор неудачи `failure` `closed` $\leftarrow$ пустое множество `fringe`
 $\leftarrow$ Insert(Make-Node(Initial-State[problem]), fringe) `loop do` `if` Empty?(fringe) `then`
`return` индикатор неудачи `failure` `node` $\leftarrow$ Remove-First(fringe) `if` Goal-Test[problem](State[node]) `then` `return` Solution(node) `if` State[node] не находится в множестве closed `then` добавить State[node] к множеству closed `fringe` $\leftarrow$ Insert-

All(Expand(node, problem), fringe) Между прочим, использование закрытого списка closed означает, что поиск в глубину и поиск с итеративным углублением больше не имеют линейных требований к пространству. Поскольку в алгоритме Graph-Search каждый узел хранится в памяти, некоторые методы поиска становятся неосуществимыми из-за недостаточного объема памяти.

3.6. ПОИСК С ЧАСТИЧНОЙ ИНФОРМАЦИЕЙ В разделе 3.3 было выдвинуто предположение, что среда является полностью наблюдаемой и детерминированной и что агент имеет информацию о том, каковы последствия каждого действия. Поэтому агент может точно вычислить, какое состояние становится результатом любой последовательности действий, и всегда знает, в каком состоянии он находится. Его восприятия не предоставляют новой информации после выполнения каждого действия. Но что произойдет, если знания о состояниях или действиях являются неполными? Авторы обнаружили, что разные типы неполноты приводят к трем перечисленным ниже типам проблем.

1. Проблемы отсутствия датчиков (называемые также проблемами совместимости).

Если агент вообще не имеет датчиков, то (насколько ему известно) может находиться в одном из нескольких возможных начальных состояний и поэтому каж-

дое действие способно перевести его в одно из нескольких возможных состояний-преемников.

2. Проблемы непредвиденных ситуаций. Если среда наблюдаема лишь частично или действия являются неопределенными, то акты восприятия агента предоставляют новую информацию после выполнения каждого действия. Каждое возможное восприятие определяет непредвиденную ситуацию, к которой необходимо подготовиться с помощью соответствующего плана. Проблема называется обусловленной сторонним воздействием, если неопределенность вызвана действиями другого агента.

3. Проблемы исследования. Если состояния и действия в среде неизвестны, агент должен действовать так, чтобы их обнаружить. Проблемы исследования могут рассматриваться как крайний случай проблем непредвиденных ситуаций.

В качестве примера мы будем использовать среду мира пылесоса. Напомним, что пространство состояний имеет восемь состояний, как показано на рис. 3.12.

Существуют три действия (Left, Right и Suck), и цель состоит в том, чтобы был убран весь мусор (состояния 7 и 8). Если среда наблюдаема, детерминирована и полностью известна, то эта задача решается тривиально с помощью любого из описанных нами алгоритмов. Например, если начальным является состояние 5, то последовательность действий [Right, Suck] обеспечивает достижение целевого состояния 8. В оставшейся части этого раздела рассматриваются версии данной

задачи, в которых отсутствуют датчики и возникают непредвиденные ситуации. Проблемы исследования описаны в разделе 4.5, а проблемы, обусловленные сторонним воздействием, — в главе 6.

Рис. 3.12. Восемь возможных состояний мира пылесоса

Проблемы отсутствия датчиков. Предположим, что агенту-пылесосу известны все последствия его действий, но он не имеет датчиков. В таком случае агент знает только, что его начальным состоянием является одно состояние из множества $\{1,2,3,4,5,6,7,8\}$. На первый взгляд можно предположить, что попытки агента предсказать будущую ситуацию окажутся бесполез-

ными, но фактически он может сделать это вполне успешно. Поскольку агент знает, к чему приводят его действия, то может, например, вычислить, что действие Right вызовет переход его в одно из состояний $\{2,4,6,8\}$, а последовательность действий [Right, Suck] всегда оканчивается в одном из состояний $\{4,8\}$. Наконец, последовательность действий [Right, Suck, Left, Suck] гарантирует достижение целевого состояния 7, независимо от того, каковым является начальное состояние. Мы утверждаем, что агент может принудительно перевести мир в состояние 7, даже если ему не известно, с какого состояния он начинается. Подведем итог: если мир не является полностью наблюдаемым, то агент должен рассуждать о том, в какое множество состояний (а не в единственное состояние) он может попасть. Мы называем каждое такое множество состояний доверительным состоянием, поскольку оно показывает, в каких возможных физических состояниях агент может считать себя находящимся в данный момент со всей уверенностью. (В полностью наблюдаемой среде каждое доверительное состояние содержит одно физическое состояние.) Для решения проблемы отсутствия датчиков необходимо выполнять поиск в пространстве доверительных, а не физических состояний. Первоначальное состояние является доверительным состоянием, а каждое действие становится отображением из одного доверительного состояния в другое. Результат применения некоторого действия к некоторому доверительному состоянию определяется путем объединения результатов применения этого действия к каждому физическому состоянию из этого доверительного состояния. Теперь любой путь объединяет несколько доверительных состояний, а решением является путь, который ведет к такому доверительному состоянию, все члены которого представляют собой целевые состояния. На рис. 3.13 показано пространство достижимых доверительных состояний для детерминированного мира пылесоса без датчиков. Существует только 12 достижимых доверительных состояний, но все пространство доверительных состояний включает каждое возможное множество физических состояний, т.е. $2^8=256$ доверительных состояний. Вообще говоря, если пространство физических состояний имеет S состояний, то пространство доверительных состояний имеет 2^S доверительных состояний.

В приведенном выше описании проблем отсутствия датчиков предполагалось, что действия являются детерминированными, но этот анализ, по сути, остается неизменным, если среда — недетерминированная, т.е. если действия могут иметь несколько возможных результатов. Причина этого состоит в том, что в отсутствие датчиков агент не способен определить, какой результат достигнут фактически, поэтому различные возможные результаты становятся просто дополнительными физическими состояниями в доверительном состоянии-приемнике. Например, предположим, что среда подчиняется закону Мэрфи (или закону "подлости"): так называемое действие Suck иногда оставляет мусор на полу, но только если на нем еще не было мусора. В таком случае, если действие Suck применяется в физическом состоянии 4 (см. рис. 3.12), то существуют два возможных результата: состояния 2 и 4. Теперь применение действия Suck в начальном доверительном состоянии, $\{1,2,3,4,5,6,7,8\}$, приводит к доверительному состоянию, представляюб Мы предполагаем, что большинство читателей сталкиваются с аналогичными проблемами и поэтому выражают сочувствие нашему агенту. Авторы приносят свои извинения владельцам современных, эффективных бытовых приборов, которые не смогут извлечь урок из этого педагогического упражнения.

щему собой объединение множеств результатов для этих восьми физических состояний. Проведя эти вычисления, можно обнаружить, что новым доверительным состоянием снова становится $\{1,2,3,4,5,6,7,8\}$. Таким образом, для агента без датчиков в мире закона Мэрфи действие Suck оставляет доверительное состояние неизменным! Это означает, что фактически данная задача неразрешима (см. упр.

3.18). Интуитивно можно понять, что причина этого состоит в том, что агент не может определить, является ли текущий квадрат грязным и поэтому не способен установить, приведет ли действие Suck к его очистке или оставит еще больше мусора.

Рис. 3.13. Достижимая часть пространства доверительных состояний для детерминированного мира пылесоса без датчиков. Каждый затененный прямоугольник соответствует одному доверительному состоянию. В любой конкретный момент времени агент находится в одном конкретном доверительном состоянии, но не знает, в каком именно физическом состоянии он находится.

Первоначальным доверительным состоянием (с полным незнанием ситуации) является верхний центральный прямоугольник. Действия обозначены дугами с метками, а петли, обозначающие возврат в одно и то же доверительное состояние, опущены для упрощения рисунка Проблемы непредвиденных ситуаций Если среда является таковой, что агент после выполнения действия может получить от своих датчиков в том же состоянии новую информацию, то агент сталкива-

ется с $\wedge$ проблемой непредвиденных ситуаций. Решение проблемы непредвиденных ситуаций часто принимает форму дерева, в котором каждая ветвь может быть выбрана в зависимости от результатов восприятий, полученных вплоть до этой позиции в дереве. Например, предположим, что агент находится в мире закона Мэрфи и имеет датчик положения и локальный датчик мусора, но не имеет датчика, способного обнаружить мусор в других квадратах. Таким образом, восприятие [L, Dirty] означает, что агент находится в одном из состояний {1,3}. Агент может сформулировать последовательность действий [Suck, Right, Suck]. Всасывание должно было бы перевести текущее состояние в одно из состояний {5,7}, и тогда передвижение вправо перевело бы данное состояние в одно из состояний {6,8}. Выполнение заключительного действия Suck в состоянии 6 переводит агента в состояние 8, целевое, но выполнение его в состоянии 8 способно снова перевести агента в состояние 6 (согласно закону Мэрфи), и в этом случае данный план оканчивается неудачей.

Исследуя пространство доверительных состояний для этой версии задачи, можно легко определить, что ни одна фиксированная последовательность действий не гарантирует решение данной задачи. Однако решение существует, при условии, что мы не будем настаивать на фиксированной последовательности действий: [Suck, Right, if [R,Dirty] then Suck] Тем самым дополняется пространство решений для включения возможности выбора действий с учетом непредвиденных ситуаций, возникающих во время выполнения. Многие задачи в реальном, физическом мире усложняются из-за наличия проблемы непредвиденных ситуаций, поскольку точные предсказания в них невозможны. По этой причине многие люди соблюдают предельную осторожность во время пеших прогулок или вождения автомобиля.

Иногда проблемы непредвиденных ситуаций допускают чисто последовательные решения. Например, рассмотрим полностью наблюдаемый мир закона Мэрфи.

Непредвиденные ситуации возникают, если агент выполняет действие Suck в чистом квадрате, поскольку при этом в данном квадрате может произойти или не произойти отложение мусора. При том условии, что агент никогда не будет очищать чистые квадраты, не будут возникать какие-либо непредвиденные ситуации и поэтому из любого начального состояния можно будет найти последовательное решение (упр. 3.18).

Алгоритмы для решения проблем непредвиденных ситуаций являются более сложными по сравнению с алгоритмами стандартного поиска, приведенными в этой главе; они рассматриваются в главе 12. Кроме того, проблемы непредвиденных ситуаций требуют применения немного иного проекта агента, в котором агент может действовать до того, как найдет гарантированный план. Это полезно, поскольку вместо предварительного обдумывания каждой возможной непредвиденной ситуации, которая могла бы возникнуть во время выполнения,

часто бывает лучше приступить к действию и узнать, какие непредвиденные ситуации действительно возникают. После этого агент может продолжать решать свою задачу, принимая в расчет эту дополнительную информацию. Такой тип $\wedge$ чередования поиска и выполнения является также полезным при решении проблем исследования (см. раздел 4.5) и при ведении игр (см. главу 6).

Резюме В настоящей главе представлены методы, которые могут использоваться агентом для выбора действий в таких вариантах среды, которые являются детерминированными, наблюдаемыми, статическими и полностью известными. В таких случаях агент может формировать последовательности из действий, позволяющих ему достичь своих целей; такой процесс называется поиском. • Прежде чем агент сможет приступить к поиску решений, он должен сформулировать цель, а затем использовать эту цель для формулировки задачи. • Задача состоит из четырех частей: начальное состояние, множество действий, функция проверки цели и функция стоимости пути. Среда задачи представлена пространством состояний, а путь через пространство состояний от начального состояния до целевого состояния представляет собой решение. • Для решения любой задачи может использоваться единый, общий алгоритм Tree-Search; конкретные варианты этого алгоритма воплощают различные стратегии. • Алгоритмы поиска оцениваются на основе полноты, оптимальности, временной и пространственной сложности. Сложность зависит от коэффициента ветвления в пространстве состояний, b , и глубины самого поверхностного решения, d . • При поиске в ширину для развертывания выбирается самый поверхностный неразвернутый узел в дереве поиска. Этот поиск является полным, оптимальным при единичных стоимостях этапов и характеризуется временной и пространственной сложностью $O(bd)$. В связи с такой пространственной сложностью в большинстве случаев он становится практически не применимым.

Поиск по критерию стоимости аналогичен поиску в ширину, но предусматривает развертывание узла с самой низкой стоимостью пути, $d(n)$. Он является полным и оптимальным, если стоимость каждого шага превышает некоторое положительное предельное значение e . • При поиске в глубину для развертывания выбирается самый глубокий неразвернутый узел в дереве поиска. Этот поиск не является ни полным, ни оптимальным, и характеризуется временной сложностью $O(b^d)$ и пространственной сложностью $O(b \cdot d)$, где d — максимальная глубина любого пути в пространстве состояний. • При поиске с ограничением глубины на поиск в глубину налагается установленный предел глубины. • При поиске с итеративным углублением вызывается поиск с ограничением глубины и каждый раз устанавливаются увеличивающиеся пределы, до тех пор, пока цель не будет найдена. Этот поиск является полным и оптимальным при единичных стоимостях этапов и характеризуется временной сложностью $O(bd)$ и пространственной

сложностью $O(bd)$. • Двухнаправленный поиск способен чрезвычайно уменьшить временную сложность, но он не всегда применим и может потребовать слишком много пространства.

• Если пространство состояний представляет собой граф, а не дерево, то может оказаться целесообразной проверка повторяющихся состояний в дереве поиска. Алгоритм Graph-Search устраняет все дублирующие состояния. • Если среда является частично наблюдаемой, то агент может применить алгоритмы поиска в пространстве доверительных состояний, или множестве возможных состояний, в которых может находиться агент. В некоторых случаях может быть сформирована единственная последовательность решения; в других случаях агенту требуется план действий в непредвиденных ситуациях, чтобы иметь возможность справиться со всеми неизвестными обстоятельствами, которые могут возникнуть.

Библиографические и исторические заметки Большинство задач поиска в пространстве состояний, проанализированных в этой главе, имеют длинную историю, отраженную в литературе, и являются менее тривиальными, чем может показаться на первый взгляд. Задача с миссионерами и каннибалами, используемая в упр. 3.9, была подробно проанализирована Амарелем [24]. До него эту задачу ввели в проблематику искусственного интеллекта Саймон и Ньюэлл [1420], и в проблематику исследования операций — Беллман и Дрейфус [96]. Работы наподобие проведенных Ньюэллом и Саймоном над программами Logic Theorist [1127] и GPS [1129] привели к тому, что алгоритмы поиска считались основным оружием в арсенале исследователей искусственного интеллекта 1960-х годов, а сами процессы решения задач стали рассматриваться в качестве канонической проблемы искусственного интеллекта. К сожалению, в то время в области автоматизации этапа формулировки задачи было предпринято слишком мало усилий. Более современная трактовка подхода к представлению и абстрагированию задач, включая применение программ искусственного интеллекта, которые (отчасти) сами выполняют эти функции, изложена в книге Кноблока [807].

Задача игры в восемь — это "младшая сестра" задачи игры в пятнадцать, которая была изобретена знаменитым американским разработчиком игр Сэмом Ллойдом [955] в 1870-х годах. Игра в пятнадцать быстро приобрела в Соединенных Штатах огромную популярность, которую можно сравнить лишь с более современной сенсацией, вызванной изобретением кубика Рубика. Она также быстро привлекла внимание математиков [739], [1486]. Редакторы American Journal of Mathematics заявили:

"Игра в пятнадцать в последние несколько недель занимает все внимание американской публики, и можно с уверенностью сказать, что ею интересуются девять из десяти людей всех полов и возрастов и всех общественных положений. Но не это побудило нас, редакторов, включить статьи, посвященные этой теме, в наш

журнал American Journal of Mathematics, а тот факт, что..." (далее следует краткое описание аспектов игры в пятнадцать, представляющих интерес с точки зрения математики).

Исчерпывающий анализ игры в восемь был проведен с помощью компьютера Шофилдом [1363]. Ратнер и Уормут [1268] показали, что общая версия игры (не в пятнадцать, а в $nl-1$) принадлежит к классу NP-полных задач.

Задача с восемью ферзями была впервые опубликована анонимно в немецком шахматном журнале Schach в 1848 году; позднее ее создание приписали некоему

Максу Беззелю. Она была повторно опубликована в 1850 году и на этот раз привлекала внимание выдающегося математика Карла Фридриха Гаусса, который попытался перечислить все возможные решения, но нашел только 72. Наук (Nauck) опубликовал все 92 решения позднее, в том же 1850 году. Нетто [1121] обобщил эту задачу до n ферзей, а Абрамсон и Янг [2] нашли алгоритм с оценкой $O(n)$.

Каждая из реальных задач поиска, перечисленных в данной главе, была предметом значительных усилий исследователей. Методы выбора оптимальных расписаний авиаперелетов по большей части остаются недоступными для общего пользования (закрытыми), но Карл де Маркен (Carl de Marcken) сообщил авторам (в личной беседе), что структуры формирования цен на авиабилеты и связанные с ними ограничения стали настолько сложными, что задача выбора оптимального авиарейса является формально неразрешимой. Задача коммивояжера (Traveling Salesperson Problem — TSP) — это стандартная комбинаторная проблема в теоретических компьютерных науках [898], [899]. Карп [772] доказал, что задача TSP является NP-трудной, но для нее были разработаны эффективные методы эвристической аппроксимации [933]. Арора [41] разработал полностью полиномиальную схему аппроксимации для евклидовых вариантов задачи TSP. Обзор методов компоновки СБИС был сделан Шахукаром и Мазумдером [1390], а в журналах по сверхбольшим интегральным микросхемам (СБИС) появилось много статей, посвященных оптимизации компоновки. Задачи робототехнической навигации и сборки обсуждаются в главе 25.

Алгоритмы неинформированного поиска для решения задач являются центральной темой классических компьютерных наук [680] и исследования операций [417]; более современные результаты исследований приведены в книгах Део и Панга [390], а также Галло и Паллоттино [516]. Метод поиска в ширину применительно к решению задач с лабиринтами был сформулирован Муром [1078]. Метод динамического программирования [96], в котором предусматривается систематическая регистрация решений для всех подзадач с возрастающей длиной, может рассматриваться как форма поиска в ширину в графах. Алгоритм определения

кратчайшего пути между двумя точками, предложенный Дейкстрой [399], является предшественником алгоритма поиска по критерию стоимости.

Одна из версий алгоритма поиска с итеративным углублением, предназначенная для эффективного ведения игры в шахматы с контролем времени, была впервые применена Слейтом и Аткином [1429] в программе ведения шахматной игры Chess 4.5, но ее применению для поиска кратчайшего пути в графе мы обязаны Корфу [835]. В некоторых случаях может также оказаться очень эффективным двунаправленный поиск, который был предложен Полом [1219], [1221].

Частично наблюдаемые и недетерминированные варианты среды не были достаточно глубоко исследованы в этом подходе к решению задач. Некоторые проблемы эффективности при поиске в пространстве доверительных состояний рассматривались Генезеретом и Нурбакшем [538]. Кёниг и Симмонс [819] изучали навигацию робота из неизвестной начальной позиции, а Эрдман и Мэйсон [439] исследовали проблему робототехнического манипулирования без датчиков, используя непрерывную форму поиска в пространстве доверительных состояний. Поиск в условиях непредвиденных ситуаций изучался в этой подобласти планирования (см. главу 12). По большей части в подходе к изучению планирования и осуществления действий с неопределенной информацией используются инструментальные средства теории вероятностей и теории решений (см. главу 17).

Книги Нильссона [1141], [1142] являются хорошим общим источником информации о классических алгоритмах поиска. Исчерпывающий и более современный обзор можно найти в [838]. Статьи о новых алгоритмах поиска (которые, как это ни удивительно, продолжают изобретаться) публикуются в таких журналах, как *Artificial Intelligence*.

Упражнения 3.1. Самостоятельно сформулируйте определения следующих понятий: состояние, пространство состояний, дерево поиска, поисковый узел, цель, действие, функция определения преемника и коэффициент ветвления.

3.2. Объясните, почему составление формулировки задачи должно осуществляться вслед за составлением формулировки цели.

3.3. Предположим, что выражение $\text{Legal-Actions}(s)$ обозначает множество действий, которые являются допустимыми в состоянии s , а выражение $\text{Result}(a, s)$ обозначает состояние, которое следует из выполнения допустимого действия a в состоянии s . Определите функцию Successor-Fn в терминах выражений Legal-Actions и Result , и наоборот.

3.4. Покажите, что в задаче игры в восемь состояния подразделяются на два непересекающихся множества, таких, что ни одно состояние из первого множества не может быть преобразовано в состояние из второго множества, даже с

применением сколь угодно большого количества ходов. (Подсказка. См. [102].) Разработайте процедуру, позволяющую узнать, к какому множеству относится данное состояние, и объясните, для чего нужно иметь под рукой такую процедуру, формируя состояния случайным образом.

3.5. Рассмотрите задачу с p ферзями, используя "эффективную" инкрементную формулировку, приведенную на с. 119. Объясните, почему размер пространства состояний равен по меньшей мере l^{fc} , и оцените наибольшее значение p , для которого является осуществимым исчерпывающее исследование.

(Подсказка. Определите нижнюю границу коэффициента ветвления, рассматривая максимальное количество клеток, которые ферзь может атаковать в любом столбце.) 3.6. Всегда ли наличие конечного пространства состояний приводит к получению конечного дерева поиска? А что можно сказать о конечном пространстве состояний, которое является деревом? Можете ли вы указать более точно, применение пространств состояний каких типов всегда приводит к получению конечных деревьев поиска? (Адаптировано из [99].) 3.7. Укажите для каждой из следующих задач ее компоненты: начальное состояние, проверку цели, функцию определения преемника и функцию стоимости.

Выберите формулировку, которая является достаточно точной, чтобы ее можно было успешно реализовать. а) Необходимо раскрасить плоскую карту, используя только четыре цвета, таким образом, чтобы никакие два смежных региона не имели один и тот же цвет.

б) Обезьяна, имеющая рост 3 фута, находится в комнате, где под потолком, на высоте 8 футов, подвешено несколько бананов. Обезьяна хочет получить бананы. В комнате находятся две проволочные корзины высотой по 3 фута каждая, которые можно передвигать, ставить друг на друга и на которые можно залезать. в) Некоторая программа выводит сообщение "недопустимая входная запись" после передачи ей некоторого файла, состоящего из входных записей.

Известно, что обработка каждой записи происходит независимо от других записей. Требуется обнаружить, какая запись является недопустимой. г) Имеются три кувшина, с емкостью 12 галлонов, 8 галлонов и 3 галлона, а также водопроводный кран. Кувшины можно заполнять или опорожнять, выливая воду из одного кувшина в другой или на землю.

Необходимо отмерить ровно один галлон. 3.8. Рассмотрите пространство состояний, в котором начальным состоянием является число 1, а функция определения преемника для состояния p возвращает два состояния: числа $2p$ и $2p+1$. а) Нарисуйте часть пространства состояний, которая относится к состояниям 1-15. б) Предположим, что целевым состоянием является 11. Перечислите последовательности, в которых будут посещаться узлы при поиске в ширину, поиске

с ограничением глубины, с пределом 3, и поиске с итеративным углублением. в) Будет ли подходящим для решения этой задачи двунаправленный поиск?

Если да, то опишите подробно, как действовал бы этот метод поиска. г) Каковым является коэффициент ветвления в каждом направлении двунаправленного поиска?

д) Не подсказывает ли вам ответ на вопрос упр. 3.8, <?, что нужно найти другую формулировку этой задачи, которая позволила бы решить проблему перехода из состояния 1 в заданное целевое состояние почти без поиска?

3.9. йё) Задача с миссионерами и каннибалами обычно формулируется следующим образом. Три миссионера и три каннибала находятся на одной стороне реки, где также находится лодка, которая может выдержать одного или двух человек.

Найдите способ перевезти всех на другой берег реки, никогда не оставляя гделибо группу миссионеров, которую превосходила бы по численности группа каннибалов. Это — известная задача в искусственном интеллекте, поскольку она была темой первой статьи, в которой был применен подход к формулировке проблемы с аналитической точки зрения [24]. а) Точно сформулируйте эту задачу, определяя только те различия, которые необходимы для обеспечения правильного решения. Нарисуйте схему полного пространства состояний. б) Реализуйте и решите эту задачу оптимальным образом, используя соответствующий алгоритм поиска. Действительно ли имеет смысл проверять наличие повторяющихся состояний? в) Почему, по вашему мнению, люди сталкиваются с затруднениями при решении этой головоломки, несмотря на то, что пространство ее состояний является чрезвычайно простым?

ЗЛО. dfe Реализуйте следующие две версии функции определения преемника для задачи игры в восемь. Первая из них должна формировать сразу всех преемников, копируя и редактируя структуру данных игры в восемь, а вторая при каждом ее вызове должна формировать по одному новому преемнику и действовать по принципу непосредственной модификации родительского состояния (отменяя эти модификации в случае необходимости). Напишите версии процедуры поиска в глубину с итеративным углублением, в которых используются эти функции, и сравните данные об их производительности.

3.11. dfe На с. 135 упоминался поиск с итеративным удлинением, итеративный аналог поиска по критерию стоимости. Его идея состоит в том, что должны использоваться увеличивающиеся пределы стоимости пути. Если сформирован узел, стоимость пути для которого превышает текущий предел, этот узел немедленно отбрасывается. При каждой новой итерации предел устанавливается равным самому низкому значению стоимости пути для любого узла, отвергнутого в предыдущей итерации. а) Покажите, что этот алгоритм является оптимальным применительно к общим методам определения стоимости пути. б) Рассмотрите однородное дерево с коэффициентом

ветвления b , глубиной решения d и единичными стоимостями этапов. Какое количество итераций требуется при итеративном удлинении? в) Рассмотрите стоимости этапов, взятые из непрерывного диапазона $[0,1]$ с минимальной положительной стоимостью e . Сколько итераций потребуется в самом неблагоприятном случае? г) Реализуйте этот алгоритм и примените его к экземплярам задачи игры в восемь и задачи коммивояжера. Сравните производительность этого алгоритма с производительностью алгоритма поиска по критерию стоимости и прокомментируйте полученные результаты.

3.12. Докажите, что поиск по критерию стоимости и поиск в ширину с постоянными значениями стоимости этапа являются оптимальными при их использовании с алгоритмом Graph-Search. Продемонстрируйте такое пространство состояний с постоянными значениями стоимости этапа, в котором алгоритм Graph-Search с использованием итеративного углубления находит неоптимальное решение.

3.13. Опишите пространство состояний, в котором поиск с итеративным углублением характеризуется гораздо более низкой производительностью по сравнению с поиском в глубину (например, $O(n^2)$, в отличие от $O(n)$).

3.14. 1ж) Напишите программу, которая принимает в качестве входных данных URL-локаторы двух Web-страниц и находит путь от одной к другой, состоящий из ссылок. В чем состоит подходящая для этого стратегия поиска? Действительно ли имеет смысл применять двунаправленный поиск? Можно ли использовать для реализации функции определения предшественника машину поиска?

3.15. йё) Рассмотрите задачу определения кратчайшего пути между двумя точками на плоскости, на которой имеются препятствия в виде выпуклых многоугольников, как показано на рис. 3.14. Это — идеализация задачи, которую должен

решать робот, прокладывая свой путь через среду, в которой очень мало свободного места. одет Рис. 3.14. Сцена с многоугольными препятствиями а) Предположим, что пространство состояний состоит из всех позиций (x, y) на плоскости. Каково количество состояний в этом пространстве?

Каково в нем количество путей к цели? б) Объясните в нескольких словах, почему в этой двумерной сцене кратчайший путь от одной вершины многоугольника до любой другой должен состоять из прямолинейных отрезков, соединяющих некоторые вершины многоугольников. Теперь определите более приемлемое пространство состояний. Насколько велико это пространство состояний? в) Определите функции, необходимые для реализации решения этой задачи поиска, включая функцию определения преемника, которая принимает в качестве входных данных координаты любой из вершин и возвращает множество вершин, достижимых по прямой линии от данной вершины.

(Не забывайте при этом о соседних вершинах того же многоугольника.) Используйте в качестве значения эвристической функции расстояние между указанными точками по прямой, г) Примените для решения ряда задач из этой области один или несколько алгоритмов, представленных в настоящей главе, и прокомментируйте данные об их производительности.

3.16. Gh Определение задачи обеспечения навигации робота, приведенной в упр. 3.15, можно преобразовать в определение среды следующим образом. • Акт восприятия будет представлять собой список позиций видимых вершин, сформированный относительно агента. Акт восприятия не предусматривает определение позиции робота! Робот должен определять свою собственную позицию по карте; на данный момент можно предположить, что каждое местоположение имеет другое "представление". • Каждое действие должно быть вектором, описывающим прямолинейный путь, по которому будет следовать робот. Если путь свободен, то действие выполняется успешно; в противном случае робот останавливается в той точке,

где его путь впервые сталкивается с препятствием. Если агент возвращает нулевой вектор движений и находится в целевой позиции (которая является постоянной и известной), то среда должна телепортировать этого агента в случайно выбранное местоположение (не находящееся в пределах препятствия). • Показатель производительности предусматривает штрафование агента на 1 пункт за каждую единицу пройденного расстояния и награждение его 1000 пунктами каждый раз после достижения цели.

Ниже перечислены предлагаемые задания. а) Реализуйте описанную среду и агента, решающего задачи для этой среды.

После каждой телепортации агент должен будет сформулировать новую задачу, которая предусматривает также определение его текущего местоположения. б) Зарегистрируйте данные о производительности предложенного вами агента (для этого предусмотрите выработку агентом соответствующих комментариев по мере его передвижения в среде) и составьте отчет о его производительности поданным, охватывающим больше 100 эпизодов. в) Модифицируйте среду так, чтобы в 30% случаев движение агента заканчивались в не предусмотренном им месте назначения (выбранном случайным образом среди других видимых вершин, если таковые имеются, а в противном случае соответствующем ситуации, в которой вообще не было выполнено никакого движения). Это — грубая модель ошибок при выполнении движений реального робота. Доработайте определение агента так, чтобы при обнаружении указанной ошибки он определял, где находится, а затем создавал план возвращения в то место, где он находился прежде, и возобновлял выполнение прежнего плана. Помните, что иногда попытка возвращения в прежнее место также может оканчиваться неудачей! Продемонстрируйте пример агента,

который успешно преодолевает две последовательные ошибки движения и все равно достигает цели. г) Опробуйте две различные схемы возобновления работы после ошибки: впервых, отправиться к ближайшей вершине из первоначального маршрута и, во-вторых, перепланировать маршрут к цели от нового местоположения.

Сравните производительность всех схем возобновления работы. Влияет ли на результаты такого сравнения включение затрат на поиск? д) Теперь предположим, что есть такие местоположения, представления среды из которых являются идентичными. (Например, предположим, что мир — это решетка с квадратными препятствиями.) С какой проблемой теперь сталкивается агент? Как должны выглядеть решения?

3.17. На с. 115 было указано, что мы не будем принимать во внимание задачи с отрицательными значениями стоимости пути. В данном упражнении эта тема рассматривается немного более подробно. а) Предположим, что действия могут иметь произвольно большие отрицательные стоимости; объясните, почему такая ситуация может вынудить любой оптимальный алгоритм исследовать полное пространство состояний. б) Удастся ли выйти из этого положения, потребовав, чтобы стоимости этапов были больше или равны некоторой отрицательной константе с?

Рассмотрите и деревья, и графы.

в) Предположим, что имеется множество операторов, образующих цикл, так что выполнение операторов этого множества в определенном порядке не приводит к какому-либо чистому изменению состояния. Если все эти операторы имеют отрицательную стоимость, то какие выводы из этого следуют применительно к оптимальному поведению агента в такой среде? г) Можно легко представить себе, что операторы с высокой отрицательной стоимостью имеются даже в таких проблемных областях, как поиск маршрута. Например, некоторые участки дороги могут оказаться настолько живописными, что стремление ознакомиться с ними намного перевесит обычные здравые рассуждения о стоимости, измеряемой в терминах затрат времени и топлива. Объясните, применяя точные термины, принятые в контексте поиска в пространстве состояний, почему все же люди не ведут свои автомобили неопределенно долго по живописным циклическим участкам пути, и укажите, каким образом нужно определить пространство состояний и операторы для задачи поиска маршрута, чтобы агенты с искусственным интеллектом также могли избежать попадания в цикл. д) Можете ли вы придумать пример такой реальной проблемной области, в которой стоимости этапов таковы, что могут вызвать возникновение цикла?

3.18. Рассмотрим мир пылесоса без датчиков, с двумя местоположениями, подчиняющийся закону Мэрфи. Нарисуйте пространство доверительных состояний,

достижимых из начального доверительного состояния $\{1,2,3,4,5,6,7,8\}$, и объясните, почему эта задача неразрешима. Покажите также, что если бы этот мир был полностью наблюдаемым, то существовала бы последовательность решения для каждого возможного начального состояния.

3.19. dfe Рассмотрим задачу в мире пылесоса, который определен на рис. 2.2. а) Какой из алгоритмов, определенных в этой главе, был бы подходящим для решения этой задачи? Должен ли этот алгоритм проверять наличие повторяющихся состояний? б) Примените выбранный вами алгоритм для вычисления оптимальной последовательности действий в мире с размером 3×3 , в начальном состоянии которого в трех верхних квадратах имеется мусор, а агент находится в центре. в) Сконструируйте агента, выполняющего поиск в этом мире пылесоса, и оцените его работу в множестве миров с размером 3×3 , характеризующемся вероятностью наличия мусора в каждом квадрате, равной 0.2. Включите в состав показателя производительности не только стоимость поиска, но и стоимость пути, используя для них подходящий "курс обмена". г) Сравните вашего лучшего поискового агента с простым рандомизированным рефлексным агентом, который всасывает мусор, если последний имеется, а в противном случае выполняет случайно выбранные перемещения. д) Рассмотрите, что произошло бы, если ли бы мир расширился до размеров $n \times n$. Как производительность данного поискового агента и рефлексного агента зависит от n ?

Глава 4. Информированный поиск и исследование пространства состояний

4 Информированный поиск и состояний В этой главе показано, как можно с помощью информации о пространстве состояний не дать алгоритмам заблудиться в темноте.

В главе 3 показано, что неинформированные стратегии поиска позволяют находить решения задач путем систематической выработки новых состояний и их проверки применительно к цели. К сожалению, в большинстве случаев эти стратегии являются чрезвычайно неэффективными. Как показано в настоящей главе, информированные стратегии поиска (в которых используются знания, относящиеся к конкретной задаче) обеспечивают более эффективный поиск решения. В разделе 4.1 описаны информированные версии алгоритмов главы 3, а в разделе 4.2 показано, как может быть получена необходимая информация, относящаяся к конкретной задаче. В разделах 4.3 и 4.4 представлены алгоритмы, которые выполняют исключительно локальный поиск в пространстве состояний, оценивая и модифицируя одно или несколько текущих состояний вместо систематического исследования путей из начального состояния. Эти алгоритмы применимы для решения задач, в которых стоимость пути не представляет интереса и требуется лишь найти состояние,

соответствующее решению. К этому семейству алгоритмов локального поиска относятся методы, созданные под влиянием исследований в области статистической физики (моделируемый отжиг) и эволюционной биологии (генетические алгоритмы).

Наконец, в разделе 4.5 рассматривается поиск в оперативном режиме, в котором агент сталкивается с полностью неизвестным пространством состояний.

4.1. Стратегии информированного (эвристического) поиска В данном разделе показано, как стратегия $\wedge$ информированного поиска (в которой кроме определения самой задачи используются знания, относящиеся к данной конкретной проблемной области) позволяет находить решения более эффективно, чем стратегия неинформированного поиска.

Общий рассматриваемый здесь подход называется $\wedge$ поиском по первому наилучшему совпадению. Поиск по первому наилучшему совпадению представляет собой разновидность общего алгоритма Tree-Search или Graph-Search, в котором узел для развертывания выбирается на основе $\wedge$ функции оценки, $f(n)$. По традиции для развертывания выбирается узел с наименьшей оценкой, поскольку такая оценка измеряет расстояние до цели. Поиск по первому наилучшему совпадению может быть реализован в рамках описанной в данной книге общей инфраструктуры поиска с помощью очереди по приоритету — структуры данных, в которой периферия поиска поддерживается в возрастающем порядке f -значений.

Название "поиск по первому наилучшему совпадению" (best first search) узаконено традицией, но неточно. В конце концов, если бы мы действительно могли развертывать наилучший узел первым, то не было бы и поиска как такового; решение задачи представляло бы собой прямое шествие к цели. Единственное, что мы можем сделать, — это выбрать узел, который представляется наилучшим в соответствии с функцией оценки. Если функция оценки действительно является точной, то выбранный узел в самом деле окажется наилучшим узлом, но фактически функция оценки иногда оказывается малоприспособленной и способной завести поиск в тупик. Тем не менее авторы будут придерживаться названия "поиск по первому наилучшему совпадению", поскольку более подходящее название "поиск по первому совпадению, которое можно считать наилучшим" было бы довольно громоздким.

Существует целое семейство алгоритмов поиска по первому наилучшему совпадению, Best-First-Search, с различными функциями оценки¹. Ключевым компонентом этих алгоритмов является $\wedge$ эвристическая функция², обозначаемая как $h(n)$: $h(n)$ = оценка стоимости наименее дорогостоящего пути от узла n до целевого узла. Например, в задаче поиска маршрута в Румынии можно было бы оценивать стоимость наименее дорогостоящего пути от Арада до Бухареста с

помощью расстояний по прямой до Бухареста, измеряемых в узловых точках маршрута от Арада до Бухареста.

Эвристические функции (или просто эвристики) представляют собой наиболее общую форму, в которой к алгоритму поиска подключаются дополнительные знания о задаче. Эвристики рассматриваются более подробно в разделе 4.2, а на данный момент мы будем определять их как произвольные функции, относящиеся к конкретной проблеме, с одним ограничением: если l — целевой узел, то $h(l) = 0$. В упр. 4.3 предложено показать, что это семейство включает несколько знакомых читателю неинформированных алгоритмов.

2 Эвристическая функция $h(n)$ принимает в качестве входного параметра некоторый узел, но зависит только от состояния данного узла.

В этой части настоящего раздела рассматриваются два способа использования эвристической информации для управления поиском.

Жадный поиск по первому наилучшему совпадению При жадном поиске по первому наилучшему совпадению предпринимаются попытки развертывания узла, который рассматривается как ближайший к цели на том основании, что он со всей вероятностью должен быстро привести к решению. Таким образом, при этом поиске оценка узлов производится с использованием только эвристической функции: $f(n) = h(n)$.

Теперь рассмотрим, как используется этот алгоритм при решении задачи поиска маршрута в Румынии на основе эвристической функции определения расстояния по прямой (Straight Line Distance — SLD), для которой принято обозначение $hSLD$.

Если целью является Бухарест, то необходимо знать расстояния по прямой от каждого прочего города до Бухареста, которые приведены в табл. 4.1. Например, $hSLD(\text{In } \{\text{Arad}\}) = 366$. Обратите внимание на то, что значения $hSLD$ не могут быть вычислены на основании описания самой задачи. Кроме того, для использования этой эвристической функции нужен определенный опыт, позволяющий узнать, каким образом значения $hSLD$ связаны с действительными дорожными расстояниями, а это означает, что данная функция исходит из практики.

Таблица 4.1. Значения $hSLD$ — расстояния по прямой до Бухареста

Обозначение узла	Название города	Расстояние по прямой до Бухареста
Arad	Арад	366
Bucharest	Бухарест	0
Craiova	Крайова	291
Drobeta	Дробета	245
Eforie	Эфорие	161
Fagaras	Фэгрэш	176
Giurgiu	Джурджу	201
Hirsova	Хыршова	151
Iasi	Яссы	226
Lugoj	Лугож	244
Mehadia	Мегадия	281
Neamt	Нямц	234
Oradea	Орадя	281
Pitesti	Питешти	171
Rimnicu	Рымнику-Вылча	191
Vilcea	Сибия	151
Si biu	Тимишоара	114
Timisoara	Урзичени	84
Urziceni	Васлуй	141
Vaslui	Зеринд	136
Zerind	Зеринд	374

На рис. 4.1 показан процесс применения жадного поиска по первому наилучшему совпадению с использованием значений

hSLD для определения пути от Арада до Бухареста. Первым узлом, подлежащим развертыванию из узла Arad, является узел Sibiu, поскольку город Сибиу находится ближе к Бухаресту, чем города Зеринд или Тимишоара. Следующим узлом, подлежащим развертыванию, является узел Fagaras, поскольку теперь ближайшим к Бухаресту является город Фэгэраш. Узел 3 В первом издании данной книги этот алгоритм поиска именовался просто жадным поиском; в книгах других авторов для него применяется название поиск по первому наилучшему совпадению.

Авторы настоящей книги используют последний термин в более общем смысле, в соответствии с трактовкой Перла [1188].

Fagaras, в свою очередь, обеспечивает формирование узла Bucharest, который является целевым. Применение в процессе решения данной конкретной задачи алгоритма жадного поиска по первому наилучшему совпадению с использованием функции hSLD позволяет найти решение без развертывания какого-либо узла, не находящегося в пути решения; это означает, что стоимость такого поиска является минимальной. Но само найденное решение не оптимально: путь до Бухареста через города Сибиу и Фэгэраш на 32 километра длиннее, чем путь через города РымникуВилча и Питешти. Это замечание показывает, почему данный алгоритм называется "жадным": на каждом этапе он пытается подойти к цели как можно ближе (фигурально выражаясь, "захватить как можно больше"). а) Начальное состояние б) После развертывания узла Arad в) После развертывания узла Sibiu Arad $AgacI^{\wedge} dSibiQ)^{\wedge} Fagaras^{\wedge} Oradea^{\wedge} 176\ 380$ (Щттси VilceI) Qjmisoaara, Zerind i) После развертывания узла Fagaras Рис. 4.1. Этапы жадного поиска пути до Бухареста по первому наилучшему совпадению с использованием эвристической функции hSLD, определяющей расстояние по прямой. Узлы обозначены с помощью их h-значений Процедура минимизации h(n) восприимчива к фальстартам (при ее использовании иногда приходится отменять начальные этапы). Рассмотрим задачу поиска пути

от города Яссы до города Фэгэраш. Эта эвристическая функция подсказывает, что в первую очередь должен быть развернут узел города Нямц, Neamt, поскольку он является ближайшим к узлу Fagaras, но этот путь становится тупиковым. Решение состоит в том, чтобы отправиться вначале в город Васлуй (а этот этап, согласно данной эвристической функции, фактически уводит дальше от цели), а затем продолжать движение через Урзичени, Бухарест и, наконец, в Фэгэраш. Поэтому в данном случае применение указанной эвристической функции вызывает развертывание ненужных узлов. Более того, если не будет предусмотрено обнаружение повторяющихся состояний, то решение так никогда не будет найдено — процедура поиска станет совершать возвратно-поступательные движения между узлами Neamt и Iasi.

Жадный поиск по первому наилучшему совпадению напоминает поиск в глубину в том отношении, что этот алгоритм предпочитает на пути к цели постоянно следовать

по единственному пути, но возвращается к предыдущим узлам после попадания в тупик. Данный алгоритм страдает от тех же недостатков, что и алгоритм поиска в глубину: он не является оптимальным, к тому же он — не полный (поскольку способен отправиться по бесконечному пути, да так и не вернуться, чтобы опробовать другие возможности). При этом в наихудшем случае оценки временной и пространственной сложности составляют $O(t^2)$, где t — максимальная глубина пространства поиска. Однако хорошая эвристическая функция позволяет существенно сократить такую сложность. Величина этого сокращения зависит от конкретной задачи и от качества эвристической функции.

Поиск A^* : минимизация суммарной оценки стоимости решения Наиболее широко известная разновидность поиска по первому наилучшему совпадению называется A^* поиском A^* (читается как "А звездочка"). В нем применяется оценка узлов, объединяющая в себе $d(n)$, стоимость достижения данного узла, и $h(n)$, стоимость прохождения от данного узла до цели: $f(n) = d(n) + h(n)$ Поскольку функция $d(n)$ позволяет определить стоимость пути от начального узла до узла n , а функция $h(n)$ определяет оценку стоимости наименее дорогостоящего пути от узла n до цели, то справедлива следующая формула: $f(n) =$ оценка стоимости наименее дорогостоящего пути решения, проходящего через узел n Таким образом, при осуществлении попытки найти наименее дорогостоящее решение, по-видимому, разумнее всего вначале попытаться проверить узел с наименьшим значением $d(n) + h(n)$. Как оказалось, данная стратегия является больше чем просто разумной: если эвристическая функция $h(n)$ удовлетворяет некоторым условиям, то поиск A^* становится и полным, и оптимальным.

Анализ оптимальности поиска A^* является несложным, если этот метод используется в сочетании с алгоритмом Tree-Search. В таком случае поиск A^* является оптимальным, при условии, что $h(n)$ представляет собой A^* допустимую эвристическую функцию, т.е. при условии, что $h(n)$ никогда не переоценивает стоимость достижения цели. Допустимые эвристические функции являются по своей сути оптимистическими функциями, поскольку возвращают значения стоимости решения за-

дачи, меньшие по сравнению с фактическими значениями стоимости. А поскольку $d(n)$ — точная стоимость достижения узла n , из этого непосредственно следует, что функция $f(n)$ никогда не переоценивает истинную стоимость достижения решения через узел n .

Очевидным примером допустимой эвристической функции является функция определения расстояния по прямой h_{SLD} , которая уже использовалась в данной главе для поиска пути в Бухарест. Расстояние по прямой является допустимым, поскольку кратчайший путь между любыми двумя точками лежит на прямой; это означает, что длина прямого пути по определению не может представлять собой

переоценку длины пути. На рис. 4.2 показан процесс поиска A^* пути в Бухарест с помощью дерева.

Значения d вычисляются на основании стоимостей этапов, показанных на рис. 3.1, а значения $hSLD$ приведены в табл. 4.1. Следует, в частности, отметить, что узел Bucharest впервые появляется в периферии на этапе, показанном на рис. 4.2, d , но не выбирается для развертывания, поскольку его f -стоимость $D50$) выше, чем стоимость узла Pitesti $D17$). Иными словами эту ситуацию можно описать так, что может существовать решение, при котором путь проходит через город Питешти со стоимостью, достигающей 417, поэтому алгоритм не останавливается на решении со стоимостью 450. Данный пример может служить общим свидетельством того, что СГЗ поиск A^* с использованием алгоритма Tree-Search является оптимальным, если функция l (l) допустима. Предположим, что на периферии поиска появился неоптимальный целевой узел $G2$, а стоимость оптимального решения равна C^* . В таком случае, поскольку узел $G2$ неоптимален, а $h(G2) = 0$ (это выражение справедливо для любого целевого узла), можно вывести следующую формулу: $f(G2) = g(G2) + h(G2) = g(G2) > C^*$ Теперь рассмотрим периферийный узел l , который находится в оптимальном пути решения, например узел $Pi\ testi$ в примере, приведенном в предыдущем абзаце.

(Если решение существует, то всегда должен быть такой узел.) Если функция l (l) не переоценивает стоимость завершения этого пути решения, то справедлива следующая формула: $f(n) = g(n) + h(n) < C^*$ Таким образом, доказано, что $f(l) < C^* < f(G2)$, поэтому узел $G2$ не развертывается и поиск A^* должен вернуть оптимальное решение.

Если бы вместо алгоритма Tree-Search использовался алгоритм GraphSearch, приведенный в листинге 3.6, то данное доказательство стало бы недействительным. Дело в том, что алгоритм Graph-Search способен отбросить оптимальный путь к повторяющемуся состоянию, если он не был сформирован в первую очередь, поэтому может возвращать неоптимальные решения (см. упр. 4.4). Существуют два способа устранения этого недостатка. Первое решение состоит в том, что алгоритм Graph-Search должен быть дополнен так, чтобы он отбрасывал наиболее дорогостоящий из любых двух найденных путей к одному и тому же узлу (см. обсуждение этой темы в разделе 3.5). Сопровождение необходимой для этого дополнительной информации связано с определенными трудностями, но гарантирует оптимальность. Второе решение состоит в обеспечении того, чтобы оптимальный путь к любому повторяющемуся состоянию всегда был первым из тех, по которым следует алгоритм, как в случае поиска по критерию стоимости.

а) Начальное состояние fr^{Arad}
 $366=0+366$ $393=140+253$ $447=118+329$
 $449=75+374$ $646=280+366$ $415=239+176$ $671=291+380$ $413=220+193$ $591=338+253$
 $450=450+0$ $526=366+160$ $417=317+100$ $553=300+253$ $418=418+0$ $615=455+160$

607=414+193 Рис. 4.2. Этапы поиска A* пути в Бухарест. Узлы отменены значениями $f = g + h$.

Значения h представляют собой расстояния по прямой до Бухареста, приведенные в табл. 4.1

Такое свойство соблюдается, если на функцию h налагается дополнительное требование, а именно требование обеспечения $\wedge$ преемственности эвристической функции (такое свойство называют также $\wedge$ монотонностью эвристической функции). Эвристическая функция h является преемственной, если для любого узла l и для любого преемника l' узла l , сформированного в результате любого действия a , оценка стоимости достижения цели из узла l не больше чем стоимость этапа достижения узла l' плюс оценка стоимости достижения цели из узла l' : $h(l) \leq c(l, a, l') + h(l')$ Это — форма общего $\wedge$ неравенства треугольника, которое указывает, что длина любой стороны треугольника не может превышать сумму длин двух других сторон.

В данном случае треугольник образован узлами l , l' и целью, ближайшей к l .

Можно довольно легко показать (упр. 4.7), что любая преемственная эвристическая функция является также допустимой. Наиболее важным следствием из определения преемственности является такой вывод: СГЗ поиск A* с использованием алгоритма Graph-Search является оптимальным, если функция h преемственна.

Несмотря на то что требование к преемственности является более строгим, чем требование к допустимости, весьма нелегко составить такие эвристические функции, которые были бы допустимыми, но не преемственными. Все допустимые эвристические функции, рассматриваемые в данной главе, являются также преемственными. Возьмем в качестве примера функцию h_{SLD} . Известно, что общее неравенство треугольника удовлетворяется, если длина каждой стороны измеряется с помощью расстояния по прямой, и что расстояние по прямой между l и l' не больше чем $c(l, a, l')$. Поэтому эвристическая функция h_{SLD} является преемственной.

Еще один важный вывод из определения преемственности является таковым: $\wedge$ = если функция h преемственна, то значения функции $f(n)$ вдоль любого пути являются неубывающими. Доказательство этого утверждения непосредственно вытекает из определения преемственности. Предположим, что узел l' — преемник узла l ; в таком случае для некоторого a справедливо выражение $g(l') = g(l) + c(l, a, l')$ и имеет место такая формула: $f(l') = g(l') + h(l') = g(l) + c(l, a, l') + h(l') > g(l) + h(l) = f(l)$ На основании этого можно сделать вывод, что последовательность узлов, развернутых в поиске A* с использованием алгоритма Graph-Search, находится в неубывающем порядке значений $f(n)$. Поэтому первый целевой узел, выбранный для

развертывания, должен представлять собой оптимальное решение, поскольку все дальнейшие узлы будут, по меньшей мере, столь же дорогостоящими.

Тот факт, что f -стоимости вдоль любого пути являются неубывающими, означает также, что могут быть очерчены контуры равных f -стоимостей в пространстве состояний, полностью аналогичные контурам равных высот на топографической карте. Пример подобной схемы приведен на рис. 4.3. Внутри контура, обозначенного как 400, все узлы имеют значения f (л), меньшие или равные 400, и т.д. В таком случае, поскольку в поиске A^* развертывается периферийный узел с наименьшей f -стоимостью, можно видеть, как поиск A^* распространяется из начального узла, добавляя узлы в виде концентрических полос с возрастающей f -стоимостью.

Рис. 4.3. Карта Румынии, на которой показаны контуры, соответствующие $f=380$, $f = 400$ и $f = 420$, притом что Arad является начальным состоянием.

Узлы в пределах данного конкретного контура имеют f -стоимости, меньшие или равные значению стоимости контура. При поиске по критерию стоимости (таковым является поиск A^* с применением $h(n) = 0$) эти полосы будут представлять собой "кольца" с центром в начальном состоянии. При использовании более точных эвристических функций полосы вытягиваются в направлении целевого состояния и становятся более узко сосредоточенными вокруг оптимального пути. Если C^* представляет собой стоимость оптимального пути решения, то можно утверждать следующее: • в поиске A^* развертываются все узлы со значениями $f(n) < C^*$; • поэтому в поиске A^* могут развертываться некоторые дополнительные узлы, находящиеся непосредственно на "целевом контуре" (где $f(n) = C^*$), прежде чем будет выбран целевой узел.

На интуитивном уровне представляется очевидным, что первое найденное решение должно быть оптимальным, поскольку целевые узлы во всех последующих контурах будут иметь более высокое значение f -стоимости и поэтому более высокое значение g -стоимости (поскольку все целевые узлы имеют значения $h(n) = 0$). Кроме того, на интуитивном уровне также очевидно, что поиск A^* является полным.

По мере добавления полос с возрастающими значениями f мы должны в конечном итоге достичь полосы, в которой значение f будет равно стоимости пути к целевому состоянию⁴.

Следует отметить, что в поиске A^* узлы со значением $f(л) > C^*$ не развертываются; например, как показано на рис. 4.2, не развертывается узел Timisoara, даже несмотря на то, что является дочерним узлом корневого узла. Эту ситуацию принято обозначать так, что происходит ^ отсечение поддерева, находящегося ниже узла 4. Для соблюдения требования полноты необходимо, чтобы количество узлов со стоимостью, меньшей или равной C^* , было конечным; это условие соблюдается, если

стоимости всех этапов превышают некоторое конечное значение b , а коэффициент ветвления b является конечным.

Timisoara; поскольку функция h_{SLD} является допустимой, рассматриваемый алгоритм может безопасно игнорировать это поддерево, гарантируя вместе с тем оптимальность. Понятие отсечения (под которым подразумевается исключение из рассмотрения некоторых вариантов в связи с отсутствием необходимости их исследовать) является важным для многих областей искусственного интеллекта.

Одно заключительное наблюдение состоит в том, что среди оптимальных алгоритмов такого типа (алгоритмов, которые развертывают пути поиска от корня) поиск A^* является $\wedge$ оптимально эффективным для любой конкретной эвристической функции. Это означает, что не гарантируется развертывание меньшего количества узлов, чем в поиске A^* , с помощью какого-либо иного оптимального алгоритма (не считая той возможности, когда осуществляется выбор на равных среди узлов с $f(n) = C^*$). Это связано с тем, что любой алгоритм, который не развертывает все узлы со значениями $f(n) < C^*$, подвержен риску потери оптимального решения.

Те соображения, что поиск A^* , как один из всех подобных алгоритмов, является действительно полным, оптимальным и оптимально эффективным, оставляют довольно приятное впечатление. Но, к сожалению, это отнюдь не означает, что поиск A^* может служить ответом на все наши потребности в поиске. Сложность заключается в том, что при решении большинства задач количество узлов в пределах целевого контура пространства состояний все еще зависит экспоненциально от длины решения. Хотя доказательство этого утверждения выходит за рамки настоящей книги, было показано, что экспоненциальный рост происходит, если ошибка эвристической функции растет не быстрее по сравнению с логарифмом фактической стоимости пути. В математических обозначениях условие субэкспоненциального роста состоит в следующем: $|h(n) - h^*(n)| < O(\log h^*(n))$ где $h^*(n)$ — истинная стоимость достижения цели из узла n . Почти для всех практически применяемых эвристических функций эта ошибка, по меньшей мере, пропорциональна стоимости пути, и происходящий в связи с этим экспоненциальный рост в конечном итоге превосходит возможности любого компьютера. По этой причине на практике стремление находить оптимальное решение часто не оправдано.

Иногда вместо этого целесообразно использовать варианты поиска A^* , позволяющие быстро находить неоптимальные решения, а в других случаях — разрабатывать эвристические функции, которые являются более точными, но не строго допустимыми. В любом случае применение хорошей эвристической функции все равно обеспечивает поразительную экономию усилий по сравнению с

использованием неинформированного поиска. Вопрос разработки хороших эвристических функций рассматривается в разделе 4.2.

Но большая продолжительность вычислений не является основным недостатком поиска A^* . Поскольку при поиске A^* все сформированные узлы хранятся в памяти (как и во всех алгоритмах Graph-Search), фактически ресурсы пространства исчерпываются задолго до того, как исчерпываются ресурсы времени. По этой причине поиск A^* не является практически применимым при решении многих крупномасштабных задач. Разработанные недавно алгоритмы позволяют преодолеть эту проблему пространства, не жертвуя оптимальностью или полнотой, за счет небольшого увеличения времени выполнения. Эти алгоритмы рассматриваются ниже.

Эвристический поиск с ограничением объема памяти. Простейший способ сокращения потребностей в памяти для поиска A^* состоит в применении идеи итеративного углубления в контексте эвристического поиска.

Реализация этой идеи привела к созданию алгоритма A^* с итеративным углублением (Iterative Deepening A^* — IDA*). Основное различие между алгоритмом IDA* и стандартным алгоритмом итеративного углубления состоит в том, что применяемым условием останова развертывания служит f -стоимость ($g+h$), а не глубина; на каждой итерации этим остановочным значением является минимальная f -стоимость любого узла, превышающая остановочное значение, достигнутое в предыдущей итерации. Алгоритм IDA* является практически применимым для решения многих задач с единичными стоимостями этапов и позволяет избежать существенных издержек, связанных с поддержкой отсортированной очереди узлов. К сожалению, этот алгоритм характеризуется такими же сложностями, связанными с использованием стоимостей с действительными значениями, как и итеративная версия поиска по критерию стоимости, которая описана в упр. 3.11. В данном разделе кратко рассматриваются два более современных алгоритма с ограничением памяти, получивших названия RBFS и MA*. ^ Рекурсивный поиск по первому наилучшему совпадению (Recursive Best-First Search — RBFS) — это простой рекурсивный алгоритм, в котором предпринимаются попытки имитировать работу стандартного поиска по первому наилучшему совпадению, но с использованием только линейного пространства. Этот алгоритм приведен в листинге 4.1. Он имеет структуру, аналогичную структуре рекурсивного поиска в глубину, но вместо бесконечного следования вниз по текущему пути данный алгоритм контролирует f -значение наилучшего альтернативного пути, доступного из любого предка текущего узла. Если текущий узел превышает данный предел, то текущий этап рекурсии отменяется и рекурсия продолжается с альтернативного пути. После отмены данного этапа рекурсии в алгоритме RBFS происходит замена f -значения каждого узла вдоль данного пути наилучшим f -значением его дочернего узла. Благодаря этому в

алгоритме RBFS запоминается f -значение наилучшего листового узла из забытого поддерева и поэтому в некоторый последующий момент времени может быть принято решение о том, стоит ли снова разворачивать это поддерево. На рис. 4.4 показано, как с помощью алгоритма RBFS происходит поиск пути в Бухарест.

Листинг 4.1. Алгоритм рекурсивного поиска по первому наилучшему совпадению

```
function Recursive-Best-First-Search{problem} returns решение result или индикатор
неудачи failure
  RBFS(problem, Make-Node(Initial-State[problem] ) , 0)
function RBFS(problem, node, f_limit) returns решение result или индикатор
неудачи failure и
новый предел  $f$ -стоимости f_limit
  if Goal-Test[problem](State[node]) then return узел node
  successors  $\leftarrow$  Expand{node, problem}
  if множество узлов-преемников successors пусто then return failure, 0
  for each s in successors do
     $f[s] \leftarrow \max(g(s) + h(s) , f[node])$ 
  repeat best  $\leftarrow$  узел с наименьшим  $f$ -значением в множестве successors
  if  $f[best] > f\_limit$  then return failure,  $f[best]$ 
```

alternative $\leftarrow$ второе после наименьшего f -значения в множестве successors
 $f[best] \leftarrow \min\{f_limit, alternative\}$
 if result $\neq$ failure then return result

Рис. 4.4. Этапы поиска кратчайшего маршрута в Бухарест с помощью алгоритма RBFS. Значение f -предела для каждого рекурсивного вызова показано над каждым текущим узлом: путь через узел RimnicuVilcea, который следует до текущего наилучшего листового узла fPitesti, имеет значение, худшее по сравнению с наилучшим альтернативным путем fFagaras, (a); рекурсия продолжается и значение наилучшего листового узла забытого поддерева D17) резервируется в узле RimnicuVilcea; затем разворачивается узел Fagaras, в результате чего обнаруживается наилучшее значение листового узла, равное 450 (б); рекурсия продолжается и значение наилучшего листового узла забытого поддерева D50) резервируется в узле Fagaras; затем разворачивается узел RimnicuVilcea; на этот раз разворачивание продолжается в сторону Бухареста, поскольку наилучший альтернативный путь (через узел Timisoara,) стоит, по меньшей мере, 447(е)

Алгоритм RBFS является немного более эффективным по сравнению с IDA*, но все еще страдает от недостатка, связанного со слишком частым повторным формированием узлов. В примере, приведенном на рис. 4.4, алгоритм RBFS вначале следует по пути через узел RimnicuVilcea, затем "меняет решение" и пытается пройти через узел Fagaras, после этого снова возвращается к отвергнутому ранее решению.

Такие смены решения происходят в связи с тем, что при каждом разворачивании текущего наилучшего пути велика вероятность того, что его f -значение увеличится, поскольку функция h обычно становится менее оптимистической по мере того, как происходит разворачивание узлов, более близких к цели. После того как возникает такая ситуация, особенно в больших пространствах поиска, путь, который был вторым после наилучшего, может сам стать наилучшим путем, поэтому в алгоритме поиска приходится выполнять возврат, чтобы проследовать по нему. Каждое

изменение решения соответствует одной итерации алгоритма IDA* и может потребовать многих повторных развертываний забытых узлов для воссоздания наилучшего пути и развертывания пути еще на один узел.

Как и алгоритм поиска A*, RBFS является оптимальным алгоритмом, если эвристическая функция $h(n)$ допустима. Его пространственная сложность равна $O(bd)$, но охарактеризовать временную сложность довольно трудно: она зависит и от точности эвристической функции, и от того, насколько часто происходила смена наилучшего пути по мере развертывания узлов. И алгоритм IDA*, и алгоритм RBFS подвержены потенциальному экспоненциальному увеличению сложности, связанной с поиском в графах (см. раздел 3.5), поскольку эти алгоритмы не позволяют определять наличие повторяющихся состояний, отличных от тех, которые находятся в текущем пути. Поэтому данные алгоритмы способны много раз исследовать одно и то же состояние.

Алгоритмы IDA* и RBFS страдают от того недостатка, что в них используется слишком мало памяти. Между итерациями алгоритм IDA* сохраняет только единственное число — текущий предел f -стоимости. Алгоритм RBFS сохраняет в памяти больше информации, но количество используемой в нем памяти измеряется лишь значением $O(bd)$: даже если бы было доступно больше памяти, алгоритм RBFS не способен ею воспользоваться.

Поэтому представляется более разумным использование всей доступной памяти.

Двумя алгоритмами, которые осуществляют это требование, являются поиск MA^* (Memory-bounded A* — поиск A* с ограничением памяти) и SMA^* (Simplified MA* — упрощенный поиск MA*). В данном разделе будет описан алгоритм SMA*, который действительно является более простым, чем другие алгоритмы этого типа. Алгоритм SMA* действует полностью аналогично поиску A*, развертывая наилучшие листовые узлы до тех пор, пока не будет исчерпана доступная память. С этого момента он не может добавить новый узел к дереву поиска, не уничтожив старый. В алгоритме SMA* всегда уничтожается наихудший листовой узел (тот, который имеет наибольшее f -значение). Как и в алгоритме RBFS, после этого в алгоритме SMA* значение забытого (уничтоженного) узла резервируется в его родительском узле. Благодаря этому предок забытого поддерева позволяет определить качество наилучшего пути в этом поддереве. Поскольку имеется данная информация, в алгоритме SMA* поддерево восстанавливается, только если обнаруживается, что все другие пути выглядят менее многообещающими по сравнению с забытым путем. Иными словами, если все потомки узла p забыты, то неизвестно, каким

путем можно следовать от l , но все еще можно получить представление о том, есть ли смысл куда-либо следовать от p .

Полный алгоритм слишком сложен для того, чтобы его можно было воспроизвести в данной книге⁵, но заслуживает упоминания один его нюанс. Как уже было сказано выше, в алгоритме SMA* разворачивается наилучший листовой узел и удаляется наихудший листовой узел. А что происходит, если все листовые узлы имеют одинаковое f -значение? В таком случае может оказаться, что алгоритм выбирает для удаления и разворачивания один и тот же узел. В алгоритме SMA* эта проблема решается путем разворачивания самого нового наилучшего листового узла и удаления самого старого наихудшего листового узла. Эти два узла могут оказаться одним и тем же узлом, только если существует лишь один листовой узел; в таком случае текущее дерево поиска должно представлять собой единственный путь от корня до листового узла, заполняющий всю память. Это означает, что если данный листовой узел не является целевым узлом, то решение не достижимо при доступном объеме памяти, даже если этот узел находится в оптимальном пути решения. Поэтому такой узел может быть отброшен точно так же, как и в том случае, если он не имеет преемников.

Алгоритм SMA* является полным, если существует какое-либо достижимое решение, иными словами, если d , глубина самого поверхностного целевого узла, меньше чем объем памяти (выраженный в хранимых узлах). Этот алгоритм оптимален, если достижимо какое-либо оптимальное решение; в противном случае он возвращает наилучшее достижимое решение. С точки зрения практики алгоритм SMA* вполне может оказаться наилучшим алгоритмом общего назначения для поиска оптимальных решений, особенно если пространство состояний представляет собой граф, стоимости этапов не одинаковы, а операция формирования узлов является более дорогостоящей в сравнении с дополнительными издержками сопровождения открытых и закрытых списков.

Однако при решении очень сложных задач часто возникают ситуации, в которых алгоритм SMA* вынужден постоянно переключаться с одного пути решения на другой в пределах множества возможных путей решения, притом что в памяти может поместиться только небольшое подмножество этого множества. (Такие ситуации напоминают проблему $\wedge$ пробуксовки в системах подкачки страниц с жесткого диска.) В таком случае на повторное формирование одних и тех узлов затрачивается дополнительное время, а это означает, что задачи, которые были бы фактически разрешимыми с помощью поиска A^* при наличии неограниченной памяти, становятся трудноразрешимыми для алгоритма SMA*. Иными словами, из-за $s^{\wedge}$ ограничений в объеме памяти некоторые задачи могут становиться трудноразрешимыми с точки зрения времени вычисления. Хотя отсутствует теория, позволяющая найти компромисс между затратами времени и памяти, создается впечатление, что зачастую избежать возникновения этой проблемы невозможно.

Единственным способом преодоления такой ситуации становится частичный отказ от требований к оптимальности решения.

Обучение лучшим способам поиска. Выше было представлено несколько стратегий поиска (поиск в ширину, жадный поиск по первому наилучшему совпадению и т.д.), которые были разработаны уже в 50-е годы. Грубый набросок этого алгоритма был приведен в первом издании настоящей книги.

Иными и специалистами по компьютерным наукам. Но может ли сам агент обучаться лучшим способам поиска? Ответ на этот вопрос является положительным, а применяемый при этом метод обучения опирается на важную концепцию, называемую пространством состояний, рассматриваемым на метауровне, или Λ метауровневым пространством состояний. Каждое состояние в метауровневом пространстве состояний отражает внутреннее (вычислительное) состояние программы, выполняющей поиск в пространстве состояний, рассматриваемом на уровне объектов, или в Λ объектно-уровневом пространстве состояний, таком как карта Румынии.

Например, внутреннее состояние алгоритма A^* включает в себя текущее дерево поиска.

Каждое действие в метауровневом пространстве состояний представляет собой этап вычисления, который изменяет внутреннее состояние, например, на каждом этапе вычисления в процессе поиска A^* разворачивается один из листовых узлов, а его преемники добавляются к дереву. Таким образом, рис. 4.2, на котором показана последовательность все больших и больших деревьев поиска, может рассматриваться как изображающий путь в метауровневом пространстве состояний, где каждое состояние в пути является объектно-уровневым деревом поиска.

В настоящее время путь, показанный на рис. 4.2, имеет пять этапов, включая один этап (разворачивание узла *Fagagas*), который нельзя назвать слишком полезным.

Может оказаться, что при решении более сложных задач количество подобных ненужных этапов будет намного больше, а алгоритм Λ метауровневого обучения может изучать этот опыт, чтобы в дальнейшем избегать исследования бесперспективных поддеревьев.

Методы, используемые при обучении такого рода, описаны в главе 21. Целью обучения является минимизация суммарной стоимости решения задач, а также поиск компромисса между вычислительными издержками и стоимостью пути.

4.2. Эвристические функции В этом разделе будут рассматриваться эвристические функции для задачи игры в восемь, что позволяет лучше продемонстрировать характерные особенности всех эвристических функций в целом.

Головоломка "игра в восемь" была одной из первых задач эвристического поиска.

Как было указано в разделе 3.2, в ходе решения этой головоломки требуется передвигать фишки по горизонтали или по вертикали на пустой участок до тех пор, пока полученная конфигурация не будет соответствовать целевой конфигурации (рис. 4.5).

13 1 \v\ 1 1 1 пг Гт] Гм |5| И LZJ ГУ] ||8 | Гм [з| |4] \6\\ Начальное состояние Целевое состояние Рис. 4.5. Типичный экземпляр головоломки "игра в восемь". Решение имеет длину 26 этапов

Средняя стоимость решения для сформированного случайным образом экземпляра головоломки "игра в восемь" составляет около 22 этапов. Коэффициент ветвления примерно равен 3. (Если пустой квадрат находится в середине коробки, то количество возможных ходов равно четырем, если находится в углу — двум, а если в середине одной из сторон — трем.) Это означает, что при исчерпывающем поиске на глубину 22 приходится рассматривать примерно $3^{22} \approx 3.1 \times 10^{10}$ состояний.

Отслеживая повторяющиеся состояния, это количество состояний можно сократить приблизительно в 170 000 раз, поскольку существует только $9! / 2 = 181\,440$ различных состояний, которые являются достижимыми (см. упр. 3.4.) Это количество состояний уже лучше поддается контролю, но соответствующее количество для игры в пятнадцать примерно равно 10^{13} , поэтому для такой головоломки с более высокой сложностью требуется найти хорошую эвристическую функцию. Если нужно находить кратчайшие решения с использованием поиска A^* , то требуется эвристическая функция, которая никогда не переоценивает количество этапов достижения цели.

История исследований в области поиска таких эвристических функций для игры в пятнадцать является довольно долгой, а в данном разделе рассматриваются два широко используемых кандидата на эту роль, которые описаны ниже.

- h_1 = количество фишек, стоящих не на своем месте. На рис. 4.5 все восемь фишек стоят не на своем месте, поэтому показанное слева начальное состояние имеет эвристическую оценку $h_1 = 8$. Эвристическая функция h_1 является допустимой, поскольку очевидно, что каждую фишку, находящуюся не на своем месте, необходимо переместить по меньшей мере один раз.
- h_2 = сумма расстояний всех фишек от их целевых позиций. Поскольку фишки не могут передвигаться по диагонали, рассчитываемое расстояние представляет собой сумму горизонтальных и вертикальных расстояний. Такое расстояние иногда называют расстоянием, измеряемым в городских кварталах, или h_2 манхэттенским расстоянием. Эвристическая функция h_2 также является допустимой, поскольку все, что может быть сделано в одном ходе, состоит лишь в перемещении одной фишки на один этап ближе к цели. Фишки от 1 до 8 в рассматриваемом начальном состоянии соответствуют такому значению манхэттенского расстояния: $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.

Как и можно было предположить, ни одна из этих функций не переоценивает истинную стоимость решения, которая равна 26.

Зависимость производительности поиска от точности эвристической функции. Одним из критериев, позволяющих охарактеризовать качество эвристической функции, является b^* — эффективный коэффициент ветвления. Если общее количество узлов, вырабатываемых в процессе поиска A^* решения конкретной задачи, равно N , а глубина решения равна d , то b^* представляет собой коэффициент ветвления, который должно иметь однородное дерево с глубиной d для того, чтобы в нем содержалось $N-1$ узлов. Поэтому справедлива следующая формула:

$$N+1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$
 Например, если алгоритм A^* находит решение на глубине 5 с использованием 52 узлов, то эффективный коэффициент ветвления равен 1,92. Эффективный коэффи-

циент ветвления может изменяться от одного экземпляра одной и той же задачи к другому, но обычно в случае достаточно трудных задач остается относительно постоянным. Поэтому экспериментальные измерения коэффициента b^* на небольшом множестве задач могут служить хорошим критерием общей полезности рассматриваемой эвристической функции. Хорошо спроектированная эвристическая функция должна иметь значение b^* , близкое к 1, что позволяет быстро решать довольно большие задачи.

Для проверки эвристических функций h_1 и h_2 авторы сформировали случайным образом 1200 экземпляров задачи с длиной решения от 2 до 24 (по 100 экземпляров для каждого четного значения длины) и нашли их решения с помощью поиска с итеративным углублением и поиска в дереве по алгоритму A^* с применением эвристических функций h_1 и h_2 . Данные о среднем количестве узлов, развернутых при использовании каждой стратегии и эффективном коэффициенте ветвления, приведены в табл. 4.2. Эти результаты показывают, что эвристическая функция h_2 лучше чем h_1 и намного лучше по сравнению с использованием поиска с итеративным углублением. Применительно к найденным авторами решениям с длиной 14 применение поиска A^* с эвристической функцией h_2 становится в 30 000 раз более эффективным по сравнению с неинформированным поиском с итеративным углублением.

Таблица 4.2. Сравнение значений стоимости поиска и эффективного коэффициента ветвления для алгоритмов Iterative-Deepening-Search и A^* с h_1 , h_2 . Данные усреднялись по 100 экземплярам задачи игры в восемь применительно к различным значениям длины решения

Стоимость поиска	Эффективный коэффициент ветвления
IDS 47127 3644035	$A^*(h_1)$ 18094 39135
$A^*(h_2)$ IDS 2,45 2,87 2,73 2,80 2,79 2,78	$A^*(h_1)$ 1,79 1,48 1,34 1,33 1,38 1,42 1,44 1,45 1,46 1,47 1,48 1,48
$A^*(h_2)$ 1,79 1,45 1,30 1,24 1,22 1,24 1,23 1,25 1,26 1,27 1,28 1,26	Интерес представляет вопрос о том, всегда ли

эвристическая функция h_2 лучше чем $h_{\pm}$. Ответ на этот вопрос является положительным. На основании определений этих двух эвристических функций можно легко прийти к выводу, что для любого узла n справедливо выражение $h_2(n) > h_{\pm}(n)$. Таким образом, можно утверждать, что эвристика h_2 б. доминирует над $h_{\pm}$. Доминирование напрямую связано с эффективностью: при поиске A^* с использованием функции h_2 никогда не происходит развертывание большего количества узлов, чем при поиске A^* с использованием $h_{\pm}$ (возможно, за исключением нескольких узлов с $f(n) = C^*$). Доказательство этого утверждения является несложным. Напомним приведенное на с. 161 замечание, что каждый узел со значением $f(n) < C^*$ наверняка должен быть развернут. Это аналогично утверждению, что должен быть развернут каждый узел со значением $h(n) < C^* - d(n)$. Но поскольку для всех узлов значение h_2 , по крайней мере, не меньше значения $h_{\pm}$ то каждый узел, который должен быть наверняка развернут в поиске A^* с $h_{\pm}$, будет также наверняка развернут при поиске с h_2 а применение эвристической функции $h_{\pm}$ может к тому же вызвать и развертывание других узлов.

Поэтому всегда лучше использовать эвристическую функцию с более высокими значениями, при тех условиях, что эта функция не переоценивает длину пути решения и что время вычисления этой эвристической функции не слишком велико.

Составление допустимых эвристических функций Выше было показано, что эвристические функции $h_{\pm}$ (в которой используется количество стоящих не на своем месте фишек) и h_2 (в которой используется манхэттенское расстояние) являются довольно неплохими эвристическими функциями для задачи игры в восемь и что функция h_2 — из них наилучшая. Но на основании чего именно была предложена функция h_2 ? Возможно ли, чтобы компьютер мог составить некоторую эвристическую функцию механически?

Составление допустимых эвристических функций Выше было показано, что эвристические функции $h_{\pm}$ (в которой используется количество стоящих не на своем месте фишек) и h_2 (в которой используется манхэттенское расстояние) являются довольно неплохими эвристическими функциями для задачи игры в восемь и что функция h_2 — из них наилучшая. Но на основании чего именно была предложена функция h_2 ? Возможно ли, чтобы компьютер мог составить некоторую эвристическую функцию механически?

Эвристические функции $h_{\pm}$ и h_2 представляют собой оценки оставшейся длины пути для задачи игры в восемь, но они, кроме того, возвращают идеально точные значения длины пути для упрощенных версий этой игры. Если бы правила игры в восемь изменились таким образом, чтобы любую фишку можно было передвигать куда угодно, а не только на соседний пустой квадрат, то эвристическая функция $h_{\pm}$ возвращала бы точное количество этапов в кратчайшем решении. Аналогичным образом, если бы любую фишку можно было перемещать на один квадрат в любом направлении, даже на занятый квадрат, то точное количество этапов в кратчайшем решении возвращала бы эвристическая функция h_2 . Задача с меньшим количеством ограничений на возможные действия называется $\wedge$ ослабленной задачей. $C^{\wedge}$ = Стоимость оптимального решения ослабленной задачи представляет собой допустимую эвристику для первоначальной задачи. Такая эвристическая функция является допустимой, поскольку оптимальное решение первоначальной задачи, по определению, служит также решением ослабленной задачи и поэтому должно быть,

по меньшей мере, таким же дорогостоящим, как и оптимальное решение ослабленной задачи. Поскольку значение производной эвристической функции представляет собой точную стоимость решения ослабленной задачи, эта функция должно подчиняться неравенству треугольника и поэтому должна быть преемственной (см. с. 160).

Если определение задачи записано на каком-то формальном языке, то существует возможность формировать ослабленные задачи автоматически^б. Например, если действия в игре в восемь описаны следующим образом:

6 В главах 8 и 11 будут описаны формальные языки, подходящие для этого назначения; возможность автоматизировать формирование ослабленных задач появляется при наличии формальных описаний, с которыми могут проводиться манипуляции, а пока для этого назначения будет использоваться естественный язык.

Фишка может быть передвинута из квадрата А в квадрат В, если квадрат А является смежным по горизонтали или по вертикали с квадратом В и квадрат В пуст то могут быть сформированы три ослабленные задачи путем удаления одного или обоих из приведенных выше условий. а) Фишка может быть передвинута из квадрата А в квадрат В, если квадрат А является смежным с квадратом в. б) Фишка может быть передвинута из квадрата А в квадрат в, если квадрат в пуст. в) Фишка может быть передвинута из квадрата А в квадрат в.

На основании ослабленной задачи а) можно вывести функцию h_2 (манхэттенское расстояние). В основе этих рассуждений лежит то, что h_2 должна представлять собой правильную оценку, если каждая фишка передвигается к месту ее назначения по очереди. Эвристическая функция, полученная на основании ослабленной задачи б), обсуждается в упр. 4.9. На основании ослабленной задачи в) можно получить эвристическую функцию h_i (фишки, стоящие не на своих местах), поскольку эта оценка была бы правильной, если бы фишки можно было передвигать в предназначенное для них место за один этап. Обратите внимание на то, что здесь существенной является возможность решать ослабленные задачи, создаваемые с помощью указанного метода, фактически без какого-либо поиска, поскольку ослабленные правила обеспечивают декомпозицию этой задачи на восемь независимых подзадач. Если бы было трудно решать и ослабленную задачу, то был бы дорогостоящим сам процесс получения значений соответствующей эвристической функции⁷.

Для автоматического формирования эвристических функций на основе определений задач с использованием метода "ослабленной задачи" и некоторых других методов может применяться программа под названием Absolver [1237]. Программа Absolver составила для задачи игры в восемь новую эвристическую функцию, лучшую по сравнению со всеми существовавшими ранее эвристическими

функциями, а также нашла первую полезную эвристическую функцию для знаменитой головоломки "кубик Рубика".

Одна из проблем, возникающих при составлении новых эвристических функций, состоит в том, что часто не удается получить эвристическую функцию, которая была бы "лучшей во всех отношениях" по сравнению с другими. Если для решения какой-либо задачи может применяться целая коллекция допустимых эвристических функций $h_1, \dots, h_m$ и ни одна из них не доминирует над какой-либо из других функций, то какая из этих функций должна быть выбрана? Оказалось, что такой выбор делать не требуется, поскольку можно взять от них всех самое лучшее, определив такой критерий выбора: $h(n) = \max\{h_1(n), \dots, h_m(n)\}$. В этой составной эвристике используется та функция, которая является наиболее точной для рассматриваемого узла. Поскольку эвристические функции, входящие в состав эвристики h , являются допустимыми, сама эта функция также является допустимой. Следует отметить, что идеальную эвристическую функцию можно также получить, просто разрешив функции h "тайком" выполнять полный поиск в ширину. Таким образом, при создании эвристических функций всегда приходится искать компромисс между точностью и временем вычисления.

пустимой; кроме того, можно легко доказать, что функция h преобладающа. К тому же h доминирует над всеми эвристическими функциями, которые входят в ее состав.

Допустимые эвристические функции могут быть также выведены на основе стоимости решения подзадачи данной конкретной задачи. Например, на рис. 4.6 показана подзадача для экземпляра игры в восемь, приведенного на рис. 4.5. Эта подзадача касается перемещения фишек 1, 2, 3, 4 в их правильные позиции.

Очевидно, что стоимость оптимального решения этой подзадачи представляет собой нижнюю границу стоимости решения полной задачи. Как оказалось, такая оценка в некоторых случаях является намного более точной по сравнению с манхэттенским расстоянием. и $[3] \mid 4 \mid \mid 1 \mid 1 \mid 3 \mid n_j \mid \sim 4 \sim \mid \mid 2 \mid$ Начальное состояние Целевое состояние Рис. 4.6. Подзадача для экземпляра игры в восемь, показанного на рис. 4.5. Задание заключается в том, чтобы передвинуть фишки 1, 2, 3 и 4 в их правильные позиции, не беспокоясь о том, что произойдет с другими фишками. В основе баз данных с шаблонами лежит такая идея, что указанные точные стоимости решений нужно хранить для каждого возможного экземпляра подзадачи — в нашем примере для каждой возможной конфигурации из четырех фишек и пустого квадрата. (Следует отметить, что при выполнении задания по решению этой подзадачи местонахождения остальных четырех фишек не нужно принимать во внимание, но ходы с этими фишками следует учитывать в стоимости решения.) После этого вычисляется допустимая эвристическая функция h_{DB} (здесь DB — Data Base) для каждого полного состояния, встретившегося в процессе поиска, путем выборки данных для соответствующей конфигурации подзадачи из базы данных. Сама база

данных заполняется путем обратного поиска из целевого состояния и регистрации стоимости каждого нового встретившегося шаблона; затраты на этот поиск окупаются за счет успешного решения в дальнейшем многих новых экземпляров задачи.

Выбор фишек 1-2-3-4 является довольно произвольным; можно было бы также создать базы данных для фишек 5-6-7-8, 2-4-6-8 и т.д. По данным каждой базы данных формируется допустимая эвристическая функция, а эти эвристические функции можно составлять в общую эвристическую функцию, как было описано выше, применяя их максимальное значение. Составная эвристическая функция такого вида является намного более точной по сравнению с манхэттенским расстоянием; количество узлов, вырабатываемых при решении сформированных случайным образом экземпляров задачи игры в пятнадцать, может быть уменьшено примерно в 1000 раз.

Представляет интерес вопрос о том, можно ли складывать значения эвристических функций, полученных из баз данных 1-2-3-4 и 5-6-7-8, поскольку очевид-

но, что соответствующие две подзадачи не перекрываются. Будет ли при этом все еще получена допустимая эвристическая функция? Ответ на этот вопрос является отрицательным, поскольку в решениях подзадачи 1-2-3-4 и подзадачи 5-6-7-8 для данного конкретного состояния должны наверняка присутствовать некоторые общие ходы. Дело в том, что маловероятна ситуация, при которой фишки 1-2-3-4 можно было бы передвинуть на свои места, не трогая фишек 5-6-7-8, и наоборот.

Но что будет, если не учитывать эти ходы? Иными словами, допустим, что регистрируется не общая стоимость решения подзадачи 1-2-3-4, а только количество ходов, в которых затрагиваются фишки 1-2-3-4. В таком случае можно легко определить, что сумма этих двух стоимостей все еще представляет собой нижнюю границу стоимости решения всей задачи. Именно эта идея лежит в основе ^ баз данных с непересекающимися шаблонами. Применение таких баз данных позволяет решать случайно выбранные экземпляры задачи игры в пятнадцать за несколько миллисекунд — количество формируемых узлов сокращается примерно в 10 000 раз по сравнению с использованием манхэттенского расстояния. Для задачи игры в 24 может быть достигнуто ускорение приблизительно в миллион раз.

Базы данных с непересекающимися шаблонами успешно применяются для решения головоломок со скользящими фишками, поскольку сама задача может быть разделена таким образом, что каждый ход влияет лишь на одну подзадачу, так как одновременно происходит перемещение только одной фишки. При решении таких задач, как кубик Рубика, подобное разделение не может быть выполнено, поскольку каждый ход влияет на положение 8 или 9 из 26 элементов кубика. В настоящее время

нет полного понимания того, как можно определить базы данных с непересекающимися шаблонами для подобных задач.

Изучение эвристических функций на основе опыта Предполагается, что эвристическая функция $h(n)$ оценивает стоимость решения, начиная от состояния, связанного с узлом n . Как может некоторый агент составить подобную функцию? Одно из решений было приведено в предыдущем разделе, а именно: агент должен сформулировать ослабленные задачи, для которых может быть легко найдено оптимальное решение. Еще один подход состоит в том, что агент должен обучаться на основании полученного опыта. Здесь под "получением опыта" подразумевается, например, решение большого количества экземпляров игры в восемь. Каждое оптимальное решение задачи игры в восемь предоставляет примеры, на основе которых можно изучать функцию $h(n)$. Каждый пример складывается из состояния, взятого из пути решения, и фактической стоимости пути от этой точки до решения. На основе данных примеров с помощью алгоритма индуктивного обучения может быть сформирована некоторая функция $h(n)$, способная (при счастливом стечении обстоятельств) предсказывать стоимости решений для других состояний, которые возникают во время поиска. Методы осуществления именно такого подхода с использованием нейронных сетей, деревьев решения и других средств описаны в главе 18. (Применимы также методы обучения с подкреплением, которые рассматриваются в главе 21.) Методы индуктивного обучения действует лучше всего, когда в них учитываются ^ характеристики состояния, релевантные для оценки этого состояния, а не просто общее описание состояния. Например, характеристика "количество стоящих не на

своих местах фишек" может быть полезной при предсказании фактического удаления некоторого состояния от цели. Назовем эту характеристику $x_1(n)$. Например, можно взять 100 сформированных случайным образом конфигураций головоломки игры в восемь и собрать статистические данные об их фактических стоимостях решений. Допустим, это позволяет обнаружить, что при $x_1(n)$, равном 5, средняя стоимость решения составляет около 14, и т.д. Наличие таких данных позволяет использовать значение x_1 для предсказания значений функции $h(n)$. Безусловно, можно также применять сразу несколько характеристик. Второй характеристикой, $x_2(n)$, может быть "количество пар смежных фишек, которые являются также смежными в целевом состоянии". Каким образом можно скомбинировать значения $x_1(n)$ и $x_2(n)$ для предсказания значения $h(n)$? Общепринятый подход состоит в использовании линейной комбинации, как показано ниже. $h(n) = c_1 x_1(n) + c_2 x_2(n)$ Константы c_1 и c_2 корректируются для достижения наилучшего соответствия фактическим данным о стоимостях решений. Предполагается, что константа c_1 должна быть положительной, а c_2 — отрицательной.

4.3. Алгоритмы локального поиска и задачи оптимизации Рассматривавшиеся до сих пор алгоритмы поиска предназначались для систематического исследования пространств поиска. Такая систематичность достигается благодаря тому, что один или несколько путей хранится в памяти и проводится регистрация того, какие альтернативы были исследованы в каждой точке вдоль этого пути, а какие нет. После того как цель найдена, путь к этой цели составляет также искомое решение данной задачи.

Но при решении многих задач путь к цели не представляет интереса. Например, в задаче с восемью ферзями (см. с. 1) важна лишь окончательная конфигурация ферзей, а не порядок, в котором они были поставлены на доску. К этому классу задач относятся многие важные приложения, такие как проектирование интегральных схем, разработка плана цеха, составление производственного расписания, автоматическое программирование, оптимизация сети связи, составление маршрута транспортного средства и управление портфелем акций.

Если путь к цели не представляет интереса, то могут рассматриваться алгоритмы другого класса, в которых вообще не требуются какие-либо данные о путях.

Алгоритмы $\wedge$ локального поиска действуют с учетом единственного $\wedge$ текущего состояния (а не многочисленных путей) и обычно предусматривают только переход в состояние, соседнее по отношению к текущему состоянию. Как правило, информация о путях, пройденных в процессе такого поиска, не сохраняется. Хотя алгоритмы локального поиска не предусматривают систематическое исследование пространства состояний (не являются систематическими), они обладают двумя важными преимуществами: во-первых, в них используется очень небольшой объем памяти, причем обычно постоянный, и, во-вторых, они часто позволяют находить приемлемые решения в больших или бесконечных (непрерывных) пространствах состояний, для которых систематические алгоритмы не применимы.

Кроме поиска целей, алгоритмы локального поиска являются полезным средством решения чистых $\wedge$ задач оптимизации, назначение которых состоит в поиске состояния, наилучшего с точки зрения $\wedge$ целевой функции. Многие задачи оптимизации не вписываются в "стандартную" модель поиска, представленную в главе 3.

Например, природа предусмотрела такую целевую функцию (пригодность для репродукции), что дарвиновская эволюция может рассматриваться как попытка ее оптимизации, но в этой задаче оптимизации нет компонентов "проверка цели" и "стоимость пути".

Авторы пришли к выводу, что для понимания сути локального поиска очень полезно рассмотреть $\wedge$ ландшафт пространства состояний (подобный показанному на рис. 4.7). Этот ландшафт характеризуется и "местонахождением" (которое определяется

состоянием), и "возвышением" (определяемым значением эвристической функции стоимости или целевой функции). Если возвышение соответствует стоимости, то задача состоит в поиске самой глубокой долины — $\wedge$ глобального минимума, а если возвышение соответствует целевой функции, то задача заключается в поиске высочайшего пика — $\wedge$ глобального максимума. (Минимум и максимум можно поменять местами, взяв их с обратными знаками.) Алгоритмы локального поиска исследуют такой ландшафт. Алгоритм полного локального поиска всегда находит цель, если она существует, а оптимальный алгоритм всегда находит глобальный минимум/максимум.

Целевая функция A $\wedge \wedge \wedge$ Глобальный максимум Локальный максимум "Плоский" локальный максимум $\wedge$ Пространство состояний Текущее состояние Рис. 4.7. Ландшафт одномерного пространства состояний, в котором возвышение соответствует целевой функции. Задача состоит в поиске глобального максимума. Как обозначено стрелкой, в процессе поиска по принципу "подъема к вершине" осуществляются попытки модификации текущего состояния в целях его улучшения. Различные топографические особенности ландшафта определены в тексте Поиск с восхождением к вершине Алгоритм поиска $\wedge$ с восхождением к вершине показан в листинге 4.2. Он представляет собой обычный цикл, в котором постоянно происходит перемещение в направлении возрастания некоторого значения, т.е. подъем. Работа этого алгоритма заканчивается после достижения "пика", в котором ни одно из соседних состояний не имеет более высокого значения. В данном алгоритме не предусмотрено сопрово-

ждение дерева поиска, поэтому в структуре данных текущего узла необходимо регистрировать только состояние и соответствующее ему значение целевой функции.

В алгоритме с восхождением к вершине не осуществляется прогнозирование за пределами состояний, которые являются непосредственно соседними по отношению к текущему состоянию. Это напоминает попытку альпиниста, страдающего от амнезии, найти вершину горы Эверест в густом тумане.

Листинг 4.2. Алгоритм поиска с восхождением к вершине (версия с наиболее крутым подъемом), который представляет собой самый фундаментальный метод локального поиска. На каждом этапе текущий узел заменяется наилучшим соседним узлом; в данной версии таковым является узел с максимальным значением Value, но если используется эвристическая оценка стоимости h , то может быть предусмотрен поиск соседнего узла с минимальным значением h function H_{i11} -Climbing(problem) returns состояние, которое представляет собой локальный максимум inputs: problem, задача local variables: current, узел neighbor, узел current $\leftarrow$ Make-Node(Initial-State[problem]) loop do neighbor $\leftarrow$ преемник узла current с наивысшим значением if Value[neighbor] < Value[current] then return State[current] current $\leftarrow$ neighbor Для

иллюстрации поиска с восхождением к вершине воспользуемся задачей с восемью ферзями, которая представлена на с. 118. В алгоритмах локального поиска обычно применяется формулировка полного состояния, в которой в каждом состоянии на доске имеется восемь ферзей, по одному ферзю в каждом столбце. Функция определения преемника возвращает все возможные состояния, формируемые путем перемещения отдельного ферзя в другую клетку одного и того же столбца (поэтому каждое состояние имеет $8 \times 7 = 56$ преемников). Эвристическая функция стоимости h определяет количество пар ферзей, которые атакуют друг друга либо прямо, либо косвенно (атака называется косвенной, если на одной горизонтали, вертикали или диагонали стоят больше двух ферзей). Глобальный минимум этой функции становится равным нулю, и это происходит только в идеальных решениях. На рис. 4.8, а показано состояние со значением $h=11$. На этом рисунке также показаны значения всех преемников данного состояния, притом что наилучшие преемники имеют значение $h=12$. Алгоритмы с восхождением к вершине обычно предусматривают случайный выбор в множестве наилучших преемников, если количество преемников больше одного.

Поиск с восхождением к вершине иногда называют $\wedge$ жадным локальным поиском, поскольку в процессе его выполнения происходит захват самого хорошего соседнего состояния без предварительных рассуждений о том, куда следует отправиться дальше. Жадность считается одним из семи смертных грехов, но, как оказалось, жадные алгоритмы часто показывают весьма высокую производительность. Во время поиска с восхождением к вершине зачастую происходит очень быстрое продвижение в направлении к решению, поскольку обычно бывает чрезвычайно легко

улучшить плохое состояние. Например, из состояния, показанного на рис. 4.8, а, достаточно сделать лишь пять ходов, чтобы достичь состояния, показанного на рис. 4.8, б, которое имеет оценку $h=1$ и очень близко к одному из решений.

ПйГ 1,5 Г4 и ЩЩ,7 17 № 16 18 14 Щ 1 14 14 113 13 1 13 др| 13 15 FFI шя 14 |14 | ^щ 16 Щ161| 15 Щ 14 16 16 ш н н «.H'ltm Fm И [Щ 18 | L"

Qi ta 1Ж I ж7 ж7! _ Wl i 1ш а) б) Рис. 4.8. Пример применения алгоритма с восхождением к вершине: диаграмма состояния задачи с восемью ферзями, характеризующегося эвристической оценкой стоимости $\bar{b}=17$; на этой диаграмме показано значение h для каждого возможного преемника, полученное путем передвижения ферзя в пределах своего столбца; отмечены наилучшие ходы (а); локальный минимум в пространстве состояний задачи с восемью ферзями; это состояние имеет оценку $h=1$, но каждый преемник характеризуется более высокой стоимостью (б) К сожалению, поиск с восхождением к вершине часто заходит в тупик по описанным ниже причинам. • Локальные максимумы. Локальный максимум представляет собой пик, более высокий по сравнению с каждым из его

соседних состояний, но более низкий, чем глобальный максимум. Алгоритмы с восхождением к вершине, которые достигают окрестностей локального максимума, обеспечивают продвижение вверх, к этому пику, но после этого заходят в тупик, из которого больше некуда двигаться. Такая проблема схематически показана на рис. 4.7. Более конкретный пример состоит в том, что состояние, показанное на рис. 4.8, б, фактически представляет собой локальный максимум (т.е. локальный минимум для оценки стоимости h); задача еще не решена, а при любом передвижении отдельно взятого ферзя ситуация становится еще хуже.

- Хребты. Пример хребта показан на рис. 4.9. При наличии хребтов возникают последовательности локальных максимумов, задача прохождения которых для жадных алгоритмов является очень трудной.
- Плато. Это область в ландшафте пространства состояний, в которой функция оценки является плоской. Оно может представлять собой плоский локальный максимум, из которого не существует выхода вверх, или $\wedge$ уступ, из которого возможно дальнейшее успешное продвижение (см. рис. 4.7). Поиск с восхождением к вершине может оказаться неспособным выйти за пределы плато.

Рис. 4.9. Иллюстрация того, почему хребты вызывают сложности при восхождении к вершине. На хребет, возвышающийся слева направо, налагается решетка состояний (полужирные дуги), создавая последовательность локальных максимумов, которые не являются непосредственно связанными друг с другом. В каждом локальном максимуме все доступные действия направлены вниз. В каждом из этих случаев рассматриваемый алгоритм достигает такой точки, из которой не может осуществляться дальнейшее успешное продвижение. Начиная со случайно сформированного состояния с восемью ферзями, алгоритм поиска с восхождением к вершине по самому крутому подъему заходит в тупик в 86% случаях, решая только 14% экземпляров этой задачи. Но он работает очень быстро, выполняя в среднем только 4 этапа в случае успешного завершения и 3 этапа, когда заходит в тупик. Это не очень плохой результат для пространства состояний с $88=17$ миллионами состояний.

Алгоритм, приведенный в листинге 4.2, останавливается, достигнув плато, на котором наилучший приемник имеет такое же значение, как и в текущем состоянии.

Имеет ли смысл продолжать движение, разрешив $\wedge$ движение в сторону в надежде на то, что это плато в действительности представляет собой уступ, как показано на рис. 4.7? Ответ на этот вопрос обычно является положительным, но необходимо соблюдать осторожность. Если будет всегда разрешено движение в сторону, притом что движение вверх невозможно, могут возникать бесконечные циклы после того, как алгоритм достигнет плоского локального максимума, не являющегося уступом.

Одно из широко применяемых решений состоит в том, что устанавливается предел количества допустимых последовательных движений в сторону. Например, можно разрешить, допустим, 100 последовательных движений в сторону в задаче с

восемью ферзями. В результате этого относительное количество экземпляров задачи, решаемых с помощью восхождения к вершине, возрастает с 14 до 94%. Но за этот успех приходится платить: алгоритм в среднем выполняет приблизительно 21 этап при каждом успешном решении экземпляра задачи и 64 этапа при каждой неудаче.

Разработано много вариантов поиска с восхождением к вершине. При $\wedge$ стохастическом поиске с восхождением к вершине осуществляется выбор случайным образом одного из движений вверх; вероятность такого выбора может зависеть от крутизны движения вверх. Обычно этот алгоритм сходится более медленно по сравнению с вариантом, предусматривающим наиболее крутой подъем, но в некоторых ландшафтах состояний он находит лучшие решения.

При $\wedge$ поиске с восхождением к вершине с выбором первого варианта реализуется стохастический поиск с восхождением к вершине путем формирования преемников случайным образом до тех пор, пока не будет сформирован преемник, лучший по сравнению с текущим состоянием. Это — хорошая стратегия, если любое состояние имеет большое количество преемников (измеряемое тысячами). В упр. 4.16 предлагается исследовать этот алгоритм.

Алгоритмы с восхождением к вершине, описанные выше, являются неполными, поскольку часто оказываются неспособными найти цель, притом что она существует, из-за того, что могут зайти в тупик, достигнув локального максимума, " $\wedge$ Поиск с восхождением к вершине и перезапуском случайным образом руководствуется широко известной рекомендацией: "Если первая попытка оказалась неудачной, пробуйте снова и снова".

В этом алгоритме предусмотрено проведение ряда поисков из сформированных случайным образом начальных состояний⁸ и остановов после достижения цели. Он является полным с вероятностью, достигающей 1, даже по той тривиальной причине, что в нем в конечном итоге в качестве начального состояния формируется одно из целевых состояний.

Если вероятность успеха каждого поиска с восхождением к вершине равна p , то ожидаемое количество требуемых перезапусков составляет $1/p$. Для экземпляров задачи с восемью ферзями, если не разрешено движение в сторону, $p=0.14$, поэтому для нахождения цели требуется приблизительно 7 итераций (7 неудачных и 1 успешная). Ожидаемое количество этапов решения равно стоимости одной успешной итерации, которая складывается с увеличенной в $A-p$ раз стоимостью неудачи, или составляет приблизительно 22 этапа. Если разрешено движение в сторону, то в среднем требуется $1/0.94=1.06$ итераций и $A \times 21 + 0.06/0.94 \times 64=25$ этапов. Поэтому алгоритм поиска с восхождением к вершине и перезапуском

случайным образом действительно является очень эффективным применительно к задаче с восемью ферзями.

Даже для варианта с тремя миллионами ферзей этот подход позволяет находить решения меньше чем за минуту⁹.

Успех поиска с восхождением к вершине в значительной степени зависит от формы ландшафта пространства состояний: если в нем есть лишь немного локальных максимумов и плато, то поиск с восхождением к вершине и перезапуском случайным образом позволяет очень быстро найти хорошее решение. С другой стороны, многие реальные задачи имеют ландшафт, который больше напоминает семейство дикобразов на плоском полу, где на вершине иглы каждого дикобраза живут другие миниатюрные дикобразы и т.д., до бесконечности. NP-трудные задачи обычно имеют экспоненциальное количество локальных максимумов, способных завести 8 Может оказаться трудной даже сама задача формирования случайным образом некоторого состояния из неявно заданного пространства состояний.

9 В [958] доказано, что в некоторых случаях лучше всего осуществлять перезапуск рандомизированного алгоритма поиска по истечении конкретной, постоянной продолжительности времени и что такой подход может оказаться гораздо более эффективным по сравнению с тем, когда разрешено продолжать каждый поиск до бесконечности. Примером такого подхода является также запрещение или ограничение количества движений в сторону.

алгоритм в тупик. Несмотря на это, часто существует возможность найти достаточно хороший локальный максимум после небольшого количества перезапусков.

Поиск с эмуляцией отжига Для любого алгоритма с восхождением к вершине, который никогда не выполняет движения "вниз по склону", к состояниям с более низкой оценкой (или более высокой стоимостью), гарантируется, что он окажется неполным, поскольку такой алгоритм всегда способен зайти в тупик, достигнув локального максимума. В отличие от этого алгоритм с чисто случайным блужданием (т.е. с перемещением к преемнику, выбираемому на равных правах случайным образом из множества преемников) является полным, но чрезвычайно неэффективным. Поэтому представляется разумной попытка скомбинировать каким-то образом восхождение к вершине со случайным блужданием, что позволит обеспечить и эффективность, и полноту.

Алгоритмом такого типа является алгоритм с [^] эмуляцией отжига. В металлургии отжигом называется процесс, применяемый для отпуска металла и стекла путем нагревания этих материалов до высокой температуры, а затем постепенного охлаждения, что позволяет перевести обрабатываемый материал в низкоэнергетическое кристаллическое состояние. Чтобы понять суть эмуляции отжига, переведем наше внимание с восхождения к вершине на [^] градиентный

спуск (т.е. минимизацию стоимости) и представим себе, что наше задание — загнать теннисный шарик в самую глубокую лунку на неровной поверхности. Если бы мы просто позволили шарiku катиться по этой поверхности, то он застрял бы в одном из локальных минимумов. А встряхивая поверхность, можно вытолкнуть шарик из локального минимума. Весь секрет состоит в том, что поверхность нужно трясти достаточно сильно, чтобы шарик можно было вытолкнуть из локальных минимумов, но не настолько сильно, чтобы он вылетел из глобального минимума. Процесс поиска решения с эмуляцией отжига заключается в том, что вначале происходит интенсивное встряхивание (аналогичное нагреву до высокой температуры), после чего интенсивность встряхивания постепенно уменьшается (что можно сравнить с понижением температуры).

Самый внутренний цикл алгоритма с эмуляцией отжига (листинг 4.3) полностью аналогичен циклу алгоритма с восхождением к вершине, но в нем вместо наилучшего хода выполняется случайно выбранный ход. Если этот ход улучшает ситуацию, то всегда принимается. В противном случае алгоритм принимает данный ход с некоторой вероятностью, меньшей 1. Эта вероятность уменьшается экспоненциально с "ухудшением" хода—в зависимости от величины ΔE , на которую ухудшается его оценка. Кроме того, вероятность уменьшается по мере снижения "температуры" т. "плохие" ходы скорее всего могут быть разрешены в начале, когда температура высока, но становятся менее вероятными по мере снижения T . Можно доказать, что если в графике `schedule` предусмотрено достаточно медленное снижение T , то данный алгоритм позволяет найти глобальный оптимум с вероятностью, приближающейся к 1.

На первых порах, в начале 1980-х годов, поиск с эмуляцией отжига широко использовался для решения задач компоновки СБИС. Кроме того, этот алгоритм нашел широкое применение при решении задач планирования производства и других крупномасштабных задач оптимизации. В упр. 4.16 предлагается сравнить его производительность с производительностью поиска с восхождением к вершине и перезапуском случайным образом при решении задачи с p ферзями.

Листинг 4.3. Алгоритм поиска с эмуляцией отжига, который представляет собой одну из версий алгоритма стохастического поиска с восхождением к вершине, в которой разрешены некоторые ходы вниз. Ходы вниз принимаются к исполнению с большей вероятностью на ранних этапах выполнения графика отжига, а затем, по мере того как проходит время, выполняются менее часто.

Входной параметр `schedule` определяет значение температуры T как функции от времени `function Simulated-Annealing(problem, schedule) returns состояние решения`
`inputs: problem`, задача `schedule`, отображение между временем и "температурой"
`local variables: current`, узел `next`, узел T , "температура", от которой зависит вероятность шагов вниз `current  $\leftarrow$  Make-Node(Initial-State[problem]) for  $t \leftarrow 1$  to`

$oo \text{ do } T \leftarrow \text{schedule}[t] \text{ if } T = 0 \text{ then return current next} \leftarrow \text{случайно выбранный}$
 $\text{преемник состояния current } AE \leftarrow \text{Value[next]} - \text{Value[current]} \text{ if } AE > 0 \text{ then current}$
 $\leftarrow \text{next else current} \leftarrow \text{next с вероятностью только } eAE/T$ Локальный лучевой поиск
 Стремление преодолеть ограничения, связанные с нехваткой памяти, привело к тому, что в свое время предпочтение отдавалось алгоритмам, предусматривающим хранение в памяти только одного узла, но, как оказалось, такой подход часто является слишком радикальным способом экономии памяти. В алгоритме $\wedge$ локального лучевого поиска¹⁰ предусмотрено отслеживание к состояний, а не только одного состояния. Работа этого алгоритма начинается с формирования случайным образом к состояний. На каждом этапе формируются все преемники всех к состояний. Если какой-либо из этих преемников соответствует целевому состоянию, алгоритм останавливается. В противном случае алгоритм выбирает из общего списка к наилучших преемников и повторяет цикл.

На первый взгляд может показаться, что локальный лучевой поиск с к состояниями представляет собой не что иное, как выполнение к перезапусков случайным образом, но не последовательно, а параллельно. Тем не менее в действительности эти два алгоритма являются полностью разными. При поиске с перезапуском случайным образом каждый процесс поиска осуществляется независимо от других, с³ А в локальном лучевом поиске полезная информация передается по к параллельным потокам поиска. Например, если в одном состоянии вырабатывается несколько хороших преемников, а во всех других к-1 состояниях вырабатываются плохие преемники, то возникает такой эффект, как если бы поток, контролирующий первое состояние, сообщил другим потокам: "Идите все сюда, здесь 10 Локальный лучевой поиск является адаптацией лучевого поиска, который представляет собой алгоритм, основанный на использовании пути.

трава зеленее!" Этот алгоритм способен быстро отказаться от бесплодных поисков и перебросить свои ресурсы туда, где достигнут наибольший прогресс.

В своей простейшей форме локальный лучевой поиск может страдать от отсутствия разнообразия между 7с состояниями, поскольку все эти состояния способны быстро сосредоточиться в небольшом регионе пространства состояний, в результате чего этот поиск начинает ненамного отличаться от дорогостоящей версии поиска с восхождением к вершине. Этот недостаток позволяет устранить вариант, называемый $\wedge$ стохастическим лучевым поиском, который аналогичен стохастическому поиску с восхождением к вершине. При стохастическом лучевом поиске вместо выбора наилучших к преемников из пула преемников-кандидатов происходит выбор к преемников случайным образом, притом что вероятность выбора данного конкретного преемника представляет собой возрастающую функцию значения его оценки.

Стохастический лучевой поиск имеет некоторое сходство с процессом естественного отбора, в котором "преемники" (потомки) некоторого "состояния" (организма) образуют следующее поколение в соответствии со "значением их оценки" (в соответствии с их жизненной пригодностью).

Генетические алгоритмы ^ Генетический алгоритм (Genetic Algorithm — GA) представляет собой вариант стохастического лучевого поиска, в котором состояния-преемники формируются путем комбинирования двух родительских состояний, а не посредством модификации единственного состояния. В нем просматривается такая же аналогия с естественным отбором, как и в стохастическом лучевом поиске, за исключением того, что теперь мы имеем дело с половым, а не бесполом воспроизводством.

Как и при лучевом поиске, работа алгоритмов GA начинается с множества T сформированных случайным образом состояний, называемых ^популяцией.

Каждое состояние, или ^ индивидуум, представлено в виде строки символов из конечного алфавита, чаще всего в виде строки из нулей 0 и единиц 1. Например, состояние задачи с восемью ферзями должно определять позиции восьми ферзей, каждый из которых стоит на вертикали с 8 клетками, и поэтому для его представления требуется $8 \times \log_2 8 = 24$ бита. Иным образом, каждое состояние может быть представлено в виде восьми цифр, каждая из которых находится в диапазоне от 1 до 8.

(Как будет показано ниже, эти две кодировки проявляют себя в ходе поиска поразному.) На рис. 4.10, а показана популяция из четырех восьмисимвольных строк, представляющих состояния с восемью ферзями.

Процесс выработки следующего поколения состояний показан на рис. 4.10, б— д.

На рис. 4.10, б каждое состояние классифицируется с помощью функции оценки, или (в терминологии GA) ^ функции пригодности. Функция пригодности должна возвращать более высокие значения для лучших состояний, поэтому в задаче с восемью ферзями используется количество неатакующих друг друга пар ферзей, которое в любом решении имеет значение 28. Для этих четырех состояний соответствующие значения равны 24, 23, 20 и 11. В данном конкретном варианте генетического алгоритма вероятность выбора для воспроизводства прямо пропорциональна оценке пригодности; соответствующие вероятности в процентах показаны рядом с исходными оценками.

24748552 32752411 24415124 32543213 а) . ? 4 31% ^ ^ _ ^ 23 29% ^ 20 26% И 14% б)
327J52411':

24748552 32752411 24415:124 в) \N 32748552 24752411 32752124 24415411 3274CЭ52
24752411 3^1152124 2441543Q | д) Рис. 4.10. Генетический алгоритм: начальная

популяция (а); функция пригодности (б); отбор (в); скрещивание (г); мутация (д). Начальная популяция классифицируется с помощью функции пригодности, в результате чего формируются пары для скрещивания. Эти пары производят потомков, которые в конечном итоге подвергаются мутации. Как показано на рис. 4.10, в, для воспроизводства случайным образом выбираются две пары в соответствии с вероятностями, показанными на рис. 4.10, б. Обратите внимание на то, что один индивидум выбирается дважды, а один вообще остается не выбранным¹¹. Для каждой пары, предназначенной для воспроизводства, среди позиций в строке случайным образом выбирается точка ^ скрещивания. На рис. 4.10 точки скрещивания находятся после третьей цифры в первой паре и после пятой цифры во второй паре¹².

Как показано на рис. 4.10, г, сами потомки создаются путем перекрестного обмена подстроками родительских строк, разорванных в точке скрещивания. Например, первый потомок первой пары получает три первые цифры от первого родителя, а остальные цифры — от второго родителя, тогда как второй потомок получает первые три цифры от второго родителя, а остальные — от первого родителя. Состояния задачи с восемью ферзями, участвующие в этом этапе воспроизводства, показаны на рис. 4.11. Данный пример иллюстрирует тот факт, что если два родительских состояния являются весьма различными, то операция скрещивания способна выработать состояние, которое намного отличается и от одного, и от другого родительского состояния. Часто происходит так, что популяция становится весьма разнообразной на самых ранних этапах этого процесса, поэтому скрещивание (как и эмуляция отжига) в основном обеспечивает выполнение крупных этапов в пространстве состояний в начале процесса поиска и более мелких этапов позднее, когда большинство индивидуумов становятся весьма похожими друг на друга.

Наконец, на рис. 4.10, д показано, что каждое местонахождение подвергается случайной ^ мутации с небольшой независимой вероятностью. В первом, третьем и четвертом потомках мутация свелась к изменению одной цифры. В задаче с восемью ферзями эта операция соответствует выбору случайным образом одного ферзя и пе¹¹ Существует много вариантов этого правила выбора. Можно показать, что при использовании метода отсеивания, в котором отбрасываются все индивидуумы с оценками ниже заданного порога, алгоритм сходится быстрее, чем при использовании версии со случайным выбором [81].

¹² Кодировка начинает играть решающую роль в процессе решения именно на этом этапе.

Если вместо 8 цифр используется 24-битовая кодировка, то вероятность попадания точки скрещивания в середину цифры становится равной $2/3$, а это приводит к тому, что мутация этой цифры по сути становится произвольной.

ремещению этого ферзя на случайно выбранную клетку в его столбце. В листинге 4.4 приведен алгоритм, который реализует все эти этапы.

Рис. 4.11. Состояния задачи с восемью ферзями, соответствующие первым двум родительским состояниям, показанным на рис. 4.10, в, и первому потомку, показанному на рис. 4.10, г.

Затененные столбцы на этапе скрещивания теряются, а незатененные сохраняются

Листинг 4.4. Генетический алгоритм. Этот алгоритм аналогичен тому, который показан схематически на рис. 4.10, за одним исключением: в этой более широко применяемой версии алгоритма каждое скрещивание двух родителей приводит к получению только одного потомка, а не двух

```
function Genetic-Algorithm{population, Fitness-Fn}
returns индивидуум individual
inputs: population, множество индивидуумов (популяция)
Fitness-Fn, функция, которая измеряет пригодность индивидуума
repeat new_population <— пустое множество
loop for i from 1 to Size{population} do
  x <rRandom-Selection {population, Fitness-Fn}
  y <rRandom-Selection {population, Fitness-Fn}
  child <— Reproduce(x, y)
  if (небольшое случайно выбранное значение вероятности) then
    child <— Mutate(child)
  добавить дочерний индивидуум child к множеству new_population
population <— new_population
until некоторый индивидуум не станет достаточно пригодным или не истечет достаточное количество времени
return наилучший индивидуум individual
в популяции population, в соответствии с функцией Fitness-Fn
function Reproduce(x, y)
returns индивидуум individual
inputs: x, y, индивидуумы-родители
n <— Length(x)
c <g~ случайное число от 1 до n
return Append(Substring(x, 1, c), Substring(y, c+1, n))
```

Эволюция И ПОИСК Теория эволюции была изложена Чарльзом Дарвином в его книге *On the Origin of Species by Means of Natural Selection* (Происхождение видов путем естественного отбора) [325]. Основная идея этой теории проста: при воспроизводстве возникают вариации (известные как мутации), которые сохраняются в следующих поколениях с частотой, приблизительно пропорциональной тому, как они влияют на пригодность к воспроизводству.

Дарвин разрабатывал свою теорию, не зная о том, благодаря чему происходит наследование и модификация характерных особенностей организмов.

Вероятностные законы, управляющие этими процессами, были впервые обнаружены Грегором Менделем [1034], монахом, проводившим эксперименты с душистым горошком с использованием метода, названного им искусственным оплодотворением. Гораздо позже Уотсон и Крик [1559] выявили структуру молекулы ДНК (дезоксирибонуклеиновой кислоты) и ее алфавит, состоящий из аминокислот АГТЦ (аденин, гуанин, тимин, цитозин). В предложенной ими стандартной модели вариации возникают и в результате точечных мутаций в последовательности этих

аминокислот, и в результате "скрещивания" (при котором ДНК потомка формируется путем объединения длинных секций ДНК от каждого родителя).

Аналогия между этим процессом и алгоритмами локального поиска уже была описана выше; принципиальное различие между стохастическим лучевым поиском и эволюцией состоит в использовании полового воспроизводства, в котором потомки формируются с участием нескольких организмов, а не только одного. Однако фактически механизмы эволюции намного богаче по сравнению с тем, что допускает большинство генетических алгоритмов.

Например, мутации могут быть связаны с обращением, дублированием и перемещением больших фрагментов ДНК; некоторые вирусы заимствуют ДНК из одного организма и вставляют в другой; кроме того, существуют взаимозаменяемые гены, которые лишь копируют самих себя в геноме много тысяч раз.

Есть даже такие гены, которые отравляют клетки потенциальных родителей, не несущие ген этого типа, повышая тем самым шансы на воспроизводство данного гена. Наиболее важным является тот факт, что гены сами кодируют те механизмы, с помощью которых геном воспроизводится и транслируется в организме. В генетических алгоритмах такие механизмы представляют собой отдельную программу, не закодированную в строках, с которыми осуществляются манипуляции.

На первый взгляд может показаться, что механизм дарвиновской эволюции является неэффективным, поскольку в его рамках на Земле за многие тысячи лет было вслепую сформировано около 10⁴⁵ или примерно столько организмов, притом что за такое время поисковые эвристики этого механизма не улучшились ни на йоту. Но за пятьдесят лет до Дарвина французский натуралист Жан Ламарк [884], получивший известность в другой области, предложил теорию эволюции, согласно которой характерные особенности организма, приобретенные в результате его адаптации на протяжении срока жизни этого организма, передаются его потомкам. Такой процесс был бы очень эффективным, но в природе, по-видимому, не происходит. Намного

позднее Джеймс Болдуин [64] предложил теорию, которая внешне кажется аналогичной; согласно этой теории, поведение, которому обучился организм на протяжении своей жизни, способствует повышению скорости эволюции. В отличие от теории Ламарка, теория Болдуина полностью согласуется с дарвиновской теорией эволюции, поскольку в ней учитываются давления отбора, действующие на индивидуумов, которые нашли локальные оптимумы среди множества возможных вариантов поведения, допустимых согласно их генетической природе. Современные компьютерные модели подтвердили, что "эффект Болдуина" является реальным, при условии, что "обычная" эволюция способна создавать организмы, внутренние

показатели производительности которых каким-то образом коррелируют с их фактической жизненной пригодностью.

Как и в алгоритмах стохастического лучевого поиска, в генетических алгоритмах тенденция к стремлению к максимуму сочетается с проводимым случайным образом исследованием и с обменом информацией между параллельными поисковыми потоками. Основное преимущество генетических алгоритмов (если таковое действительно имеется) связано с применением операции скрещивания. Тем не менее можно доказать математически, что если позиции в генетическом коде первоначально устанавливаются в случайном порядке, то скрещивание не дает никаких преимуществ.

На интуитивном уровне можно предположить, что преимуществом была бы способность комбинировать в результате скрещивания большие блоки символов, которые были независимо сформированы в ходе эволюции для выполнения полезных функций, что позволило бы повысить степень детализации, с которой действует этот алгоритм поиска. Например, преимущество достиглось бы, если в качестве полезного блока была выделена строка символов, предусматривающая размещение первых трех ферзей в позициях 2, 4 и 6 (где они не атакуют друг друга), после чего можно было бы объединять этот блок с другими блоками для формирования решения.

В теории генетических алгоритмов описано, как действует этот подход, в котором используется идея $\wedge$ схемы, представляющей собой такую подстроку, где некоторые из позиций могут оставаться не заданными. Например, схема 246***** описывает такие состояния всех восьми ферзей, в которых первые три ферзя находятся соответственно в позициях 2, 4 и 6. Строки, соответствующие этой схеме (такие как 24613578), называются экземплярами схемы. Можно доказать, что если средняя пригодность экземпляров некоторой схемы находится выше среднего, то количество экземпляров этой схемы в популяции будет расти со временем. Очевидно, что данный эффект вряд ли окажется существенным, если смежные биты полностью не связаны друг с другом, поскольку в таком случае будет лишь немного непрерывных блоков, которые предоставляли бы какое-то постоянное преимущество.

Генетические алгоритмы работают лучше всего в сочетании со схемами, соответствующими осмысленным компонентам решения. Например, если строка представляет какую-либо радиотехническую антенну, то схемы могут соответствовать компонентам этой антенны, таким как рефлекторы и дефлекторы. Хороший компонент, по всей вероятности, будет оставаться хорошим во многих разных проектах. Из этого следует, что для успешного использования генетических алгоритмов требуется тщательное конструирование представления задачи.

На практике генетические алгоритмы оказали глубокое влияние на научные методы, применяющиеся при решении таких задач оптимизации, как компоновка электронных схем и планирование производства. В настоящее время уже не так ясно, вызвана ли притягательность генетических алгоритмов их высокой производительностью или обусловлена эстетически привлекательными истоками в теории эволюции. Но все еще необходимо проделать большой объем работы для выяснения условий, при которых генетические алгоритмы обладают наиболее высокой производительностью.

4.4. Локальный поиск в непрерывных пространствах В главе 2 было описано различие между дискретными и непрерывными вариантами среды, а также указано, что большинство реальных вариантов среды являются непрерывными. Но еще ни один из описанных выше алгоритмов не способен действовать в непрерывных пространствах состояний, поскольку в этих алгоритмах в большинстве случаев функция определения преемника возвращала бы бесконечно большое количество состояний! В настоящем разделе приведено очень краткое введение в некоторые методы локального поиска, предназначенные для нахождения оптимальных решений в непрерывных пространствах. Литература по этой теме весьма обширна; многие из этих основных методов впервые были созданы в XVII веке после разработки первых математических исчислений Ньютоном и Лейбницем¹³.

Применение данных методов рассматривается в нескольких главах настоящей книги, включая главы, касающиеся обучения, машинного зрения и робототехники. Короче говоря, эти методы касаются всего, что связано с реальным миром.

Начнем изложение этой темы с примера. Предположим, что где-то в Румынии требуется найти место для размещения трех новых аэропортов таким образом, чтобы сумма квадратов расстояний от каждого города на карте (см. рис. 3.1) до ближайшего к нему аэропорта была минимальной. В таком случае пространство состояний определено координатами аэропортов: $\{x_1, y_1\}$, $\{x_2, y_2\}$ и $\{x_3, y_3\}$. Это — шестимерное пространство; иными словами можно выразить данную мысль так, что состояния определяются шестью переменными. (Вообще говоря, состояния определяются n -мерным вектором переменных, x .) Перемещение в этом пространстве соответствует переносу одного или нескольких из этих аэропортов в другое место на карте.

Целевую функцию $f(x_1, y_1, x_2, y_2, x_3, y_3)$ после определения ближайших городов можно вычислить довольно легко, но гораздо сложнее составить общее выражение, соответствующее искомому решению.

Один из способов предотвращения необходимости заниматься непрерывными задачами состоит в том, чтобы просто дискретизировать окрестности каждого состояния. Например, можно предусмотреть перемещение одновременно только

одного аэропорта в направлении либо x , либо y на постоянную величину $\pm D$. При наличии шести переменных это соответствует двенадцати возможным преемникам для каждого состояния. После этого появляется возможность применить любой из алгоритмов локального поиска, описанных выше. Кроме того, алгоритмы стохастиче¹³ Для чтения этого раздела желательно обладать элементарными знаниями о системах исчисления в многомерном пространстве и векторной арифметике.

ского поиска с восхождением к вершине и поиска с эмуляцией отжига можно применять непосредственно, без дискретизации этого пространства. Такие алгоритмы выбирают преемников случайным образом, что может быть осуществлено путем формирования случайным образом векторов с длиной A .

Имеется также много методов, в которых при осуществлении попыток найти максимум используется ∇ градиент ландшафта. Градиент целевой функции представляет собой вектор V_f , позволяющий определить величину и направление наиболее крутого склона. Для рассматриваемой задачи справедливо следующее соотношение:

$(df \ df_1 \ df_2 \ df_3 \ df_4 \ df_5 \ df_6) \ (dx_1 \ dy_1 \ dx_2 \ dy_2 \ dx_3 \ dy_3 \ \dots)$ В некоторых случаях можно найти максимум, решая уравнение $V_f=0$. (Это можно сделать, например, при размещении только одного аэропорта; решение представляет собой арифметическое среднее координат всех городов.) Но во многих случаях это уравнение не может быть решено в замкнутой форме. Например, при наличии трех аэропортов выражение для градиента зависит от того, какие города являются ближайшими к каждому аэропорту в текущем состоянии. Это означает, что мы можем вычислить этот градиент локально, но не глобально. Даже в таком случае остается возможность выполнять поиск с восхождением к вершине по самому крутому склону, обновляя текущее состояние с помощью следующей формулы: $x \leftarrow x + \alpha V_f(x)$ где α — небольшая константа. В других случаях целевая функция в дифференцируемой форме может оказаться вообще не доступной, например, значение конкретного множества местонахождений аэропортов может быть определено в результате вызова на выполнение какого-то пакета крупномасштабного экономического моделирования. В таких случаях можно определить так называемый ∇ эмпирический градиент, оценивая отклик на небольшие увеличения и уменьшения каждой координаты.

Поиск по эмпирическому градиенту является аналогичным поиску с восхождением к вершине по самому крутому склону в дискретизированной версии данного пространства состояний.

За фразой "а — небольшая константа" скрывается огромное разнообразие методов определения значения α . Основная проблема состоит в том, что если значение α

слишком мало, то требуется слишком много этапов поиска, а если слишком велико, то в поиске можно проскочить максимум. Попытка преодолеть эту дилемму предпринимается в методе $\wedge$ линейного поиска, который предусматривает продолжение поиска в направлении текущего градиента (обычно путем повторного удвоения a) до тех пор, пока f не начнет снова уменьшаться. Точка, в которой это происходит, становится новым текущим состоянием. Сформировалось несколько научных школ, в которых доминируют разные взгляды на то, каким образом в этой точке следует выбирать новое направление.

Для многих задач наиболее эффективным алгоритмом является почтенный метод $\wedge$ Ньютона-Рафсона [1132], [1266]. Это — общий метод поиска корней функций, т.е. метод решения уравнений в форме $d(x) = 0$. Этот алгоритм действует на основе вычисления новой оценки для корня x в соответствии с формулой Ньютона: $x \leftarrow x - d(x) / d'(x)$

Чтобы найти максимум или минимум f , необходимо найти такое значение x , для которого градиент равен нулю (т.е. $\nabla f(x) = 0$). Поэтому функция $d(x)$ в формуле Ньютона принимает вид $\nabla f(x)$, и уравнение обновления состояния может быть записано в матрично-векторной форме следующим образом: $x \leftarrow x - H^{-1}(x) \nabla f(x)$ где $H(x)$ — представляет собой $\wedge$ гессиан (матрицу вторых производных), элементы которого H_{ij} задаются выражением $d^2 f / dx_i dx_j$. Поскольку гессиан имеет n^2 элементов, то метод Ньютона-Рафсона в многомерных пространствах становится дорогостоящим, поэтому было разработано множество способов приближенного решения этой задачи.

Локальные максимумы, хребты и плато создают затруднения в работе методов локального поиска не только в дискретных пространствах состояний, но и в непрерывных пространствах состояний. Для преодоления этих затруднений могут применяться алгоритмы с перезапуском случайным образом и с эмуляцией отжига, которые часто оказываются достаточно полезными. Тем не менее многомерные непрерывные пространства характеризуются очень большим объемом, в котором легко потеряться.

Последней темой, с которой необходимо кратко ознакомиться, является $\wedge$ оптимизация с ограничениями. Задача оптимизации называется задачей оптимизации с ограничениями, если решения в ней должны удовлетворять некоторым жестким ограничениям на значения каждой переменной. Например, в рассматриваемой задаче с размещением аэропортов можно ввести ограничения, чтобы места для аэропортов находились в пределах Румынии, причем на суше (а не, допустим, на середине озера). Сложность задач оптимизации с ограничениями зависит от характера ограничений и от целевой функции. Наиболее широко известной категорией таких задач являются задачи $\wedge$ линейного программирования, в которых ограничения должны представлять собой линейные неравенства,

образующие выпуклую область, а целевая функция также является линейной. Задачи линейного программирования могут быть решены за время, полиномиально зависящее от количества переменных.

Кроме того, проводились исследования задач с другими типами ограничений и целевых функций: квадратичное программирование, коническое программирование второго порядка и т.д.

4.5. Поисковые агенты, действующие в оперативном режиме, и неизвестные варианты среды До сих пор в данной книге изложение было сосредоточено на описании агентов, в которых используются алгоритмы $\wedge$ поиска в автономном режиме. Эти агенты вычисляют полное решение, прежде чем ступить в реальный мир (см. листинг 3.1), а затем выполняют это решение, не обращаясь к данным своих восприятий.

В отличие от этого любой агент, выполняющий $\wedge$ поиск в оперативном режиме¹⁴,
¹⁴ В компьютерных науках термин "работающий в оперативном режиме" обычно используется по отношению к алгоритмам, которые должны обрабатывать входные данные по мере их получения, а не ожидать, пока станет доступным все множество входных данных.

функционирует по методу чередования вычислений и действий: вначале предпринимает действие, затем обзореваает среду и вычисляет следующее действие. Поиск в оперативном режиме целесообразно применять в динамических или полудинамических проблемных областях; таковыми являются проблемные области, в которых назначается штраф за то, что агент ведет себя пассивно и вычисляет свои действия слишком долго. Еще более оправданным является использование поиска в оперативном режиме в стохастических проблемных областях. Вообще говоря, результаты любого поиска в автономном режиме должны сопровождаться экспоненциально большим планом действий в непредвиденных ситуациях, в котором учитываются все возможные варианты развития событий, тогда как при поиске в оперативном режиме необходимо учитывать лишь то, что действительно происходит. Например, агент, играющий в шахматы, должен быть настолько хорошо проконсультирован, чтобы мог сделать свой первый ход задолго до того, как станет ясен весь ход игры.

Применение поиска в оперативном режиме является необходимым при решении любой $\wedge$ задачи исследования, в которой агенту не известны состояния и действия.

Агент, находящийся в таком положении полного неведения, должен использовать свои действия в качестве экспериментов для определения того, что делать дальше, и поэтому вынужден чередовать вычисления и действия.

Каноническим примером применения поиска в оперативном режиме может служить робот, который помещен в новое здание и должен его исследовать, чтобы составить карту, которую может затем использовать для перехода из точки А в точку В.

Примерами алгоритмов поиска в оперативном режиме являются также методы выхода из лабиринтов (как известно, такие знания всегда были нужны вдохновляющим нас на подвиги героям древности). Однако исследование пространства— это не единственная форма познания окружающего мира. Рассмотрим поведение новорожденного ребенка: в его распоряжении есть много возможных действий, но он не знает, к чему приведет выполнение какого-либо из них, а эксперименты проводит лишь в немногих возможных состояниях, которых он может достичь. Постепенное изучение ребенком того, как устроен мир, отчасти представляет собой процесс поиска в оперативном режиме.

Задачи поиска в оперативном режиме Любая задача поиска в оперативном режиме может быть решена только агентом, выполняющим и вычисления, и действия, а не осуществляющим лишь вычислительные процессы. Предполагается, что агент обладает только описанными ниже знаниями.

- Функция $Actions(s)$, которая возвращает список действий, допустимых в состоянии s .
- Функция стоимости этапа $c(s, a, s')$; следует отметить, что она не может использоваться до тех пор, пока агент не знает, что результатом является состояние s' .
- Функция $Goal-Test(s)$.

Следует, в частности, отметить, что агент не может получить доступ к преемникам какого-либо состояния, иначе чем путем фактического опробования всех действий в этом состоянии. Например, в задаче с лабиринтом, показанной на рис. 4.12,

агент не знает, что переход в направлении Up из пункта A,1) приводит в пункт A,2), а выполнив это действие, не знает, позволит ему действие Down вернуться назад в пункт A,1). Такая степень неведения в некоторых приложениях может быть уменьшена, например, робот-исследователь может знать, как работают его действия по передвижению, и оставаться в неведении лишь в отношении местонахождения препятствий.

Мы будем предполагать, что агент всегда может распознать то состояние, которое он уже посещал перед этим, кроме того, будем руководствоваться допущением, что все действия являются детерминированными. (Последние два допущения будут ослаблены в главе 17.) Наконец, агент может иметь доступ к некоторой допустимой эвристической функции $h(s)$, которая оценивает расстояние от текущего состояния до целевого. Например, как показано на рис. 4.12, агент может знать местонахождение цели и быть способным использовать эвристику с манхэттенским расстоянием.

з Г I П1 I i S 1 2 3 Рис. 4.12. Простая задача с лабиринтом. Агент начинает движение с квадрата S и должен достичь квадрата G, но ничего не знает о самой

среде. Как правило, назначение агента состоит в том, чтобы достичь целевого состояния, минимизируя при этом стоимость. (Еще одно возможное назначение агента может заключаться в том, чтобы он просто исследовал всю эту среду.) Стоимость представляет собой суммарную стоимость пути, фактически пройденного агентом. Обычно принято сравнивать эту стоимость со стоимостью пути, по которому следовал бы агент, если бы он заранее знал пространство поиска, т.е. со стоимостью фактического кратчайшего пути (или кратчайшего полного обследования). В терминологии алгоритмов поиска в оперативном режиме это соотношение называется α коэффициентом конкурентоспособности; желательно, чтобы этот коэффициент был как можно меньше.

Хотя такое требование по минимизации коэффициента конкурентоспособности может показаться резонным, можно легко доказать, что в некоторых случаях наилучший достижимый коэффициент конкурентоспособности (competitive ratio) является бесконечным. Например, если некоторые действия необратимы, то поиск в оперативном режиме может в конечном итоге перейти в тупиковое состояние, из которого не достижимо целевое состояние. Возможно, читатель сочтет выражение "в конечном итоге" неубедительным; в конце концов, должен же существовать такой алгоритм, который окажется способным не упираться в тупик в ходе проведения исследований с его помощью! Поэтому уточним приведенное выше утверждение таким образом: d^α ни один алгоритм не позволяет избежать тупиков во всех возможных

пространствах состояний. Рассмотрим два пространства состояний с тупиками, показанные на рис. 4.13, а. Для алгоритма поиска в оперативном режиме, который посетил состояния БиА, оба эти пространства состояний представляются идентичными, поэтому он должен принять одинаковое решение в обоих из них. Поэтому в одном из этих состояний алгоритм потерпит неудачу. Это — один из примеров α возражения противника (adversary argument), поскольку легко себе представить, как противник формирует пространство состояний в ходе того, как это пространство исследуется агентом, и может размещать цели и тупики везде, где пожелает.

С -п G а) б) Рис. 4.13. Примеры ситуаций, характеризующихся бесконечным значением коэффициента конкурентоспособности: два пространства состояний, которые способны завести агента, выполняющего поиск в оперативном режиме, в тупик; любой конкретный агент потерпит неудачу, по меньшей мере, в одном из этих пространств (а); двухмерная среда, которая может вынудить агента, выполняющего поиск в оперативном режиме, следовать по маршруту к цели, который может оказаться сколь угодно неэффективным; какой бы выбор ни сделал агент, противник блокирует его маршрут еще одной длинной, тонкой стеной, чтобы путь, по которому следует агент, стал намного длиннее по сравнению с наилучшим возможным путем

(б) Тупики представляют собой одну из реальных сложностей в области исследований, проводимых с помощью робота, поскольку лестницы, пандусы, обрывы и многие другие формы естественного ландшафта создают предпосылки для необратимых действий. Для того чтобы добиться прогресса, мы будем предполагать, что данное пространство состояний является $\wedge$ безопасно исследуемым, т.е. что некоторые целевые состояния достижимы из каждого достижимого состояния.

Пространства состояний с обратимыми действиями, такие как лабиринты и задачи игры в восемь, могут рассматриваться как неориентированные графы и, вполне очевидно, являются безопасно исследуемыми.

Но даже в безопасно исследуемых вариантах среды нельзя гарантировать достижение какого-либо ограниченного значения коэффициента конкурентоспособности, если в них имеются маршруты с неограниченной стоимостью. Это утверждение легко доказать применительно к вариантам среды с необратимыми действиями, но

фактически оно остается истинным также и для обратимого случая (см. рис. 4.13, б).

По этой причине обычно принято описывать производительность алгоритмов поиска в оперативном режиме с учетом размеров всего пространства состояний, а не только глубины сдмой поверхностной цели.

Агенты, выполняющие поиск в оперативном режиме После каждого действия агент, выполняющий поиск в оперативном режиме, получает результаты акта восприятия, с помощью которых может узнать, какого состояния он достиг; на основании этой информации агент может дополнить карту своей среды. Текущая карта используется для принятия решения о том, куда двигаться дальше. Такое чередование планирования и выполнения действий означает, что алгоритмы поиска в оперативном режиме весьма значительно отличаются от алгоритмов поиска в автономном режиме, которые рассматривались выше.

Например, алгоритмы поиска в автономном режиме, такие как A^* , обладают способностью развертывать узел в одной части пространства, а затем немедленно развертывать узел в другой части пространства, поскольку для развертывания узла требуются моделируемые, а не реальные действия. С другой стороны, любой алгоритм поиска в оперативном режиме позволяет агенту развертывать только тот узел, который он физически занимает. Для предотвращения необходимости путешествовать через все дерево, чтобы развернуть следующий узел, представляется более удобным развертывание узлов в локальном порядке. Именно таким свойством обладает поиск в глубину, поскольку (за исключением операции возврата) следующий развертываемый узел является дочерним узлом ранее развернутого узла.

Алгоритм агента, выполняющего поиск в глубину в оперативном режиме, приведен в листинге 4.5. Агент хранит составленную им карту в таблице `result [a, s]`, в которой регистрируются состояния, явившиеся следствием выполнения действия `a` в состоянии `s`. Этот агент предпринимает попытку выполнения из текущего состояния любого действия, которое еще не было исследовано в этом состоянии.

Сложности возникают после того, как агент опробовал все действия в некотором состоянии. При поиске в глубину в автономном режиме это состояние удаляется из очереди, а при поиске в оперативном режиме агент должен выполнить возврат физически. При поиске в глубину это равносильно возврату в последнее по времени состояние, из которого агент перешел в текущее состояние. Соответствующая организация работы обеспечивается за счет ведения такой таблицы, где для каждого состояния перечисляются состояния-предшественники, к которым агент еще не выполнял возврат. Если агент исчерпал список состояний, к которым может выполнить возврат, это означает, что проведенный агентом поиск является полным.

Листинг 4.5. Агент, выполняющий поиск в оперативном режиме, который проводит исследование с использованием поиска в глубину. Этот агент может применяться только в пространствах двунаправленного поиска `function Online-DFS-Agent(s*)` returns действие `action inputs: s'`, восприятие, позволяющее идентифицировать текущее состояние `static: result`, таблица, индексированная по действиям и состояниям, первоначально пустая

`unexplored`, таблица, в которой для каждого посещенного состояния перечислены еще не опробованные действия `unbacktracked`, таблица, в которой для каждого посещенного состояния перечислены еще не опробованные возвраты `s, a`, предыдущие состояние и действие, первоначально неопределенные `if Goal-Test(s')` then return stop if `s'` представляет собой новое состояние then `unexplored[sy] <— Actions(s')` if `s` не является неопределенным then do `result[a, s] <— s'` добавить `s` в начало таблицы `unbacktracked[s*]` if элемент `unexplored[s']` пуст then if элемент `unbacktracked[s']` пуст then return stop else `a <— действие b`, такое что `result[b, s'] = Pop(unbacktracked[s'])` else `a <— Pop(imexplored[s'] )` `s <— s'` return `a` Рекомендуем читателю провести трассировку хода выполнения процедуры `Online-DFS-Agent` применительно к лабиринту, показанному на рис. 4.12. Можно довольно легко убедиться в том, что в наихудшем случае агент в конечном итоге пройдет по каждой связи в данном пространстве состояний точно два раза. При решении задачи обследования такая организация работы является оптимальной; а при решении задачи поиска целей, с другой стороны, коэффициент конкурентоспособности может оказаться сколь угодно неэффективным, если агент будет отправляться в длительные экскурсии, притом что целевое состояние находится непосредственно рядом с начальным состоянием. Такая проблема решается с использованием варианта метода итеративного углубления для работы в оперативном режиме; в

среде, представляющей собой однородное дерево, коэффициент конкурентоспособности подобного агента равен небольшой константе.

В связи с тем что в алгоритме Online-DFS -Agent используется метод с возвратами, он может применяться только в таких пространствах состояний, где действия являются обратимыми. Существуют также немного более сложные алгоритмы, применимые в пространствах состояний общего вида, но ни один из подобных алгоритмов не характеризуется ограниченным коэффициентом конкурентоспособности.

Локальный поиск в оперативном режиме Как и поиск в глубину, поиск с восхождением к вершине обладает свойством локализации применительно к операциям развертывания в нем узлов. В действительности сам поиск с восхождением к вершине уже можно считать алгоритмом поиска в оперативном режиме, поскольку предусматривает хранение в памяти только одного текущего состояния! К сожалению, в своей простейшей форме этот алгоритм не очень полезен, так как оставляет агента в таком положении, что последний не может покинуть локальный максимум и отправиться куда-то еще. Более того, в этом алгоритме не может использоваться перезапуск случайным образом, поскольку агент не способен перенести самого себя в новое состояние.

Вместо перезапуска случайным образом для исследования среды может быть предусмотрено использование $\wedge$ случайного блуждания (random walk). В методе случайного блуждания просто выбирается случайным образом одно из действий, доступных из текущего состояния; предпочтение отдается действиям, которые еще не были опробованы. Легко доказать, что метод случайного блуждания позволяет агенту в конечном итоге найти цель или выполнить свою задачу исследования, при условии, что пространство является конечным¹⁵. С другой стороны, этот процесс может оказаться очень продолжительным. На рис. 4.14 показана среда, в которой для поиска цели с помощью метода случайного блуждания может потребоваться количество этапов, измеряемое экспоненциальной зависимостью, поскольку на каждом этапе вероятность шага назад вдвое превышает вероятность шага вперед. Безусловно, что этот пример является надуманным, но существует много реальных пространств состояний, топология которых способствует возникновению "ловушек" такого рода при случайном блуждании.

Рис. 4.14. Среда, в которой применение метода случайного блуждания для поиска цели требует выполнения такого количества этапов, которое определяется экспоненциальной зависимостью. Как оказалось, более эффективным подходом является дополнение метода поиска с восхождением к вершине способностью запоминать, а не способностью выбирать следующее действие случайным образом. Основная идея состоит в том, что необходимо хранить в памяти "текущую

наилучшую оценку" $H(s)$ стоимости достижения цели из каждого состояния, которое уже было посещено. На первых порах $H(s)$ представляет собой только эвристическую оценку $h(s)$ и обновляется по мере того, как агент приобретает опыт в исследовании пространства состояний. На рис.

4.15 показан простой пример одномерного пространства состояний. На рис. 4.15, а показано, что агент, по-видимому, зашел в тупик, попав в плоский локальный минимум, соответствующий затененному состоянию. Вместо того чтобы оставаться там, где находится, агент должен последовать по тому пути, который может рассматриваться как наилучший путь к цели согласно текущим оценкам стоимостей для соседних состояний. Оценка стоимости достижения цели через соседнее состояние s' равна оценке стоимости достижения s' , которая складывается с оценкой стоимости достижения цели из s^* , т.е. равна $c(s, a, s^*) + H(s^*)$. В данном примере имеются два действия, с оценками стоимости $1+9$ и $1+2$, поэтому, по всей видимости, лучше всего двигаться вправо. После этого становится очевидно, что оценка стоимости для 15. Этот вариант бесконечного пространства фактически является гораздо более сложным.

Методы случайного блуждания являются полными в бесконечных одной-двухмерных решетках, но не в трехмерных! В последнем случае вероятность того, что блуждающий агент в конечном итоге возвратится в начальную точку, составляет лишь около 0,3405 (для ознакомления с общим введением в эту тему см. [703]).

этого затененного состояния, равная 2, была слишком оптимистической.

Поскольку стоимость наилучшего хода равна 1 и он ведет в состояние, которое находится по меньшей мере на расстоянии 2 шагов от цели, то затененное состояние должно находиться по меньшей мере на расстоянии 3 шагов от цели, поэтому его оценка J должна быть обновлена соответствующим образом, как показано на рис.

4.15, б. Продолжая этот процесс, агент перейдет вперед и назад еще дважды, каждый раз обновляя оценку J и "сглаживая" локальный минимум до тех пор, пока не вырвется из него вправо.

Рис. 4.15. Пять итераций алгоритма $LRTA^*$ в одномерном пространстве состояний. Каждое состояние обозначено значением $H(s)$, текущей оценкой стоимости достижения цели, а каждая дуга обозначена соответствующей ей стоимостью этапа. Затененное состояние показывает местонахождение агента, а значения, обновленные после каждой итерации, обозначаются двойным кружком. Алгоритм агента, в котором реализована эта схема, получивший название поиска A^* в реальном времени с обучением (Learning Real-Time A^* — $LRTA^*$), показан в листинге 4.6. Как и в алгоритме Online-DFS-Agent, в данном алгоритме составляется карта среды с использованием таблицы result. Этот алгоритм обновляет оценку стоимости для состояния, которое он только что оставил, а затем выбирает ход,

"который представляется наилучшим" в соответствии с его текущими оценками стоимости. Следует отметить одну важную деталь, что действия, которые еще не были опробованы в состоянии s , всегда рассматриваются как ведущие непосредственно к цели с наименьшей возможной стоимостью, а именно $h(s)$. Такой ^ оптимизм в отношении неопределенности побуждает агента исследовать новые пути, которые могут действительно оказаться перспективными.

Листинг 4.6. Функция LRTA*-Agent выбирает действие в соответствии со значениями соседних состояний, которые обновляются по мере того, как агент передвигается в пространстве состояний function LRTA*-Agent(s') returns действие a inputs: s' , восприятие, позволяющее идентифицировать текущее состояние

```
static: result, таблица, индексированная по действиям и состояниям, первоначально
пустая Я, таблица оценок стоимостей, индексированная по состояниям,
первоначально пустая  $s$ ,  $a$ , предыдущие состояние и действие, первоначально
неопределенные if Goal-Test( $s'$ ) then return stop if  $s'$  является одним из новых
состояний (отсутствующим в Я) then  $H[s'] < rhis1$  ) unless  $s$  является неопределенным
 $result[a, s] \leftarrow s' H[s] \leftarrow \min LRTA^*-Cost(s, b, result[b, s], Я) b \in Actions(s)$   $a \leftarrow$  родно из
действий  $b$  среди действий  $Actions(s)$ / которое минимизирует значение  $LRTA^*-Cost(s', b, result[b, s], Я)$   $s \leftarrow s'$  return  $a$  function LRTA*-Cost( $s, a, s', Я$ ) returns оценка
стоимости if  $s'$  является неопределенным then return  $h(s)$  else return  $0E,3,5' + H[s']$ 
```

Гарантируется, что агент LRTA* найдет цель в любой конечной, безопасно исследуемой среде. Однако, в отличие от A*, в бесконечных пространствах состояний применяемый в этом агенте алгоритм становится неполным, поскольку в некоторых случаях этот алгоритм может вовлечь агента в бесконечные блуждания. В наихудшем случае данный алгоритм позволяет исследовать среду с p состояниями за $O(p^2)$ этапов, но чаще всего действует намного лучше. Агент LRTA* представляет собой лишь один пример из большого семейства агентов, выполняющих поиск в оперативном режиме, которые могут быть определены путем задания правила выбора действия и правила обновления различными способами. Это семейство агентов, которое было первоначально разработано для стохастических вариантов среды, рассматривается в главе 21.

Обучение в ходе поиска в оперативном режиме То, что агенты, выполняющие поиск в оперативном режиме, на первых порах находятся в полном неведении, открывает некоторые возможности для обучения. Во-первых, агенты изучают "карту" своей среды (а точнее, результаты каждого действия в каждом состоянии), регистрируя результаты каждого из своих опытов. (Обратите внимание на то, что из предположения о детерминированности среды следует, что для изучения любого действия достаточно проведения одного эксперимента.) Во-вторых, агенты, выполняющие локальный поиск, получают все более точные оценки значения

каждого состояния, используя локальные правила обновления, как в алгоритме LRTA*.

В главе 21 будет показано, что в конечном итоге такие обновления сходятся к точным значениям для каждого состояния, при условии, что агент исследует пространство состояний правильным способом. А после того как станут известными точные значения, оптимальные решения могут быть приняты путем перемещения к преемнику с наи-

высшим значением, т.е. в таком случае оптимальной стратегией становится метод поиска с восхождением к вершине в чистом виде.

Если читатель выполнил нашу рекомендацию провести трассировку поведения алгоритма Online-DFS-Agent в среде, показанной на рис. 4.12, то должен был заметить, что этот агент не слишком умен. Например, после того как агент обнаружил, что действие Up ведет из пункта A,1) в пункт A, 2), он еще не знает, что действие Down возвратит его назад в пункт A,1) или что следующее действие Up приведет его из пункта B,1) в пункт B, 2), из пункта B,2) в пункт B,3) и т.д. Вообще говоря, было бы желательно, чтобы агент освоил в результате обучения, что действие Up приводит к увеличению координаты y , если на этом пути нет стены, и что действие Down приводит к уменьшению этой координаты, и т.д. Для того чтобы это произошло, требуются две составляющие. Во-первых, необходимо формальное и явно манипулируемое представление общих правил такого рода; до сих пор мы скрывали эту информацию внутри "черного ящика", называемого функцией определения преемника. Данному вопросу посвящена часть III. Во-вторых, нужны алгоритмы, позволяющие формировать подходящие общие правила из конкретных наблюдений, сделанных агентом. Эта тема рассматривается в главе 18.

Резюме В данной главе рассматривалось применение эвристических функций для уменьшения стоимости поиска. В ней исследовался целый ряд алгоритмов, в которых используются эвристические функции, и было установлено, что даже при использовании хороших эвристических функций достижение оптимальности связано с увеличением стоимости поиска. • Поиск по первому наилучшему совпадению сводится просто к применению алгоритма Graph-Search, в котором для развертывания выбираются неразвернутые узлы с минимальной стоимостью (в соответствии с некоторым критерием). В алгоритмах поиска по первому наилучшему совпадению обычно используется эвристическая функция $h(n)$, которая оценивает стоимость достижения решения из узла n . • При жадном поиске по первому наилучшему совпадению развертываются узлы с минимальным значением $h(n)$. Этот поиск не оптимален, но часто является эффективным. • При поиске A^* развертываются узлы с минимальным значением $f(n)=g(n)+h(n)$. Алгоритм A^* является полным и оптимальным, при условии, что гарантирована допустимость (для

алгоритма Tree-Search) или преимущество (для алгоритма Graph-Search) функции $h(n)$.

Пространственная сложность алгоритма A^* все еще остается слишком высокой. • Производительность алгоритмов эвристического поиска зависит от качества эвристической функции. Иногда хорошие эвристические функции можно составить путем ослабления определения задачи, предварительного вычисления стоимости решения подзадач и сохранения этой информации в базе данных шаблонов или обучения на основе опыта решения данного класса задач.

• Алгоритмы RBFS и SMA* представляют собой надежные, оптимальные алгоритмы поиска, в которых используются ограниченные объемы памяти; при наличии достаточного количества времени эти алгоритмы могут решать такие задачи, которые не позволяет решать алгоритм A^* , поскольку исчерпывает доступную память. • Такие методы локального поиска, как поиск с восхождением к вершине, действуют на основе формулировок полного состояния и предусматривают хранение в памяти лишь небольшого количества узлов. Было разработано также несколько стохастических алгоритмов, включая поиск с эмуляцией отжига, которые возвращают оптимальные решения при наличии подходящего графика "охлаждения" (т.е. графика постепенного уменьшения величины случайного разброса). Кроме того, многие методы локального поиска могут использоваться для решения задач в непрерывных пространствах. • Генетический алгоритм представляет собой стохастический поиск с восхождением к вершине, в котором сопровождается большая популяция состояний.

Новые состояния формируются с помощью мутации и скрещивания; в последней операции комбинируются пары состояний, взятых из этой популяции. • Проблемы исследования возникают, если агент не имеет никакого представления о том, каковы состояния и действия в его среде. Для безопасно исследуемых вариантов среды агенты, выполняющие поиск в оперативном режиме, могут составить карту и найти цель, если она существует. Эффективным методом выхода из локальных минимумов является обновление эвристических оценок на основе опыта.

Библиографические и исторические заметки Пример использования эвристической информации при решении задач впервые появился в одной из ранних статей Саймона и Ньюэлла [1419], но само выражение "эвристический поиск" и обоснование применения эвристических функций, которые оценивают расстояние до цели, появились немного позже [932], [1126]. Доран и Мичи [404] осуществили обширные экспериментальные исследования в области применения эвристического поиска к решению ряда задач, в частности задач игры в восемь и игры в пятнадцать. Хотя Доран и Мичи провели теоретический анализ длины пути и "проникновения" (penetrance) (отношения длины пути к общему количеству узлов, исследованных к данному моменту) в процессе эвристического поиска, они, по-видимому,

игнорировали ту информацию, которую может предоставить текущая длина пути. Алгоритм A^* , предусматривающий использование в эвристическом поиске текущего значения длины пути, был разработан Хартом, Нильссоном и Рафаэлем [623], с некоторыми дальнейшими поправками [624]. Декстер и Перл [371] продемонстрировали оптимальную эффективность алгоритма A^* .

В указанной выше оригинальной статье, посвященной алгоритму A^* , было впервые представлено условие преемственности (consistency condition), которому должны удовлетворять эвристические функции. В качестве более простой замены этого условия Поллом [1223] было предложено условие монотонности, но Перл [1188] показал, что эти два условия эквивалентны. Аналоги открытых и закрытых списков использовались

во многих алгоритмах, явившихся предшественниками алгоритма A^* ; к ним относятся алгоритмы поиска в ширину, поиска в глубину и поиска по критерию стоимости [97], [399]. Важность применения аддитивных стоимостей путей для упрощения алгоритмов оптимизации особо ярко была показана в работах Беллмана [97].

Пол [1220], [1223] впервые провел исследования связи между ошибками в эвристических функциях и временной сложностью алгоритма A^* . Доказательство того, что работа алгоритма A^* завершается за линейное время, если ошибка в эвристической функции ограничена некоторой константой, можно найти в работах Пола [1223] и Гашнига [522]. Перл [1188] развил этот результат, что позволило учитывать логарифмический рост ошибки. Применение "эффективного коэффициента ветвления" в качестве критерия эффективности эвристического поиска было предложено Нильссоном [1141].

Существует много вариантов алгоритма A^* . Пол [1222] предложил использовать в алгоритме A^* динамическое взвешивание (dynamic weighting), в котором в качестве функции оценки применяется взвешенная сумма текущей длины пути и эвристической функции $fw(n) = w_{gg}\{n\} + w_{hh}\{n\}$, а не просто сумма $f(n) = d(n) + h(n)$. Веса w_g и w_h корректируются динамически по мере развития поиска. Можно доказать, что алгоритм Пола является ϵ -допустимым (т.е. гарантирует нахождение решений с отклонением от оптимального решения на коэффициент $1+\epsilon$), где ϵ — параметр, предусмотренный в алгоритме. Таким же свойством обладает алгоритм Ae^* [1188], который способен выбрать из периферии любой узел, при условии, что его f -стоимость находится в пределах коэффициента $1+\epsilon$ от стоимости периферийного узла с наименьшей f -стоимостью. Выбор соответствующего коэффициента может быть сделан таким образом, чтобы существовала возможность минимизировать стоимость поиска.

Алгоритм A^* и другие алгоритмы поиска в пространстве состояний тесно связаны с методами ветвей и границ, которые широко используются в исследованиях операций [901]. Соотношения между алгоритмами поиска в пространстве состояний и методами ветвей и границ были глубоко исследованы в [867], [869], [1115]. Мартелли и Монтанари [990] показали связь между динамическим программированием (см. главу 17) и некоторыми типами поиска в пространстве состояний. Кумар и Канал [868] предприняли попытку "великой унификации" методов эвристического поиска, динамического программирования, а также методов ветвей и границ под общим названием CDP (Composite Decision Process — комплексный процесс принятия решений).

Поскольку компьютеры в конце 1950-х — начале 1960-х годов имели не больше нескольких тысяч слов оперативной памяти, темой ранних исследовательских работ часто служил эвристический поиск с ограничением памяти. Одна из самых первых программ поиска, Graph Traverser [404], фиксирует свои результаты в виде некоторого оператора после выполнения поиска по первому наилучшему совпадению вплоть до заданного предела объема памяти. Алгоритм IDA^* [835], [836] стал одним из первых широко применяемых оптимальных алгоритмов эвристического поиска с ограничением памяти, после чего было разработано большое количество его вариантов. Анализ эффективности алгоритма IDA^* и сложностей, возникающих при его применении с эвристическими функциями, которые встречаются в реальных задачах, приведен в работе Патрика и др. [1182].

Алгоритм RBFS [840], [841] фактически является лишь немного более сложным по сравнению с алгоритмом, приведенным в листинге 4.1. Последний вари-

ант ближе к независимо разработанному алгоритму, получившему название алгоритма $\wedge$ итеративного развертывания, или сокращенно IE (Iterative Expansion) [1324].

В алгоритме RBFS используется не только верхняя, но и нижняя граница; эти два алгоритма при использовании допустимых эвристических функций ведут себя одинаково, но RBFS развертывает узлы в порядке первого наилучшего совпадения даже при наличии недопустимой эвристической функции. Идея отслеживания наилучшего альтернативного пути появилась еще раньше в изящной реализации алгоритма A^* на языке Prolog, предложенной Братко [174], и в алгоритме DTA^* [1332]. В последней работе обсуждаются также метауровневые пространства состояний и метауровневое обучение.

Алгоритм MA^* впервые опубликован в [229]. Появление алгоритма SMA^* , или Simplified MA^* , стало результатом попытки реализовать MA^* в качестве алгоритма сравнения для метода IE [1324]. Кайндл и Корсанд [763] применили алгоритм SMA^* для создания алгоритма двунаправленного поиска, который функционирует

значительно быстрее по сравнению с предшествующими алгоритмами. В [845] описан подход по принципу "разделяй и властвуй", а в [1645] представлен алгоритм A^* поиска в графе с ограничением памяти. Обзор методов поиска с ограничением памяти приведен в [842].

Идея о том, что допустимые эвристические функции могут быть получены с помощью ослабления условий задачи, впервые опубликована в оригинальной статье [645], авторы которой использовали эвристику на основе минимального связующего дерева для решения задачи TSP (см. упр. 4.8).

Автоматизация процесса ослабления условий задачи была успешно осуществлена Придитисом [1237] на основе его более ранней совместной работы с Мосту [1092].

Использование баз данных шаблонов для составления допустимых эвристических функций предложено в [313] и [523]; базы данных с непересекающимися шаблонами описаны в [844]. Вероятностная интерпретация эвристических функций глубоко исследована в [616] и [1188].

Безусловно, наиболее исчерпывающим источником сведений об эвристических функциях и алгоритмах эвристического поиска является книга Перла *Heuristics* [1188]. В этой книге приведено особенно хорошее описание широкого разнообразия версий и вариантов алгоритма A^* , включая строгие доказательства их формальных свойств. В работе Канала и Кумара представлена антология важных статей по эвристическому поиску [767]. Результаты новейших исследований в области алгоритмов поиска регулярно публикуются в журнале *Artificial Intelligence*.

Методы локального поиска имеют долгую историю в математике и компьютерных науках. Безусловно, как очень эффективный метод локального поиска в непрерывных пространствах, в которых доступна информация о градиенте, может рассматриваться метод Ньютона—Рафсона [1132], [1266]. В [182] можно найти классический справочник по алгоритмам оптимизации, для которых не требуется такая информация. Алгоритм лучевого поиска, представленный в данной книге как алгоритм локального поиска, впервые появился в виде одного из вариантов методов динамического программирования с ограничением по ширине, предназначенного для распознавания речи, в системе Harpy [952]. Еще один алгоритм подобного типа был В последние годы снова пробудился интерес к теме локального поиска под влиянием исключительно качественных результатов, полученных при решении таких круп-

ных задач удовлетворения ограничений, как задача с p -ферзями [1058] и задача формирования логических рассуждений [1382], а также под влиянием успешного включения в алгоритмы методов рандомизации, методов множественного одновременного поиска и других усовершенствований. Это возрождение интереса к алгоритмам, названным Кростосом Пападимитриу алгоритмами "новой эры",

вызвало также повышенный интерес теоретиков в области компьютерных наук [13], [848]. В области исследования операций завоевал широкую популярность один из вариантов поиска с восхождением к вершине, получивший название $\wedge$ поиска с запретами [563], [564].

В этом алгоритме, основанном на моделях ограниченной кратковременной памяти человека, ведется список запретов, состоящий из k ранее посещенных состояний, которые нельзя посещать повторно; это позволяет данному алгоритму не только выполнять поиск в графах более эффективно, но и выходить из некоторых локальных минимумов. Еще одним полезным усовершенствованием алгоритма поиска с восхождением к вершине является алгоритм Stage [163]. Идея этого алгоритма состоит в том, что для получения представления об общей форме ландшафта должны использоваться локальные максимумы, найденные в результате восхождения к вершине с перезапуском случайным образом. Данный алгоритм согласует гладкую поверхность с множеством локальных максимумов, а затем аналитическим путем вычисляет глобальный максимум этой поверхности. Полученный глобальный максимум становится новой точкой перезапуска. Успешное функционирование этого алгоритма было испытано на практике при решении трудных задач. В [573] показано, что распределения времени выполнения алгоритмов, в которых систематически применяются возвраты, часто имеют форму $\wedge$ распределений с широким хвостом (heavy-tailed distribution), а это означает, что вероятность очень продолжительного времени выполнения выше, чем можно было бы предположить в случае нормального распределения значений времени выполнения. Данные результаты могут служить теоретическим обоснованием применимости алгоритмов с перезапуском случайным образом.

Метод поиска с эмуляцией отжига был впервые описан Кирпатриком и др. [799], которые заимствовали соответствующую идею непосредственно из алгоритма Метрополиса (этот алгоритм применялся для эмуляции сложных систем в физике [1036] и был, как многие полагают, изобретен на одном из званых обедов в Лос-Аламосе).

Методы поиска с эмуляцией отжига теперь лежат в основе отдельного научного направления, в котором ежегодно публикуются сотни статей.

Поиск оптимальных решений в непрерывных пространствах является предметом исследований в нескольких научных областях, включая теорию оптимизации, теорию оптимального управления и вариационное исчисление. Подходящие для нашей темы (и практически применимые) вступительные сведения приведены в [133] и [1236].

Одним из первых приложений для компьютеров было линейное программирование (Linear Programming— LP); относящийся к этой области симплексный алгоритм [322],

[1611] все еще используется, несмотря на то, что в наихудшем случае он характеризуется экспоненциальной сложностью. Кармаркар [771] разработал практически применимый алгоритм для задач LP с полиномиальной временной сложностью.

Важной работой, предшествующей разработке генетических алгоритмов, было исследование концепции ландшафта пригодности (fitness landscape), проведенное Сьюэллом Райтом [1623]. В 1950-х годах некоторые специалисты в области статистики, включая Бокса [161] и Фридмана [504], использовали эволюционные методы для решения задач оптимизации, но этот подход приобрел популярность только по-

сле того, как Рехенберг [1270], [1271] предложил использовать $\wedge$ стратегии эволюции (evolution strategy) для решения задач оптимизации аэродинамических поверхностей.

В 1960-х и 1970-х годах генетические алгоритмы глубоко исследовал Джон Холланд [669], применяя их и в качестве полезного инструментального средства, и в качестве способа расширения знаний в области адаптации, как биологической, так и связанной с другими научными направлениями [670]. Сторонники движения, доказывающие возможность $\wedge$ искусственной жизни [886], развили эту идею на один шаг дальше, рассматривая продукты генетических алгоритмов как организмы, а не как решения задач. В результате исследований в этой области, проведенных Хинтоном и Ноуланом [656], а также Экли и Литтманом [3], было сделано очень многое для выяснения последствий действия эффекта Болдуина. Для ознакомления с общими сведениями об эволюции авторы настоятельно рекомендуют книгу [1439].

Сравнение генетических алгоритмов с другими подходами (особенно с алгоритмом стохастического поиска с восхождением к вершине) чаще всего показывает, что генетические алгоритмы сходятся более медленно [66], [754], [1060], [1158]. Такие исследования не находят всеобщего признания в сообществе приверженцев алгоритмов GA, но недавние попытки представителей этого сообщества трактовать поиск на основе популяций как приближенный аналог байесовского обучения (см. главу 20) могут способствовать преодолению разногласий между представителями этой области и ее критиками [1200]. Для анализа производительности алгоритмов GA может также применяться теория квадратичных динамических систем [1262].

В [944] можно найти пример применения GA для проектирования антенн, а в [890] — пример применения этих алгоритмов для решения задачи коммивояжера.

С генетическими алгоритмами тесно связана область $\wedge$ генетического программирования. Принципиальное различие между ними состоит в том, что представлениями, к которым применяются операции мутации и комбинирования, являются программы, а не битовые строки. Программы представлены в форме

деревьев выражений; выражения могут быть сформированы на таком стандартном языке, как Lisp, или специально спроектированы для представления технических схем, контроллеров роботов и т.д. Операция скрещивания предусматривает соединение друг с другом поддеревьев, а не подстрок. Такая форма мутации гарантирует, что потомки будут представлять собой правильно построенные выражения, а это не было бы возможно в случае проведения манипуляций с программами, представленными в виде строк.

Недавно возникший широкий интерес к генетическому программированию был стимулирован работой Джона Козы [855], [856], но исследования в этой области проводились уже давно, начиная с экспериментов с машинным кодом [502] и с конечными автоматами [476]. Как и в отношении генетических алгоритмов, еще не утихли споры по поводу эффективности методов генетического программирования. В [857] описаны результаты ряда экспериментов по автоматизированному проектированию схмотехнических устройств с использованием генетического программирования.

Работы в области генетических алгоритмов и генетического программирования публикуются в журналах *Evolutionary Computation* и *IEEE Transactions on Evolutionary Computation*] статьи на эти темы можно также найти в журналах *Complex Systems*, *Adaptive Behavior* и *Artificial Life*. Основными конференциями являются *International Conference on Genetic Algorithms* и *Conference on Genetic Programming*, а недавно произошло их слияние и создана конференция *Genetic and Evolutionary Computation Conference*. Хорошие обзоры этой области приведены в книгах Мелани Митчелл [1059] и Дэвида Фогеля [475].

Алгоритмы исследования неизвестных пространств состояний привлекали интерес ученых в течение многих столетий. Поиск в глубину в лабиринте можно осуществить, постоянно держась левой рукой за стену; циклов можно избежать, отмечая каждую развилку. При наличии необратимых действий поиск в глубину оканчивается неудачей; более общая задача исследования $\wedge$ графов Эйлера (т.е. графов, в которых каждый узел имеет одинаковое число входящих и исходящих ребер) была решена с помощью алгоритма, предложенного Хирхольцером [652]. Первое исчерпывающее описание алгоритмов решения задачи исследования для произвольных графов было приведено в [385]; авторы этой работы предложили полностью общий алгоритм, но показали, что невозможно определить ограниченный коэффициент конкурентоспособности для задачи исследования графа общего вида. В [1171] приведены результаты исследования проблемы поиска путей к цели в геометрических вариантах среды планирования пути (где все действия являются обратимыми). Эти результаты показали, что достичь того, чтобы коэффициент конкурентоспособности оставался небольшим, можно при наличии квадратных препятствий, но если препятствия имеют произвольную прямоугольную форму, то

невозможно добиться того, чтобы значения коэффициента конкурентоспособности оставались ограниченными (см. рис. 4.13).

Алгоритм LRTA* был разработан Корфом [839] в ходе исследований в области $\wedge$ поиска в реальном времени для вариантов среды, в которых агент должен действовать после проведения поиска лишь в течение ограниченного времени (эта ситуация встречается гораздо чаще в играх с двумя участниками). По сути, алгоритм LRTA* представляет собой частный случай алгоритмов обучения с подкреплением для стохастических вариантов среды [74]. Применяемый в нем принцип оптимистического отношения к неопределенности (согласно которому следует всегда отправляться к ближайшему непосещенному состоянию) может в случае отсутствия информации о среде привести к формированию такого подхода к исследованию, который является менее эффективным по сравнению с простым поиском в глубину [818]. В [327] показано, что поиск с итеративным углублением в оперативном режиме обладает оптимальной эффективностью поиска цели в однородном дереве при отсутствии эвристической информации. В сочетании с различными методами поиска и обновления в известной части графа было разработано несколько вариантов алгоритма LRTA*, предусматривающих использование информации о среде [1201]. Тем не менее еще нет полного понимания того, как следует находить цели с оптимальной эффективностью при использовании эвристической информации.

Тема алгоритмов $\wedge$ параллельного поиска в этой главе не рассматривалась, отчасти потому, что для этого требуется привести подробное описание параллельных компьютерных архитектур. Параллельный поиск становится важной темой и в искусственном интеллекте, и в теоретических компьютерных науках. Краткое введение в тематику литературы по искусственному интеллекту можно найти в книге Маханты и Дэниелса [969].

Упражнения 4.1. Выполните трассировку работы алгоритма поиска A^* применительно к решению задачи проезда в город Бухарест из города Лугож с использованием эвристической функции определения расстояния по прямой. Иными словами, по-

кажите последовательность узлов, которые рассматриваются этим алгоритмом, и приведите оценки f , d и h для каждого узла.

4.2. Эвристический алгоритм поиска пути представляет собой алгоритм поиска по первому наилучшему совпадению, в котором целевая функция равна $f(n) = \{2-w\}g(n) + wh(n)$. Для каких значений w гарантируется оптимальность этого алгоритма? (Может быть принято предположение, что функция h является допустимой.) Какого рода поиск выполняется в этом алгоритме, когда $w=0$? Когда $w=1$? Когда $w=2$?

4.3. Докажите каждое из приведенных ниже утверждений. а) Поиск в ширину представляет собой частный случай поиска по критерию стоимости. б) Поиск в ширину, поиск в глубину и поиск по критерию стоимости (uniform-cost search) — специальные случаи поиска по первому наилучшему совпадению. в) Поиск по критерию стоимости — специальный случай поиска A^* .

4.4. (§E) Предложите такое пространство состояний, в котором применение поиска A^* с использованием алгоритма Graph-Search приводит к получению неоптимального решения при наличии функции $h(n)$, которая является допустимой, но непреемственной.

4.5. На с. 156 было показано, что применение эвристической функции определения расстояния по прямой приводит к тому, что жадный поиск по первому наилучшему совпадению безвозвратно углубляется в дерево поиска при решении задачи проезда от города Яссы до города Фэгэраш. Однако эта эвристическая функция работает идеально при решении противоположной задачи — поиска пути проезда от города Фэгэраш до города Яссы. Существуют ли такие задачи, в которых эта эвристическая функция исключает возможность успешного выполнения задачи поиска пути в любом из направлений?

4.6. Предложите эвристическую функцию для задачи игры в восемь, которая иногда допускает переоценку, и покажите, каким образом это может привести к получению неоптимального решения некоторых конкретных экземпляров задачи. (Для упрощения работы при выполнении упражнения можете использовать компьютер.) Докажите, что если функция h никогда не переоценивает стоимость больше чем на c , то алгоритм A^* с использованием функции h возвращает решение, стоимость которого превышает оптимальное решение не больше чем на c .

4.7. Докажите, что если эвристическая функция является преемственной, то она должна быть допустимой. Составьте допустимую эвристическую функцию, которая не является преемственной.

4.8. (§D) Задача коммивояжера (Traveling Salesperson Problem — TSP) может быть решена с помощью эвристической функции, основанной на определении минимального связующего дерева (Minimum Spanning Tree — MST), которая используется для оценки стоимости завершения обхода, при условии, что уже был сформирован частичный путь обхода. Стоимость MST для множества городов представляет собой минимальную сумму стоимостей соединений в любом дереве, которое связывает все города.

а) Покажите, как можно вывести эту эвристическую функцию на основе одной из ослабленных версий задачи TSP. б) Покажите, что эвристическая функция MST доминирует над эвристической функцией определения расстояния по прямой. в) Напишите генератор задач для создания экземпляров задачи TSP, в которых города

представлены случайно выбранными точками в единичном квадрате. г) Найдите в литературе эффективный алгоритм формирования MST и примените его в сочетании с допустимым алгоритмом поиска для решения экземпляров задачи TSP.

4.9. На с. 171 был определен ослабленный вариант задачи игры в восемь, в котором любая фишка может перемещаться из квадрата A в квадрат B, если квадрат B является пустым. Точное решение этой задачи предусматривает определение эвристической функции Гашнига [522]. Объясните, почему эвристическая функция Гашнига является, по меньшей мере, такой же точной, как и функция $h_{\pm}$ (в которой учитываются фишки, стоящие не на месте), и продемонстрируйте случай, в котором она является более точной по сравнению с h_1 и с h_2 (в которой используется манхэттенское расстояние). Можете ли вы предложить способ эффективного вычисления эвристической функции Гашнига?

4.10. (И) В данной главе были описаны две простые эвристические функции для задачи игры в восемь; в одной из них учитывается манхэттенское расстояние, а в другой — стоящие не на месте фишки. В литературе предложено также несколько других эвристических функций, предназначенных для использования в качестве улучшенной замены этих двух (см., например, [617], [1092] и [1141]).

Проверьте обоснованность этих претензий, реализовав соответствующие эвристические функции и сравнив производительность полученных алгоритмов.

4.11. Укажите название алгоритма, который фактически становится следствием применения каждого из перечисленных ниже частных случаев. а) Локальный лучевой поиск $sk=1$. б) Локальный лучевой поиск с одним начальным состоянием и без ограничений на количество хранимых состояний. в) Моделируемый отжиг с постоянным значением $T=0$ (и исключенной проверкой завершения). г) Генетический алгоритм с размером популяции $N=1$.

4.12. Иногда не существует хорошей функции оценки для некоторой задачи, но имеется хороший метод сравнения — способ определения того, является ли один узел лучше другого, без присваивания числовых значений тому и другому. Покажите, что этого достаточно для выполнения поиска по первому наилучшему совпадению. Есть ли соответствующий аналог алгоритма A^* ?

4.13. Определите, как связана временная сложность алгоритма $LRTA^*$ с его пространственной сложностью.

4.14. Предположим, что некоторый агент находится в среде лабиринта 3×3 , подобного приведенному на рис. 4.12. Агенту известно, что его первоначальным местонахождением является пункт A,1), что цель находится в пункте C,3) и

что четыре действия, Up, Down, Left, Right, приводят к своим обычным последствиям, если не будут заблокированы стеной. Агент не знает о том, где находятся внутренние

стены. В любом конкретном состоянии агент воспринимает информацию о множестве допустимых действий; он также может определить, является ли данное состояние тем, которое он уже посещал ранее, или представляет собой новое состояние. а) Объясните, каким образом к этой задаче поиска в оперативном режиме можно применить такую трактовку, чтобы она могла рассматриваться как поиск в автономном режиме в пространстве доверительных состояний, где первоначальное доверительное состояние включает все возможные конфигурации среды. Насколько велико это первоначальное доверительное состояние? Насколько велико все пространство доверительных состояний? б) Какое количество различных вариантов восприятия возможно в первоначальном состоянии? в) Опишите первые несколько ветвей плана действий в непредвиденных ситуациях для этой задачи. Насколько велик этот план в полном виде (дайте приблизительную оценку)?

Обратите внимание на то, что этот план действий в непредвиденных ситуациях является решением для каждой возможной среды, соответствующей данному описанию. Поэтому, строго говоря, чередование поиска и выполнения не требуется даже в неизвестных вариантах среды.

4.15. (И) В данном упражнении исследуется направление использования локальных методов поиска для решения задач TSP такого типа, который определен в упр. 4.8. а) Предложите подход к решению задач TSP на основе поиска с восхождением к вершине. Сравните эти результаты с оптимальными решениями, полученными на основе алгоритма A^* с эвристической функцией MST (упр. 4.8). б) Предложите подход к решению задачи коммивояжера с помощью генетического алгоритма. Сравните результаты обоих подходов. Для ознакомления с некоторыми рекомендациями по выбору представлений вы можете обратиться к [890].

4.16. (ё) Сформируйте большое количество экземпляров игры в восемь и задачи с восемью ферзями и найдите их решения (там, где это возможно) с помощью поиска с восхождением к вершине (в варианте подъема по самому крутому склону и в варианте поиска по первому наилучшему совпадению), поиска с восхождением к вершине с перезапуском случайным образом и поиска с эмуляцией отжига. Измерьте стоимость поиска и процентное соотношение решенных задач, а также нанесите эти данные на график наряду с данными об оптимальной стоимости решения. Прокомментируйте полученные вами результаты.

4.17. ds В данном упражнении исследуется применение поиска с восхождением к вершине в контексте робототехнической навигации, с использованием в качестве примера среды, показанной на рис. 3.14. а) Повторите упр. 3.16 с использованием поиска с восхождением к вершине.

Зашел ли когда-либо предложенный вами агент в тупик в локальном ми-

нимуме? Возможно ли, чтобы он зашел в тупик при наличии выпуклых препятствий? б) Сформируйте среду с препятствиями в виде невыпуклых многоугольников, в которой агент может оказаться в тупике. в) Модифицируйте алгоритм поиска с восхождением к вершине таким образом, чтобы вместо выполнения поиска на глубину 1 для принятия решения о том, куда двигаться дальше, агент предпринимал бы поиск на глубину k . Он должен находить наилучший путь, состоящий из k этапов, проходить по нему на один этап, а затем повторять процесс. г) Существует ли такое значение k , при котором новый алгоритм гарантирует выход из локальных минимумов? д) Объясните, благодаря чему алгоритм LRTA* позволяет агенту выходить из локальных минимумов в этом случае.

4.18. (И) Сравните производительность алгоритмов A* и RBFS на множестве случайно сформированных экземпляров задач в проблемных областях задачи игры в восемь (с манхэттенским расстоянием) и задачи TSP (с MST — см. упр. 4.8).

Обсудите полученные вами результаты. Как изменяется производительность алгоритма RBFS после добавления небольшого случайного числа к эвристическим значениям в проблемной области задачи игры в восемь?

Глава 5. Задачи удовлетворения ограничений

5 Задачи ограничений Бэтой главе показано, что, рассматривая состояния на более высоком уровне детализации, чем просто как маленькие "черные ящики", можно прийти к созданию целого ряда мощных новых методов поиска и более глубокому пониманию структуры и сложности задачи.

В главах 3 и 4 рассматривался подход, согласно которому задачи можно решать, выполняя поиск в пространстве состояний. Для реализации этого подхода состояния необходимо оценивать с помощью эвристических функций, соответствующих данной проблемной области, а также проверять для определения того, не являются ли они целевыми состояниями. Однако с точки зрения таких алгоритмов поиска каждое состояние представляет собой ^черный ящик с неразличимой внутренней структурой. Они представлены с помощью произвольно выбранной структуры данных, доступ к которой может осуществляться только с применением процедур, относящихся к данной проблемной области, — функции определения преемника, эвристической функции и процедуры проверки цели.

В настоящей главе рассматриваются задачи удовлетворения ограничений, в которых состояния и проверка цели соответствуют стандартному, структурированному и очень простому ^представлению (см. раздел 5.1). Это позволяет определять алгоритмы поиска, способные воспользоваться преимуществом такой структуры состояний, и применять для решения крупных задач эвристические функции общего назначения, а не функции, относящиеся к конкретной задаче (см. разделы 5.2, 5.3).

Но, возможно, еще более важно то, что применяемое стандартное представление проверки цели раскрывает структуру самой задачи (см. раздел 5.4). Это позволяет создать методы декомпозиции задачи и понять внутреннюю связь между структурой задачи и сложностью ее решения.

5.1. Задачи удовлетворения ограничений Формально говоря, $\wedge$ любая задача удовлетворения ограничений (Constraint Satisfaction Problem— CSP) определена множеством $\wedge$ переменных, $x_1, x_2, \dots, x_n$, и множеством $\wedge$ ограничений, $C_1, C_2, \dots, C_m$. Каждая переменная x_i имеет непустую $\wedge$ область определения D_i возможных $\wedge$ значений. Каждое ограничение C_i включает некоторое подмножество переменных и задает допустимые комбинации значений для этого подмножества. Состояние задачи определяется путем $\wedge$ присваивания значений некоторым или всем этим переменным, $\{x_i = v_i, x_j = v_j, \dots\}$. Присваивание, которое не нарушает никаких ограничений, называется $\wedge$ совместным, или допустимым присваиванием. Полным называется такое присваивание, в котором участвует каждая переменная, а решением задачи CSP является полное присваивание, которое удовлетворяет всем ограничениям. Кроме того, для некоторых задач CSP требуется найти решение, которое максимизирует $\wedge$ целевую функцию.

Как же все это описание перевести на язык практики? Предположим, что устав от Румынии, мы рассматриваем карту Австралии, на которой показаны все ее штаты и территории (рис. 5.1, а), и что мы получили задание раскрасить каждый регион в красный, зеленый или синий цвет таким способом, чтобы ни одна пара соседних регионов не имела одинаковый цвет. Чтобы сформулировать это задание в виде задачи CSP, определим в качестве переменных сокращенные обозначения этих регионов: wa, NT, o, NSW, v, SA и т. Областью определения каждой переменной является множество цветов {red, green, blue}. Ограничения требуют, чтобы все пары соседних регионов имели разные цвета; например, допустимыми комбинациями для WA и NT являются следующие пары: {{red, green}, {red, blue}, {green, red}, {green, blue}, {blue, red}, {blue, green}} (Это ограничение можно также представить более сжато в виде неравенства $WA \neq NT$, при условии, что в данном алгоритме удовлетворения ограничений предусмотрен некоторый способ вычисления таких выражений.) Количество возможных решений достаточно велико; например, одним из таких решений является следующее: {WA=red, NT= green, Q=red, NSW=green, V=red, SA=blue, T=red} Задачу CSP удобно представить визуально в виде $\wedge$ графа ограничений, как показано на рис. 5.1, б. Узлы этого графа соответствуют переменным задачи, а дуги — ограничениям.

Рассматривая некоторую задачу в виде задачи CSP, можно достичь нескольких важных преимуществ. Представление состояний в задаче CSP соответствует некоторому стандартному шаблону (т.е. выражается в виде множества переменных с присвоенными значениями), поэтому функцию определения преемника и

проверку цели можно записать в универсальной форме, применимой ко всем задачам CSP.

Более того, могут быть разработаны эффективные, универсальные эвристические функции, для создания которых не требуются дополнительные знания о конкретной проблемной области. Наконец, для упрощения процесса решения может использоваться сама структура графа ограничений, что позволяет в некоторых случаях добиться экспоненциального уменьшения сложности. Это представление задачи CSP является первым и простейшим из ряда схем представления, которые будут разрабатываться на протяжении данной книги.

Тасмания а) б) Рис. 5.1. Пример определения задачи CSP: основные штаты и территории на карте Австралии (а); раскраска этой карты может рассматриваться как задача удовлетворения ограничений; задание состоит в том, чтобы назначить цвет каждому региону так, что ни одна пара соседних регионов не будет иметь одинаковый цвет; рассматриваемая задача раскраски карты, представленная в виде графа ограничений (б) Можно довольно легко показать, что любой задаче CSP может быть дана инкрементная формулировка, как и любой стандартной задаче поиска, следующим образом. • Начальное состояние. Пустое присваивание {}, в котором ни одной переменной не присвоено значение. • Функция определения преемника. Значение может быть присвоено любой переменной с неприсвоенным значением, при условии, что переменная не будет конфликтовать с другими переменными, значения которым были присвоены ранее. • Проверка цели. Текущее присваивание является полным. • Стоимость пути. Постоянная стоимость (например, 1) для каждого этапа.

Каждое решение должно представлять собой полное присваивание и поэтому должно находиться на глубине n , если имеется n переменных. Кроме того, дерево поиска распространяется только на глубину n . По этим причинам для решения задач CSP широко применяются алгоритмы поиска в глубину (см. раздел 5.2). К тому же такие задачи характерны тем, что s^* = сам путь, по которому достигается некоторое решение, не представляет интереса. Поэтому может также использоваться формулировка с полным состоянием, в которой каждое состояние представляет собой полное присваивание, удовлетворяющее или не удовлетворяющее ограничениям. На основе такой формулировки хорошо действуют методы локального поиска (см. раздел 5.3).

Задачи CSP простейшего вида характеризуются тем, что в них используются переменные, которые являются дискретными и имеют $\wedge$ конечные области определения. К такому виду относится задача раскраски карты. Задача с восемью ферзями, описанная в главе 3, также может рассматриваться как задача CSP с конечной областью определения, где переменные $Q_1, \dots, Q_8$ представляют собой

позиции каждого ферзя на столбцах 1,...,8, а каждая переменная имеет область определения {1,2,3,4,5,6,7,8}. Если максимальный размер области определения любой переменной в задаче CSP равен d , то количество возможных полных присваиваний измеряется величиной $O(d^n)$, т.е. зависит экспоненциально от количества переменных. К категории задач CSP с конечной областью определения относятся $\wedge$ булевы задачи CSP, в которых переменные могут иметь значения либо true, либо false. Булевы задачи CSP включают в качестве частных случаев некоторые NP-полные задачи, такие как 3SAT (см. главу 7).

Поэтому в худшем случае нельзя рассчитывать на то, что мы сможем решать задачи CSP с конечной областью определения за время меньше экспоненциального. Но в большинстве практических приложений алгоритмы CSP общего назначения позволяют решать задачи, на несколько порядков величины более крупные по сравнению с теми, которые могут быть решены с помощью алгоритмов поиска общего назначения, представленных в главе 3.

Дискретные переменные могут также иметь $\wedge$ бесконечные области определения, например, такие, как множество всех целых чисел или множество всех строк. В частности, при календарном планировании строительных работ дата начала каждой работы представляет собой переменную, а ее возможными значениями являются целочисленные интервалы времени в сутках, отсчитываемые от текущей даты. При решении задач с бесконечными областями определения больше нет возможности описывать ограничения, перечисляя все допустимые комбинации значений. Вместо этого должен использоваться $\wedge$ язык ограничений. Например, если работа Job¹ которая занимает пять дней, должна предшествовать работе Job³, то для описания этого условия потребуется язык ограничений, представленных в виде алгебраических неравенств, таких как $StartJob^1 + 5 < StartJob^3$. Кроме того, больше нет возможности решать такие ограничения, просто перечисляя все возможные присваивания, поскольку количество подобных возможных присваиваний бесконечно велико. Для $\wedge$ линейных ограничений с целочисленными переменными (т.е. ограничений, подобных только что приведенному, в которых каждая переменная представлена только в линейной форме) существуют специальные алгоритмы поиска решений (которые здесь не рассматриваются). Можно доказать, что не существует алгоритмов решения общих $\wedge$ нелинейных ограничений с целочисленными переменными. В некоторых случаях задачи с целочисленными ограничениями можно свести к задачам с конечной областью определения, устанавливая пределы значений всех этих переменных. Например, в задаче планирования можно установить верхний предел, равный общей продолжительности всех работ, которые должны быть запланированы.

В реальном мире очень часто встречаются задачи удовлетворения ограничений с "JSk непрерывными областями определения, и эти задачи интенсивно изучаются в

области исследования операций. Например, для планирования экспериментов на космическом телескопе Хаббл требуется очень точная привязка наблюдений по времени; начало и конец каждого наблюдения и каждого маневра представляют собой переменные со значениями из непрерывной области опре-

деления, которые должны подчиняться всевозможным астрономическим ограничениям, ограничениям предшествования и ограничениям по мощности двигателей. Одной из широко известных категорий задач CSP с непрерывной областью определения являются задачи $\wedge$ линейного программирования, в которых ограничения должны представлять собой линейные неравенства, образующие выпуклую область. Задачи линейного программирования могут быть решены за время, которое зависит полиномиально от количества переменных. Кроме того, проводились исследования задач с другими типами ограничений и целевых функций — задачи квадратичного программирования, конического программирования второго порядка и т.д.

Кроме исследования типов переменных, которые могут присутствовать в задачах CSP, полезно заняться изучением типов ограничений. Простейшим типом ограничения является $\wedge$ унарное ограничение, которое ограничивает значение единственной переменной. Например, может оказаться, что жители штата Южная Австралия очень не любят зеленый цвет, {green}. Каждое унарное ограничение можно устранить, выполняя предварительную обработку области определения соответствующей переменной, чтобы удалить любое значение, нарушающее это ограничение. $\wedge$ Бинарное ограничение связывает между собой две переменные. Например, бинарным ограничением является SAтNSW. Бинарной задачей CSP называется задача, в которой предусмотрены только бинарные ограничения; она может быть представлена в виде графа ограничений, подобного приведенному на рис. 5.1,5.

В ограничениях высокого порядка участвуют три или больше переменных.

Одним из известных примеров таких задач являются $\wedge$ криптоарифметические головоломки, называемые также числовыми ребусами (рис. 5.2, а). Обычно предъявляется требование, чтобы каждая буква в криптоарифметической головоломке представляла отдельную цифру. В случае задачи, показанной на рис. 5.2, а, такое требование может быть выражено с помощью ограничения с шестью переменными AllDiff(F, T, U, W, R, O). Иным образом это требование может быть представлено в виде коллекций бинарных ограничений, таких как $F \neq T$. Ограничения сложения для четырех столбцов этой головоломки также включают несколько переменных и могут быть записаны следующим образом:

$O + O = R + 10 \cdot X_1$
 $X_1 + w + w = i + 10 \cdot x_2$
 $X_2 + T + T = O + 10 \cdot x_3$
 $x_3 = F$ где x_1, x_2, x_3 — вспомогательные переменные, представляющие цифру @ или 1), которая

переносится в следующий столбец. Ограничения высокого порядка могут быть представлены в виде $\wedge$ гиперграфа ограничений, подобного приведенному на рис. 5.2, б. Внимательный читатель заметит, что ограничение Alldiff может быть разбито на бинарные ограничения— $f \neq t$, $F \neq uk$ т.д. И действительно, в упр. 5.11 предлагается доказать, что каждое ограничение высокого порядка с конечной областью определения можно свести к множеству бинарных ограничений, введя достаточное количество вспомогательных переменных. В связи с этим в данной главе будут рассматриваться только бинарные ограничения.

TWO + T W o FOUR а) б) Рис. 5.2. Пример криптоарифметической задачи: каждая буква обозначает отдельную цифру; задание состоит в том, чтобы найти такую замену букв цифрами, чтобы результирующее выражение суммирования было арифметически правильным, с дополнительным ограничением, что наличие ведущих нулей не допускается (а); гиперграф ограничений для этой криптоарифметической задачи, на котором показано ограничение Alldiff f , а также ограничения сложения по столбцам; каждое ограничение обозначено квадратом, который соединен с ограничиваемыми им переменными (б) Все ограничения, рассматривавшиеся до сих пор, были абсолютными ограничениями, нарушение которых равносильно тому, что какое-то потенциальное решение больше не рассматривается как таковое. С другой стороны, во многих реальных задачах CSP применяются ограничения $\wedge$ предпочтения, которые указывают, какие решения являются предпочтительными. Например, в задаче составления расписания занятий в университете может потребоваться учесть, что профессор x предпочитает проводить занятия по утрам, а профессор y — после полудня. Расписание занятий, в котором предусмотрено проведение профессором x занятия в 14:00, все еще может рассматриваться как решение (если не окажется, что профессор x должен в это время председательствовать на заседании кафедры), но уже не будет оптимальным. Ограничения предпочтения часто можно закодировать как стоимости присваиваний отдельных переменных; например, за назначение профессору x послеполуденного интервала придется заплатить 2 пункта, которые будут учтены в суммарном значении целевой функции, в то время как утренний интервал имеет стоимость 1 пункт. При использовании такой формулировки задачи CSP с предпочтениями можно решать, используя методы поиска с оптимизацией, либо основанные на поиске пути, либо локальные. Подобные задачи CSP в данной главе больше не рассматриваются, но в разделе с библиографическими замечками приведены некоторые ссылки.

5.2. Применение поиска с возвратами для решения задач CSP В предыдущем разделе приведена формулировка задач CSP в виде задач поиска.

Если используется такая формулировка, то появляется возможность решать задачи CSP с помощью любых алгоритмов поиска, приведенных в главах 3 и 4.

Предположим, что мы применяем алгоритм поиска в ширину к универсальной формулировке задачи CSP, приведенной в предыдущем разделе. Но мы быстро обнаружим нечто ужасное: коэффициент ветвления на верхнем уровне равен pd , поскольку любое из d значений может быть присвоено любой из p переменных. На следующем уровне коэффициент ветвления равен $(p-1)d$ и т.д., на всех p уровнях. При этом создается дерево $p^{p-1}d^p$ ветвями, даже несмотря на то, что имеется только d возможных полных присваиваний!

В применяемой нами формулировке задачи, которая внешне казалась разумной, но оказалась плохо продуманной, не было учтено существенно важное свойство, общее для всех задач CSP, — $\wedge$ коммутативность. Задача называется коммутативной, если порядок применения любого конкретного множества действий в процессе ее решения не влияет на результат. Именно это свойство характерно для задач CSP, поскольку, присваивая значения переменным, мы достигаем одного и того же частичного присваивания независимо от порядка присваивания. Поэтому $\wedge$ во всех алгоритмах поиска CSP преемники формируются с учетом возможных присваиваний только для единственной переменной в каждом узле дерева поиска. Например, в корневом узле дерева поиска в задаче раскрашивания карты Австралии можно иметь выбор между $SA=red$, $SA=green$ и $SA=blue$, но мы никогда не должны выбирать между $SA=red$ и $WA=blue$.

Определив такое условие, можно надеяться сократить количество листьев до d^p .

Поиск в глубину, в котором каждый раз выбираются значения для одной переменной и выполняется возврат, если больше не остается допустимых значений, которые можно было бы присвоить переменной, называется $\wedge$ поиском с возвратами. Этот алгоритм поиска приведен в листинге 5.1. Обратите внимание на то, что в этом алгоритме по существу используется метод инкрементного формирования преемников по одному преемнику за один раз, который описан на с. 131. Кроме того, для формирования преемника текущее присваивание дополняется, а не копируется. Поскольку это представление задач CSP стандартизировано, то в функции Backtracking-Search не требуется учитывать начальное состояние, функцию определения преемника или проверку цели, характерные для рассматриваемой проблемной области. Часть дерева поиска для задачи раскраски карты Австралии показана на рис. 5.3, где значения присваиваются переменным в порядке $wa, nt, Q, \dots$.

Листинг 5.1. Простой алгоритм поиска с возвратами для решения задач удовлетворения ограничений csp. Этот алгоритм составлен по принципу рекурсивного поиска в глубину, описанного в главе 3. Функции Select-Unassigned-Variable и Order-Domain-Values могут использоваться для реализации эвристических функций общего назначения, которые рассматриваются в тексте данной главы

function Backtracking-Search(csp) returns решение result или индикатор отказа failure

return Recursive-Backtracking({}, csp) function Recursive-Backtracking{assignment, csp}
 returns решение result или индикатор отказа failure if присваивание assignment
 является полным then return assignment var $\leftarrow$ Select-Unassigned-
 Variable(Variables[csp], assignment, csp)

for each value in Order-Domain-Values(var, assignment, csp) do if значение value
 является совместимым с присваиванием assignment согласно ограничениям
 Constraints[csp] then добавить {var = value} к присваиванию assignment result $\leftarrow$
 Recursive-Backtracking{assignment, csp} if result $\neq$ failure then return result удалить {var
 = value} из присваивания assignment return failure Рис. 5.3. Часть дерева поиска,
 сформированного путем простого поиска с возвратами при решении задачи
 раскрашивания карты, приведенной на рис. 5.1 Согласно определениям,
 приведенным в главе 3, алгоритм простого поиска с возвратами представляет собой
 неинформированный алгоритм, поэтому не следует рассчитывать на то, что он
 окажется очень эффективным при решении крупных задач. Результаты его
 применения к некоторым примерам задач показаны в первом столбце табл. 5.1 и
 подтверждают эти предположения.

В главе 4 было показано, что такой недостаток неинформированных алгоритмов
 поиска, как низкая производительность, можно устранить, предусмотрев
 использование в них эвристических функций, соответствующих данной проблемной
 области, которые основаны на наших знаниях о данной задаче. Как оказалось,
 задачи CSP можно решать эффективно без таких знаний о конкретной проблемной
 области.

Вместо этого в данной главе будут разработаны методы общего назначения,
 позволяющие найти ответ на перечисленные ниже вопросы. • Какой переменной
 должно быть присвоено значение в следующую очередь и в каком порядке
 необходимо пытаться присваивать эти значения? • Как влияют текущие
 присваивания значений переменным на другие переменные с неприсвоенными
 значениями? • Если какой-то путь оказался неудачным (т.е. достигнуто состояние, в
 котором ни одна переменная не имеет допустимых значений), позволяет ли поиск
 избежать повторения этой неудачи при прохождении последующих путей?

В приведенных ниже подразделах по очереди даны ответы на каждый из этих
 вопросов.

Таблица 5.1. Результаты применения различных алгоритмов CSP для решения
 разных задач. Слева направо показаны алгоритмы простого поиска с возвратами,
 поиска с возвратами на основе эвристической функции MRV, локального поиска с
 предварительной проверкой, локального поиска с предварительной проверкой на
 основе MRV и локального поиска с минимальными конфликтами. В каждой ячейке
 показано среднее количество проверок совместимости (за пять прогонов), которые

потребовались для решения данной задачи; обратите внимание на то, что все эти числа, кроме двух, находящихся вверху справа, выражены в тысячах (K). Числа в круглых скобках показывают, что за назначенное количество проверок не было найдено ни одного ответа. Первой задачей является поиск раскраски в 4 цвета для 50 штатов США. Остальные задачи взяты из [56, табл. 1]. Во второй задаче подсчитывается общее количество проверок, необходимых для решения всех задач с п ферзями для л от 2 до 50.

Третьей задачей является "головоломка с зеброй", которая описана в упр. 5.13. Последние две задачи представляют собой искусственные задачи, формируемые случайным образом (алгоритм с минимальными конфликтами к ним не применялся). Эти результаты показывают, что предварительная проверка на основе эвристической функции MRV является лучшим способом решения всех этих задач по сравнению с другими алгоритмами поиска с возвратами, но этот метод не всегда превосходит локальный поиск с минимальными конфликтами. Задача Поиск с воз- Поиске Локальный по Локальный поиск Локальный по - вратами возвратами иск с предварис предварительной иск с минимальна основе тельной проверпроверкой на осными конфликMRV кой нове MRV тами Определение рас- (> 1000K) (> 1000K) 2K 60 64 краски в 4 цвета для 50 штатов США Задача сп ферзями (>40 000K) 13 500K (>40 000K) 817K 4K Головоломка с 3859K 1K 35K 0,5K 2K зеброй Сформированная 415K 3K 26K 2K - случайным образом задача 1 Сформированная 942K 27K 77K 15K случайным образом задача 2 Упорядочение переменных и значений Приведенный выше алгоритм поиска с возвратами содержит следующую строку: `var <— Select-Unassigned-Variable(Variables[csp] , assignment, csp)` По умолчанию функция `Select-Unassigned-Variable` выбирает следующую переменную с неприсвоенным значением в порядке, указанном в списке `Variables [csp]`. Такое статическое упорядочение переменных редко приводит к наиболее эффективному поиску. Например, после присваиваний `WA=red` и `NT= green` остается только одно возможное значение для `sa9` поэтому имеет смысл на следующем этапе выполнить присваивание `SA=blue`, а не присваивать значение переменной `Q`. В действительности после присваивания значения переменной `SA` все варианты выбора значений для `Q`, `NSW` и `V` становятся вынужденными. Эта интуитивная идея (согласно которой в первую очередь следует выбирать переменную с наименьшим количеством "допустимых" значений) называется эвристикой с ^минимальным количеством оставшихся значений (`Minimum Remaining Values —`

`MRV`). Эту эвристику называют также эвристикой с "переменной, на которую распространяется наибольшее количество ограниченной", или эвристикой "до первого неудачного завершения"; последнее название применяется потому, что такая эвристическая функция предусматривает выбор переменной, которая с наибольшей вероятностью вскоре приведет к неудаче, усекая тем самым дерево

поиска. Если существует переменная x с нулевым количеством оставшихся допустимых значений, эвристическая функция MRV выберет x и неудача будет обнаружена немедленно; это позволяет избежать бессмысленных поисков среди других переменных, которые всегда будут оканчиваться неудачей после того, как в конечном итоге будет выбрана переменная x . Производительность поиска с использованием этой эвристической функции показана во втором столбце табл. 5.1, обозначенном как Поиск с возвратами на основе MRV. Производительность поиска повышается от 3 до 3000 раз по сравнению с простым поиском с возвратами, в зависимости от задачи. Следует отметить, что в применяемом здесь показателе производительности не учитывается дополнительная стоимость вычисления значений эвристической функции; в следующем подразделе описан метод, который позволяет удерживать это значение стоимости в приемлемых рамках.

Эта эвристическая функция MRV вообще не оказывает никакой помощи при выборе для окрашивания в Австралии первого региона, поскольку первоначально каждый регион имеет три допустимых цвета. В таком случае удобной становится n -степенная эвристика. В этой эвристике предпринимается попытка уменьшить коэффициент ветвления в будущих вариантах за счет выбора переменной, которая участвует в наибольшем количестве ограничений на другие переменные с неприсвоенными значениями. На рис. 5.1 переменной с наибольшей степенью, 5, является переменная 5A; другие переменные имеют степень 2 или 3, за исключением T, которая имеет степень 0. В действительности после выбора переменной SA применение степенной эвристики позволяет решить задачу без каких-либо неудачных этапов — теперь можно выбрать любой согласованный цвет в каждой точке выбора и все равно прийти к решению без поиска с возвратами. Эвристика с минимальным количеством оставшихся значений обычно обеспечивает более мощное руководство, но степенная эвристика может оказаться полезной в качестве средства разрешения неопределенных ситуаций.

После выбора одной из переменных в этом алгоритме необходимо принять решение о том, в каком порядке должны проверяться ее значения. Для этого в некоторых случаях может оказаться эффективной эвристика с n -наименее ограничительным значением. В ней предпочитается значение, в котором из рассмотрения исключается наименьшее количество вариантов выбора значений для соседних переменных в графе ограничений. Например, предположим, что на рис. 5.1 сформировано частичное присваивание с WA-red и NT= green и что следующий вариант выбора предназначен для Q. Синий цвет был бы плохим вариантом, поскольку исключил бы последнее допустимое значение, оставшееся для соседней переменной относительно Q, т.е. переменной SA. Поэтому эвристика с "наименее ограничительным значением" предпочитает значение red, а не blue. Вообще говоря, в этой эвристике предпринимается попытка сохранить

максимальную гибкость для последующих присваиваний значений переменным. Безусловно, если бы мы стремились найти все решения некоторой задачи, а не первое из них, то такое упорядочение не имело бы значения,

поскольку нам так или иначе пришлось бы рассмотреть каждое значение. Такое же замечание остается справедливым, если данная задача вообще не имеет решений.

Распространение информации с помощью ограничений До сих пор в рассматриваемом алгоритме поиска ограничения, распространяющиеся на какую-то переменную, учитывались только в тот момент, когда происходил выбор этой переменной с помощью функции `Select-Unassigned-Variable`. Но рассматривая некоторые ограничения на предыдущих этапах поиска или даже до начала поиска, можно резко уменьшить пространство поиска.

Предварительная проверка Один из способов лучшего использования ограничений во время поиска получил название $\wedge$ предварительной проверки (`forward checking`). При каждом присваивании значения переменной X в процессе предварительной проверки просматривается каждая переменная Y с неприсвоенным значением, которая соединена с X с помощью некоторого ограничения, и из области определения переменной Y удаляется любое значение, которое является несовместимым со значением, выбранным для X . На рис. 5.4 показан ход поиска решения задачи раскрашивания карты с помощью предварительной проверки.

На основании данного примера можно сделать два важных вывода. Прежде всего следует отметить, что после присваивания $WA=red$ и $Q=green$ области определения переменных NT и SA сокращаются до единственного значения; таким образом, ветвление, связанное с поиском значений для этих переменных, было полностью устранено путем распространения информации, касающейся переменных WA и Q . Применение эвристики `MRV`, которая, безусловно, хорошо сочетается с предварительной проверкой, позволяет на следующем этапе автоматически выбрать значение для переменных SA и NT . (Разумеется, что предварительная проверка может рассматриваться как эффективный способ инкрементного вычисления той информации, которая требуется эвристике `MRV` для выполнения своей работы.) Вторым выводом, заслуживающим внимания, состоит в том, что после присваивания $V=blue$ область определения SA становится пустой. Поэтому предварительная проверка позволила обнаружить, что частичное присваивание $\{WA=red, Q=green, V=blue\}$ является несовместимым с ограничениями этой задачи, значит, алгоритм немедленно выполняет возврат.

Начальные области определения После присваивания $WA=red$ После присваивания $Q=green$ После присваивания $V=blue$ Рис. 5.4. Поиск решения задачи с раскрашиванием карты на основе предварительной проверки. Вначале выполняется присваивание $WA=red$, затем предварительная проверка приводит к удалению

значений red из областей определения соседних переменных NT и SA. После присваивания Q=green значение green удаляется из областей определения NT, SA и NSW. После присваивания V=blue из областей определения NSW и SA удаляется значение blue, в результате чего переменная SA остается без допустимых значений WA RGB Ф Ф ^_ NT RGB G B в в Q RGB RGB NSW RGB RGB R B R V RGB RGB RGB SA RGB G B B T RGB RGB RGB RGB

Распространение ограничения Хотя предварительная проверка обнаруживает много несовместимостей, она не позволяет обнаружить их все. Например, рассмотрим третью строку на рис. 5.4, которая показывает, что если переменная WA имеет значение red, а Q— green, то обеим переменным, NT и SA, должно быть присвоено значение blue. Но соответствующие им регионы являются смежными и поэтому не могут иметь одно и то же значение цвета. Предварительная проверка не позволяет обнаружить эту ситуацию как несовместимость, поскольку не предусматривает достаточно далекий просмотр наперед. ^ Распространение ограничения (constraint propagation) — это общее название методов распространения на другие переменные последствий применения некоторого ограничения к одной переменной; в данном случае необходимо распространить ограничение с WA и Q на NT и SA (как было сделано с помощью предварительной проверки), а затем — на ограничение между NT и SA, чтобы обнаружить указанную несовместимость. К тому же желательно, чтобы такая операция выполнялась быстро: нет смысла ограничивать таким образом объем поиска, если будет расходоваться больше времени на распространение ограничений, чем на выполнение простого поиска.

Идея проверки ^ совместимости дуг легла в основу быстрого метода распространения ограничений, который является значительно более мощным по сравнению с предварительной проверкой. В данном случае термин дуга обозначает ориентированное ребро в графе ограничений, такое как дуга от SA до NSW. Если рассматриваются текущие области определения SA и NSW, то дуга является совместимой, если для каждого значения x переменной SA существует некоторое значение y переменной NSW, которое совместимо с x. В третьей строке на рис. 5.4 текущими областями определения SA и NSW являются {blue} и {red, blue} соответственно. При SA=blue существует совместимое присваивание для NSW, а именно NSW=red, поэтому дуга от SA до NSW совместима. С другой стороны, обратная дуга от NSW к SA несовместима, поскольку применительно к присваиванию NSW=blue не существует совместимого присваивания для SA. Эту дугу можно сделать совместимой, удалив значение blue из области определения NSW.

На том же этапе в процессе поиска проверку совместимости дуг можно также применить к дуге от SA до NT Третья строка таблицы, приведенной на рис. 5.4,

показывает, что обе переменные имеют область определения {blue}. Результатом становится то, что значение blue должно быть удалено из области определения SA, после чего эта область определения остается пустой. Таким образом, применение проверки совместимости дуг привело к раннему обнаружению той несовместимости, которая не была обнаружена с помощью предварительной проверки, применяемой в чистом виде.

Проверку совместимости дуг можно использовать либо в качестве этапа предварительной обработки перед началом процесса поиска, либо в качестве этапа распространения ограничения (аналогично предварительной проверке) после каждого присваивания во время поиска. (Последний алгоритм иногда называют МАС, сокращенно обозначая метод поддержки совместимости дуг— Maintaining Arc Consistency.) И в том и в другом случае процесс проверки необходимо выполнять повторно, до тех пор, пока не перестанут обнаруживаться какие-либо несовместимости. Это связано с тем, что при удалении (в целях устранения некоторой несовместимости дуг) любого значения из области определения некоторой переменной может

появиться новая несовместимость дуг в тех дугах, которые указывают на эту переменную. В полном алгоритме проверки совместимости дуг, AC-3, используется очередь для отслеживания дуг, которые должны быть проверены на несовместимость (листинг 5.2). Каждая дуга (X_iX_j) по очереди "снимается с повестки дня" и проверяется; если из области определения x_L необходимо удалить какие-либо значения, то каждая дуга (X_k, x_L), указывающая на x_L , должна быть повторно вставлена в очередь для проверки. Сложность проверки совместимости дуг можно проанализировать следующим образом: любая бинарная задача CSP имеет самое большее $O(n^2)$ дуг; каждая дуга (X_k, x_L) может быть "внесена в повестку дня" только d раз, поскольку область определения x_L имеет самое большее d значений, доступных для удаления; проверка совместимости любой дуги может быть выполнена за время $O(d^2)$, поэтому в наихудшем случае затраты времени составляют $O(n^2d^3)$. Хотя такой метод является значительно более дорогостоящим по сравнению с предварительной проверкой, все эти дополнительные затраты обычно окупаются¹.

Листинг 5.2. Алгоритм проверки совместимости дуг AC-3. После применения алгоритма AC-3 либо каждая дуга является совместимой, либо некоторая переменная имеет пустую область определения, указывая на то, что эту задачу CSP невозможно сделать совместимой по дугам (и поэтому данная задача CSP не может быть решена). Обозначение "AC-3" предложено разработчиком данного алгоритма [967], поскольку это — третья версия, представленная в его статье function AC-3 (csp) returns определение задачи CSP, возможно, с сокращенными областями определения переменных inputs: csp, бинарная задача CSP с переменными $\{X_i, X_2, \dots$,

X_n local variables: queue, очередь, состоящая из дуг, которая первоначально включает все дуги из определения задачи CSP while очередь queue не пуста do $(X_i, X_j) \leftarrow \text{Remove-First}(\text{queue})$ if $\text{Remove-Inconsistent-Values}(X_i, X_j)$ then for each X_k in $\text{Neighbors}[X_i]$ do добавить (X_k, X_i) к очереди queue function $\text{Remove-Inconsistent-Values}(X_i, X_j)$ returns значение true тогда и только тогда, когда произошло удаление некоторого значения removed $\leftarrow \text{false}$ for each x in $\text{Domain}[X_i]$ do if ни одно из значений y в области определения $\text{Domain}[X_j]$ не позволяет использовать (x, y) для удовлетворения ограничения, установленного между X_i и X_j then удалить x из области определения $\text{Domain}[X_i]$; removed $\leftarrow \text{true}$ return removed Поскольку задачи CSP включают задачу 3SAT в качестве частного случая, не следует рассчитывать на то, что удастся найти алгоритм с полиномиальной временной сложностью 1 Алгоритм AC-4, предложенный Мором и Хендерсоном [1068], выполняется за время $O(n^2d^2)$; см. упр. 5.10.

сложностью, позволяющий определить, является ли данная конкретная задача CSP совместимой по дугам. Таким образом, можно сделать вывод, что метод проверки совместимости дуг не позволяет обнаружить все возможные несовместимости. Например, как показано на рис. 5.1, частичное присваивание $\{WA=\text{red}, NSW=\text{red}\}$ несовместимо, но алгоритм AC-3 не обнаруживает такую несовместимость. Более строгие формы распространения ограничения можно определить с помощью понятия $\wedge$ k-совместимости.

Задача CSP является k-совместимой, если для любого множества $k-1$ переменных и для любого совместимого присваивания значений этим переменным любой k-й переменной всегда можно присвоить некоторое совместимое значение. Например, 1-совместимость означает, что совместимой является каждая отдельная переменная сама по себе; это понятие называют также $\wedge$ совместимостью узла. Далее, 2-совместимость — то же, что и совместимость дуги, а 3-совместимость означает, что любая пара смежных переменных всегда может быть дополнена третьей соседней переменной; это понятие именуется также $\wedge$ совместимостью пути.

Любой граф называется $\wedge$ строго k-совместимым, если он является k-совместимым, а также (k-1)-совместимым, (k-2)-совместимым, ... и т.д. вплоть до 1-совместимого.

Теперь предположим, что имеется некоторая задача CSP с узлами l , которая сделана строго n-совместимой (т.е. строго k-совместимой для $k=n$). Тогда эту задачу можно решить без возвратов. Для этого вначале можно выбрать совместимое значение для x_1 . В таком случае существует гарантия, что удастся выбрать значение для x_2 , поскольку граф является 2-совместимым, для x_3 , поскольку он — 3-совместимый, и т.д. Для каждой переменной x_i необходимо выполнить поиск только среди d значений в ее области определения, чтобы найти значение, совместимое с $x_1, \dots, x_{i-1}$. Это означает, что гарантируется нахождение решения за время $O(nd)$. Безусловно, за такую возможность также приходится платить: любой алгоритм обеспечения n-

совместимости в наихудшем случае должен требовать времени, экспоненциально зависящего от p .

Между методами обеспечения p -совместимости и совместимости дуг существует целый ряд промежуточных методов, выбор которых осуществляется с учетом того, что для выполнения более строгих проверок совместимости потребуется больше времени, но это позволяет получить больший эффект с точки зрения сокращения коэффициента ветвления и обнаружения несовместимых частичных присваиваний.

Существует возможность рассчитать такое наименьшее значение k , что выполнение алгоритма проверки k -совместимости будет гарантировать решение данной задачи без возвратов (см. раздел 5.4), но применение данного расчета на практике не всегда оправдано. В действительности определение подходящего уровня проверки совместимости в основном относится к области эмпирических методов.

Обработка специальных ограничений В реальных задачах часто встречаются некоторые типы ограничений, которые могут обрабатываться с помощью алгоритмов специального назначения, более эффективных по сравнению с методами общего назначения, которые рассматривались до сих пор. Например, ограничение Alldiff указывает, что все участвующие в нем переменные должны иметь разные значения (как в криптоарифметической задаче).

Одна из простых форм проверки несовместимости для ограничений Alldiff применяется следующим образом: если в данном ограничении участвуют t переменных

и все они вместе взятые имеют p возможных разных значений, притом что $t > p$, то это ограничение не может быть удовлетворено.

Применение данной проверки приводит к созданию следующего простого алгоритма: вначале удалить из ограничения каждую переменную, имеющую одноэлементную область определения, затем удалить значение этой переменной из областей определения оставшихся переменных. Повторять эту операцию до тех пор, пока имеются переменные с одноэлементными областями определения. Если в какой-то момент времени появится пустая область определения или останется больше переменных, чем областей определения, это будет означать, что обнаружена несовместимость.

Этот метод можно применить для обнаружения несовместимости в частичном присваивании $\{WA=red, NSW=red\}$ на рис. 5.1. Обратите внимание на то, что переменные SA, NT к 0 фактически связаны ограничением Alldiff, поскольку каждая пара соответствующих регионов должна быть обозначена различными цветами. После применения алгоритма AC-3 в сочетании с этим частичным присваиванием область определения каждой переменной сокращается до $\{green, blue\}$. Это означает,

что имеются три переменные и только два цвета, поэтому ограничение Alldiff нарушается. Таким образом, иногда простая процедура проверки совместимости для ограничения более высокого порядка эффективнее по сравнению с процедурой проверки совместимости дуг, применяемой к эквивалентному множеству бинарных ограничений.

Возможно, более важным ограничением высокого порядка является $\wedge$ ресурсное ограничение (resource constraint), иногда называемое ограничением "самое большее"

(atmost). Например, допустим, что $PA_1, \dots, PA_4$ обозначают количество персонала, назначенного для выполнения каждого из четырех заданий. Ограничение, согласно которому может быть всего назначено не больше 10 членов персонала, записывается как $atmost\ A_0, PA_1, PA_2, PA_3, PA_4$ • Несовместимость можно обнаружить, проверяя сумму минимальных значений текущих областей определения; например, если каждая переменная имеет область определения {3, 4, 5, 6}, то ограничение $atmost$ не может быть удовлетворено. Кроме того, можно принудительно добиться совместимости, удаляя максимальное значение из любой области определения, если оно не совместимо с минимальными значениями других областей определения. Таким образом, если каждая переменная в данном примере имеет область определения {2, 3, 4, 5, 6}, то из каждой области определения можно удалить значения 5 и 6.

При решении крупных задач проверки ресурсных ограничений с целочисленными значениями (таких как задачи снабжения, в которых предусматривается перемещение тысяч людей в сотнях транспортных средств) обычно не существует возможности представлять область определения каждой переменной в виде большого множества целых чисел и постепенно сокращать это множество с применением методов проверки совместимости. Вместо этого области определения представляются в виде верхнего и нижнего пределов и управляются с помощью метода распространения этих пределов. Например, предположим, что имеются два рейса, Flight271 и Flight272, в которых самолеты имеют соответственно вместимость 165 и 385 пассажиров. Поэтому начальные области определения для количества пассажиров в каждом рейсе определяются следующим образом:

Flight271 $G [0,165]$ и Flight272 $G [0,385]$ Теперь допустим, что имеется дополнительное ограничение, согласно которому в этих двух рейсах необходимо перевести 420 человек:

Flight271+Flight272 $e [420,420]$. Распространяя ограничения пределов, можно сократить области определения до таких величин:

Flight271 $e [35,165]$ и Flight272 $G [255,385]$ Задача CSP называется совместимой с пределами (bounds-consistent), если для каждой переменной X , а также

одновременно для нижнего и верхнего предельных значений X существует некоторое значение Y , которое удовлетворяет заданному ограничению между x и Y для каждой переменной Y . Такого рода $\wedge$ распространение пределов (bounds propagation) широко используется в практических задачах с ограничениями.

Интеллектуальный поиск с возвратами: поиск в обратном направлении В алгоритме Backtracking-Search, приведенном в листинге 5.1, применялось очень простое правило, касающееся того, что делать, если какая-то ветвь поиска оканчивается неудачей: вернуться к предыдущей переменной и попытаться использовать для нее другое значение. Такой метод называется $\wedge$ хронологическим поиском с возвратами, поскольку повторно посещается пункт, в котором было принято последнее по времени решение. В данном подразделе будет показано, что существуют намного лучшие способы поиска с возвратами.

Рассмотрим, что произойдет в случае применения простого поиска с возвратами в задаче, показанной на рис. 5.1, с постоянным упорядочением переменных Q , NSW , V , T , SA , WA , NT . Предположим, что сформировано частичное присваивание $\{Q=\text{red}, NSW=\text{green}, V=\text{blue}, T=\text{red}\}$. А после попытки присвоить значение следующей переменной, SA , будет обнаружено, что любое значение нарушает какое-то ограничение. Алгоритм возвращается к узлу T и пытается назначить новый цвет для Тасмании! Очевидно, что это — бессмысленное действие, поскольку смена цвета Тасмании не позволяет решить проблему с Южной Австралией.

Более интеллектуальный подход к поиску с возвратами состоит в том, чтобы вернуться к одному из множеств переменных, которые стали причиной неудачи. Это множество называется $\wedge$ конфликтным множеством; в данном случае конфликтным множеством для SA является $\{Q, NSW, V\}$. Вообще говоря, конфликтное множество для переменной X представляет собой множество переменных с ранее присвоенными значениями, которые связаны с X ограничениями. Метод $\wedge$ обратного перехода выполняет обратный переход к переменной с последним по времени присвоенным значением из конфликтного множества; в данном случае в обратном переходе следует перескочить через узел Тасмании и попытаться применить новое значение для V .

Такая операция может быть легко реализована путем модификации алгоритма Backtracking-Search таким образом, чтобы он накапливал данные о конфликтном множестве, одновременно проверяя одно из значений, допустимых для присваивания. Если не будет найдено ни одного допустимого значения, алгоритм должен возвратиться к последнему по времени элементу конфликтного множества и наряду с этим установить индикатор неудачи.

Внимательный читатель уже должен был заметить, что предварительная проверка позволяет определить конфликтное множество без дополнительной работы,

поскольку каждый раз, когда процедура предварительной проверки, основанная на присваивании значения переменной X , удаляет некоторое значение из области определения u , она должна добавить X к конфликтному множеству U . Кроме того, каждый раз, когда это последнее значение удаляется из области определения u , переменные из конфликтного множества U добавляются к конфликтному множеству X .

В этом случае после перехода к Y можно сразу же установить, куда должен быть выполнен обратный переход в случае необходимости.

А проницательный читатель уже должен был заметить нечто странное: обратный переход происходит, когда каждое значение в некоторой области определения конфликтует с текущим присваиванием, но предварительная проверка обнаруживает этот случай и вообще исключает возможность достижения такого узла! В действительности можно показать, что s^3 каждая ветвь, отсекаемая с помощью обратного перехода, отсекается также с помощью предварительной проверки. Поэтому простой поиск с обратным переходом в сочетании с поиском с предварительной проверкой становится избыточным, а фактически обратный переход избыточен в любом поиске, в котором используется более строгая проверка совместимости, такая как MAC.

Несмотря на замечания, сделанные в предыдущем абзаце, идея, лежащая в основе обратного перехода, остается перспективной, если возврат осуществляется с учетом причин неудачи. При обратном переходе неудача обнаруживается после того, как область определения некоторой переменной становится пустой, но во многих случаях какая-то ветвь поиска становится бесперспективной задолго до того, как это происходит. Еще раз рассмотрим частичное присваивание $\{WA=red, NSW=red\}$ (которое согласно приведенному выше описанию является несовместимым).

Предположим, что на следующем этапе предпринимается попытка присваивания $T=red$, а затем осуществляется присваивание значений переменным jvt , Q , V , SA . Известно, что ни одно присваивание не применимо для этих последних четырех переменных, поэтому в конечном итоге не остается доступных значений, которые можно было бы попытаться присвоить переменной NT . Теперь возникает вопрос, куда выполнить возврат?

Обратный переход не может применяться, поскольку в области определения переменной NT имеются значения, совместимые со значениями, ранее присвоенными переменным, — переменная NT не имеет полного конфликтного множества из переменных с ранее присвоенными значениями, которое вызвало бы неудачу при попытке присваивания ей значения. Однако известно, что неудачу вызывают четыре переменные, NT , Q , U и SA , вместе взятые, поскольку во множестве переменных с ранее присвоенными значениями должны существовать такие

переменные, которые непосредственно конфликтуют с этими четырьмя. Эти рассуждения приводят к созданию более глубокого определения понятия конфликтного множества для такой переменной, как NT. конфликтным множеством называется множество переменных с ранее присвоенными значениями, которые, наряду со всеми переменными со значениями, присваиваемыми в дальнейшем, становятся причиной того, что для NT не существует совместимого решения. В таком случае конфликтное множество состоит из переменных WA и NSW, поэтому алгоритм должен выполнить возврат к NSW и пропустить переменную, соответствующую Тасмании. Алгоритм обратного перехода, в котором используются конфликтные множества, определенный таким образом, называется $\wedge$ алгоритмом обратного перехода, управляемого конфликтами (conflict-directed backjumping).

Теперь необходимо объяснить, как вычисляются эти новые конфликтные множества. Применяемый для этого метод фактически очень прост. "Окончательная" неудача в какой-то ветви поиска всегда возникает из-за того, что область определения некоторой переменной становится пустой; эта переменная имеет типичное

конфликтное множество. В данном примере неудача возникает при присваивании значения переменной SA, а ее конфликтным множеством является (скажем) {WA, NT, Q}. Выполняется обратный переход к переменной Q, и эта переменная поглощает конфликтное множество, полученное от SA (безусловно, после исключения из него самой переменной Q), объединяя его со своим собственным прямым конфликтным множеством, представляющим собой {NT, NSW}', новым конфликтным множеством становится {WA, NT, NSW}. Это означает, что, продолжая поиск от Q, нельзя найти решение при наличии ранее выполненного присваивания переменным {WA, NT, NSW}. Поэтому обратный переход выполняется к переменной NT, присваивание значения которой выполнено последним по времени по сравнению с другими этими переменными. Переменная NT поглощает множество {WA, NT, NSW} - {NT}, объединяя его со своим собственным прямым конфликтным множеством [WA]9 что приводит к получению {WA, NSW} (как было указано в предыдущем абзаце). Теперь алгоритм осуществляет обратный переход к NSW, как и предполагалось. Подведем итог: допустим, что X_j — текущая переменная, а $\text{conf}(X_j)$ — ее конфликтное множество. Если все попытки присваивания каждого возможного значения переменной X_j оканчиваются неудачей, то необходимо выполнить возврат к переменной x_i с последним по времени присвоенным значением во множестве $\text{conf}(X_j)$ и установить: $\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_i\}$ Обратный переход, управляемый конфликтами, позволяет вернуться в правильную точку дерева поиска, но не предотвращает возможности возникновения таких же ошибок в другой ветви того же дерева. Метод $\wedge$ определения ограничений с помощью обучения (constraint learning) по сути представляет собой метод модификации задачи CSP путем

добавления нового ограничения, выдвинутого на основе логического анализа этих конфликтов.

5.3. Применение локального поиска для решения задач Как оказалось, алгоритмы локального поиска (см. раздел 4.3) являются очень эффективными средствами решения многих задач CSP. В них используется следующая формулировка с полным состоянием: в начальном состоянии присваивается значение каждой переменной, а функция определения преемника обычно действует по принципу изменения за один раз значения одной переменной. Например, в задаче с восемью ферзями начальное состояние может представлять собой случайную конфигурацию из 8 ферзей, стоящих на 8 столбцах, а функция определения преемника выбирает одного ферзя и рассматривает возможность перемещения его на какую-то другую клетку в том же столбце. Еще одна возможность предусматривает, чтобы поиск начинался с 8 ферзей (по одному на каждом столбце) в виде какой-то перестановки из 8 строк, а преемник формировался путем обмена двумя строками с двумя ферзями². В этой книге уже фактически был приведен пример применения 2 Локальный поиск можно легко дополнить для использования его в задачах CSP с помощью целевой функции. В таком случае для оптимизации целевой функции могут применяться все методы поиска с восхождением к вершине и с эмуляцией отжига.

локального поиска для решения задачи CSP — применение поиска с восхождением к вершине для решения задачи с p ферзями (с. 1). Еще одним примером может служить использование алгоритма WalkSAT (с. 1) для решения задач удовлетворения ограничений, которые представляют собой частный случай задач CSP.

Наиболее очевидная эвристика, которая может применяться при выборе нового значения для какой-либо переменной, состоит в определении того, приводит ли выбранное значение к минимальному количеству конфликтов с другими переменными; таковой является эвристика с $\wedge$ минимальными конфликтами. Соответствующий алгоритм приведен в листинге 5.3, пример его применения в задаче с восемью ферзями схематически показан на рис. 5.5, а полученные при этом результаты даны в табл. 5.1.

Листинг 5.3. Алгоритм Min-Conflicts, применяемый для решения задач CSP с помощью локального поиска. Начальное состояние может быть выбрано либо случайным образом, либо с помощью жадного процесса присваивания, в котором выбирается значение с минимальными конфликтами для каждой переменной по очереди. Функция Conflicts вычисляет количество ограничений, нарушенных данным конкретным значением, с учетом остальной части текущего присваивания

```
function Min-Conflicts(csp, max_steps) returns решение current или индикатор неудачи failure
inputs: csp, определение задачи удовлетворения ограничений max_steps, количество шагов, которые разрешается выполнить, прежде чем отказаться от дальнейших попыток
current ← первоначальное полное присваивание для csp for
```

$i = 1$ to max_steps do if current представляет собой решение для задачи csp then return current
var $\leftarrow$ выбранная случайным образом, конфликтующая переменная из $\text{Variables}[\text{csp}]$
value $\leftarrow$ значение v для var, которое минимизирует значение $\text{Conflicts}(\text{var}, v, \text{current}, \text{csp})$
задать значение $\text{var} = \text{value}$ в решении current
return failure

Рис. 5.5. Двухэтапный процесс решения задачи с восемью ферзями на основе эвристики с минимальными конфликтами. На каждом этапе выбирается ферзь для повторного назначения ему места в том же столбце. Количество конфликтов (в данном случае количество атакующих ферзей) показано в каждой клетке. Применяемый алгоритм предусматривает перемещение ферзя в клетку с минимальными конфликтами, разрешая неопределенность с помощью случайного выбора

Алгоритм с минимальными конфликтами показал себя чрезвычайно эффективным при решении многих задач CSP, особенно при наличии подходящего начального состояния. Его производительность показана в последнем столбце табл. 5.1.

Больше всего достойно удивления то, что при решении задачи с p ферзями время прогона алгоритма с минимальными конфликтами остается почти независимым от размера задачи (если не учитываются затраты времени на первоначальную расстановку ферзей). Данный алгоритм решает в среднем за 50 этапов (после начального присваивания) даже задачу с миллионом ферзей. Это замечательное достижение стало стимулом, побудившим к проведению значительной части исследований по локальному поиску в 1990-х годах, а также позволило уточнить различие между легкими и трудными задачами, которое дополнительно будет рассматриваться в главе 7.

Грубо говоря, задача с p ферзями является легкой для локального поиска, поскольку решения плотно распределены по всему пространству состояний. Кроме того, алгоритм с минимальными конфликтами успешно применяется при решении трудных задач. Например, он использовался для планирования наблюдений на космическом телескопе Хаббл, что позволило сократить затраты времени на планирование наблюдений, намеченных для проведения в течение одной недели, с трех недель (!) примерно до 10 минут.

Еще одним преимуществом локального поиска является то, что он может использоваться для оперативной корректировки в случае изменения условий задачи. Это особенно важно при решении задач планирования. Недельный график работы авиакомпании может включать тысячи рейсов и десятки тысяч персональных назначений, но из-за плохой погоды в одном аэропорту весь этот график может стать невыполнимым. Поэтому желательно иметь возможность откорректировать график с минимальным количеством изменений. Такую задачу легко выполнить с помощью алгоритма локального поиска, начинающего свою работу с текущего графика. Поиск с возвратами, выполняемый с учетом нового

множества ограничений, обычно требует гораздо больше времени и может найти менее удачное решение, требующее внести много изменений в текущий график.

5.4. Структура задач В данном разделе рассматриваются способы, позволяющие использовать для быстрого поиска решений структуру самой задачи, представленную в виде графа ограничений. Большинство описанных здесь подходов являются очень общими и применимыми для решения других задач, кроме CSP, например задач вероятностного формирования рассуждений. В конечном итоге единственный способ, с применением которого можно надеяться справиться с задачами реального мира, состоит в том, чтобы разложить их на множество подзадач. Еще раз обратившись к рис. 5.1, б с целью определить структуру задачи, можно обнаружить еще один факт: Тасмания не связана с материком³. Интуитивно ясно, что задачи раскрашивания Тасмании и рас³ Педантичный картограф или патриотически настроенный житель Тасмании мог бы возразить, что Тасманию не следует окрашивать в такой же цвет, как и ее ближайшего соседа по материку, чтобы исключить вероятность, что создается такое впечатление, будто Тасмания является частью этого штата.

крашивания материка представляют собой $\wedge$ независимые подзадачи — любое решение для материка, скомбинированное с любым решением для Тасмании, приводит к получению решения для всей карты. В том, что эти подзадачи являются независимыми, можно убедиться, рассматривая " $\wedge$ связанные компоненты графа ограничений.

Каждый компонент соответствует одной подзадаче CSP_i. Если присваивание SL является решением CSP_i то (J_i-S_i является решением (J_iCSP_i. Почему это так важно? Рассмотрим следующее: предположим, что каждая подзадача CSP_i имеет с переменных из общего количества п переменных, где с — константа. В таком случае должно быть п/с подзадач, и для решения каждой из них требуется, самое большее, объем работы dс. Поэтому общий объем работы измеряется величиной $O(d^n/s)$, которая линейно зависит от л; без такой декомпозиции общий объем работы измерялся бы величиной $O(d^n)$, которая экспоненциально зависит от п. Приведем более конкретный пример: разделение булевой задачи CSP с л=80 на четыре подзадачи с с=2 0 сокращает продолжительность поиска решения в наихудшем случае от такой величины, которая равна сроку существования всей вселенной, до величины меньше одной секунды.

Поэтому полностью независимые подзадачи являются привлекательными, но встречаются редко. В большинстве случаев подзадачи любой задачи CSP связаны друг с другом. Простейшим случаем является тот, в котором граф ограничений образует дерево: любые две переменные связаны не больше чем одним путем. На рис. 5.6, а показан один из примеров в схематическом виде⁴. Ниже будет показано, что сг* любая задача CSP с древовидной структурой может быть решена за время,

линейно зависящее от количества переменных. Этот алгоритм имеет перечисленные ниже этапы. • Выбрать в качестве корня дерева любую переменную и упорядочить переменные от корня до листьев таким образом, чтобы родительский узел каждого узла в дереве предшествовал этому узлу в таком упорядочении (см. рис. 5.6, б).

Обозначить эти переменные по порядку как $x_1, \dots, x_n$. Теперь каждая переменная, кроме корня, имеет только одну родительскую переменную.

Рис. 5.6. Пример задачи CSP с древовидной структурой: граф ограничений задачи CSP с древовидной структурой (а); линейное упорядочение переменных, совместимых с деревом, корнем которого является переменная А (б) • В цикле по j от 1 до n применять проверку совместимости к дугам (x_i, x_j) , где x_i — родительский узел узла x_j , удаляя значения из области определения $Domain[x_i]$ по мере необходимости.

4 К сожалению, карты с древовидной структурой имеют лишь немногие регионы мира; в качестве одного из возможных исключений следует назвать Сулавеси.

• В цикле по j от 1 до n присваивать x_j любое значение, совместимое со значением, присвоенным x_i где x_i — родительский узел узла x_j .

Следует учитывать два важных соображения. Во-первых, после выполнения этапа 2 задача CSP становится совместимой по дугам с учетом ориентации дуг, поэтому присваивание значений на этапе 3 не требует возврата. (См. описание понятия k -совместимости на с. 222.) Во-вторых, благодаря применению проверок совместимости дуг в обратном порядке на этапе 2 алгоритм гарантирует, что никакие удаленные значения не смогут нарушить совместимость тех дуг, которые уже были обработаны. Весь этот алгоритм выполняется за время $O(n^2)$.

Теперь, после создания эффективного алгоритма для деревьев, следует рассмотреть вопрос о том, можно ли каким-то образом приводить к древовидным структурам более общие графы ограничений. Существуют два основных способа выполнения этой задачи; один из них основан на удалении узлов, а другой — на слиянии узлов друг с другом.

Первый подход предусматривает присваивание значений некоторым переменным так, чтобы оставшиеся переменные образовывали дерево. Рассмотрим граф ограничений для карты Австралии, который еще раз показан на рис. 5.7, а. Если с этой карты можно было бы удалить узел SA, соответствующий Южной Австралии, то граф превратился бы в дерево, как показано на рис. 5.7, б. К счастью, это можно сделать (в графе, а не на самом континенте), зафиксировав некоторое значение для SA и удалив из областей определения других переменных все значения, несовместимые со значением, выбранным для SA.

0 0 а) б) Рис. 5.7. Первый способ преобразования графа ограничений в дерево: первоначальный граф ограничений, впервые приведенный на рис. 5.1 (а); граф ограничений после удаления узла SA (б) Теперь, после удаления узла SA и связанных с ним ограничений, любое решение данной задачи CSP будет совместимым со значением, выбранным для SA. (Такой способ удобен при решении бинарных задач CSP; при наличии ограничений более высокого порядка ситуация усложняется.) Поэтому появляется возможность решить задачу, представленную оставшимся деревом, с помощью приведенного выше алгоритма и таким образом решить всю проблему. Безусловно, в общем случае (в отли-

чие от задачи раскрашивания карты) значение, выбранное для узла SA, может оказаться неподходящим, поэтому придется проверить каждое из возможных значений.

Общий алгоритм решения указанным способом описан ниже.

- Выбрать подмножество S из множества Variables [csp], такое, что граф ограничений после удаления S становится деревом. Подмножество S называется $\wedge$ множеством разрыва цикла (cycle cutset).
- Для каждого возможного присваивания переменным в S , которое удовлетворяет всем ограничениям в S , выполнить следующее:
 - удалить из областей определения оставшихся переменных любые значения, несовместимые с данным присваиванием для S ;
 - если оставшаяся задача CSP имеет решение, вернуть это решение вместе с присваиванием для S .

Если множество разрыва цикла имеет размер s , то общее время прогона алгоритма составляет $O(c^{n-s})$ (л-с $d2$). В том случае, если граф по своей форме "очень близок к дереву", то множество s будет небольшим, а экономия времени по сравнению с прямым поиском с возвратами окажется огромной. Но в наихудшем случае количество элементов s может достигать $(n-2)$. Задача поиска наименьшего множества разрыва цикла является NP-трудной, но известно несколько эффективных алгоритмов решения этой задачи. В целом данный алгоритмический подход носит название $\wedge$ определения условий выбора множества разрыва цикла (cutset conditioning); мы снова столкнемся с этим понятием в главе 14, где оно используется при формировании рассуждений о вероятностях.

Второй подход основан на формировании $\wedge$ древовидной декомпозиции (tree decomposition) графа ограничений и создании множества связанных подзадач.

Каждая подзадача решается независимо, а затем итоговые решения комбинируются. Как и большинство алгоритмов, действующих по принципу "разделяй и властвуй", этот алгоритм работает успешно, если подзадачи не слишком велики.

На рис. 5.8 показана древовидная декомпозиция задачи раскрашивания карты на пять подзадач. Любая древовидная декомпозиция должна удовлетворять трем приведенным ниже требованиям.

- Каждая переменная из первоначальной задачи

появляется по меньшей мере в одной из подзадач. • Если две переменные первоначальной задачи связаны ограничением, то должны появиться вместе (наряду с этим ограничением) по меньшей мере в одной из подзадач. • Если какая-то переменная появляется в двух подзадачах в дереве, то должна появляться в каждой подзадаче вдоль пути, соединяющего эти подзадачи.

Первые два условия гарантируют, что в декомпозиции будут представлены все переменные и ограничения. Третье условие на первый взгляд кажется довольно формальным, но просто отражает то ограничение, что любая конкретная переменная должна иметь одно и то же значение в каждой подзадаче, в которой появляется; соблюдение этого ограничения гарантируют связи, соединяющие подзадачи в дереве.

Например, переменная SA появляется во всех четырех связанных подзадачах на рис. 5.8.

На основании рис. 5.7 можно убедиться, что эта декомпозиция имеет смысл.

Каждая подзадача решается независимо; если какая-либо из них не имеет решения, то известно, что вся задача также не имеет решения. Если удастся решить все подзадачи, то может быть предпринята попытка составить глобальное решение следующим образом. Прежде всего, каждая подзадача рассматривается как "мегапеременная", областью определения которой является множество всех решений этой подзадачи. Например, самая левая подзадача на рис. 5.8 представляет задачу раскрашивания карты с тремя переменными и поэтому имеет шесть решений; одним из них является {WA=red, SA=blue, NT= green}. Затем решается задача с ограничениями, связывающими подзадачи; для этого используется эффективный алгоритм для деревьев, приведенный выше. Ограничения, связывающие подзадачи, указывают на то, что решения подзадач должны быть согласованными по их общим переменным.

Например, если за основу берется решение первой подзадачи {WA=red, SA=blue, NT=green}, то единственным совместимым решением для следующей подзадачи становится {SA=blue, NT= green, Q=red}.

Рис. 5.8. Древовидная декомпозиция графа ограничений, показанного на рис. 5.7, а Любой конкретный граф ограничений допускает большое количество древовидных декомпозиций; при выборе декомпозиции нужно стремиться к тому, чтобы подзадачи были как можно меньше. ^ Ширина дерева древовидной декомпозиции графа на единицу меньше размера наибольшей подзадачи; ширина дерева самого графа определяется как минимальная ширина дерева среди всех его древовидных декомпозиций. Если граф имеет ширину дерева w и дана соответствующая древовидная декомпозиция, то соответствующая задача может быть решена за время $O(ndw+1)$. Это означает, что c^3 задачи CSP с графами ограничений,

характеризующимися конечной шириной дерева, могут быть решены за полиномиальное время. К сожалению, задача поиска декомпозиции с минимальной шириной дерева является NP-трудной, но существуют эвристические методы, которые хорошо работают на практике.

Резюме Задачи удовлетворения ограничений (Constraint Satisfaction Problems — CSP) состоят из переменных с налагаемыми на них ограничениями. В виде задач CSP могут быть описаны многие важные задачи реального мира. Структуру любой задачи CSP можно представить в виде ее графа ограничений.

Для решения задач CSP обычно применяется поиск с возвратами — одна из форм поиска в глубину.

Независимыми от области определения методами выявления того, какая переменная должна быть выбрана следующей в ходе поиска с возвратами, являются эвристики, основанные на определении минимального количества оставшихся значений, и степеннь/е эвристики. Для упорядочения значений переменной может применяться эвристика с наименее ограничительным значением.

В алгоритме поиска с возвратами можно намного сократить коэффициент ветвления задачи, распространяя последствия частичных присваиваний, сформированных в ходе работы этого алгоритма. Простейшим методом такого распространения является предварительная проверка. Более мощным методом является обеспечение совместимости дуг, но применение этого метода может оказаться более дорогостоящим.

Возврат происходит после того, как для переменной нельзя больше найти допустимое присваивание. При использовании обратного перехода, управляемого конфликтами, возврат происходит непосредственно к источнику проблемы, возникшей в процессе поиска.

Для решения задач с ограничениями весьма успешно используется локальный поиск на основе эвристики с минимальными конфликтами.

Сложность решения задачи CSP в значительной степени зависит от структуры ее графа ограничений. Задачи с древовидной структурой могут быть решены за линейное время. Метод определения условий формирования множества разрыва цикла позволяет преобразовать задачу CSP общего вида в задачу с древовидной структурой и является очень эффективным, если существует возможность найти небольшое множество разрыва цикла. Методы древовидной декомпозиции предусматривают преобразование задачи CSP в дерево подзадач и являются эффективными, если ширина дерева графа ограничений мала.

Библиографические и исторические заметки В самых ранних работах, относящихся к проблематике удовлетворения ограничений, главным образом рассматривались

числовые ограничения. Выраженные в виде уравнений ограничения с целочисленными областями определения были исследованы индийским математиком Брахмагуптой в VII веке; их часто называют "диофантовыми уравнениями в честь греческого математика Диофанта (около 200—284 гг.), который в своих исследованиях фактически рассматривал область определения положительных рациональных чисел. Систематические методы решения линейных уравнений путем удаления переменных исследовались Гауссом [527]; истоки

подходов к решению ограничений с линейными неравенствами можно найти в работах Фурье [487].

Задачи удовлетворения ограничений с конечными областями определения также имеют долгую историю. Например, одной из старых задач в математике является ^Λ раскрашивание графа (раскрашивание карты — частный случай данной задачи).

Согласно данным, приведенным в [125], гипотезу о четырех цветах (в соответствии с которой любой плоский граф можно раскрасить с помощью четырех или меньшего количества цветов) впервые выдвинул Френсис Гутри, студент де Моргана, в 1852 году. Эта задача не поддавалась решению (несмотря на то, что в некоторых публикациях утверждалось обратное) до тех пор, пока доказательство не было получено с помощью компьютера Appel'ом и Хакеном [35].

Специальные классы задач удовлетворения ограничений рассматривались на протяжении всей истории развития компьютерных наук. Одним из самых первых примеров решения таких задач, оказавшим наибольшее влияние, была система Sketchpad [1476], которая решала геометрические ограничения на чертежах и была предшественником современных программ рисования и инструментальных средств САПР. Выявлению того, что задачи CSP представляют собой общий класс задач, мы обязаны Уго Монтанари [1073]. Первые попытки приведения задач CSP высокого порядка к чисто бинарным задачам CSP с помощью вспомогательных переменных (см. упр. 5.11) были впервые предприняты в XIX веке логиком Чарльзом Сандерсом Пирсом. Соответствующие методы были введены в тематику CSP Дектером [367] и доработаны Бакусом и ван Биком [55]. Задачи CSP с предпочтениями между решениями широко рассматривались в исследованиях по оптимизации; обобщение инфраструктуры CSP, позволяющее использовать предпочтения, приведено в [134].

Для решения задач оптимизации может также применяться алгоритм устранения интервалов [369].

Использование поиска с возвратами для удовлетворения ограничений было предложено Битнером и Рейнгольдом [135], но эти ученые проследили истоки идей своего основного алгоритма до работ XIX века. Кроме того, Битнер и Рейнгольд впервые предложили использовать эвристику MRV, названную ими эвристикой с наиболее ограниченной переменной. Брелаз [181] использовал степенную

эвристику для устранения неопределенности, возникающей после применения эвристики MRV.

Полученный в итоге алгоритм, несмотря на его простоту, все еще остается наилучшим методом раскрашивания произвольных графов в k цветов. Харалик и Эллиот [618] предложили эвристику с наименее ограничительным значением.

Расширению популярности методов распространения ограничений способствовало успешное решение Вальцем [1552] задач полиэдральной линейной разметки для систем компьютерного зрения. Вальц показал, что применение методов распространения ограничений при решении многих задач позволяет полностью исключить необходимость в использовании возвратов. Монтанари [1073] ввел понятие сетей ограничений и предложил метод распространения ограничений на основе понятия совместимости пути. Ален Макворт [967] предложил алгоритм AC-3 для обеспечения совместимости дуг, а также выдвинул общую идею о том, что поиск с возвратами необходимо сочетать с обеспечением совместимости некоторой степени. Более эффективный алгоритм обеспечения совместимости дуг, AC-4, был разработан Мором и Хендерсоном [1068]. Вскоре после появления статьи Макворта исследователи приступили к экспериментам в области поиска компромисса между затратами на обес-

печение совместимости и достигаемыми за счет этого преимуществами с точки зрения сокращения объема поиска. Харалик и Эллиот [618] предпочли минимальный алгоритм предварительной проверки, описанный Макгрегором [1028], а Гашниг [522] предложил применять полную проверку совместимости дуг после присваивания значения каждой переменной; в дальнейшем соответствующему алгоритму в работе Сабина и Фрейдера [1335] было присвоено название MAC. В последней статье представлены некоторые убедительные свидетельства того, что при решении более трудных задач CSP полная проверка совместимости дуг вполне окупается. Фрейдер [497], [498] исследовал понятие k -совместимости и ее связь со сложностью решения задач CSP.

Апт [36] описал универсальную алгоритмическую инфраструктуру, в рамках которой могут быть проанализированы алгоритмы распространения совместимости.

Специальные методы обработки ограничений высокого порядка были созданы в основном в контексте логического программирования в ограничениях. Превосходный обзор исследований в этой области приведен в [987]. Ограничение Alldiff исследовалось в [1272]. Ограничения с пределами были включены в тематику логического программирования в ограничениях ван Хентенриком и др. [1533].

Основной метод обратного перехода был разработан Джоном Гашнигом [521], [522]. Кондрак и ван Бик [831] показали, что этот алгоритм по сути перекрывается алгоритмом предварительной проверки. Обратный переход, управляемый

конфликтами, был предложен Проссером [1240]. Наиболее общая и мощная форма интеллектуального поиска с возвратами фактически разработана еще довольно давно Сталманом и Зюссманом [1456]. Предложенный ими метод $\wedge$ поиска с возвратами, управляемого зависимостями, привел к разработке систем обеспечения истинности [409], которые будут рассматриваться в разделе 10.8. Связь между этими двумя научными областями проанализирована в [348].

В указанной работе Сталмана и Зюссмана была также предложена идея $\wedge$ регистрации ограничения, в соответствии с которой частичные результаты, полученные в ходе поиска, можно сохранять и повторно использовать на последующих этапах этого поиска. Данная идея была введена формально в поиск с возвратами Дектером [366]. Особенно простым методом является $\wedge$ проставление обратных отметок [522], в котором для предотвращения повторной проверки ограничений сохраняются и используются совместимые и несовместимые попарные присваивания.

Проставление обратных отметок можно комбинировать с обратным переходом, управляемым конфликтами; в [831] представлен гибридный алгоритм, который превосходит любой из этих методов, отдельно взятых (и это подтверждается формальным доказательством). В методе $\wedge$ динамического поиска с возвратами [558] сохраняются успешные частичные присваивания из полученных позднее подмножеств переменных при поиске с возвратами по предыдущему варианту, который не влияет на дальнейший успех.

Применение локального поиска при решении задач удовлетворения ограничений стало популярным после публикации [799] с описанием метода эмуляции отжига (см. главу 4), который широко используется для решения задач планирования.

Эвристика с минимальными конфликтами была впервые предложена в [601] и независимо разработана в [1058]. В [1447] показано, как эта эвристика может использоваться для решения задачи с 3 000 000 ферзей меньше чем за минуту. Этот поразительный успех локального поиска на основе эвристики с минимальными конфликтами при решении задачи с p ферзями привел к переоценке характера и распространения "легких"

и "трудных" задач. В [244] исследовалась сложность задач CSP, сформированных случайным образом, и было обнаружено, что почти все такие задачи являются либо тривиально легкими, либо не имеют решений. "Трудные" экземпляры задач встречаются, только если параметры генератора задач устанавливаются в некотором узком диапазоне, в пределах которого лишь примерно половина задач является разрешимой. Этот феномен рассматривается дополнительно в главе 7.

Исследования, касающиеся структуры и сложности задач CSP, начались с [499], где было показано, что поиск в деревьях с совместимыми дугами происходит без каких-

либо возвратов. Аналогичные результаты применительно к ациклическим гиперграфам были получены в области теории баз данных [92]. Со времени публикации этих статей достигнут значительный прогресс в части получения более общих результатов, касающихся связи между сложностью решения задачи CSP и структурой ее графа ограничений. Понятие ширины дерева было введено специалистами по теории графов Робертсоном и Сеймуром [1295]. Дектер и Перл [372], [373], основываясь на работе Фрейдера, применяли то же понятие (названное ими индуцированной шириной) к задачам удовлетворения ограничений и разработали подход с древовидной декомпозицией, кратко описанный в разделе 5.4. Опираясь на эту работу и на результаты из области теории баз данных, Готлобб и др. [585], [586] разработали понятие ширины гипердерева, которое основано на методе характеристики задачи CSP как гиперграфа. Они не только показали, что любую задачу CSP с шириной гипердерева w можно решить за время $O(nw + 1 \log n)$, но и обосновали утверждение, что критерий ширины гипердерева превосходит все ранее предложенные критерии "ширины" в том смысле, что в некоторых случаях ширина гипердерева является конечной, тогда как ширина, определяемая другими критериями, — неограниченной.

В литературе можно найти несколько хороших обзоров с описанием методов решения задач CSP, включая [73], [370] и [866], а также энциклопедические сборники статей [368] и [968]. В [1194] приведен обзор разрешимых классов задач CSP и описаны как методы структурной декомпозиции, так и методы, основанные на свойствах областей определения или свойствах самих ограничений. В [831] приведен аналитический обзор алгоритмов поиска с возвратами, а в [56] приведен обзор, характеризующийся большей практической направленностью. В [987] и [1514] эта тематика рассматривается гораздо более глубоко, чем было возможно ее представить в данной главе. Некоторые интересные приложения описаны в сборнике статей [500], который вышел под редакцией Фрейдера и Макворта. Статьи на тему удовлетворения ограничений регулярно публикуются в журнале *Artificial Intelligence* и в специализированном журнале *Constraints*. Основным местом встречи специалистов в этой области является конференция *International Conference on Principles and Practice of Constraint Programming*, часто называемая просто CP.

Упражнения 5.1. Самостоятельно сформулируйте определения следующих понятий: проблема удовлетворения ограничений, ограничение, поиск с возвратами, совместимость дуги, обратный переход и минимизация конфликтов.

5.2. Сколько решений имеет задача раскрашивания карты, показанная на рис. 5.1?

5.3. Объясните, почему при поиске решения задачи CSP одна из хороших эвристик состоит в том, что следует выбирать переменную, которая является наиболее ограниченной, и вместе с тем выбирать наименее ограничительное значение.

5.4. Рассмотрите задачу составления (а не решения) кроссвордов⁵: подбора слов, которые укладываются в прямоугольную сетку. Эта сетка, которая указывается как часть задания, определяет, какие квадраты пусты и какие затенены.

Предположим, что предоставлен список слов (например, словарь) и задача состоит в том, чтобы заполнить пустые квадраты с использованием любого подмножества из этого списка. Точно сформулируйте эту задачу следующими двумя способами. а) Как общую задачу поиска. Выберите соответствующий алгоритм поиска и определите эвристическую функцию, если, по вашему мнению, она необходима. Как лучше заполнять пустые квадраты: одновременно по одной букве или по одному слову? б) Как задачу удовлетворения ограничений. Должны ли переменные быть представлены в виде слов или букв?

Какая формулировка, по вашему мнению, является наилучшей? Объясните, почему?

5.5. Приведите точные формулировки каждой из перечисленных ниже задач в виде задач удовлетворения ограничений. а) $\wedge$ Планирование покрытия пола прямоугольниками. Найти в большом прямоугольнике неперекрывающиеся места для размещения меньших прямоугольников. б) $\wedge$ Составление расписания занятий. Определены следующие исходные данные: постоянное количество преподавателей и классов, список предлагаемых занятий и список возможных временных интервалов для занятий. С каждым преподавателем связано множество занятий, которые он может проводить.

5.6. Решите криптоарифметическую задачу, приведенную на рис. 5.2, вручную, с помощью поиска с возвратами, предварительной проверки, а также на основе эвристики MRV и наименее ограничительным значением.

5.7. (ё) В табл. 5.1 приведены результаты проверки производительности различных алгоритмов при решении задачи с p ферзями. Проведите испытания тех же алгоритмов при решении задач раскрашивания карты, формируемых случайным образом, с помощью следующего метода: разбросать p точек на единичном квадрате; выбрать точку X случайным образом, соединить X прямой линией с ближайшей точкой g , такой, что x еще не соединена с Y , и соединительная линия не пересекает какую-либо другую линию; повторять предыдущий шаг до тех пор, пока не будет исключена возможность проводить дальнейшие соединения. Сформируйте таблицу производительности для наибольшего значения l , с которым удастся справиться с помощью этих алгоритмов, используя как $d=4$, так и $d=3$ цветов. Прокомментируйте полученные вами результаты.

5 В [560] обсуждается несколько методов составления кроссвордов. В [938] рассматривается более сложная задача их решения.

5.8. Воспользуйтесь алгоритмом AC-3, чтобы показать, что проверка совместимости дуг позволяет обнаружить несовместимость частичного присваивания $\{W_A=\text{red}, V=\text{blue}\}$ для задачи, показанной на рис. 5.1.

5.9. Какова в наихудшем случае временная сложность при прогоне алгоритма AC3 применительно к задаче CSP с древовидной структурой?

5.10. Алгоритм AC-3 помещает обратно в очередь каждую дугу (x_k, x_L) каждый раз, когда любое значение удаляется из области определения x_i ? даже если каждое значение x_k совместимо с несколькими оставшимися значениями x_L .

Предположим, что для каждой дуги $(X_k|X_i)$ отслеживается количество оставшихся значений x_L , которые являются совместимыми с каждым значением x_k .

Объясните, как можно эффективно обновлять эти числа, и тем самым докажите, что совместимость дуг можно обеспечить за суммарное время $O(n^2d^2)$.

5.11. Покажите, как можно преобразовать одно тернарное ограничение, такое как " $A+B=C$ ", в три бинарных ограничения с использованием вспомогательной переменной. Может быть принято предположение о конечных областях определения. (Подсказка. Предусмотрите использование новой переменной, которая принимает значения, являющиеся парами других значений, и примените такие ограничения, как " x является первым элементом пары Y ".) Затем покажите, как можно трактовать аналогичным образом ограничения больше чем с тремя переменными. Наконец, покажите, как можно устранить унарные ограничения, модифицируя области определения переменных. На этом завершается демонстрация того, что любую задачу CSP можно преобразовать в задачу CSP только с бинарными ограничениями.

5.12. Ш Предположим, что известно, будто граф имеет множество разрыва цикла, содержащее не больше k узлов. Опишите простой алгоритм поиска минимального множества разрыва цикла, время прогона которого не намного превышает $O(n^k)$ для задачи CSP с n переменными. Найдите в литературе методы обнаружения приближенно минимальных множеств разрыва цикла за время, которое полиномиально зависит от размера множества разрыва. Обеспечивает ли наличие таких алгоритмов практическую применимость метода поиска на основе множества разрыва цикла?

5.13. Рассмотрите приведенную ниже логическую головоломку. В пяти домах, имеющих разный цвет, живут лица пяти национальностей, которые предпочитают разные сорта сигарет, пьют разные напитки и держат разных домашних питомцев. На основе следующих фактов найдите ответ на вопрос: "В каком доме держат зебру и в каком доме пьют воду?" • Англичанин живет в доме красного цвета. • Испанец держит собаку. • Норвежец живет в первом доме слева. • Сигареты "Куле" курят в

доме желтого цвета. • Человек, курящий "Честерфилд", живет рядом с домом человека, который держит лису. • Норвежец живет рядом с домом синего цвета. • Человек, курящий "Уинстон", держит улиток.

• Человек, курящий "Лаки Страйк", пьет апельсиновый сок. • Украинец пьет чай. • Японец курит "Парламент". • Сигареты "Куле" курят в доме, находящемся рядом с домом, где держат лошадь. • Кофе пьют в доме зеленого цвета. • Дом зеленого цвета находится непосредственно справа (с точки зрения того, кто решает эту задачу) от дома цвета слоновой кости. • Молоко пьют в среднем доме.

Обсудите различные представления этой задачи в виде задачи CSP. По каким причинам следовало бы предпочесть одно представление другому?

Глава 6. Поиск в условиях противодействия

6 Поиск В УСЛОВИЯХ В этой главе рассматриваются проблемы, возникающие при попытке агента планировать наперед в мире, где другие агенты составляют планы, направленные против него.

6А. Игры В главе 2 представлены мультиагентные варианты среды, в которых каждый конкретный агент вынужден принимать во внимание действия других агентов и устанавливать, как они повлияют на его собственное благополучие. Непредсказуемость действий этих прочих агентов может потребовать в процессе решения задачи агентом учета многих возможных непредвиденных ситуаций, как было описано в главе 3.

Различие между кооперативными и конкурентными мультиагентными вариантами среды также было показано в главе 2. Наличие конкурентных вариантов среды, в которых цели агентов конфликтуют, приводит к возникновению задач поиска в условиях противодействия, часто называемых $\wedge$ играми.

В математической теории игр, одной из ветвей экономики, любые мультиагентные варианты среды рассматриваются как игры, при условии, что влияние каждого агента на других является "значительным", независимо от того, являются ли агенты кооперативными или конкурентными¹. В искусственном интеллекте "играми" обычно называют довольно специфические формы взаимодействия агентов, которые теоретиками игр именуются как детерминированные, поочередные, охватывающие двух игроков $\wedge$ игры с нулевой суммой и с полной информацией. В терминологии, принятой в данной книге, это соответствует детерминированным, полностью наблюдаемым вариантам среды, в которых имеются два агента, обязанных чередовать свои действия, и в которых значения полезности в конце игры всегда равны и противоположны. Например, если один игрок выигрывает игру в шахматы (+1), другой игрок обязательно проигрывает (-1). В подобной ситуации

условия против¹ Варианты среды с очень большим количеством агентов лучше рассматривать как экономики, а не как игры.

действия возникают именно из-за такого противопоставления функций полезности агентов. В данной главе будут кратко рассматриваться игры с несколькими игроками, игры с ненулевой суммой и стохастические игры, но надлежащее обсуждение теории игр откладывается до главы 17.

Игры заставляли людей напрягать свои интеллектуальные способности (иногда до угрожающей степени) на протяжении всего существования цивилизации. В силу своего абстрактного характера игры являются привлекательным объектом исследований и в области искусственного интеллекта. Состояние игры можно легко представить, а поведение агентов обычно ограничено небольшим количеством действий, результаты которых определяются с помощью точных правил. Спортивные игры, такие как крокет и хоккей с шайбой, имеют гораздо более сложные описания, значительно больший диапазон возможных действий и довольно неточные правила, определяющие допустимость действий. За исключением проблематики создания робота-футболиста эти спортивные игры не привлекают значительного интереса в сообществе специалистов по искусственному интеллекту.

Ведение игр было одной из первых задач, рассматриваемых в области искусственного интеллекта. К 1950 году, почти сразу же после того, как компьютеры стали программируемыми, шахматами уже интересовались Конрад Цузе (изобретатель первого программируемого компьютера и разработчик первого языка программирования), Клод Шеннон (основоположник теории информации), Норберт Винер (создатель современной теории управления) и Алан Тьюринг. С тех пор уровень игры с применением компьютеров неуклонно повышался и достиг того, что компьютеры превзошли людей в шашках и игре "Отелло" ("реверси"), побеждали чемпионов (но не всегда) в шахматах и нардах, а также стали конкурентоспособными во многих других играх. Основным исключением остается игра го, в которой компьютеры пока еще выступают на любительском уровне.

Игры, в отличие от большинства учебных задач, которые рассматривались в главе 3, интересны тем, что в них очень трудно найти решение. Например, шахматы характеризуются в среднем коэффициентом ветвления, примерно равным 35, а игра часто продолжается до 50 ходов со стороны каждого игрока, поэтому дерево поиска имеет приблизительно 35100 или 10154 узлов (хотя граф поиска включает "всего лишь" около 1040 различных узлов). Поэтому игры, как и реальная жизнь, требуют способности принимать хоть какие-то решения, даже если вычисление оптимального решения неосуществимо. Кроме того, игры сурово наказывают за неэффективность. Притом что реализация поиска A^* , в два раза менее эффективная по сравнению с другой реализацией, просто потребует вдвое больше времени для

получения окончательного решения, шахматная программа, вдвое менее эффективно использующая отведенное ей время, по-видимому, потерпит поражение на самых ранних этапах игры, даже при всех прочих равных условиях. Поэтому исследователи, работающие в области ведения игр, стали авторами многих интересных идей, касающихся того, как обеспечить наилучшее возможное использование времени.

Начнем описание данной темы с определения понятий оптимального хода игры и алгоритма его поиска. Затем рассмотрим методы выбора хорошего хода в условиях ограниченного времени. Отсечение позволяет игнорировать те части дерева поиска, которые не оказывают влияния на окончательный выбор, а эвристические функции оценки позволяют приближенно рассчитывать истинную полезность состояния без проведения полного поиска. В разделе 6.5 рассматриваются такие игры, включающие элемент случайности, как нарды; кроме того, в данной главе рассматривается

бридж, который включает элементы $\wedge$ неполной информации, поскольку не все карты видны каждому игроку. Наконец, в этой главе будет описано, как новейшие программы ведения игр постепенно преодолевают сопротивление людей в борьбе с этими программами и каковы направления будущих разработок.

6.2. Принятие оптимальных решений в играх В данной главе речь идет об играх с двумя игроками, которые будут именоваться MAX и MIN по причинам, которые вскоре станут очевидными. Игрок MAX ходит первым, после чего игроки делают ходы по очереди до тех пор, пока игра не закончится.

В конце игры игроку-победителю присваиваются очки, а на побежденного налагается штраф. Игра может быть формально определена как своего рода задача поиска с описанными ниже компонентами. • Начальное состояние, которое включает позицию на доске и определяет игрока, который должен ходить. • Функция определения преемника, возвращающая список пар (move, state), каждая из которых указывает допустимый ход и результирующее состояние. • $\wedge$ Проверка терминального состояния, которая определяет, что игра закончена.

Состояния, в которых игра закончена, называются терминальными состояниями. • Функция полезности (называемая также целевой функцией, или функцией вознаграждения), которая сообщает числовое значение терминальных состояний.

В шахматах результатом является победа, поражение или ничья, со значениями +1, -1 или 0. Некоторые игры имеют более широкий диапазон возможных результатов; например, количество очков в нардах может составлять от +192 до -192. В настоящей главе в основном рассматриваются игры с нулевой суммой, хотя будут также кратко упоминаться игры с ненулевой суммой.

Начальное состояние и допустимые ходы каждой стороны определяют $\wedge$ дерево игры для данной игры. На рис. 6.1 показана часть дерева игры в крестики-нолики.

Из начального состояния игрок MAX имеет девять возможных ходов. Ходы чередуются так, что MAX ставит значок X, а MIN значок O, до тех пор пока не будут достигнуты листовые узлы, соответствующие терминальным состояниям, таким, что один игрок поставил три своих значка в один ряд или заполнены все клетки. Число под каждым листовым узлом указывает значение полезности соответствующего терминального состояния с точки зрения игрока MAX; предполагается, что высокие значения являются благоприятными для игрока MAX и неблагоприятными для игрока MIN (именно поэтому в теории этим игрокам присвоены такие имена). Задача игрока MAX состоит в использовании дерева поиска (особенно данных о полезности терминальных состояний) для определения наилучшего хода.

Оптимальные стратегии При решении обычных задач поиска оптимальное решение для игрока MAX должно представлять собой последовательность ходов, ведущих к цели — к терминальному состоянию, которое соответствует выигрышу. С другой стороны, в игре

участвует также игрок MIN, который имеет другое мнение по этому поводу. Это означает, что игрок MAX должен найти надежную $\wedge$ стратегию, позволяющую определить ход игрока MAX в начальном состоянии, затем ходы игрока MAX в состояниях, ставших результатом любого возможного ответа игрока MIN, а затем ходы MAX в состояниях, ставших результатом любого возможного ответа MIN на те ходы, и т.д. Грубо говоря, оптимальная стратегия приводит к итогу, по меньшей мере, такому же благоприятному, как и любая другая стратегия, в тех условиях, когда приходится играть с противником, не допускающим ошибок. Прежде всего рассмотрим, как найти эту оптимальную стратегию, даже притом что для MAX часто будет неосуществимой задача ее исчерпывающего вычисления в играх, более сложных, чем крестики-нолики.

MAX(X) MIN (O) MAX (K) MIN (O) Заключительное состояние TERMINAL Полезность X
X X -1 X X O X X X O o X X O X X O +1 Рис. 6.1. (Частичное) дерево поиска для игры крестики-нолики. Верхний узел представляет собой начальное состояние, а первым ходит игрок MAX, ставя значок X в пустой клетке. На этом рисунке показана часть дерева поиска, в которой демонстрируются чередующиеся ходы игроков MIN (значок O) и MAX (значок X). Ходы выполняются до тех пор, пока в конечном итоге не будет достигнуто одно из терминальных состояний, которым могут быть назначены данные о полезности в соответствии с правилами игры. Даже такие простые игры, как крестики-нолики, являются слишком сложными, чтобы можно было привести в этой книге для них полное дерево игры, поэтому перейдем к описанию тривиальной игры, показанной на рис. 6.2. Возможные ходы игрока MAX из корневого узла обозначены как a_1 и a_2 . Возможными ответами на ход a_i для

игрока MIN являются b_1, b_2, b_3 и т.д. Данная конкретная игра заканчивается после того, как каждый игрок, MAX и MIN, сделают по одному ходу. (Согласно терминологии теории игр, это дерево имеет глубину в один ход и состоит из сделанных двумя участниками ходов, каждый из которых называется $\wedge$ полуходом.) Полезности терминальных состояний в этой игре находятся в пределах от 2 до 14.

MAX MIN Рис. 6.2. Дерево игры с двумя полуходами. Узлы A представляют собой "узлы MAX", в которых очередь хода принадлежит игроку MAX, а узлы B рассматриваются как "узлы MIN". Терминальные узлы показывают значения полезности для MAX; остальные узлы обозначены их минимаксными значениями. Лучшим ходом игрока MAX от корня является a_1 , поскольку ведет к преемнику с наибольшим минимаксным значением, а наилучшим ответом MIN является b_1 , поскольку ведет к преемнику с наименьшим минимаксным значением. При наличии дерева игры оптимальную стратегию можно определить, исследуя $\wedge$ минимаксное значение каждого узла, которое можно записать как $\text{MinimaxValue}(n)$. Минимаксным значением узла является полезность (для MAX) пребывания в соответствующем состоянии, при условии, что оба игрока делают ходы оптимальным образом от этого узла и до узла, обозначающего конец игры. Очевидно, что минимаксным значением терминального состояния является просто его полезность.

Более того, если есть выбор, игрок MAX должен предпочесть ход, ведущий в состояние с максимальным значением, а игрок MIN — ведущий в состояние с минимальным значением. Поэтому имеет место приведенное ниже соотношение.

$\text{Minimax-Value}(n) = \wedge \text{Utility}(n)$, если n - терминальное состояние
 $\text{Max Minimax-Value}(s)$
 $s \in \text{Successors}(n)$ если n - узел MAX
 $\text{Min Minimax-Value}(s)$, если $n \in \text{Successors}(n)$ узел MIN
 Применим эти определения к дереву игры, показанному на рис. 6.2.

Терминальные узлы на низшем уровне уже обозначены числами, которые указывают их полезность. Первый узел MIN, обозначенный как B, имеет трех преемников со значениями 3, 12 и 8, поэтому его минимаксное значение равно 3. Аналогичным образом, другие два узла MIN имеют минимаксное значение, равное 2. Корневым узлом является узел MAX; его преемники имеют минимаксные значения 3, 2 и 2, поэтому сам корневой узел имеет минимаксное значение 3. Можно также определить понятие $\wedge$ минимаксного решения, принимаемого в корне дерева: действие a_1 является оптимальным выбором для игрока MAX, поскольку ведет к преемнику с наивысшим минимаксным значением.

В этом определении оптимальной игры для игрока MAX предполагается, что игрок MIN также играет оптимальным образом: он максимизирует результат, соответствующий наихудшему исходу игры для MAX. А что было бы, если игрок MIN не играл оптимальным образом? В таком случае можно легко показать (упр. 6.2), что

игрок МАХ добился бы еще большего. Могут существовать другие стратегии игры против соперников, играющих неоптимальным образом, которые позволяют добиться большего, чем минимаксная стратегия; но эти стратегии обязательно действуют хуже против соперников, играющих оптимально.

Минимаксный алгоритм ^Эк Минимаксный алгоритм (листинг 6.1) вычисляет минимаксное решение из текущего состояния. В нем используется простое рекурсивное вычисление минимаксных значений каждого состояния-преемника с непосредственной реализацией определяющих уравнений. Рекурсия проходит все уровни вплоть до листьев дерева, а затем минимаксные значения ^ резервируются по всему дереву по мере обратного свертывания рекурсии. Например, в дереве, показанном на рис. 6.2, алгоритм вначале выполнил рекурсию с переходом на нижние уровни до трех нижних левых узлов и применил к ним функцию полезности utility, чтобы определить, что значения полезности этих узлов соответственно равны 3, 12 и 8. Затем алгоритм определяет минимальное из этих значений, 3, и возвращает его в качестве зарезервированного значения узла В. Аналогичный процесс позволяет получить зарезервированные значения 2 для узла С и 2 для узла D. Наконец, берется максимальное из значений 3, 2 и 2 для получения зарезервированного значения 3 корневого узла.

Листинг 6.1. Алгоритм вычисления минимаксных решений. Он возвращает действие, соответствующее наилучшему возможному ходу, т.е. действие, ведущее к результату с наилучшей полезностью, в соответствии с предположением, что противник играет с целью минимизации полезности.

Функции Max-Value и Min-Value используются для прохождения через все дерево игры вплоть до листьев, что позволяет определить зарезервированное значение некоторого состояния function Minimax-Decision(state) returns действие action inputs: state, текущее состояние игры $v \leftarrow \text{Max-Value}(\text{state})$ return действие action в множестве Successors(state) со значением v function Max-Value(state) returns значение полезности if Terminal-Test(state) then return Utility(state) $v \leftarrow -\infty$ for a, s in Successors(state) do $v \leftarrow \text{Max}(v, \text{Min-Value}(s))$ return v function Min-Value(state) returns значение полезности if Terminal-Test(state) then return Utility(state) $v \leftarrow \infty$ for a, s in Successors(state) do $v \leftarrow \text{Min}(v, \text{Max-Value}(s))$ return v

Этот минимаксный алгоритм выполняет полное исследование дерева игры в глубину. Если максимальная глубина дерева равна $\ln$ и в каждом состоянии имеются b допустимых ходов, то временная сложность этого минимаксного алгоритма составляет $O(b^{\ln})$. Для алгоритма, который формирует сразу всех преемников, пространственная сложность равна $O(b^{\ln})$, а для алгоритма, который формирует преемников по одному, — $O(\ln)$ (см. с. 131). Безусловно, в реальных играх такие затраты времени полностью неприемлемы, но данный алгоритм служит в качестве основы математического анализа игр, а также более практичных алгоритмов.

Оптимальные решения в играх с несколькими игроками Многие популярные игры допускают наличие больше чем двух игроков.

Рассмотрим, как можно распространить идею минимаксного алгоритма на игры с несколькими игроками. С точки зрения технической реализации это сделать несложно, но при этом возникают некоторые новые и интересные концептуальные проблемы.

Вначале необходимо заменить единственное значение для каждого узла вектором значений. Например, в игре с тремя игроками, в которой участвуют игроки А, В и С, с каждым узлом ассоциируется вектор $\langle v_A, v_B, v_C \rangle$. Для терминальных состояний этот вектор задает полезность данного состояния с точки зрения каждого игрока.

(В играх с двумя игроками и нулевой суммой двухэлементный вектор может быть сокращен до единственного значения, поскольку значения в нем всегда противоположны.) Простейшим способом реализации такого подхода является применение функции Utility, которая возвращает вектор значений полезности.

Теперь необходимо рассмотреть нетерминальные состояния. Рассмотрим узел в дереве игры, показанном на рис. 6.3, который обозначен как X. В этом состоянии игрок С выбирает, что делать. Два варианта ведут к терминальным состояниям с векторами полезности $\langle v_A=1, v_B=2, v_C=6 \rangle$ и $\langle v_A=4, v_B=2, v_C=3 \rangle$. Поскольку 6 больше чем 3, игрок С должен выбрать первый ход. Это означает, что если достигнуто состояние X, то дальнейшая игра приведет к терминальному состоянию с полезностями $\langle v_A=1, v_B=2, v_C=6 \rangle$. Следовательно, зарезервированным значением X является этот вектор. Вообще говоря, зарезервированное значение узла p представляет собой вектор полезности такого узла-преемника, который имеет наивысшее значение для игрока, выбирающего ход в узле p.

Любой, кто играет в игры с несколькими игроками, такие как Diplomacy™ ("Дипломатия"), быстро узнает, что в них происходит гораздо больше событий, чем в играх с двумя игроками. В играх с несколькими игроками обычно создаются альянсы между игроками, либо формальные, либо неформальные. К тому же иногда по мере развития игры альянсы то формируются, то разрушаются. Как можно понять такое поведение? Являются ли альянсы естественным следствием выбора оптимальных стратегий для каждого игрока в игре с несколькими игроками? Как оказалось, они действительно могут стать таким следствием. Например, предположим, что игроки Айв имеют слабые позиции, а игрок С — более сильную позицию.

В таком случае и для А, и для В часто бывает оптимальным решение атаковать С, а не друг друга, поскольку иначе С уничтожит каждого из них по отдельности. Таким образом, сотрудничество становится следствием чисто эгоистичного поведения.

Безусловно, как только игрок С ослабнет под совместным натиском, альянс потеряет свой

смысл и соглашение может нарушить либо игрок А, либо игрок В. В некоторых случаях явно выраженные альянсы просто становятся конкретным выражением того, что и так произошло бы. А в других случаях попытка нарушить альянс вызывает общественное осуждение, поэтому игроки должны класть на весы немедленно достигаемую выгоду от нарушения альянса и долговременный убыток, возникающий изза того, что их не будут считать заслуживающими доверия. Дополнительная информация об этих сложностях в игре приведена в разделе 17.6.

А,2,6) D,2,3) F,1,2) G,4,1) E,1,1) A,5,2) G,7,1) E,4,5) Рис. 6.3. Первые три полухода игры с тремя игроками (А, В, С). Каждый узел обозначен значениями, достигаемыми с точки зрения каждого участника. Наилучший ход обозначен стрелкой, исходящей из корня. Если игра не имеет нулевую сумму, то сотрудничество может также возникать даже при наличии всего двух игроков. Допустим, что имеется терминальное состояние с полезностями $\langle v_A=1000, v_B=1000 \rangle$ и что 1000 — максимальная возможная полезность для каждого игрока. В таком случае оптимальная стратегия для обоих игроков заключается в том, чтобы делать все возможное для достижения этого состояния; это означает, что игроки автоматически вступают в сотрудничество для достижения обоюдной желаемой цели.

6.3. АЛЬФА-БЕТА-ОТСЕЧЕНИЕ При минимаксном поиске проблема состоит в том, что количество состояний игры, которые должны быть исследованы в процессе поиска, зависит экспоненциально от количества ходов. К сожалению, такую экспоненциальную зависимость устранить невозможно, но фактически существует возможность сократить ее наполовину. Весь секрет состоит в том, что вычисление правильного минимаксного решения возможно без проверки каждого узла в дереве игры. Это означает, что можно позаимствовать идею отсечения, описанную в главе 4, чтобы исключить из рассмотрения большие части дерева.

Конкретный метод, рассматриваемый в данной главе, называется α альфа-бета-отсечением.

Будучи применен к стандартному минимаксному дереву, этот метод возвращает такие же ходы, которые вернул бы минимаксный метод, но отсекает ветви, по всей вероятности, не способные повлиять на окончательное решение.

Снова рассмотрим дерево игры с двумя полуходами (см. рис. 6.2) и еще раз проведем расчет оптимального решения, на сей раз обращая особое внимание на то, что известно на каждом этапе этого процесса. Пояснения к данным этапам вычисления

приведены на рис. 6.4. Результат состоит в том, что минимаксное решение можно выявить, даже не приступая к вычислению значений двух листовых узлов.

Рис. 6.4. Этапы вычисления оптимального решения для дерева игры, показанного на рис. 6.2; в каждой точке известен ряд возможных значений для каждого узла: первый лист, расположенный ниже узла В, имеет значение 3. Поэтому В, который является узлом MIN, имеет, самое большее, значение 3 (а); второй лист, расположенный ниже узла В, имеет значение 12 (б); игрок МI N должен избегать этого хода, поэтому значение В, все еще самое большее, равно 3; третий лист, расположенный ниже узла В, имеет значение 8; мы проверили всех преемников узла В, поэтому значение В в точности равно 3 (в). Теперь можно сделать вывод, что значение корня, самое меньшее, равно 3, поскольку игрок МАХ в корне делает выбор со значением 3; первый лист, находящийся ниже С, имеет значение 2.

Поэтому С, который представляет собой узел MIN, имеет, самое большее, значение 2. Но известно, что узел В позволяет достичь значения 3, поэтому игрок МАХ ни в коем случае не должен выбирать узел С. Это означает, что нет смысла проверять остальных преемников узла С. Это — пример применения альфа-бета-отсечения (г). Первый лист, находящийся ниже D, имеет значение 14, поэтому D имеет, самое большее, значение 14. Оно все еще выше, чем наилучшая альтернатива для игрока МАХ (т.е. 3), поэтому необходимо продолжить исследование преемников узла D. Следует также отметить, что теперь определены предельные значения всех преемников корневого узла, поэтому значение корня также равно, самое большее, 14 (д); второй преемник D имеет значение 5, поэтому снова приходится продолжать исследование. Значение третьего преемника равно 2, поэтому теперь значение D точно равно 2. В корневом узле игрок МАХ принимает решение сделать ход, ведущий к узлу В, что позволяет ему получить значение 3 (е) Этот подход может также рассматриваться под другим углом — как упрощение формулы для получения минимаксного значения Minimax-Value. Допустим, что

два преемника узла С на рис. 6.4, еще не обработанные в процессе вычисления, имеют значения x и y , и предположим, что z — минимальное значение среди x и y .

В таком случае значение корневого узла можно найти следующим образом:

$$\text{Minimax-Value}(\text{root}) = \max(\min(C, 12, 8), \min(B, x, y), \min(A, 4, 5, 2)) = \max(C, \min(B, x, y), 2) = \max(C, z, 2)$$
 где $z < 2 = 3$ Иными словами, значение корневого узла, а следовательно, и минимаксное решение не зависит от значений отсеченных листовых узлов x и y .

Альфа-бета-отсечение может применяться к деревьям любой глубины; к тому же часто возникает возможность отсекал целые поддеревья, а не просто листья. Общий принцип состоит в следующем: рассмотрим узел l , находящийся где-либо в дереве (рис. 6.5), такой, что участник игры со стороны наблюдателя (назовем его

Игрок) имеет возможность выбрать ход, ведущий к этому узлу. Но если Игрок имеет лучший выбор t либо в родительском узле l , либо в любой другой точке выбора, находящейся еще выше в дереве, то s^* узел p никогда не будет достигнут в игре, происходящей в действительности. Поэтому после получения достаточной информации об узле l (путем исследования некоторых из его потомков) для того, чтобы с полной уверенностью прийти к этому заключению, можно выполнить его отсечение.

Игрок Противник Игрок Противник Рис. 6.5. Альфа-бета-отсечение: общий случай. Если для Игрока узел m лучше чем p , то узел p никогда не встретится в игре. Напомним, что минимаксный поиск осуществляется в глубину, поэтому в любой момент времени достаточно рассматривать узлы вдоль единственного пути в дереве.

Алгоритм альфа-бета-отсечения получил свое название по следующим двум параметрам, которые представляют пределы в зарезервированных значениях, присутствующих во всех узлах вдоль этого пути: α = значение наилучшего варианта (т.е. варианта с самым высоким значением), который был до сих пор найден в любой точке выбора вдоль пути для игрока MAX;

- β = значение наилучшего варианта (т.е. варианта с самым низким значением), который был до сих пор найден в любой точке выбора вдоль пути для игрока MIN.

Алгоритм альфа-бета-поиска в процессе своей работы обновляет значения α и β , а также отсекает оставшиеся ветви в узле (т.е. прекращает рекурсивные вызовы), как только становится известно, что значение текущего узла хуже по сравнению с текущим значением α или β для игрока MAX или MIN соответственно. Полный алгоритм приведен в листинге 6.2. Рекомендуем читателю проследить за его поведением применительно к дереву, показанному на рис. 6.4.

Листинг 6.2. Алгоритм альфа-бета-поиска. Обратите внимание на то, что применяемые здесь процедуры остаются такими же, как и процедуры алгоритма Minimax, приведенного в листинге 6.1, за исключением двух строк, введенных как в процедуру Min-Value, так и в Max-Value, которые сопровождают значения α и β (а также выполняют соответствующие действия по дальнейшей передаче этих параметров) function Alpha-Beta-Search(state) returns действие action inputs: state, текущее состояние в игре $v \leftarrow \text{Max-Value}(\text{state}, -\infty, +\infty)$ return действие action из множества Successors(state) со значением v function Max-Value(state, α , β) returns значение полезности inputs: state, текущее состояние в игре α , значение наилучшей альтернативы для игрока MAX вдоль пути к состоянию state β , значение наилучшей альтернативы для игрока MIN вдоль пути к состоянию state if Terminal-Test(state) then return Utility(state) $v \leftarrow -\infty$ for a, s in Successors(state) do $v \leftarrow \text{Max}(v, \text{Min-Value}(s, \alpha, \beta))$ if $v > \beta$ then return v $\alpha \leftarrow \text{Max}(\alpha, v)$ return v function Min-Value(state, α , β) returns

значение полезности inputs: state, текущее состояние в игре a, значение наилучшей альтернативы для игрока MAX вдоль пути к состоянию state C, значение наилучшей альтернативы для игрока MIN вдоль пути к состоянию state if Terminal-Test(state) then return Utility(state) for a, s in Successors(state) do $v < \text{rMin}(v, \text{Max-Value}(s, a, |3))$ if $v < a$ then return v P $< \text{rMin}(P, v)$ return v

Эффективность алгоритма альфа-бета-отсечения в высшей степени зависит от того, в каком порядке происходит проверка преемников. Например, на рис. 6.4, д, е невозможно было бы вообще выполнить отсечение каких-либо преемников узла d поскольку в первую очередь были бы сформированы наихудшие преемники (с точки зрения игрока MIN). А если бы в первую очередь был сформирован третий преемник, то была бы возможность отсечь двух остальных. На основании этого можно сделать вывод, что имеет смысл стремиться исследовать в первую очередь таких преемников, которые, по всей вероятности, могут стать наилучшими.

Если принять допущение, что это может быть сделано², то окажется, что в алгоритме альфа-бета-отсечения для определения наилучшего хода достаточно исследовать только $O(b^{1/2})$ узлов, а не $O(b^1)$ узлов, как при использовании минимаксного алгоритма. Это означает, что эффективный коэффициент ветвления становится равным $\sqrt{b}$, а не b ; например, для шахмат он равен 6, а не 35. Иными словами, за такое же время альфа-бета-поиск позволяет заглянуть в дерево игры примерно в два раза дальше по сравнению с минимаксным поиском. А если исследование преемников происходит в случайном порядке, а не по принципу первоочередного выбора наилучших вариантов, то при умеренных значениях b общее количество исследованных узлов будет составлять примерно $O(b^{3/4})$. В случае шахмат применение довольно простой функции упорядочения (например, такой, в которой в первую очередь рассматриваются взятия фигур, затем угрозы, затем ходы вперед, а после этого ходы назад) позволяет оставаться в пределах, не превышающих удвоенное значение результата $O(b^{1/2})$, который может быть получен в наилучшем случае. Добавление динамических схем упорядочения ходов, в частности, таких, в которых в первую очередь проверяются ходы, обозначенные как наилучшие на предыдущем этапе, позволяют подойти совсем близко к этому теоретическому пределу.

Как было отмечено в главе 3, наличие повторяющихся состояний в дереве поиска может вызвать экспоненциальное увеличение стоимости поиска. В играх повторяющиеся состояния встречаются часто из-за возникновения $\wedge$ транспозиций — различных перестановок последовательностей ходов, которые оканчиваются в одной и той же позиции. Например, если белые имеют в своем распоряжении ход a15 на который черные могут ответить ходом bх, а также еще один не связанный с ним ход a2 на другой стороне доски, на который может быть дан ответ b2, то обе последовательности, [a1b1ra2fb2] и [a1,b2,a2,b1], оканчиваются в одной и той же

позиции (как и перестановки, начинающиеся с a_2). Поэтому целесообразно сохранять оценку каждой конкретной позиции в хэш-таблице при первом ее возникновении, чтобы не приходилось вычислять ее повторно при последующих возникновениях. По традиции хэш-таблица с ранее встретившимися позициями называется $\wedge$ -таблицей транспозиций; она по сути идентична списку closed в алгоритме Graph-Search (см. с. 139).

Использование таблицы транспозиций может оказать чрезвычайно эффективное воздействие, которое иногда выражается в удваивании достижимой глубины поиска в шахматах. С другой стороны, если существует возможность вычислять оценки со скоростью в несколько миллионов узлов в секунду, то практически нет смысла хранить данные обо всех этих узлах в таблице транспозиций. Для выбора наиболее ценных из этих узлов были опробованы различные стратегии.

2 Очевидно, что при этом невозможно достичь идеальных результатов, поскольку в противном случае функцию упорядочения можно было бы использовать для ведения идеальной игры!

6.4. Неидеальные решения, принимаемые в реальном времени Минимаксный алгоритм формирует все пространство поиска игры, а алгоритм альфа-бета-отсечения позволяет отсекаать значительные его части. Тем не менее при использовании алгоритма альфа-бета-отсечения все еще приходится выполнять поиск вплоть до терминальных состояний, по крайней мере, в некоторой области пространства поиска. Обычно задача достижения такой глубины на практике не осуществима, поскольку ходы должны быть сделаны за какое-то приемлемое время, как правило, не больше чем за несколько минут. Вместо этого в статье Шеннона Programming a computer for playing chess от 1950 года было предложено, чтобы программы прекращали поиск раньше времени и применяли к состояниям какую-то эвристическую функцию оценки, по сути, преобразуя нетерминальные узлы в терминальные листья. Другими словами, в этой статье было предложено модифицировать минимаксный поиск или альфа-бета-поиск в двух отношениях: заменить функцию полезности эвристической функцией оценки Eval, которая дает оценку полезности данной позиции, а проверку терминальной позиции заменить $\wedge$ проверкой останова, которая позволяет решить, когда следует применять функцию Eval.

Функции оценки Функция оценки возвращает прогноз ожидаемой полезности игры из данной конкретной позиции по аналогии с тем, как эвристические функции, описанные в главе 4, возвращают прогнозируемое значение расстояния до цели. Идея такого "оценщика" в то время, когда Шеннон предложил его воспользоваться, была не нова. В течение многих столетий шахматисты (и поклонники других игр) разработали способы выработки суждений о стоимости позиции, поскольку люди

еще более ограничены в объемах поиска, который может быть ими выполнен, чем компьютерные программы.

Должно быть очевидно, что производительность любой программы ведения игры зависит от качества применяемой функции оценки. Неточная функция оценки приведет агента к позициям, которые окажутся проигрышными. Поэтому возникает важный вопрос — как именно следует проектировать хорошие функции оценки?

Во-первых, функция оценки должна упорядочивать терминальные состояния таким же образом, как и настоящая функция полезности; в противном случае использующий ее агент может выбрать неоптимальные ходы, даже обладая способностью просчитывать все ходы до конца игры. Во-вторых, вычисления не должны занимать слишком много времени! (В функции оценки можно было бы вызывать MinimaxDecision в качестве процедуры и вычислять точную стоимость данной позиции, но это поставило бы под сомнение то, к чему мы стремимся, — экономию времени.) В-третьих, для нетерминальных состояний значения этой функции оценки должны строго коррелировать с фактическими шансами на выигрыш.

Выражение "шансы на выигрыш" на первый взгляд может показаться странным.

В конце концов, шахматы — это же не игра с элементами случайности: в ней безусловно известно текущее состояние и для определения следующего хода не нужно бросать жребий. Но если поиск должен прекращаться в нетерминальных состояниях, то в данном алгоритме будет обязательно оставаться неопределенность в отно-

шении окончательных исходов для этих состояний. Неопределенность такого рода вызвана вычислительными, а не информационными ограничениями. Из-за ограниченного объема вычислений, которые разрешено выполнить в функции оценки для данного конкретного состояния, лучшее, что она может сделать, — это принять какое-то предположение в отношении конечного результата.

Рассмотрим эту идею немного более конкретно. Функции оценки чаще всего действуют по принципу вычисления различных ^ характеристик данного состояния, например, в шахматах одной из таких характеристик является количество пешек, принадлежащих каждой из сторон. Эти характеристики, вместе взятые, определяют различные категории, или классы эквивалентности состояний: состояния из каждой категории имеют одни и те же значения для всех своих характеристик. Вообще говоря, любая конкретная категория включает некоторые состояния, которые ведут к победе, к ничьей или поражению. Функция оценки не позволяет определить, какими являются те или иные состояния, но способна вернуть единственное значение, которое отражает процентную долю этих состояний в каждом результате. Например, предположим, полученный опыт показывает, что 72% состояний,

встретившихся в данной категории, ведут к победе (полезность +1); 20% — к поражению (-1) и 8% к ничьей @). В таком случае приемлемой оценкой для состояний этой категории становится взвешенное среднее, или ожидаемое значение: $@ .72x+1) + @ .20x-1) + @ .08x0) = 0.52$.

В принципе, ожидаемое значение можно определить для каждой категории, получив в итоге функцию оценки, применимую для любого состояния. Как и в случае терминальных состояний, функция оценки не обязана возвращать фактические ожидаемые значения, при условии, что упорядочение состояний остается тем же самым.

На практике для проведения анализа такого рода требуется учитывать слишком много категорий и поэтому накопить слишком много опыта, чтобы можно было оценить все вероятности выигрыша. Вместо этого в большинстве функций оценки вычисляются отдельные представленные в числовом виде значения вклада, зависящего от каждой характеристики, после чего эти значения комбинируются для поиска суммарного значения. Например, в учебниках по шахматам для начинающих можно найти приближенные оценки ^ стоимости материала для каждой фигуры: например, такие, что пешка имеет стоимость 1, конь или слон — 3, ладья — 5, а ферзь — 9. Другие характеристики, такие как "хорошая пешечная структура" и "безопасность короля", могут оцениваться как равные, скажем, половине стоимости пешки. После этого стоимости таких характеристик просто складываются для получения оценки позиции. Надежное преимущество, эквивалентное стоимости пешки, расценивается как значительная вероятность выигрыша, а надежное преимущество в три пешки должно почти наверняка обеспечить победу, как показано на рис. 6.6, а.

В математике функция оценки такого типа называется ^ взвешенной линейной функцией, поскольку она может быть представлена следующим образом: $Eval(s) = W_1 f_1(s) + W_2 f_2(s) + \dots + W_n f_n(s) = \sum_{i=1}^n W_i f_i(s)$ где каждый коэффициент w_i представляет собой вес, а каждая функция f_i оценивает некоторую характеристику позиции. В шахматах функция f_L может определять количество на доске фигур каждого вида, а коэффициент w_i — оценивать стоимости этих фигур А за пешку, 3 за слона и т.д.).

а) Ход белых б) Ход белых Рис. 6.6. Две немного разные шахматные позиции: черные имеют преимущество в одного коня и двух пешек и должны выиграть партию (а); черные проигрывают после того, как белые берут ферзя (б) На первый взгляд метод вычисления суммы стоимостей характеристик может показаться приемлемым, но в действительности он основан на очень радикальном допущении, что вклад каждой характеристики не зависит от стоимости других характеристик. Например, присваивая слону стоимость 3, мы игнорируем тот факт, что слоны становятся более мощными в конце игры, когда имеют большой объем пространства для маневра. По

этой причине в современных программах для шахмат и других игр используются также нелинейные комбинации характеристик. Например, пара слонов может стоить немного больше по сравнению с удвоенной стоимостью одного слона, а слон стоит немного больше в конце игры, чем в начале.

Внимательный читатель должен был заметить, что все эти характеристики и веса не входят в состав шахматных правил! Они были выработаны в течение столетий на основе опыта игры людей в шахматы. Применение этих характеристик и весов на основе описанной линейной формы оценки позволяет добиться наилучшей аппроксимации по отношению к истинному упорядочению состояний по стоимости. В частности, опыт показывает, что надежное материальное преимущество больше чем в один пункт, по всей вероятности, приводит к выигрышу при всех прочих равных условиях; преимущество в три пункта является достаточным почти для безусловной победы. В таких играх, где опыт указанного вида отсутствует, веса функции оценки могут быть получены с помощью методов машинного обучения, приведенных в главе 18. Является обнадеживающим тот факт, что применение указанных методов к шахматам подтвердило, что слон действительно имеет стоимость, примерно равную трем пешкам.

Прекращение поиска Следующий этап состоит в том, что алгоритм Alpha-Beta-Search должен быть модифицирован так, чтобы он вызывал эвристическую функцию Eval, когда возникает необходимость остановить поиск. С точки зрения реализации необходимо

заменить две строки в листинге 6.2, в которых упоминается функция TerminalTest, следующей строкой: `if Cutoff-Test(state, depth) then return Eval(state)` Необходимо также предусмотреть выполнение определенных технических операций для того, чтобы текущее значение глубины `depth` наращивалось при каждом рекурсивном вызове. Наиболее прямолинейный подход к управлению объемом поиска состоит в том, что должен устанавливаться фиксированный предел глубины, чтобы функция Cutoff-Test (state, depth) возвращала значение `true` при всех значениях `depth`, превышающих некоторую фиксированную глубину `d`. (Она должна также возвращать `true` для всех терминальных состояний, как было предусмотрено и в функции Terminal-Test.) Глубина `d` выбрана таким образом, чтобы используемое время не превышало допустимое по правилам игры.

Более надежный подход состоит в использовании метода итеративного углубления, который определен в главе 3. По окончании отведенного времени программа возвращает ход, выбранный по итогам наиболее глубокого завершенного поиска. Однако подобные подходы могут приводить к ошибкам, обусловленным приближенным характером функции оценки. Еще раз рассмотрим простую функцию оценки для шахмат, основанную на учете преимущества в материале.

Предположим, что программа выполняет поиск до предела глубины, достигая позиции, приведенной на рис. 6.6, б, где черные имеют перевес на одного коня и две пешки. Программа сообщила бы об этом как об эвристическом значении данного состояния, объявив тем самым, что это состояние, по всей вероятности, приведет к победе черных. Но на следующем ходу белые берут ферзя черных без компенсации. Поэтому в действительности данная позиция является выигрышной для белых, но об этом можно было бы узнать, только заглянув вперед еще на один полуход.

Очевидно, что требуется более сложная проверка останова. Функция оценки должна применяться только к позициям, которые являются $\wedge$ спокойными, т.е. характеризующимися низкой вероятностью того, что в них в ближайшем будущем произойдут резкие изменения в стоимости. Например, в шахматах такие позиции, в которых могут быть сделаны желательные взятия фигур, не являются спокойными для такой функции оценки, в которой учитывается лишь материал. Неспойкие позиции могут быть дополнительно развернуты до тех пор, пока не будут достигнуты спокойные позиции. Подобный дополнительный поиск называется $\wedge$ поиском спокойных позиций; иногда он ограничивается тем, что в нем рассматриваются только ходы определенных типов, такие как ходы со взятием фигур, что позволяет быстро устранять все неопределенности в этой позиции.

Задача устранения $\wedge$ эффекта горизонта является более сложной. Этот эффект возникает, если программа сталкивается с каким-то ходом противника, который причиняет серьезный ущерб и в конечном итоге является неизбежным.

Рассмотрим шахматную позицию, приведенную на рис. 6.7. Черные превосходят белых по количеству материала, но если белые смогут продвинуть свою пешку с седьмой горизонтали на восьмую, то пешка станет ферзем и обеспечит легкую победу белых. Черные могут отсрочить этот итог на 14 полуходов, объявляя шах белым с помощью ладьи, но пешка неизбежно станет ферзем. Одним из недостатков поиска на фиксированную глубину является то, что применяемый при

этом алгоритм не позволяет определить, что такие отвлекающие ходы не способны предотвратить ход с превращением пешки в ферзя. В этом случае принято считать, что отвлекающие ходы выводят неизбежный ход с превращением пешки в ферзя "за пределы горизонта поиска" — в то место, где опасный ход невозможно обнаружить.

По мере того как совершенствование аппаратных средств, применяемых для ведения игры в шахматы, приводит к увеличению глубины поиска, становится все более возможным то, что эффект горизонта будет возникать не так часто, поскольку очень длинные последовательности ходов, позволяющие отсрочить выполнение нежелательного хода, возникают крайне редко. Для предотвращения эффекта горизонта без слишком значительного увеличения стоимости поиска оказалось

также весьма эффективным использование $\wedge$ одинарных расширений. Одинарным расширением называется ход, который "безусловно лучше" по сравнению со всеми другими ходами в данной конкретной позиции. Поиск с одинарным расширением позволяет выйти за обычные пределы глубины поиска без внесения значительных издержек, поскольку в нем коэффициент ветвления равен 1. (Поиск спокойной позиции может рассматриваться как один из вариантов одинарных расширений.) На рис. 6.7 поиск с одинарным расширением позволяет найти выполняемый в конечном итоге ход превращения пешки в ферзя, при условии, что ходы черных с объявлением шаха и ходы белых королем могут быть определены как "безусловно лучшие" по сравнению с другими вариантами.

Ход черных Рис. 6.7. Проявление эффекта горизонта. Серия шахов черной ладьей приводит к принудительному выводу неизбежного хода белых с превращением пешки в ферзя "за горизонт", в связи с чем эта позиция начинает казаться допускающей выигрыш черных, тогда как фактически она является выигрышной для белых. До сих пор речь шла о том, что прекращение поиска на определенном уровне и выполнение альфа-бета-отсечения, по-видимому, не влияет на результат.

Существует также возможность выполнять $\wedge$ предварительное отсечение, а это означает, что некоторые ходы в данном конкретном узле отсекаются немедленно, без дальнейшего рассмотрения. Очевидно, что большинство людей, играющих в шахматы, рассматривают лишь несколько ходов из каждой позиции (по крайней мере, сознательно). К сожалению, этот подход является довольно опасным, поскольку нет никакой гарантии того, что не произойдет отсечение лучшего хода.

А если отсечение применяется недалеко от корня, результат может оказаться катастрофическим, поскольку слишком часто возникают такие ситуации, что программа пропускает некоторые "очевидные" ходы. Предварительное отсечение может использоваться безопасно в особых ситуациях (например, если два хода являются симметричными или эквивалентными по каким-то другим признакам, то необходимо рассматривать только один из них) или при анализе узлов, которые находятся глубоко в дереве поиска.

В результате совместного использования всех методов, описанных выше, появляется возможность создать программу, которая неплохо играет в шахматы (или другие игры). Предположим, что реализована функция оценки для шахмат, предусмотрена разумная проверка останова в сочетании с поиском спокойной позиции, а также предусмотрена большая таблица транспозиций. Кроме того, предположим, что, затратив целые месяцы на скрупулезную разработку программ, функционирующих на уровне битов, мы получили возможность формировать и оценивать примерно миллион узлов в секунду на новейшем персональном компьютере, что позволяет выполнять поиск приблизительно среди 200 миллионов узлов в расчете на каждый

ход при стандартном контроле времени (три минуты на каждый ход). Коэффициент ветвления для шахмат составляет в среднем примерно 3 5, а355 равно приблизительно 50 миллионам, поэтому при использовании минимаксного поиска мы получим возможность заглядывать вперед лишь приблизительно на пять полуходов. Такая программа, хотя и не совсем некомпетентная, может быть легко обманута человеком, игроком в шахматы среднего уровня, который иногда способен планировать на шесть или восемь полуходов вперед. Алфавита-поиск позволяет достичь глубины приблизительно в 10 полуходов, что приводит к усилению игры до уровня мастера. В разделе 6.7 описаны дополнительные методы отсечения, которые позволяют увеличить эффективную глубину поиска примерно до 14 полуходов. Чтобы достичь уровня гроссмейстера, требуется тщательно настроенная функция оценки и большая база данных с записями оптимальных ходов в дебюте и эндшпиле. Не мешало бы также иметь суперкомпьютер для эксплуатации на нем такой программы!

6.5. Игры, которые включают элемент случайности В реальной жизни происходит много непредсказуемых внешних событий, из-за которых люди попадают в непредвиденные ситуации. Эта непредсказуемость отражается во многих играх за счет включения элемента случайности, такого как метание жребия. Таким образом, игры с элементов случайности позволяют нам на один шаг приблизиться к реальности и поэтому имеет смысл рассмотреть, как это отражается на процессе принятия решений.

Одной из типичных игр, в которых сочетается удача и искусство, являются нарды. Перед каждым своим ходом игрок бросает кубики для определения допустимых ходов. Например, в позиции игры в нарды, приведенной на рис. 6.8, белым выпали очки 6-5 и они имеют четыре возможных хода.

О 12 3 4 5 6 7 8 9 10 И 12 25 24 23 22 21 20 19 18 17 16 15 14 13 Рис. 6.8. Типичная позиция игры в нарды. Цель игры состоит в том, чтобы снять с доски все фишки. Белые ходят по часовой стрелке к полю 25, а черные — против часовой стрелки к полю 0. Фишка может переходить на любое поле, если на нем не находится несколько фишек противника; если же на этом поле есть только одна фишка противника, она попадает в плен и должна начать свое движение с самого начала. В показанной здесь позиции белые после метания жребия получили очки 6-5 и должны выбрать среди четырех допустимых ходов:

Е-10,5-11), Е-11,19-24), Е-10,10-16) и Е-11,11-16) Хотя белым известно, каковы их допустимые ходы, они не знают, какие очки принесет метание жребия черным, и поэтому не могут определить, какими будут допустимые ходы черных. Это означает, что белые не имеют возможности сформировать стандартное дерево игры такого типа, которое встретилось нам в шахматах, а также крестиках-ноликах. Дерево игры

в нарды, кроме узлов MAX и MIN, должно включать $\wedge$ узлы жеребьевки. Узлы жеребьевки обозначены на рис. 6.9 кружками.

Ветви, ведущие от каждого узла жеребьевки, обозначают возможные результаты метания жребия, поэтому каждая из них отмечена надписью с указанием количества очков и вероятности, с которой могут быть получены эти очки. Существуют 36 различных сочетаний очков на двух кубиках, и все они являются равновероятными; но поскольку такие сочетания очков, как 6-5 и 5-6, являются одинаковыми, имеется только 21 различимое сочетание очков. Вероятность появления шести дублей (от 11 до 6-6) равна $1/36$, а каждое из 15 остальных различных сочетаний очков имеет вероятность $1/18$.

TERMINAL 2-11 -11 Рис. 6.9. Схематическое изображение дерева игры для одной из позиций игры в нарды Следующий этап состоит в том, чтобы понять, как следует принимать правильные решения. Безусловно, и в этом случае требуется найти такой ход, который ведет к наилучшей позиции. Однако результирующие позиции не имеют определенных минимаксных значений. Вместо этого существует возможность вычислить только ожидаемое значение, в котором ожидаемый результат устанавливается с учетом всех возможных выпадений жребия, которые могут произойти. Это приводит к обобщению минимаксного значения для детерминированных игр до $\wedge$ ожидаемого минимаксного значения (expectiminimax value) для игр с узлами жеребьевки.

Терминальные узлы и узлы max и MIN (для которых известны результаты жеребьевки) применяются точно так же, как и прежде, а узлы жеребьевки оцениваются путем получения взвешенного среднего значений, полученных в результате всех возможных выпадений жребия, т.е. следующим образом:

$Expectiminimax(l) = \wedge Utility(l)$, если l - терминальное состояние
 $Expectiminimax(s)$, если l - узел MAX
 $\max_{s \in Successors(n)} Expectiminimax(s)$, если l - узел MIN
 $\max_{s \in Successors(n)} \sum P(s) \cdot Expectiminimax(s)$, если n - узел жеребьевки
 $\sum_{s \in Successors(n)} P(s) \cdot Expectiminimax(s)$

где функция определения преемника для узла жеребьевки n просто дополняет состояние n каждым возможным выпадением жребия для формирования каждого преемника s , а $P(s)$ — вероятность, с которой происходит выпадение жребия.

Результаты вычисления этих уравнений могут резервироваться рекурсивно во всех узлах вплоть до корня дерева точно так же, как и в минимаксном алгоритме. Оставляем читателю проработку всех деталей этого алгоритма в качестве упражнения.

Оценка позиции в играх с узлами жеребьевки По аналогии с минимаксными значениями, очевидный подход к использованию ожидаемых минимаксных

значений состоит в том, чтобы останавливать поиск в некоторой точке и применять функцию оценки к каждому листу. На первый взгляд может показаться, что функции оценки для таких игр, как нарды, должны быть полностью подобными функциям оценки для шахмат, ведь от них требуется лишь то, чтобы они присваивали более высокие оценки лучшим позициям. Но в действительности наличие узлов жеребьевки означает, что требуется более тщательный анализ смысла таких оценочных значений. На рис. 6.10 показано, что происходит в играх с элементами случайности — при использовании функции оценки, которая присваивает листьям значения $[1,2,3,4]$, наилучшим является ход А1, а если присваиваются значения $[1,20,30,400]$, наилучшим становится ход А2. Поэтому программа ведет себя полностью по-разному, если вносятся изменения в шкалу некоторых оценочных значений! Как оказалось, такой чувствительности к изменению шкалы можно избежать при условии, что функция оценки представляет собой положительную линейную трансформацию вероятности выигрыша из некоторой позиции (или, в более общем смысле, ожидаемой полезности данной позиции). В этом состоит важное и общее свойство ситуаций, связанных с наличием неопределенности, и это свойство дополнительно рассматривается в главе 16.

2 2 3 3 1 14 4 20 20 30 30 1 1 400 400 Рис. 6.10. Пример того, что в результате трансформации оценочных значений листьев, при которой не изменяется упорядочение этих значений, наилучший ход становится другим

Сложность оценки ожидаемых минимаксных значений Если бы в программе были заранее известны все выпадения жребия, которые должны произойти в течение остальной части игры, то поиск решения в игре с жеребьевкой был бы полностью аналогичным поиску решения в игре без жеребьевки, который осуществляется минимаксным алгоритмом поиска за время $O(b^d)$. Но поскольку в алгоритме, использующем ожидаемые минимаксные значения, рассматриваются также все возможные последовательности выпадения жребия, для его работы требуется время $O(p^d)$, где p — количество различных вариантов выпадения жребия.

Даже если глубина поиска ограничена некоторой небольшой величиной d , из-за таких дополнительных затрат времени, намного более значительных по сравнению с минимаксным алгоритмом, в большинстве игр с элементами случайности становится неосуществимым стремление заглянуть в дерево поиска на очень большую глубину. В нардах p равно 21, а d обычно составляет приблизительно 20, но в некоторых ситуациях может достигать величины 4000 для выпадений жребия, включающих повторяющиеся очки. По-видимому, все, на что можно рассчитывать, — это заглянуть вперед на три полухода.

Еще один способ размышления об этой проблеме состоит в следующем: преимущество альфа-бета-поиска является то, что в нем игнорируются будущие направления развития игры, которые просто не должны быть реализованы, если в

игре всегда выбирается наилучший ход. Поэтому альфа-бета-поиск сосредоточивается на наиболее вероятных событиях. В играх с жеребьевкой вероятных последовательностей ходов не может быть, поскольку, для того, чтобы были сделаны данные ходы, вначале должен выпасть правильный жребий, который сделал бы их допустимыми.

В этом заключается общая проблема, возникающая в любой такой ситуации, когда в картину мира вмешивается неопределенность: количество вариантов дальнейших действий умножается в чрезвычайной степени, а формирование подробных планов действий становится бессмысленным, поскольку мир, скорее всего, будет играть совсем в другую игру.

Несомненно, читателю уже приходила в голову мысль, что к деревьям игр с узлами жеребьевки, по-видимому, можно было бы применить какой-то метод, подобный альфа-бета-отсечению. Как оказалась, такая возможность действительно существует. Анализ узлов MIN и MAX остается неизменным, но можно также обеспечить отсечение узлов жеребьевки с использованием некоторой доли изобретательности.

Рассмотрим узел жеребьевки C, приведенный на рис. 6.9, и определим, что будет происходить со значением этого узла по мере исследования и оценки его дочерних узлов.

Можно ли найти верхний предел значения C до того, как будут проверены все его дочерние узлы? (Напомним, что именно это требуется в альфа-бета-поиске для того, чтобы можно было выполнить отсечение узла и его поддеревя.) На первый взгляд такое требование может показаться невыполнимым, поскольку значение узла C представляет собой среднее значение его дочерних узлов. А до тех пор пока не будут рассмотрены все выпадения жребия, это среднее может представлять собой что угодно, поскольку сами неисследованные дочерние узлы могут иметь вообще любое значение. Тем не менее, если будут установлены пределы допустимых значений функции полезности, то появится возможность определять пределы и этого среднего.

Например, если будет установлено, что все значения полезности находятся в пределах от

+3 до -3, то значения листовых узлов становятся ограниченными, а это, в свою очередь, позволяет установить верхний предел значения узла жеребьевки, не рассматривая все его дочерние узлы.

Карточные игры Карточные игры интересны не только тем, что они часто бывают связаны с денежными ставками, но и по многим другим причинам. Количество различных вариантов карточных игр чрезвычайно велико, но в данной главе мы

сосредоточимся на таких вариантах, в которых карты тасуют случайным образом в начале игры и каждый игрок получает на руки карты, которые он не показывает другим игрокам. К таким играм относятся бридж, вист, "Червы" и некоторые виды покера.

На первый взгляд может показаться, что карточные игры полностью аналогичны играм с жеребьевкой: раздача карт происходит случайным образом, а сами карты определяют, какие ходы могут быть сделаны каждым игроком. Однако в картах вся жеребьевка происходит с самого начала! Это замечание будет обсуждаться ниже более подробно и будет показано, что такая особенность рассматриваемых карточных игр весьма полезна на практике. Вместе с тем, это замечание одновременно является совершенно неправильным по очень интересным причинам.

Представьте себе, что два игрока, MAX и MIN, разыгрывают какую-то реальную раздачу по четыре карты в бридже с двумя игроками, в котором открыты все карты.

На руках находятся следующие карты, и игрок MAX должен ходить первым:

MAX: $\heartsuit 6$ $\spadesuit 6$ * 9 8 MIN: $\heartsuit 4$ A 2 * 10 5 Предположим, что игрок MAX ходит с карты * 9. Теперь должен ходить игрок MIN, который может выбросить карту * 10 или * 5. Игрок MIN кладет карту * 10 и забирает взятку. Затем очередь хода переходит к игроку MIN, который ходит картой A 2. У игрока MAX масти пика нет (и поэтому он не может забрать эту взятку), следовательно, он обязан выбросить какую-то другую карту. Очевидным вариантом является карта $\heartsuit 6$, поскольку две другие оставшиеся карты являются старшими.

Теперь, какой бы картой не ходил игрок MIN во время следующего розыгрыша, игрок MAX возьмет две оставшиеся взятки и игра закончится ничьей, при двух взятках у каждого игрока. С использованием подходящего варианта минимаксного поиска (см. упр. 6.12) можно легко показать, что фактически ход картой * 9, сделанный игроком MAX, был оптимальным.

Теперь заменим карты на руках игрока MIN и вместо карты $\heartsuit 4$ введем карту $\heartsuit 4$:

MAX: $\heartsuit 6$ $\spadesuit 6$ * 9 8 MIN: $\heartsuit 4$ A 2 * 10 5 Рассматриваемые два случая являются полностью симметричными: ход игры будет одинаковым, за исключением того, что при розыгрыше второй взятки игрок MAX выбросит карту $\heartsuit 6$. Игра снова закончится ничьей при двух взятках у каждого игрока, и ход картой * 9 является оптимальным.

До сих пор все шло хорошо. А теперь скроем одну из карт игрока MIN и допустим, что игрок MAX знает, что у игрока MIN на руках либо первая раздача (с картой $\heartsuit 4$), либо вторая раздача (с картой $\heartsuit 6$), но он не знает, какая именно из них. Игрок MAX рассуждает следующим образом.

Ход картой * 9 является оптимальным решением в игре против первой и второй раздачи на руках игрока MIN, поэтому теперь этот ход должен быть оптимальным, поскольку известно, что на руках у игрока MIN имеется один из этих двух вариантов раздачи.

На более обобщенном уровне можно сказать, что игрок мах использует подход, который может быть назван "усреднением по прогнозам". Идея его состоит в том, чтобы при наличии карт на руках у противника, которые не видны игроку, оценивать каждый возможный вариант действий, вначале вычисляя минимаксное значение этого действия применительно к каждой возможной раздаче карт, а затем вычисляя ожидаемое значение по всем раздам с использованием вероятности каждой раздачи.

Если читатель посчитает такой подход разумным (или если он не может судить о нем, поскольку не знаком с бриджем), то ему следует поразмыслить над приведенным ниже рассказом.

Первый день. Дорога А ведет к куче золотых слитков; дорога в ведет к развилке. Если вы от развилки пойдете налево, то найдете гору драгоценностей, а если пойдете направо, то попадете под автобус.

Второй день. Дорога А ведет к куче золотых слитков; дорога В ведет к развилке. Если вы от развилки пойдете направо, то найдете гору драгоценностей, а если пойдете налево, то попадете под автобус.

Третий день. Дорога А ведет к куче золотых слитков; дорога в ведет к развилке. Если вы от развилки выберете правильное направление, то найдете гору драгоценностей, а если неправильное, то попадете под автобус.

Очевидно, что в первые два дня решение выбрать дорогу в не лишено смысла, но в третий день ни один человек в здравом уме не выберет дорогу в. Однако именно такой вариант подсказывает метод усреднения по прогнозам: дорога В является оптимальной в ситуациях, возникающих в первый и второй дни, поэтому она оптимальна и в третий день, поскольку должна иметь место одна из двух предыдущих ситуаций. Вернемся к карточной игре: после того как игрок МАХ пойдет картой * 9, игрок MIN заберет взятку картой * 10. Как и прежде, MIN пойдет с карты а 2, но теперь игрок мах окажется перед развилкой на дороге без каких-либо указаний. Если игрок МАХ выбросит карту v 6, а у игрока MIN все еще будет оставаться карта v 4, то эта карта у 4 станет козырной и игрок МАХ проиграет игру. Аналогичным образом, если игрок МАХ выбросит карту | 6, а у игрока MIN все еще будет оставаться карта | 4, игрок МАХ также проиграет.

Поэтому игра с первым ходом картой * 9 ведет к ситуации, в которой игрок мах имеет 50%- ную вероятность проигрыша. (Для него было бы гораздо лучше вначале

сыграть картами v 6 и $\frac{1}{6}$, гарантируя для себя ничейную игру.) Из всего этого можно извлечь урок, что при недостатке информации игрок должен учитывать, какую информацию он будет иметь в каждый момент игры.

Недостаток алгоритма, применяемого игроком MAX, состоит в том, что в нем предполагается, будто при каждой возможной раздаче игра будет развиваться так, как если бы все карты оставались видимыми. Как показывает данный пример, это вынуждает игрока MAX действовать таким образом, как будто неопределенность разрешится, когда настанет время. Кроме того, в алгоритме игрока MAX никогда не принимается решение, что нужно собирать информацию (или предоставлять информацию партнеру), поскольку этого не требуется делать в рамках каждой отдельной раздачи; тем не менее в таких играх, как бридж, часто бывает целесообразно сыграть такой кар-

той, которая помогла бы кое-что узнать о картах противника или сообщить партнеру о своих собственных картах. Такие стереотипы поведения формируются автоматически оптимальным алгоритмом ведения игр с неполной информацией. Подобный алгоритм выполняет поиск не в пространстве состояний мира (под этим подразумеваются карты, находящиеся на руках у игроков), а в пространстве доверительных состояний (представлений о том, кто какие карты имеет и с какими вероятностями).

Авторы смогут объяснить этот алгоритм должным образом в главе 17 после разработки всего необходимого вероятностного инструментария. А в данной главе необходимо также остановиться на одном заключительном и очень важном замечании: в играх с неполной информацией лучше всего выдавать противнику как можно меньше информации, а наилучший способ сделать это чаще всего состоит в том, чтобы действовать непредсказуемо. Вот почему санитарные врачи, посещая для проверки предприятия общественного питания, не сообщают заранее о своих визитах.

6.6. Современные игровые программы Публичная демонстрация программ ведения игр представляет собой для искусственного интеллекта примерно то же, что и участие в автогонках международного масштаба для автомобильной промышленности — суперсовременные игровые программы действуют поразительно быстро, чрезвычайно тщательно настроенные компьютеры воплощают в себе наиболее качественные инженерные решения, но это все не предназначено для рядового покупателя. А некоторые исследователи даже считают, что ведение игр представляет собой нечто, не имеющее отношения к основному направлению развития искусственного интеллекта. Тем не менее эта область продолжает порождать не только всеобщее восхищение, но и постоянный поток инноваций, которые затем усваиваются более широким сообществом разработчиков. ^ Шахматы. В 1957 году Герберт Саймон предсказал, что через 10

лет компьютеры победят человека — чемпиона мира по шахматам. Через сорок лет программа Deep Blue победила Гарри Каспарова в показательном матче из шести игр. Саймон ошибся, но лишь с коэффициентом 4. Каспаров писал:

Решающей игрой в этом матче была вторая партия, которая оставила глубокий след в моей памяти... Мы наблюдали события, которые намного превосходили самые невероятные ожидания в отношении того, насколько хорошо компьютер будет способен предвидеть долговременные позиционные последствия своих решений. Машина отказалась перейти в позицию, имеющую явное, но кратковременное преимущество, продемонстрировав вполне человеческое ощущение опасности [774].

Программа Deep Blue была создана Мьюрреем Кэмпбеллом, Фенгсюнг Су и Джозефом Хоаном из компании IBM [216] на основе проекта Deep Thought, разработанного ранее Кэмпбеллом и Су в университете Карнеги—Меллона (Carnegie—Mellon University— CMU). Компьютер-победитель представлял собой параллельный компьютер с 30 процессорами IBM RS/6000. На этом компьютере эксплуатировались средства "программного поиска" и 480 специализированных СБИС шахматных процессоров, которые осуществляли выработку ходов (включая упорядочение ходов), "аппаратного поиска" для последних нескольких уровней дерева и проводилась оценка листовых узлов. В программе Deep Blue в среднем осуществлялся поиск 126 миллионов узлов в секунду, а пиковая скорость достигала 330 миллионов узлов в

секунду. Эта программа формировала вплоть до 30 миллиардов позиций в расчете на каждый ход, обычно достигая глубины поиска 14. Основой этой программы является стандартный альфа-бета-поиск с итеративным углублением на основе таблицы транспозиций, но ключом к успеху этой программы, по-видимому, стала ее способность вырабатывать расширения, выходящие за пределы глубины поиска для достаточно интересных линий форсирующих/форсированных ходов. В некоторых случаях этот поиск достигал глубины в 40 полуходов. Функция оценки охватывала свыше 8000 характеристик, причем многие из них описывали в высшей степени специфичные шаблоны расположения фигур. Использовался справочник дебютов, состоящий примерно из 4000 позиций, а также база данных с 700 000 игр гроссмейстеров, из которой программа могла извлекать согласованные рекомендации. Кроме того, в этой системе применялась большая база данных эндшпилей, состоящая из позиций с решениями, в которой содержались все позиции с пятью фигурами и многие позиции с шестью фигурами. Использование этой базы данных привело к значительному увеличению эффективной глубины поиска, что позволило программе Deep Blue в некоторых случаях играть идеально даже за много ходов до мата.

Успех программы Deep Blue укрепил и без того широко распространенное мнение, что прогресс в области ведения компьютерных игр достигается главным образом за счет все более мощного аппаратного обеспечения; к тому же распространение таких взглядов стимулировалось компанией IBM. Создатели программы Deep Blue, с другой стороны, утверждают, что важную роль сыграло также расширение поиска и применение продуманной функции оценки [216]. Более того, нам известно, что некоторые новейшие алгоритмические усовершенствования позволяют программам, работающим на стандартных персональных компьютерах, побеждать на каждом мировом чемпионате по компьютерным шахматам с 1992 года и часто наносить поражение противникам с массовой параллельной архитектурой, способным выполнять поиск в 1000 раз большего количества узлов в секунду. Для сокращения эффективного коэффициента ветвления меньше чем до 3 (по сравнению с фактическим коэффициентом ветвления, составляющим около 35) применяются всевозможные эвристики отсечения. Наиболее важной из них является эвристика с Δ пустым ходом, в которой вырабатывается хороший нижний предел значения позиции с использованием поверхностного поиска, в котором противнику в начале игры разрешается сделать ход дважды. Этот нижний предел часто позволяет выполнять алфавита-отсечение без затрат на полный поиск в глубину. Является также важным метод Δ отсечения ненужных ходов, который позволяет решить заранее, какие ходы вызовут альфа-бета-отсечение в узлах-преемниках.

Группа разработчиков Deep Blue отказалась воспользоваться шансом провести матч-реванш, предложенный Каспаровым. Вместо этого в одном из самых последних крупных соревнований в 2002 году против чемпиона мира Владимира Крамника выступила программа Fritz. Матч из восьми игр окончился ничьей. Условия этого матча были гораздо более благоприятными для человека, и в качестве аппаратного обеспечения использовался обычный персональный компьютер, а не суперкомпьютер. Тем не менее Крамник прокомментировал этот матч так: "Теперь очевидно, что эта самая лучшая программа и чемпион мира играют примерно на равных". Δ Шашки. Начиная с 1952 года Артур Самюэл из компании IBM в свое свободное время занимался разработкой программы игры в шашки, которая совершенствовалась с помощью обучения свою собственную функцию оценки, играя сама с собой

тысячи раз. Эта идея будет описана более подробно в главе 21. Программа Самюэла вначале играла на уровне новичка, но всего лишь через несколько дней игры с самой собой усовершенствовалась до такого уровня, что стала побеждать Самюэла (хотя он не был сильным игроком). В 1962 году эта программа победила Роберта Нили, чемпиона игры в шашки "вслепую", благодаря ошибке с его стороны. Многие в то время посчитали, что компьютеры уже играют в шашки лучше людей, но

фактически этого еще не произошло. Тем не менее, если учесть то, что вычислительное оборудование, использовавшееся Самюэлом (компьютер IBM 704), имело 10 000 слов основной памяти, магнитную ленту для долговременного хранения и процессор с частотой 0,000001 ГГц, эта победа остается большим достижением.

Превзойти данное достижение пытались многие, и, наконец, Джонатан Шеффер со своими коллегами разработал программу Chinook, которая работает на обычных персональных компьютерах и использует альфа-бета-поиск. В программе Chinook применяется заранее вычисленная база данных из всех 444 миллиардов позиций с восьмью или меньшим количеством шашек на доске, что позволяет ей играть в эндшпиль безошибочно. Программа Chinook заняла второе место в 1990 году на открытом чемпионате США и завоевала право сделать заявку на участие в мировом чемпионате.

Но затем эта программа столкнулась с проблемой в лице Мэриона Тинсли. Доктор Тинсли был чемпионом мира свыше 40 лет, проиграв за все это время только три партии. В первом матче против программы Chinook Тинсли потерпел свое четвертое и пятое поражение, но выиграл матч со счетом 20,5—18,5. Матч на звание чемпионата мира в августе 1994 года между Тинсли и программой Chinook закончился преждевременно, поскольку Тинсли был вынужден сдаться из-за ухудшения состояния здоровья.

Программа Chinook была официально признана чемпионом мира.

Шеффер считает, что при наличии достаточной вычислительной мощности база данных с эндшпилями может быть увеличена до такой степени, что прямой поиск из начальной позиции будет всегда достигать решенных позиций, т.е. задача игры в шашки должна быть полностью решена. (Программа Chinook иногда объявляла о своем выигрыше на пятом ходу.) Исчерпывающий анализ такого рода может быть выполнен вручную для игры в крестики-нолики 3x3 и с помощью компьютера для игр Qubic (объемные крестики-нолики 4x4x4), гомоку (пять в ряд) и Nine-Men's Morris (Мельница) [524]. В замечательной работе Кена Томпсона и Льюиса Стиллера [1464] приведены решения всех шахматных эндшпилей с пятью фигурами и некоторых эндшпилей с шестью фигурами, причем эти результаты предоставлены для всеобщего доступа в Internet. Стиллер обнаружил один вариант, в котором достигался форсированный мат, но он состоял из 262 ходов; этот результат вызвал некоторый переполох, поскольку в шахматных правилах установлено, чтобы в течение 50 ходов происходил хоть какой-то "прогресс", иначе засчитывается ничья.

Игра ^ "Отелло", называемая также "Реверси", по-видимому, более популярна как компьютерная игра, а не настольная. Она имеет меньшее пространство поиска, чем шахматы, поскольку в ней обычно имеется от 5 до 15 допустимых ходов, но опыт в

оценке позиций в ней пришлось накапливать буквально с нуля. В 1997 году программа Logistello [209] победила человека — чемпиона мира, Такеси Мураками, со счетом 6:0. Теперь общепризнано, что люди не могут соревноваться с компьютерами в игре "Отелло". ^ Нарды. В разделе 6.5 описано, почему из-за включения элемента неопределенности, вызванного метанием жребия, глубокий поиск становится дорогостоящей

роскошью. В области игры в нарды было предпринято много усилий по усовершенствованию функции оценки. Гэри Тесауро [1499] использовал сочетание метода обучения с подкреплением Самюэла и методов нейронных сетей (глава 20) для разработки удивительно точного средства оценки, которое использовалось при поиске на глубину 2 или 3. Сыграв против самой себя больше миллиона тренировочных игр, разработанная Гэри Тесауро программа TD-Gammon по праву заняла место среди лучших трех игроков мира. Некоторые рекомендации этой программы по выбору дебютных ходов в начальной стадии игры в нарды радикально расходились с "мудрыми" советами, передававшимися из поколения в поколение в течение многих веков. ^ Го — это наиболее популярная настольная игра в Азии, которая для полного овладения мастерством требует от профессионалов столько же усилий, как шахматы.

Поскольку в ней доска имеет размеры 19x19, коэффициент ветвления начинается с 361, а эта величина слишком обременительна для обычных методов поиска. Вплоть до 1997 года вообще не было ни одной достаточно компетентной программы, но теперь программы часто делают ходы, достойные уважения. В большинстве наилучших программ сочетаются методы распознавания шаблонов (в которых используется принцип — при появлении такого-то шаблона из камней необходимо рассмотреть такой-то ход) с ограниченным поиском (для выработки решения о том, следует ли стремиться к захвату таких-то камней, не выходя за пределы данной локальной области). Ко времени написания данной книги, по-видимому, самыми сильными программами были Goemate Чен Жишинга и Go4++ Майкла Рейса, каждая из которых достигает рейтинга, примерно равного 10 кю (уровень слабого любителя).

Ведение игры го — это такая область, которая, скорее всего, достигнет развития в результате интенсивных исследований с использованием более сложных методов формирования рассуждений. Успех может быть достигнут в результате обнаружения способов интеграции нескольких линий локальных рассуждений о каждой из многих слабо связанных "субигр", на которые может быть выполнена декомпозиция всей игры го. Подобные методы имели бы колоссальную ценность для всех интеллектуальных систем в целом. ^ Бридж — это игра с неполной информацией: карты любого игрока скрыты от других игроков. Кроме того, бридж — это игра с

несколькими игроками, в которой участвуют четыре игрока, а не два, хотя игроки разбиты по парам на две команды.

Как было показано в разделе 6.5, оптимальная игра в бридж может включать элементы сбора информации, передачи информации, введения в заблуждение и тщательного взвешивания вероятностей. Многие из этих методов используются в программе Bridge Baron™ [1441], которая выиграла в 1997 году чемпионат мира по бриджу среди компьютеров. Хотя программа Bridge Baron не играет оптимальным образом, она представляет собой одну из немногих успешно действующих систем ведения игры, в которой используются сложные, иерархические планы (см. главу 12), основанные на таких идеях высокого уровня, как импас (finessing) и сквиз (squeezing), которые знакомы игрокам в бридж.

Чемпионат мира 2000 года с большим отрывом от соперников выиграла программа GIB [559]. В программе GIB используется метод "усреднения по прогнозам" с двумя важными модификациями. Во-первых, в этой программе вместо исследования того, насколько удачным окажется каждый вариант игры по отношению к каждому варианту возможного расположения скрытых карт (количество которых может достигать 10 миллионов), исследуется случайно выбранный образец из 100 расположений. Во-вторых, в программе GIB используется обобщение на основе объяснения для вычисления и кэширования общих правил оптимальной игры в виде определений различных стандартных классов ситуаций. Это позволяет данной программе принимать точное решение в отношении каждой раздачи. Тактическая точность программы GIB компенсирует ее неспособность формировать рассуждения в отношении имеющейся информации. Она финишировала на 12-м месте среди 35 участников в парных соревнованиях (в которых предусматривался только розыгрыш карт, имеющихся на руках) в чемпионате мира среди людей в 1998 году, значительно превзойдя ожидания многих специалистов в этой области.

6.7. Обсуждение изложенных сведений Поскольку вычисление оптимальных решений в играх чаще всего является неосуществимым, во всех алгоритмах необходимо использовать некоторые предположения и допущения. Стандартный подход, основанный на использовании минимаксных значений, функций оценки и альфа-бета-отсечения, представляет собой лишь один из способов достижения этой цели. По-видимому, из-за того, что он был предложен уже давно, этот стандартный подход интенсивно развивался и стал доминировать над другими методами, участвующими в борьбе за право на существование.

Некоторые специалисты в этой области считают, что такое состояние дел стало причиной отделения проблематики ведения игр от основного направления развития исследований по искусственному интеллекту, поскольку этот стандартный подход больше не предоставляет большого простора для новых открытий,

позволяющих найти ответы на общие вопросы в области принятия решений. В настоящем разделе описаны некоторые альтернативные варианты.

Вначале рассмотрим минимаксный поиск. Алгоритмы минимаксного поиска позволяют выбрать оптимальный ход в данном конкретном дереве поиска, при условии, что оценки листовых узлов являются абсолютно правильными. Но в действительности оценки обычно представляют собой грубые прогнозы стоимости той или иной позиции, поэтому можно считать, что с ними связаны существенные ошибки. На рис. 6.11 показано дерево игры с двумя полуходами, для которого минимаксный поиск представляется неподходящим. Минимаксный поиск подсказывает, что должна быть выбрана правая ветвь, тогда как весьма вероятно, что истинное значение левой ветви должно быть выше. Минимаксный выбор основывается на предположении, что все узлы, отмеченные значениями 100, 101, 102 и 100, действительно лучше, чем узел, отмеченный значением 99. Однако тот факт, что узел с отметкой 99 имеет сестринские узлы с отметками 1000, наводит на мысль, что фактически он может иметь более высокое истинное значение. Один из способов справиться с этой проблемой состоит в том, чтобы использовать какую-то оценку, которая возвращает распределение вероятностей среди возможных значений. В таком случае появляется возможность вычислить распределение вероятностей для значения родительского узла с использованием стандартных статистических методов. К сожалению, обычно значения сестринских узлов являются в высшей степени коррелированными, поэтому такое вычисление может оказаться дорогостоящим и требующим больших усилий для получения информации.

Затем рассмотрим алгоритм поиска, в котором формируется дерево. Каждый проектировщик алгоритмов стремится создать такой способ вычислений, который действует быстро и позволяет определить хороший ход. Наиболее очевидным недостатком альфа-бета-алгоритма является то, что он предназначен не только для выбора хорошего хода, но и для расчета пределов значений всех допустимых ходов. Чтобы понять, почему эта лишняя информация часто не нужна, рассмотрим позицию, в которой имеется только один допустимый ход.

99 1000 1000 1000 100 101 102 100 Рис. 6.11. Дерево игры с двумя полуходами, для которого минимаксный поиск может оказаться неподходящим. Альфа-бета-поиск все еще вырабатывает и оценивает большое и полностью бесполезное дерево поиска. Безусловно, можно предусмотреть в этом алгоритме какую-то проверку подобной ситуации, но она просто маскирует основную проблему: многие вычисления, выполняемые в альфа-бета-алгоритме, практически ничего не дают для решения задачи. Ситуация, в которой имеется только один допустимый ход, ненамного отличается от той, где имеется несколько допустимых ходов, притом что один из них является правильным, а остальные — явно катастрофическими. В

подобной ситуации наличия "четко выраженного фаворита" было бы желательно быстро достигать решения после проведения небольшого объема поиска, чем терять время, которое можно было бы использовать продуктивнее в дальнейшем, в более проблематичной позиции. Это ведет к идее полезности развертывания узла.

Хороший алгоритм поиска должен выбирать варианты развертывания узлов с высокой полезностью, т.е. те варианты, которые, со всей вероятностью, приведут к обнаружению гораздо лучшего хода. Если же отсутствуют варианты развертывания узлов, полезность которых превышает их стоимость (измеряемую в затратах времени), то алгоритм должен прекратить поиск и выбрать ход. Следует отметить, что такое усовершенствование касается не только ситуаций с явными фаворитами, но и тех случаев, когда имеются симметричные ходы, для которых сколь угодно большой объем поиска не позволит показать, что один ход лучше другого.

Рассуждения о том, какие вычисления следует выполнять, а какие нет, называются $\wedge$ метарассуждениями (рассуждениями о рассуждениях). Этот подход распространяется не только на ведение игр, но и в целом на рассуждения любого рода. Все вычисления выполняются для того, чтобы выработать лучшие решения, все они имеют стоимость и характеризуются определенной вероятностью достижения конкретного улучшения качества решения. Альфа-бета-поиск представляет собой реализацию метарассуждений простейшего вида, а именно теоремы о том, что некоторые ветви дерева можно игнорировать без ущерба для всей игры. Но такие метарассуждения могут проводиться гораздо лучше. В главе 16 будет показано, как сделать изложенные идеи более точными и реализуемыми.

Наконец, еще раз рассмотрим природу самого поиска. Алгоритмы эвристического поиска и ведения игр действуют путем выработки последовательностей конкретных состояний (начиная от начального состояния), а затем применения функции оценки. Безусловно, люди играют в игры иначе. В шахматах игрок часто руководствуется конкретной целью (например, поймать ферзя противника) и может использовать эту цель для избирательной выработки осуществимых планов ее достижения. Иногда такого рода подход на основе рассуждений, управляемых целью, или планирования, позволяет полностью устранить комбинаторный поиск (см. часть IV).

Программа Paradise Дэвида Уилкинса [1592] — это единственная программа, в которой с успехом использовались рассуждения, управляемые целью, для игры в шахматы; она оказалась способной решать некоторые шахматные задачи, для которых требовались комбинации из 18 ходов. Тем не менее еще нет полного понимания того, как объединить эти два типа алгоритмов в надежную и эффективную систему, хотя программа Bridge Baron может рассматриваться как шаг в правильном направлении.

Полностью интегрированная система стала бы значительным достижением не только в области исследований ведения игр, но и для всех исследований по искусственному интеллекту в целом, поскольку послужила бы хорошей основой для создания интеллектуального агента общего назначения.

Резюме В этой главе были проанализированы самые разные игры с тем, чтобы можно было понять, что означают слова "оптимальная игра", а также узнать, как научиться хорошо играть на практике. Ниже изложены наиболее важные идеи, которые рассматривались в данной главе.

- Любая игра может быть определена с помощью начального состояния (которое показывает, как осуществляется подготовка доски к игре), допустимых действий в каждом состоянии, проверки терминального состояния (позволяющей определить, когда игра окончена) и функции полезности, которая применяется к терминальным состояниям.
- В играх с двумя игроками и нулевой суммой, характеризующихся полной информацией, для выбора оптимальных ходов с помощью перебора узлов в глубину в дереве игры может использоваться алгоритм минимаксного поиска.
- Алгоритм альфа-бета-поиска вычисляет такие же оптимальные ходы, как и алгоритм минимаксного поиска, но позволяет достичь гораздо большей эффективности, удаляя поддеревья, которые, по всей вероятности, не нужны для поиска решения.
- Обычно не представляется возможным рассматривать все дерево игры (даже с помощью альфа-бета-поиска), поэтому необходимо в какой-то точке останавливать поиск и применять функцию оценки, позволяющую определить приближенное значение полезности некоторого состояния.
- Ведение игр с элементами случайности можно осуществить с помощью расширения алгоритма минимаксного поиска, в котором оцениваются узлы же-

ребьевки путем определения средней полезности всех их дочерних узлов с учетом вероятности каждого дочернего узла.

- Для определения оптимальных ходов в играх с неполной информацией, таких как бридж, необходимо формировать рассуждения о текущем и будущем доверительных состояниях для каждого игрока. Одна из простых аппроксимаций может быть получена путем усреднения значения данного действия по всем возможным конфигурациям недостающей информации.
- Программы способны соревноваться на равных или побеждать лучших игроков-людей в шашках, игре "Отелло" и в нардах, а также вплотную приблизились к ним в бридже. Программа победила чемпиона мира по шахматам в одном показательном матче. В игре го программы до сих пор остаются на любительском уровне.

Библиографические и исторические заметки Ранняя история развития механических средств ведения игр была запятнана многочисленными случаями мошенничества. Наиболее известным из них был "Турок" барона Вольфганга фон Кемпелена A734—1804). Это устройство выдавали за автомат для игры в шахматы и смогли с его помощью обмануть очень многих, в том числе Наполеона. Лишь в дальнейшем

выяснилось, что это устройство представляет собой хитроумный ящик фокусника, в котором сидит человек, хорошо играющий в шахматы [919]. Игры с участием этого устройства проводились с 1769 по 1854 годы. В 1846 году Чарльз Бэббидж (который был в восторге от "Турка") участвовал в одной из первых серьезных дискуссий на тему осуществимости задачи ведения игр в шахматы и шашки с помощью вычислительного устройства [1089]. Он также спроектировал, но не построил машину специального назначения для игры в крестики-нолики. Первая настоящая машина для ведения игры была построена примерно в 1890 году испанским инженером Леонардо Торресом и Кеведо. Она была специально предназначена для ведения шахматного эндшпиля "KRK" (King and Rook vs. King — король и ладья против короля) и гарантировала победу из любой позиции в игре с этими тремя фигурами.

В качестве первоисточника идеи алгоритма минимаксного поиска часто называют статью, опубликованную в 1912 году Эрнстом Цермело, основоположником современной теории множеств [1642]. К сожалению, эта статья содержала несколько ошибок, а алгоритм минимаксного поиска не был в ней описан правильно.

Солидный фундамент теории игр был создан в оригинальной работе Theory of Games and Economic Behavior [1546], которая содержала анализ, показывающий, что для некоторых игр требуются стратегии, которые являются рандомизированными (или становятся непредсказуемыми для противников по каким-то иным причинам).

Дополнительная информация на эту тему приведена в главе 17.

На заре компьютерной эры возможность создания компьютерных шахмат интересовала многих важных деятелей в этой области. Конрад Цузе [1651] (первый, кто спроектировал программируемый компьютер) разработал довольно подробные предложения, касающиеся того, как может быть решена эта задача. Во влиятельной книге Cybernetics (Кибернетика) Норберта Винера [1589] обсуждался один возможный

Часть II. Решение проблем проект создания шахматной программы и были высказаны идеи минимаксного поиска, остановка на заданной глубине и применения функций оценки. Клод Шеннон [1395] изложил основные принципы создания современных программ ведения игр гораздо более подробно, чем Винер. Он ввел понятие поиска спокойной позиции, а также высказал некоторые идеи в отношении избирательного (неисчерпывающего) поиска в дереве игры. Слейтор [1430] и комментаторы его статьи также рассматривали возможности ведения игры в шахматы с помощью компьютера. В частности, Гуд [574] разработал понятие спокойной позиции независимо от Шеннона.

В 1951 году Алан Тьюринг написал первую компьютерную программу, способную вести полную игру в шахматы [1521]. Но программа Тьюринга фактически никогда

не эксплуатировалась на компьютере; она была испытана путем моделирования вручную в партии против очень слабого игрока-человека, который ее победил. Между тем, Д. Г. Принц [1238] написал и проверил на практике программу, которая решала шахматные задачи, но не могла вести полную игру. Первая программа³, способная вести полную игру в стандартные шахматы, была написана Алексом Бернстейном [112], [113].

Джон Маккарти сформулировал идею альфа-бета-поиска в 1956 году, но не опубликовал свое открытие. В шахматной программе NSS [1128] используется упрощенная версия альфа-бета-поиска; она представляет собой первую шахматную программу, в которой было введено это усовершенствование. По данным Нильссона [1141], в программе игры в шашки Артура Самюэла [1349], [1350] также использовался альфа-бета-поиск, хотя Самюэл об этом не упоминал в опубликованных отчетах о своей системе. Статьи с описанием альфа-бета-поиска были опубликованы в начале 1960-х годов [198], [625], [1427]. Одна из реализаций полного альфа-бетапоиска описана Слаглем и Диксоном [1428] применительно к программе ведения игры калах. Кроме того, альфа-бета-поиск использовался в шахматной программе "Kotok-McCarthy", написанной студентом Джоном Маккарти [847]. Кнут и Мур [810] изложили историю создания алгоритма альфа-бета-поиска, а также привели доказательство его правильности и представили результаты анализа временной сложности. Проведенный ими анализ альфа-бета-поиска со случайным упорядочиванием преемников показал его асимптотическую сложность $O(b/\log b)d$, что представляло собой довольно мрачный прогноз, поскольку соответствующий ему эффективный коэффициент ветвления $b/\log b$ не намного меньше, чем само значение b . После этого они отметили, что указанная асимптотическая формула является точной только для $b > 10$ или таких порядков величин, тогда как часто упоминаемое значение $O(b^2d)$ относится к тому диапазону коэффициентов ветвления, который чаще всего встречается в настоящих играх. Перл [1187] показал, что альфа-бета-поиск является асимптотически оптимальным среди всех алгоритмов поиска в дереве игры на постоянную глубину.

В первом шахматном матче между компьютерами участвовала программа KotokMcCarthy и программа "ITEP" (Institute of Theoretical and Experimental Physics), написанная в середине 1960-х годов в Московском институте теоретической и экспериментальной физики (ИТЕФ) [4]. Этот межконтинентальный матч проводился по телеграфу. Он закончился в 1967 году победой программы ITEP со счетом 3:1.

Первой шахматной программой, которая успешно соревновалась с людьми, была программа NSS [1128] упоминается программа, написанная в Советском Союзе для компьютера БЭСМ, которая могла предшествовать программе Бернстейна.

программа MacHack 6 [594]. Ее рейтинг, примерно равный 1400, был намного выше 1000 — уровня начинающего, но все же далеко не достигал рейтинга 2800 или больше,

который требовался для выполнения предсказания Герберта Саймона, высказанного в 1957 году, что компьютерная программа станет чемпионом мира по шахматам через 10 лет [1419].

Начиная с первого Североамериканского чемпионата ACM по компьютерным шахматам, проведенного в 1970 году, между шахматными программами разгорелась серьезная конкуренция. Программы, разработанные в начале 1970-х годов, стали чрезвычайно сложными, насыщенными всевозможными ухищрениями, которые были предназначены для устранения некоторых ветвей поиска, выработки приемлемых ходов и т.д. В 1974 году в Стокгольме проводился первый чемпионат мира по компьютерным шахматам, в котором победила Kaissa [5], еще одна программа, разработанная в ИТЕФ. В программе Kaissa использовался гораздо более прямолинейный подход к организации исчерпывающего альфа-бета-поиска в сочетании с поиском спокойных позиций.

Перспективность этого подхода была подтверждена убедительной победой программы Chess 4.6 на чемпионате мира по компьютерным шахматам в 1977 году. Программа Chess 4.6 исследовала до 400 000 позиций за каждый ход и имела рейтинг 1900.

Последняя версия разработанной Гринблаттом программы MacHack 6 была первой шахматной программой, которая эксплуатировалась на специализированном аппаратном обеспечении, разработанном специально для шахмат [1094], но первой программой, в которой удалось добиться заметного успеха благодаря использованию заказного аппаратного обеспечения, была программа Belle [286]. Применявшееся в программе Belle аппаратное обеспечение для выработки ходов и оценки позиции позволяло ей исследовать несколько миллионов позиций за каждый ход. Программа Belle достигла рейтинга 2250 и стала первой программой уровня мастера. В университете CMU бывшим чемпионом мира по игре в шахматы по переписке Хансом Берлинером и его студентом Карлом Эбелингом была разработана система Hitech, также представлявшая собой компьютер специального назначения, который обеспечивал быстрое вычисление функций оценки [428], [108]. Система Hitech, которая вырабатывала около 10 миллионов позиций за ход, стала чемпионом Северной Америки по компьютерным шахматам в 1985 году и оказалась первой программой, которая смогла победить гроссмейстера-человека в 1987 году. Система Deep Thought, которая также была разработана в университете CMU, явилась еще одним шагом в направлении чистого ускорения поиска [697]. Она достигла рейтинга 2551 и стала предшественником системы Deep Blue. В 1980 году была основана премия Фредкина (Fredkin), в которой было предложено 5000 долларов за первую программу, достигшую уровня мастера, 10 000 долларов за первую программу, достигшую рейтинга 2500 USCF (United States Chess Federation — Шахматная федерация США) (что примерно соответствует уровню

гроссмейстера), и 100 000 долларов за первую программу, победившую чемпиона мира по шахматам. Премию 5000 долларов получила программа Belle в 1983 году, премию 10 000 долларов — система Deep Thought в 1989 году, а премию 100 000 долларов — система Deep Blue за победу над Гарри Каспаровым в 1997 году. Необходимо учитывать, что успех Deep Blue был обусловлен не только усовершенствованием аппаратных средств, но и применением лучших алгоритмов [216], [696]. Применение таких методов, как эвристика нулевого хода [90], привело к созданию программ, которые стали весьма избирательными в своих поисках. В последних трех чемпионатах мира по компьютерным шахматам, проводив-

шихся в 1992, 1995 и 1999 годах, победили программы, работающие на стандартных персональных компьютерах. По-видимому, наиболее полное описание современной шахматной программы предоставлено Эрнстом Хейнцем [644], чья программа DarkThought получила самый высокий рейтинг среди некоммерческих программ для персональных компьютеров на чемпионате мира 1999 года.

Для преодоления проблем, связанных со "стандартным подходом", кратко описанных в разделе 6.7, было предпринято несколько попыток. По-видимому, первым избирательным алгоритмом поиска, в котором учитывались некоторые теоретические обоснования методов сокращения поиска, был B^* [105], в котором предпринята попытка устанавливать интервальные пределы возможных значений узла в дереве игры, а не давать единственную точечную оценку. Листовые узлы выбираются для развертывания в целях уточнения пределов верхнего уровня до тех пор, пока не обнаруживается ход, который "безусловно является наилучшим". Палей [1165] развил идею алгоритма B^* , используя распределения вероятностей значений вместо интервалов. В алгоритме поиска конспиративного числа (conspiracy number search) Дэвида Маккаллестера [1004] предусмотрено развертывание листовых узлов, которые, изменяя свои значения, могут вынудить программу предпочесть новый ход от корня. В алгоритме MGSS* [1331] используются описанные в главе 16 методы теории решений для оценки стоимости развертывания каждого листа с точки зрения ожидаемого усовершенствования качества решения, формируемого от корня. Этот алгоритм превзошел альфа-бета-алгоритм, применяемый в игре "Отелло", несмотря на то, что в нем проводился поиск среди узлов, количество которых было на порядок меньше. Вообще говоря, подход, принятый в алгоритме MGSS*, может применяться для управления размышлениями в любой форме.

Альфа-бета-поиск во многих отношениях представляет собой аналог поиска на основе метода ветвей и границ, который был превзойден поиском A^* в случае с одним агентом, если не считать того, что альфа-бета-поиск рассчитан на двух игроков.

Алгоритм SSS* [1466] может рассматриваться как поиск A^* для двух игроков; в нем для достижения того же решения никогда не развертывается больше узлов, чем в

алгоритме альфа-бета-поиска. В его первоначальной форме алгоритм SSS* предъявлял чрезмерные требования к памяти и характеризовался значительными вычислительными издержками на сопровождение очереди, поэтому был практически не применимым. Тем не менее на основе алгоритма RBFS была разработана версия SSS* с линейно возрастающими потребностями в пространстве [843]. Плат и др. [1214] разработали новый подход к реализации алгоритма SSS* как комбинации альфа-бета-поиска и таблиц транспозиции, показали, как преодолеть недостатки первоначального алгоритма, и создали новый вариант, называемый MTD(f), который нашел свое применение во многих превосходных программах.

Д.Ф. Бил [89] и Дана Хау [1113], [1114] изучали недостатки минимаксного поиска, касающиеся приближенных оценок. Они показали, что при некоторых предположениях о независимости распределения листовых значений в дереве применение минимаксного поиска может привести к получению таких значений для корня, которые фактически являются менее надежными, чем полученные с помощью непосредственного использования самой функции оценки. В книге Перла *Heuristics* [1188] дано частичное объяснение этого кажущегося парадокса и приведен анализ многих алгоритмов ведения игр. Баум и Смит [83] предложили заменить алгоритм минимаксного поиска алгоритмом, основанном на учете вероятностей, и показали, что он приводит к осуществлению лучших вариантов выбора в некоторых играх.

Но объем теоретических результатов исследования влияния останова поиска на разных уровнях и применения функций оценки все еще остается недостаточным.

Алгоритм поиска на основе ожидаемого минимаксного значения был предложен Дональдом Мичи [1043], хотя, безусловно, он непосредственно следует из принципов оценки дерева игры, выдвинутых фон Нейманом и Моргенштерном. Брюс Боллард [65] распространил метод альфа-бета-отсечения на деревья с узлами жеребьевки. Первый успешно действующей программой игры в нарды была разработанная Берлинером программа BKG [104], [107]; в ней использовалась сложная, сконструированная вручную функция оценки, а поиск осуществлялся только на глубину 1. Это была первая программа, которая смогла победить чемпиона мира — человека в одной из основных классических игр [106]. По завершении соревнований Берлинер сразу же объявил, что это был очень короткий показательный матч (а не матч чемпионата мира) и что программе BKG очень повезло со жребием. Работа Герри Тесауро, приведшая вначале к созданию программы *Neurogammon* [1498], а затем *TD-Gammon* [1500], показала, что гораздо лучшие результаты могут быть достигнуты с помощью обучения с подкреплением. Эта тема будет рассматриваться более подробно в главе 21.

Первой из классических игр, полное решение которой было найдено компьютером, оказались не шахматы, а шашки. Кристофер Стрейчи [1469] написал первую

работоспособную программу игры в шашки. Шеффер [1357] составил очень доступный отчет о разработке программы Chinook, ставшей чемпионом мира по шашкам, в котором все факты изложены "без прикрас".

Первые программы для игры го были разработаны немного позднее по сравнению с программами для шашек и шахмат [906], [1281], а их развитие происходило более медленно. Райдер [1334] использовал подход, основанный исключительно на использовании поиска в сочетании с всевозможными методами избирательного отсека для преодоления колоссального коэффициента ветвления, характерного для этой игры. Зобрист [1650] применил правила "условие—действие", которые способны предложить приемлемые ходы при обнаружении известных шаблонов. Рейтман и Уилкоккс [1280] добились хороших результатов, сочетая применение правил и поиска, поэтому большинство современных программ было разработано на основе этого гибридного подхода. Мюллер [1103] подытожил современное состояние работы в области компьютеризации игры го и привел в своей книге много ссылок. Аншелевич [33] использовал подобные методы для создания программы для игры гекс.

Описания современных разработок можно найти в журнале Computer Go Newsletter, который публикует организация Computer Go Association.

Работы по компьютерному ведению игры появляются во многих разных источниках. В трудах конференции Heuristic Programming in Artificial Intelligence, имеющей довольно дезориентирующее название, публикуются отчеты о компьютерных олимпиадах, в рамках которых проводятся весьма разнообразные игры. Имеется также несколько отредактированных сборников важных статей об исследованиях в области ведения игр [920], [921], [988]. Международная ассоциация компьютерных шахмат (International Computer Chess Association— ICCA), основанная в 1977 году, публикует ежеквартальный журнал ICGA Journal (который раньше носил название ICCA Journal).

В серийно выпускаемой антологии Advances in Computer Chess публикуются важные статьи, начиная со статьи Кларке [268]. В томе 134 журнала Artificial Intelligence B002) содержатся описания современных программ для шахмат, "Отелло", гекса, сёги (японские шахматы), го, нарда, покера, Scrabble™ ("Эрудит") и др.

Упражнения 6.1. В этой задаче рассматриваются основные концепции ведения игр с использованием в качестве примера игры в крестики-нолики. Определим X_p как количество строк, столбцов или диагоналей, в которых находится точно p значков x и ни одного значка O . Аналогичным образом, O_p — это количество строк, столбцов или диагоналей, в которых находятся только p значков O . Функция полезности присваивает $+1$ любой позиции с $X_3=1$ и -1 любой позиции с $O_3=1$. Все остальные терминальные позиции имеют полезность 0 . Для нетерминальных позиций

используется линейная функция оценки, определенная $KaKEval(s) = 3X2(s) + Xi(s) - C02(s) + 01(s)$. а) Сколько приблизительно существует возможных вариантов игры в крестики-нолики? б) Покажите полное дерево игры, начиная от пустой доски вплоть до глубины 2 (т.е. глубины, на которой на доске имеется один значок X и один значок O), учитывая симметрию позиций. в) Проставьте в этом дереве оценки всех позиций на глубине 2. г) С использованием алгоритма минимаксного поиска проставьте в дереве зарезервированные значения для позиций на глубине 1 и 0 и воспользуйтесь этими значениями для выбора наилучшего начального хода. д) Обведите кружками узлы на глубине 2, которые не оценивались бы в случае применения альфа-бета-отсечения, исходя из предположения, что эти узлы вырабатываются в порядке, оптимальном для альфа-бета-отсечения.

6.2. Докажите следующее утверждение: для каждого дерева игры значение полезности, полученное игроком MAX с использованием минимаксных решений в игре против неоптимально действующего игрока MIN, никогда не будет ниже значений полезности, полученных в игре против оптимально действующего игрока MIN. Можете ли вы показать дерево игры, в котором игрок MAX может действовать еще лучше, используя неоптимальную стратегию против неоптимально действующего игрока MIN?

6.3. Рассмотрим игру с двумя игроками, показанную на рис. 6.12.

ЩмШ ЩйР Рис. 6.12. Начальная позиция простой игры. Игрок А ходит первым. Два игрока ходят по очереди, и каждый игрок должен переместить свою фишку в открытый смежный квадрат в любом направлении. Если противник занимает смежный квадрат, то игрок может перенести свою фишку через фишку противника на следующий открытый квадрат, если он имеется. (Например, если фишка А стоит в квадрате 3, а фишка В — в квадрате 2, то А снова может перейти в квадрат 1.) Игра заканчивается после того, как один из игроков достигает противоположного конца доски. Если игрок А достигает квадрата 4 первым, то значение этой игры для А равно +1; если игрок В достигает квадрата 1 первым, то значение этой игры для игрока А равно -1. а) Нарисуйте полное дерево игры с использованием следующих соглашений:

- записывайте каждое состояние как (sA/ sB) , где sA и sB обозначают местонахождения фишек;
- обводите квадратом каждое терминальное состояние и записывайте для него значение игры в кружке;
- обводите двойными квадратами циклические состояния (состояния, которые уже появлялись на пути к корню).

Поскольку не ясно, как присваивать значения циклическим состояниям, обозначайте каждое из них знаком ? в кружке. б) Обозначьте каждый узел его зарезервированным минимаксным значением (также в кружке). Объясните, как вы учитывали значения ? и почему. в) Объясните, почему стандартный алгоритм минимаксного поиска в этом дереве игры потерпит неудачу, и вкратце опишите,

каким образом вы могли бы его исправить, опираясь на ваш ответ на пункт б). Приведет ли модифицированный вами алгоритм к получению оптимальных решений для всех игр с циклами? г) Эту игру с четырьмя клетками можно обобщить до p клеток для любого $p > 2$. Докажите, что игрок А побеждает, если число p — четное, и проигрывает, если число p — нечетное.

6.4. Реализуйте генераторы хода и функции оценки для одной или нескольких из следующих игр: калах, "Отелло", шашки и шахматы. Сконструируйте агента общего назначения для ведения игры на основе альфа-бета-поиска, в котором используется ваша реализация. Сравните эффективность увеличения глубины поиска, усовершенствования способа упорядочения ходов и улучшения функции оценки. Насколько близко приближается полученный вами эффективный коэффициент ветвления к идеальному случаю самого совершенного упорядочения ходов?

6.5. Разработайте формальное доказательство правильности алгоритма альфа-бета-отсечения. Для этого рассмотрите ситуацию, показанную на рис. 6.13.

Вопрос заключается в том, следует ли выполнять отсечение в узле ψ , который представляет собой узел MAX и является одним из потомков узла π_1 . Основная идея такова, что отсечение должно выполняться тогда и только тогда, когда можно показать, что минимаксное значение l_x не зависит от значения ψ . а) Значение узла π_1 определяется с помощью выражения $L_1 = \min(L_2, L_{21}, \dots, L_{2b2})$. Найдите аналогичное выражение для узла π_2 , а следовательно, выражение для узла π_i в терминах значений узла ψ . б) Допустим, что l_i — минимальное (или максимальное) значение узлов слева от узла π_L на глубине i , минимаксное значение которого уже известно. Аналогичным образом, допустим, что g^i — минимальное (или максимальное) значение неисследованных узлов справа от g^i на глубине i . Перепишите ваше выражение для узла π_x в терминах значений l_L и g_L . в) Теперь переформулируйте это выражение для того, чтобы показать, что значение узла ψ не должно превышать некоторый предел, полученный на основе значений l_y для того, чтобы оно могло повлиять на значение узла π_x .

г) Повторите этот процесс для того случая, когда ψ является узлом MIN. 'А Рис. 6.13. Ситуация, возникающая при определении того, следует ли отсекал узел π_j 6.6. Sfe Ш Реализуйте алгоритм поиска по ожидаемому минимаксному значению и алгоритм *-альфа-бета-поиска, который описан Баллардом [65], для отсечения деревьев игры с узлами жеребьевки. Испытайте эти алгоритмы в такой игре, как нарды, и проведите измерения эффективности отсечения в *-альфа-бета-поиске.

6.7. Докажите, что после положительной линейной трансформации листовых значений (т.е. трансформации значения x в $ax+b$, где $a > 0$) выбор хода в дереве игры остается неизменным, даже если в нем имеются узлы жеребьевки.

6.8. Рассмотрите описанную ниже процедуру выбора ходов в играх с узлами жеребьевки: • Сформируйте некоторые последовательности метания жребия (скажем, 50) вплоть до подходящей глубины (скажем, 8). • После того как результаты метания жребия становятся известными, дерево игры преобразуется в детерминированное. Для каждой последовательности метания жребия найдите решение результирующего детерминированного дерева игры с помощью альфа-бета-поиска. • С использованием этих результатов оцените значение каждого хода и выберите наилучший.

Будет ли эта процедура действовать успешно? Почему да (или почему нет)?

6.9. Опишите и реализуйте среду ведения игры в реальном времени с несколькими игроками, где время образует часть состояния среды, а игрокам выделяются фиксированные промежутки времени.

6.10. Опишите или реализуйте описания состояний, генераторы ходов, проверки терминальных состояний, функции полезности и функции оценки для одной или нескольких из следующих игр: монополия, Scrabble ("Эрудит"), бридж

(под этим подразумевается вариант бриджа с заключением соглашения) и покер (выберите свой любимый вариант).

6.11. Тщательно изучите, как взаимодействуют события жеребьевки и частичная информация в каждой из игр, упомянутых в упр. 6.10. а) Для какой из них подходит стандартная модель на основе ожидаемых минимаксных значений? Реализуйте этот алгоритм и примените его в вашем агенте для ведения игры после внесения соответствующих изменений в среду ведения игры. б) Для какой из этих игр подходит схема, описанная в упр. 6.8? в) Обсудите вопрос о том, как можно было бы воспользоваться фактом, что в некоторых играх игроки не имеют одинаковые знания о текущем состоянии.

6.12. В алгоритме минимаксного поиска принято предположение, что игроки делают ходы по очереди, но в таких карточных играх, как вист и бридж, следующую взятку разыгрывает первым тот, кто взял предыдущую взятку. а) Модифицируйте алгоритм таким образом, чтобы он правильно действовал в этих играх. Для этого можно принять предположение, что имеется функция Winner (s), которая сообщает, какой игрок взял только что разыгранную взятку (если она имеется). б) Нарисуйте дерево игры для первой пары карт, имеющихся на руках, показанной нас. 262.

6.13. В программе игры в шашки Chinook широко используются базы данных с эндшпилями, которые предоставляют точные значения для каждой позиции с восемью или меньшим количеством шашек. Каким образом можно обеспечить эффективное формирование таких баз данных?

6.14. Обсудите вопрос о том, насколько успешно может применяться стандартный подход к ведению игры в таких играх, как теннис, пул (род игры на бильярде) и крокет, которые происходят в непрерывном физическом пространстве состояний.

6.15. Опишите, как изменятся алгоритмы минимаксного поиска и альфа-бета-поиска для случая игр с ненулевой суммой с двумя игроками, в которых каждый игрок имеет собственную функцию полезности. Может быть принято предположение, что каждый игрок знает функцию полезности другого игрока. Если в этих двух функциях нет ограничений полезности терминальных состояний, возможно ли отсечение какого-либо узла с помощью альфа-бета-поиска?

6.16. Предположим, что имеется шахматная программа, способная оценивать 1 миллион узлов в секунду. Выберите компактное представление состояния игры для хранения в таблице транспозиций. Сколько примерно записей удастся вам поместить в таблицу на 500 Мбайт, находящуюся в памяти? Будет ли этого достаточно для поиска в течение трех минут, отведенных на один ход? Сколько операций поиска в таблице вы сможете выполнить за время, которое потребуется для одной оценки? Теперь предположим, что таблица транспозиций больше, чем для нее отведено оперативной памяти. Сколько узлов вы сможете оценить за время, которое требуется для выполнения одного поиска на диске с использованием стандартного аппаратного обеспечения жесткого диска?

(empty page - no text in the layer)